

DATA MINING 1

Classification

Dino Pedreschi, Riccardo Guidotti

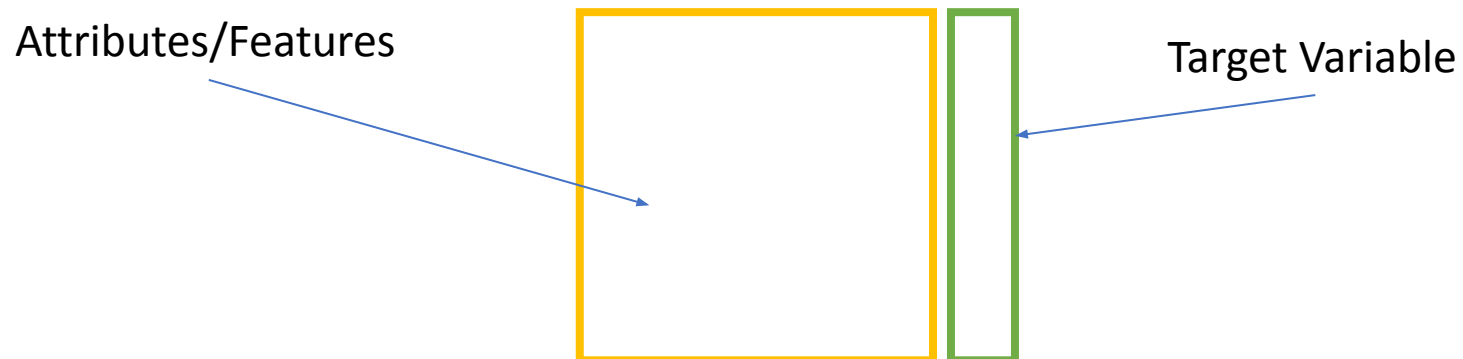
Revisited slides from Lecture Notes for Chapter 3 “Introduction to Data Mining”, 2nd Edition by Tan, Steinbach, Karpatne, Kumar



UNIVERSITÀ DI PISA

Supervised Learning

- Cluster analysis and association rules are not concerned with a specific target attribute.
- Supervised learning refers to problems where the value of a target attribute should be predicted based on the values of other attributes.
- Problems with a ***categorical target*** attribute are called **classification**, problems with a ***numerical target*** attribute are called **regression**.



What is Machine Learning?

- Machine Learning (ML) is the science (and art) of programming computers that can learn from data.



“ML is the field of study that gives computers the ability to learn without being explicitly programmed”
(Arthur Samuel, 1959)

What is Machine Learning?

- Machine Learning (ML) is the science (and art) of programming computers that can learn from data.



“A computer program is said to learn from **experience E** with respect to some **task T** and some **performance measure P**, if its performance on T, as measured by P, improves with experience E.”

(Tom Mitchell, 1997)

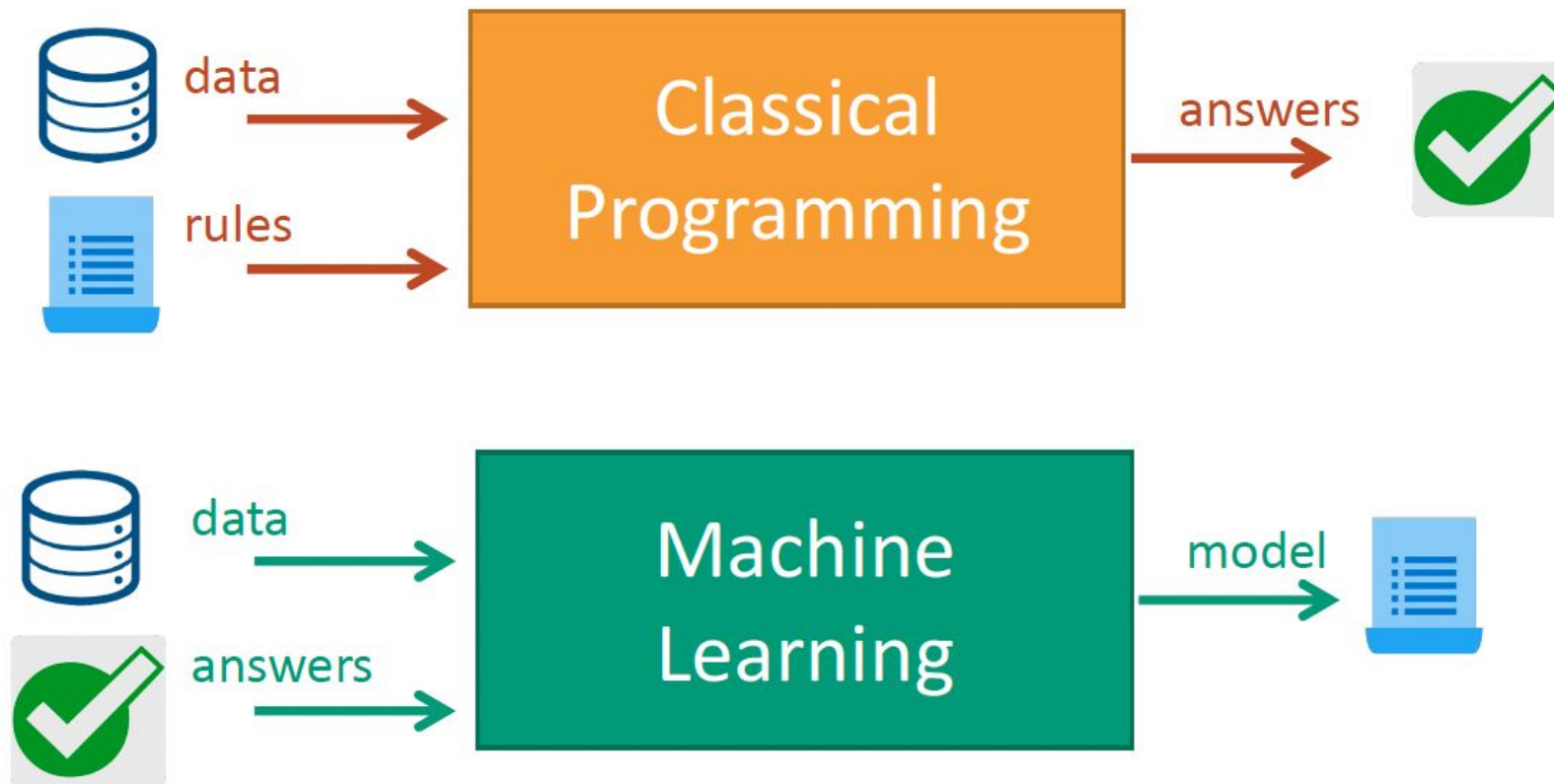
Classical Programming vs Machine Learning

- A ML system is trained rather than programmed



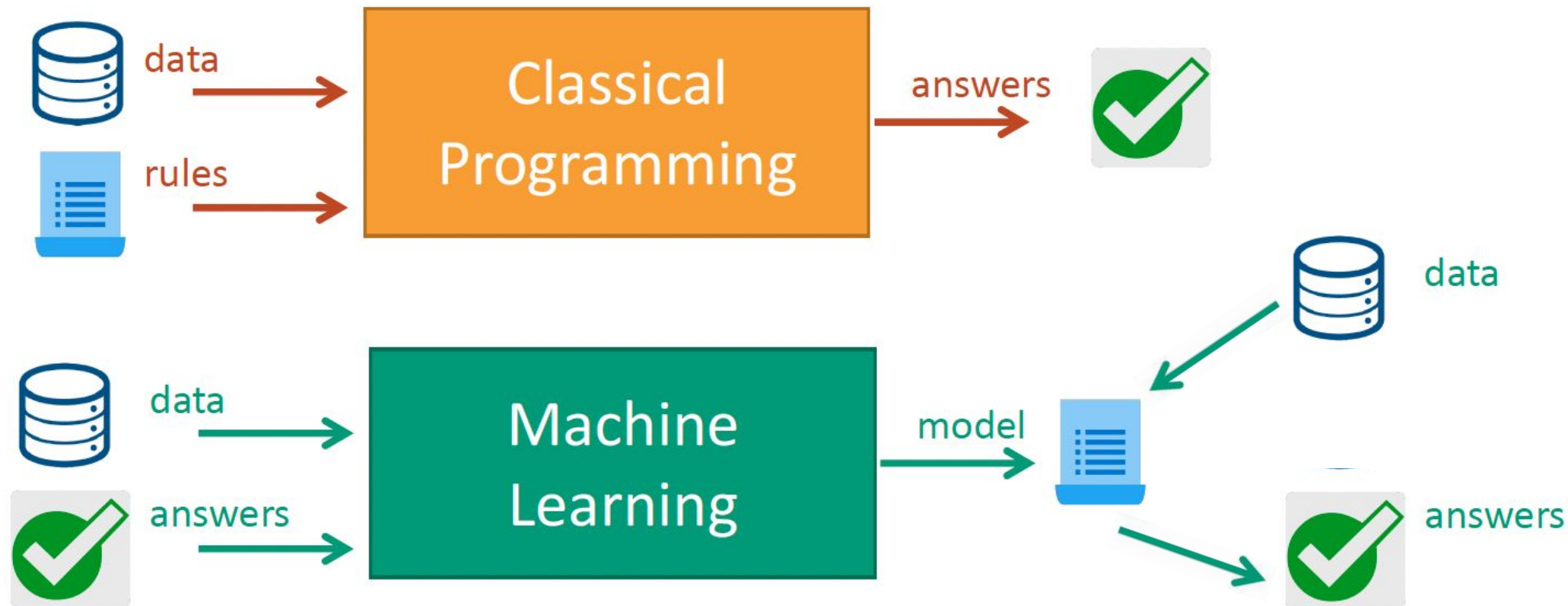
Classical Programming vs Machine Learning

- A ML system is trained rather than programmed

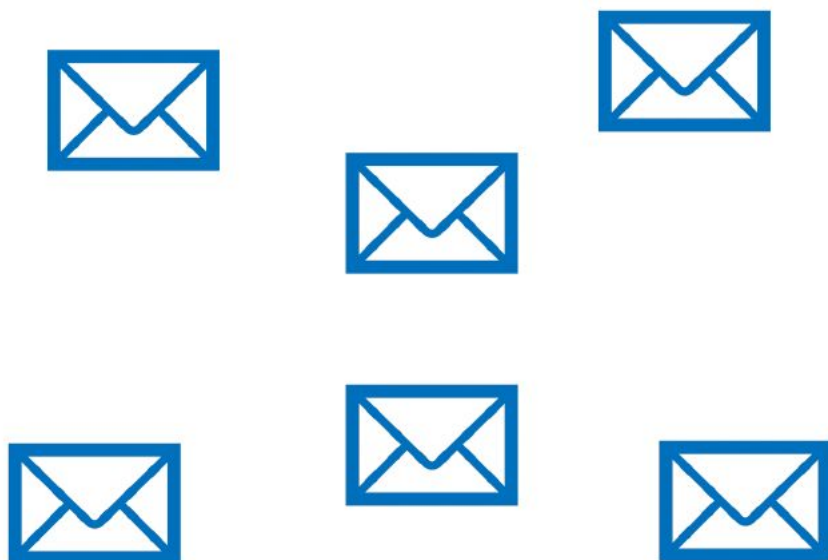


Classical Programming vs Machine Learning

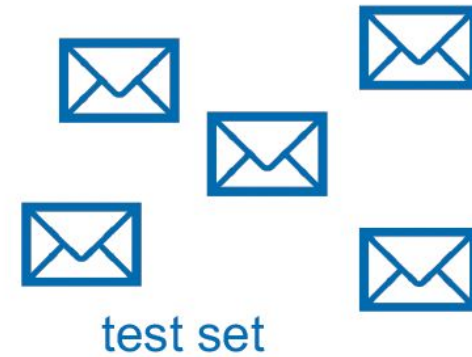
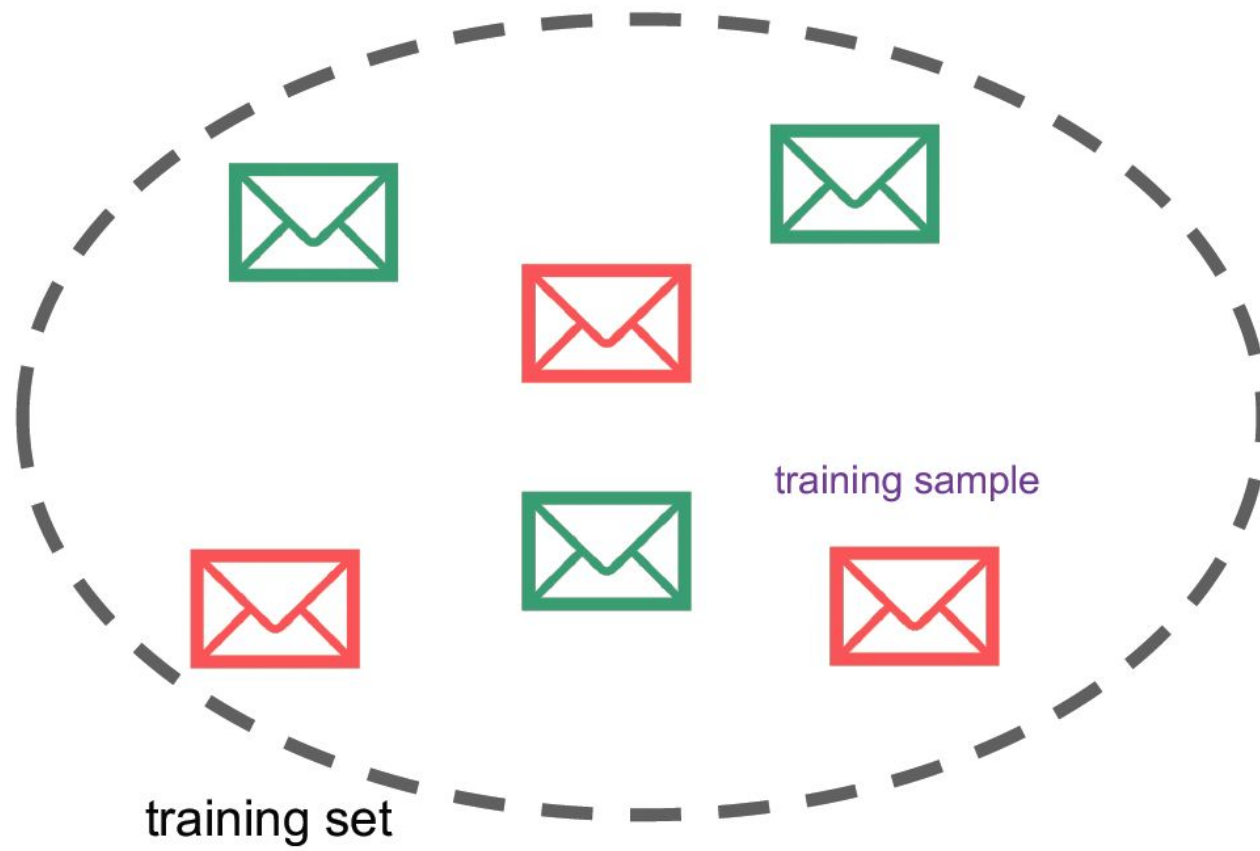
- A ML system is trained rather than programmed



Example Spam Filter



Example Spam Filter

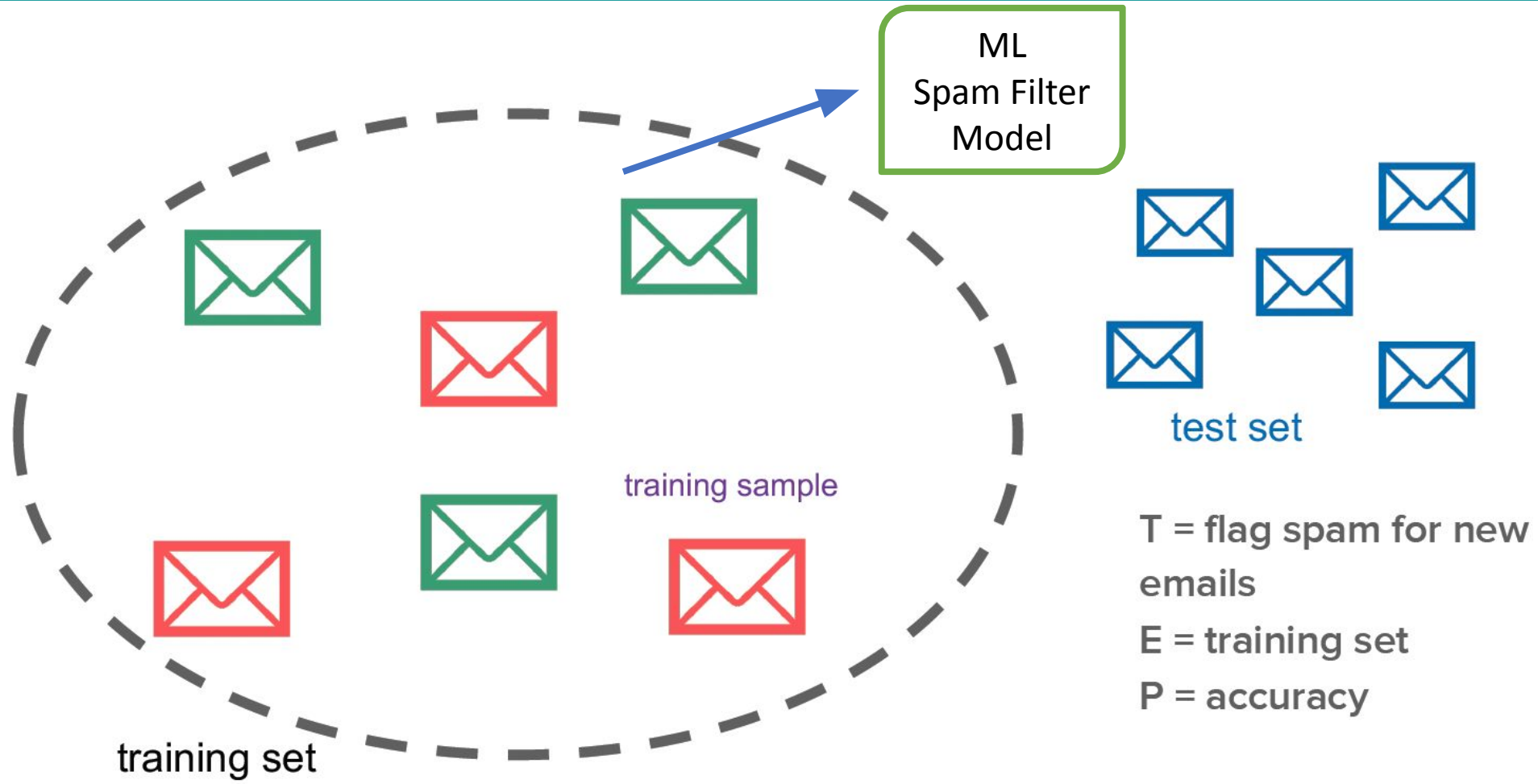


T = flag spam for new emails

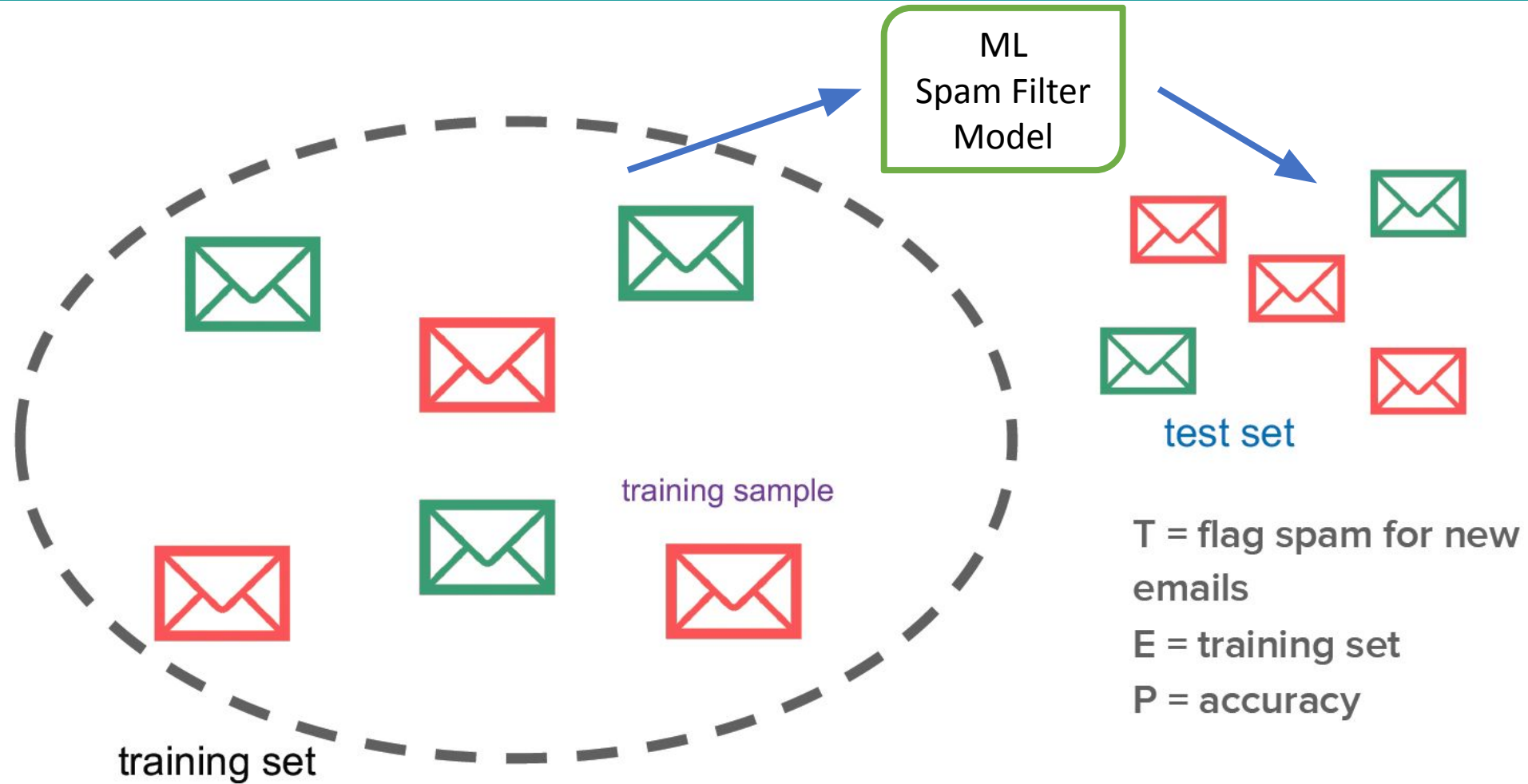
E = training set

P = accuracy

Example Spam Filter



Example Spam Filter

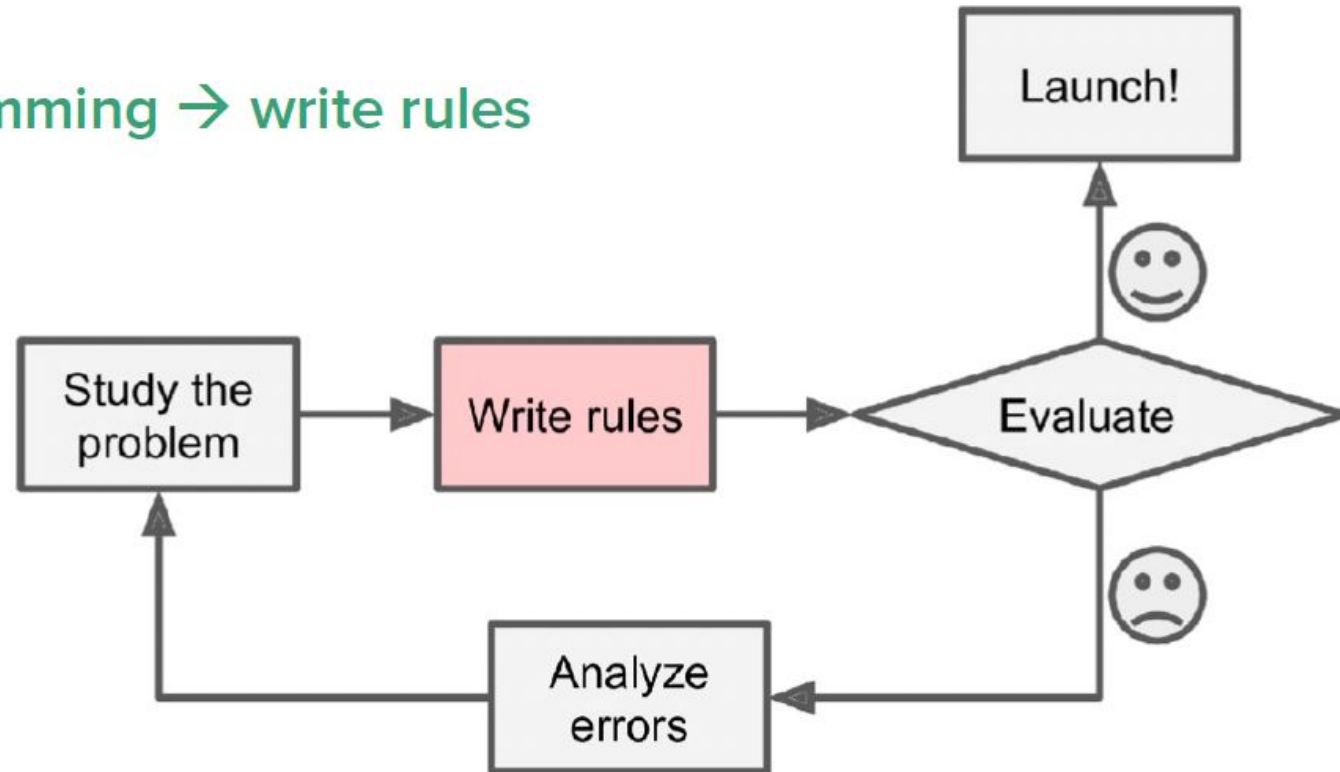


Examples of Classification Task

Task	Attribute set, x	Class label, y
Categorizing email messages	Features extracted from email message header and content	spam or non-spam
Identifying tumor cells	Features extracted from MRI scans	malignant or benign cells
Cataloging galaxies	Features extracted from telescope images	Elliptical, spiral, or irregular-shaped galaxies

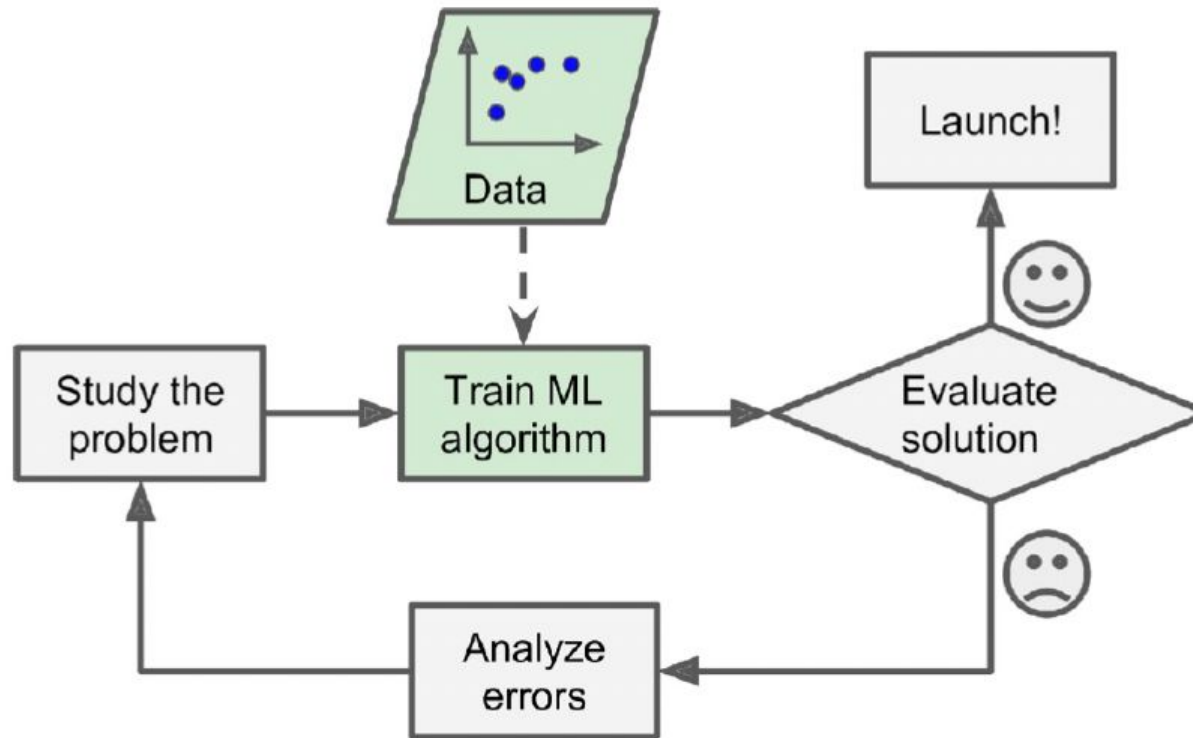
Why do we want to use Machine Learning?

Traditional Programming → write rules



Why do we want to use Machine Learning?

Machine Learning: train based on data (examples)



Why do we want to use Machine Learning?

- Problems for which existing solutions require a lot of finetuning or a long list of rules
- Complex problems for which a traditional approach yields no good solution
- Changing environments
- Getting insights about complex problems and large amount of data



What is Classification?

- Classification consists in learning a **model/function** f that maps each attribute set $\mathbf{x}$ into one of the predefined class labels $\mathbf{y}$: $f(\mathbf{x}) = \mathbf{y}$

What is Classification?

- Classification consists in learning a **model/function** f that maps each attribute set $\mathbf{x}$ into one of the predefined class labels $\mathbf{y}$: $f(\mathbf{x}) = \mathbf{y}$
- The **learning** is performed on a given a collection of records named **training set** where each record is by characterized by a tuple $(\mathbf{x}, \mathbf{y})$, where $\mathbf{x}$ is the attribute set and $\mathbf{y}$ is the class label
 - $\mathbf{x}$: attribute, predictor, independent variable, input
 - $\mathbf{y}$: class, response, dependent variable, output

What is Classification?

- Classification consists in learning a **model/function** f that maps each attribute set $\mathbf{x}$ into one of the predefined class labels $\mathbf{y}$: $f(\mathbf{x}) = \mathbf{y}$
- The **learning** is performed on a given a collection of records named **training set** where each record is by characterized by a tuple $(\mathbf{x}, \mathbf{y})$, where $\mathbf{x}$ is the attribute set and $\mathbf{y}$ is the class label
 - $\mathbf{x}$: attribute, predictor, independent variable, input
 - $\mathbf{y}$: class, response, dependent variable, output
- **Goal**: previously unseen records should be assigned a class as accurately as possible. A **test set** is used to determine the accuracy of the model f .

What is Classification?

- Classification consists in learning a **model/function** f that maps each attribute set x into one of the predefined class labels y : $f(x) = y$
- The **learning** is performed on a given a collection of records named **training set** where each record is by characterized by a tuple (x, y) , where x is the attribute set and y is the class label
 - x : attribute, predictor, independent variable, input
 - y : class, response, dependent variable, output
- **Goal**: previously unseen records should be assigned a class as accurately as possible. A **test set** is used to determine the accuracy of the model f .
- Usually, the given data set is divided into training and test sets, with training set used to build the model and test set used to evaluate it.

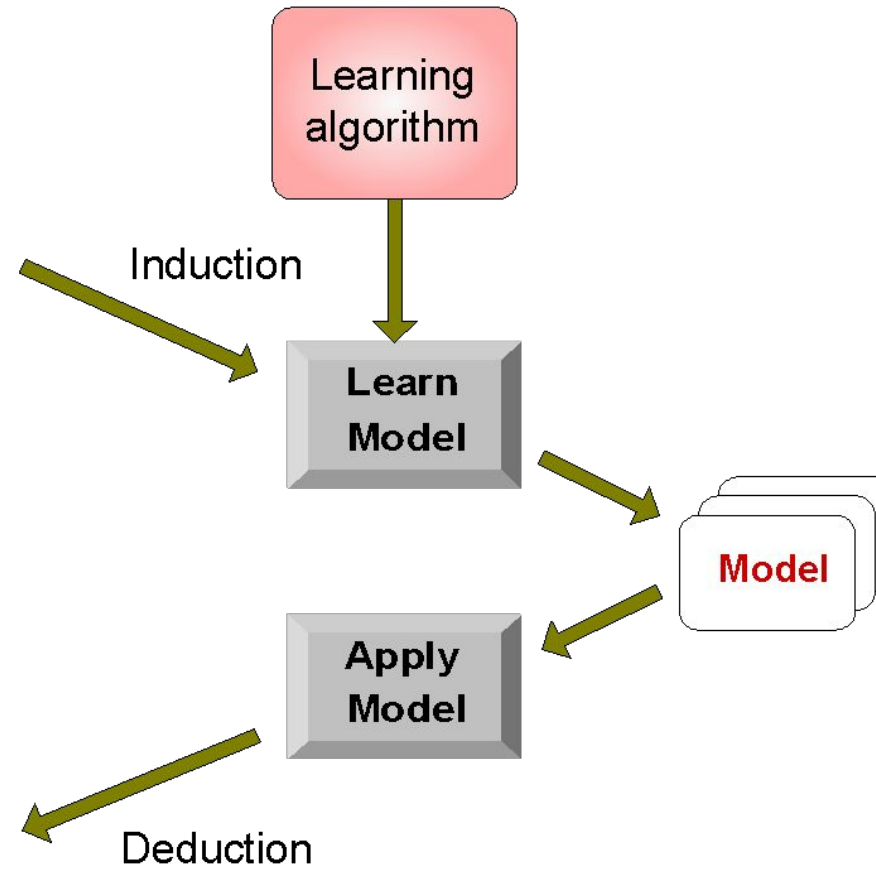
General Approach for Building Classification Model

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

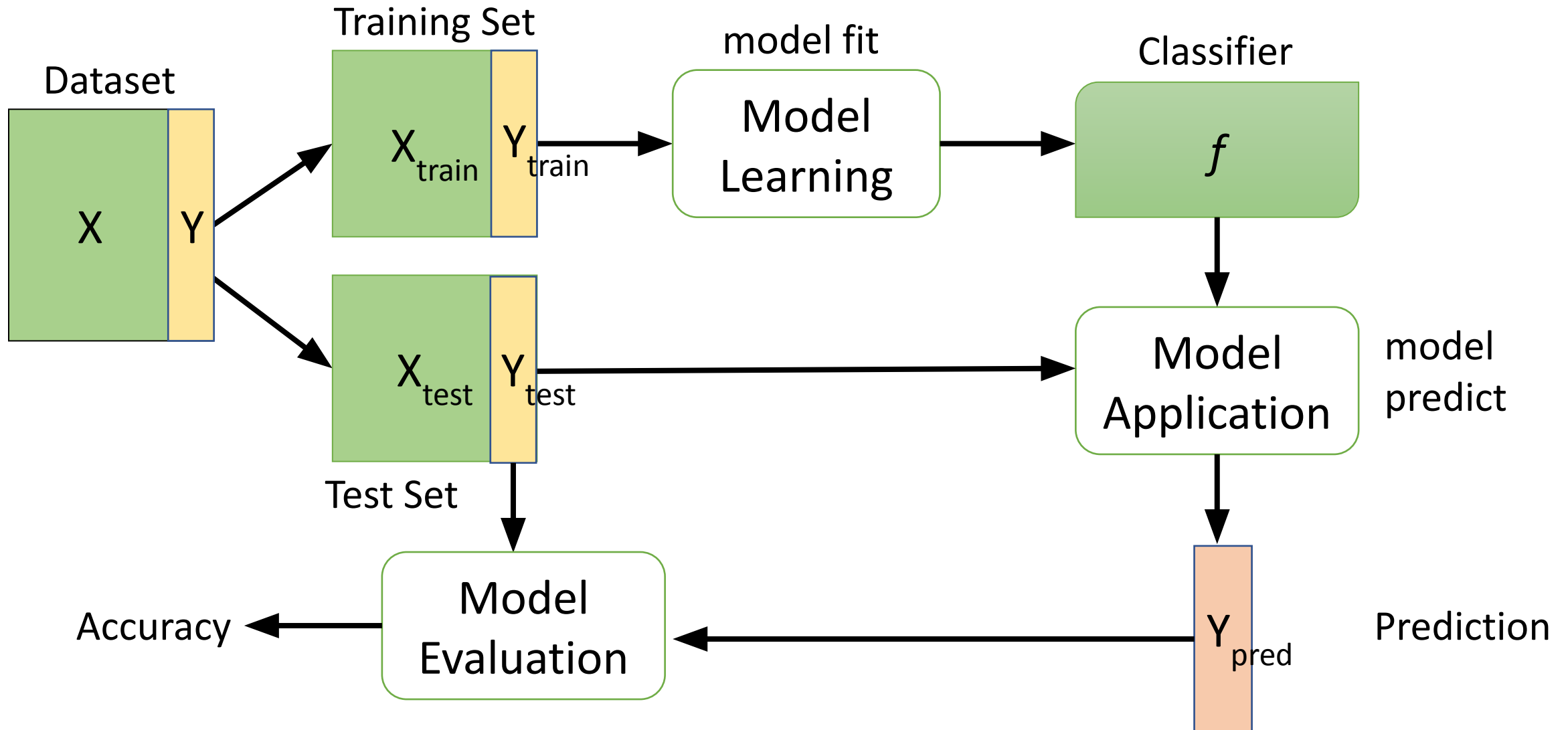
Test Set



Classification Techniques

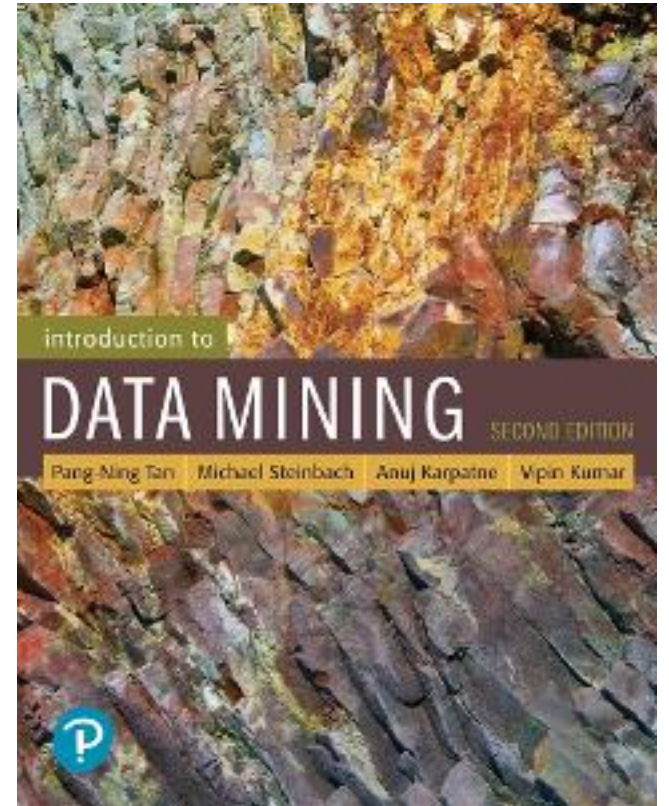
- Base Classifiers
 - Decision Tree based Methods
 - Rule-based Methods
 - Nearest-neighbor
 - Neural Networks
 - Deep Learning
 - Naïve Bayes and Bayesian Belief Networks
 - Support Vector Machines
- Ensemble Classifiers
 - Boosting, Bagging, Random Forests

What is Classification?



References

- Chapter 3. Classification: Basic Concepts and Techniques.



DATA MINING 1

Instance-based Classifiers

Dino Pedreschi, Riccardo Guidotti

Slides edited from Tan, Steinbach, Kumar, Introduction to Data Mining



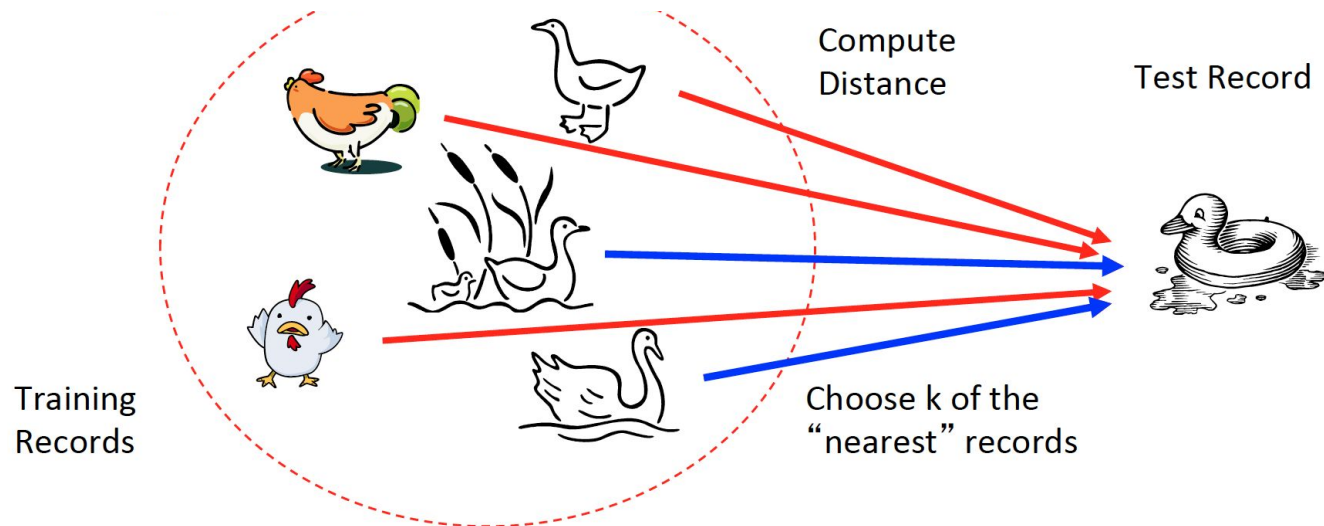
UNIVERSITÀ DI PISA

Nearest-Neighbor Classifier (K-NN)

Basic idea: If it walks like a duck, quacks like a duck, then it's probably a duck.

Requires three things

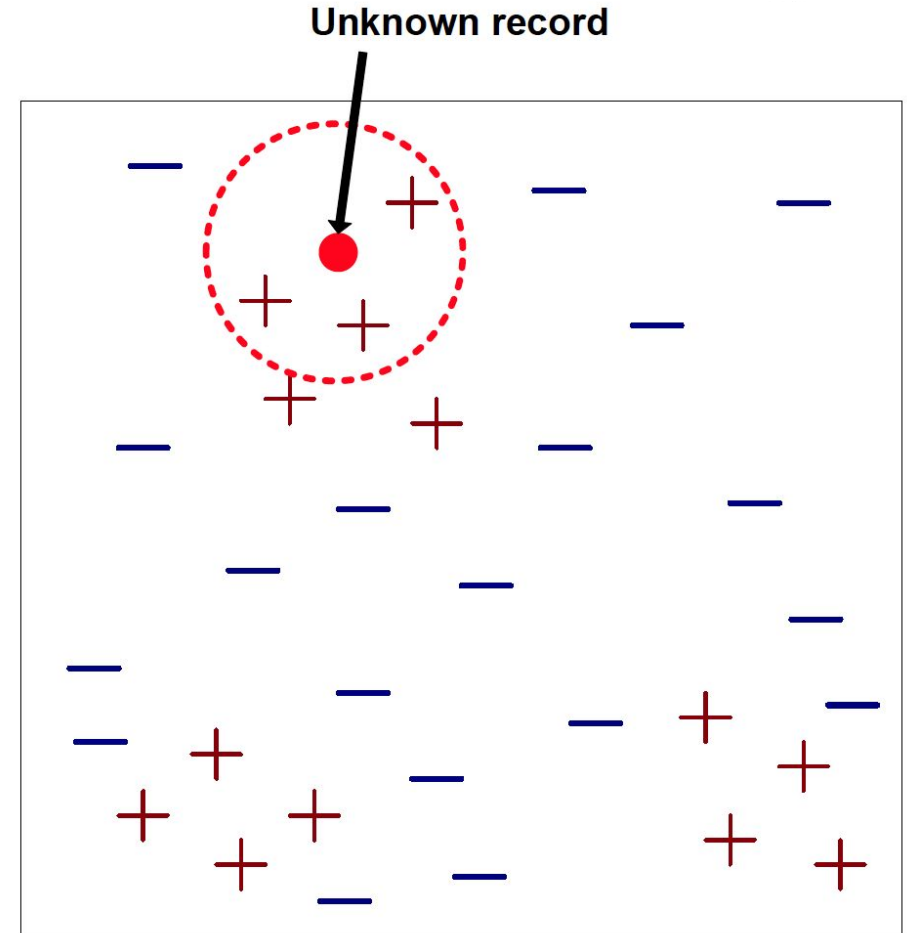
1. **Training set** of stored records
2. **Distance metric** to compute distance between records
3. **The value of k** , the number of nearest neighbors to retrieve



Nearest-Neighbor Classifier (K-NN)

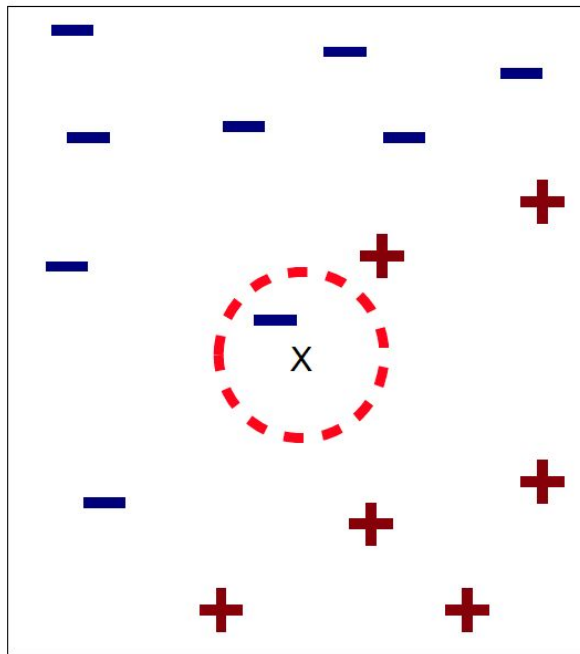
Given a set of training records (memory),
and a test record:

1. ***Compute the distances*** from the records in the training to the test.
2. ***Identify the k “nearest” records.***
3. Use class labels of nearest neighbors to ***determine the class label*** of unknown record (e.g., by taking majority vote).

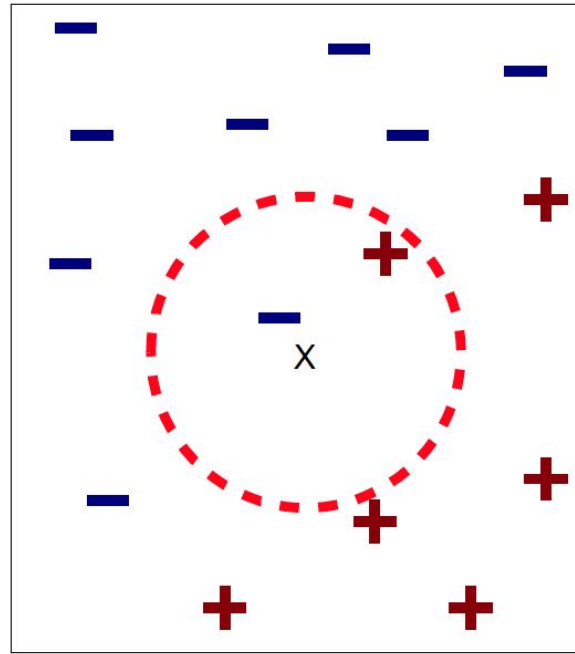


Definition of Nearest Neighbor

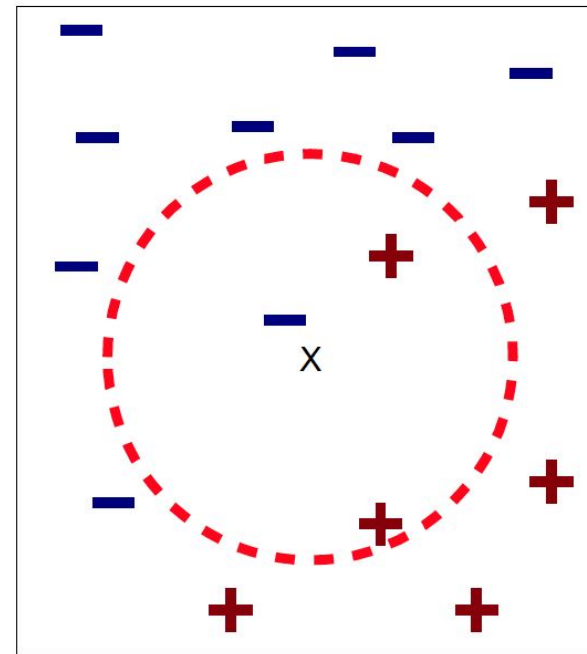
- K -nearest neighbors of a record x are data points that have the k smallest distance to x .



(a) 1-nearest neighbor



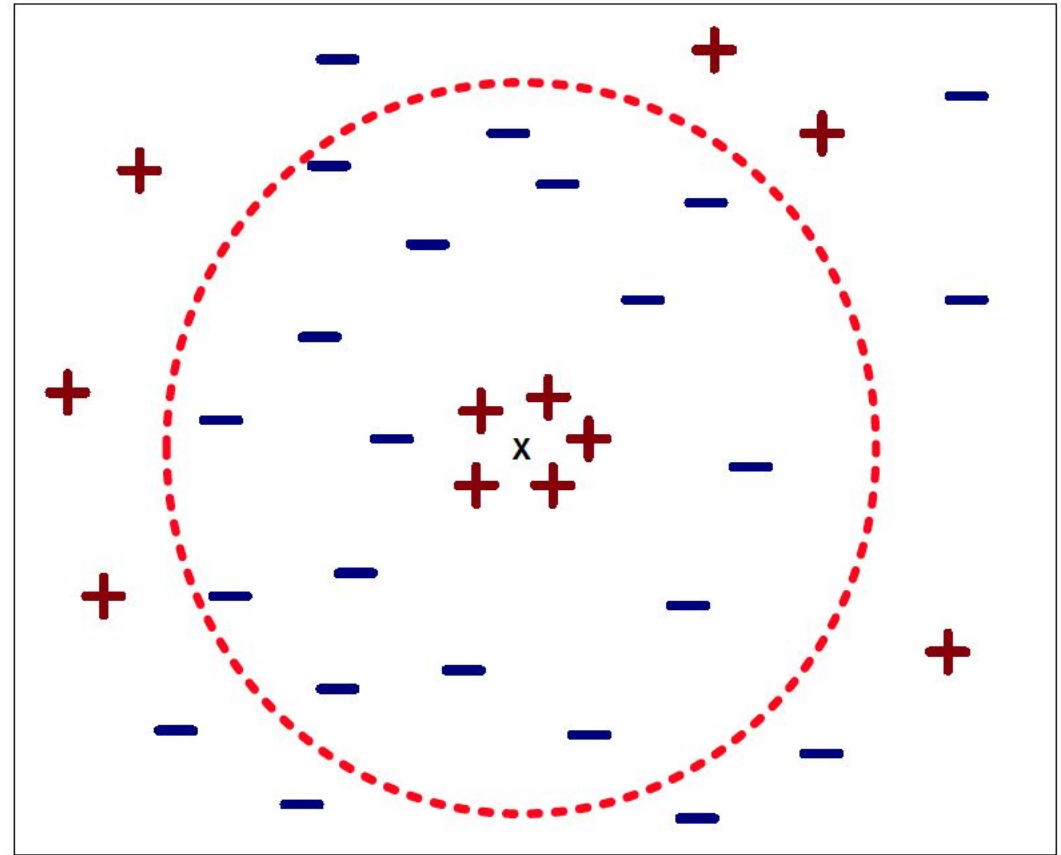
(b) 2-nearest neighbor



(c) 3-nearest neighbor

Choosing the Value of K

- If k is too small, it is sensitive to noise points and it can lead to overfitting to the noise in the training set.
- If k is too large, the neighborhood may include points from other classes.
- General practice $k = \sqrt{N}$ where N is the number of samples in the training dataset.



Nearest Neighbor Classification

Compute distance between two points:

- Euclidean distance $d(p, q) = \sqrt{\sum_i (p_i - q_i)^2}$

Determine the class from nearest neighbors

- take the majority vote of class labels among the k nearest neighbors
- weigh the vote according to distance (e.g. weight factor, $w = 1/d^2$)

Dimensionality and Scaling Issues

- Problem with Euclidean measure: high dimensional data can cause curse of dimensionality.
- Attributes may have to be scaled to prevent distance measures from being dominated by one of the attributes.
 - Solution: normalize the vectors to unit length
- Example:
 - height of a person may vary from 1.5m to 1.8m
 - weight of a person may vary from 10kg to 200kg
 - income of a person may vary from \$10K to \$1M

Instance-based Classifiers

- Instead of performing explicit generalization, compare new instances with instances seen in training, which have been stored in memory.
- Sometimes called *memory-based* learning.
- **Advantages**
 - Adapt its model to previously unseen data by storing a new instance or throwing an old instance away.
- **Disadvantages**
 - Lazy learner: it does not build a model explicitly.
 - Classifying unknown records is relatively expensive: in the worst case, given n training items, the complexity of classifying a single instance is $O(n)$.

Parallel Exemplar-Based Learning System (PEBLS)

- PEBLS is a nearest-neighbor learning system ($k=1$) designed for applications where the instances have symbolic feature values.
- Works with both continuous and nominal features.
- For nominal features, the distance between two nominal values is computed using Modified Value Difference Metric (MVDM)
- $d(V_1, V_2) = \sum_i \left| \frac{n_{1i}}{n_1} - \frac{n_{2i}}{n_2} \right|$
- Where n_1 is the number of records that consists of nominal attribute value V_1 and n_{1i} is the number of records whose target label is class i .

Distance Between Nominal Attribute Values

- $d(\text{Status}=\text{Single}, \text{Status}=\text{Married}) = |2/4 - 0/4| + |2/4 - 4/4| = 1$
- $d(\text{Status}=\text{Single}, \text{Status}=\text{Divorced}) = |2/4 - 1/2| + |2/4 - 1/2| = 0$
- $d(\text{Status}=\text{Married}, \text{Status}=\text{Divorced}) = |0/4 - 1/2| + |4/4 - 1/2| = 1$
- $d(\text{Refund}=\text{Yes}, \text{Refund}=\text{No}) = |0/3 - 3/7| + |3/3 - 4/7| = 6/7$

Class	Marital Status		
	Single	Married	Divorced
Yes	2	0	1
No	2	4	1

Class	Refund	
	Yes	No
Yes	0	3
No	3	4

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Distance Between Records

- $\delta(X, Y) = w_X w_Y \sum_{i=0}^d d(X_i, Y_i)$
- Each record X is assigned a weight $w_X = \frac{N_{X_{predict}}}{N_{X_{correct}}}$, which represents its reliability
- $N_{X_{predict}}$ is the number of times X is used for prediction
- $N_{X_{correct}}$ is the number of times the prediction using X is correct
- If $w_X \cong 1$ X makes accurate prediction most of the time
- If $w_X > 1$, then X is not reliable for making predictions. High $w_X > 1$ would result in high distance, which makes it less possible to use X to make predictions.

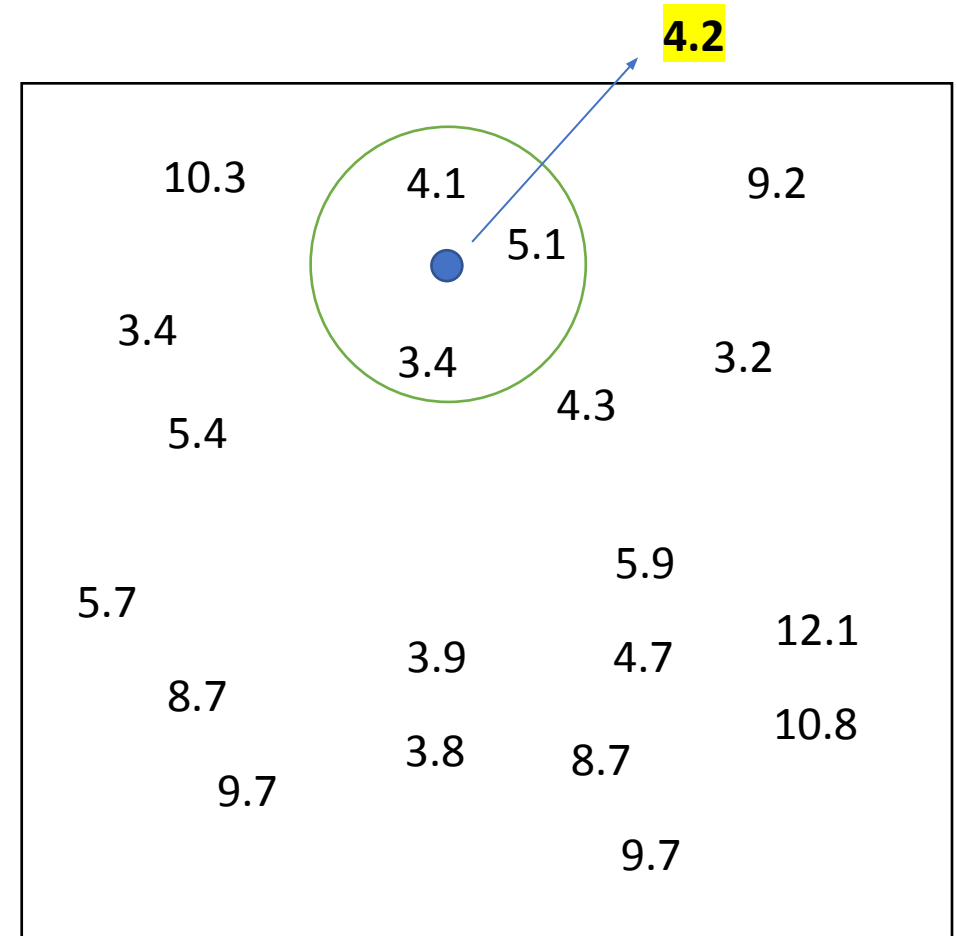
Characteristics of Nearest Neighbor Classifiers

- Instance-based learner: makes predictions without maintaining abstraction, i.e., building a model like decision trees.
- It is a lazy learner: classifying a test example can be expensive because need to compute the proximity values between test and training examples.
- In contrast eager learners spend time in building the model but then the classification is fast.
- Make their prediction on local information and for low k they are susceptible to noise.
- Can produce wrong predictions if inappropriate distance functions and/or preprocessing steps are performed.

k-NN for Regression

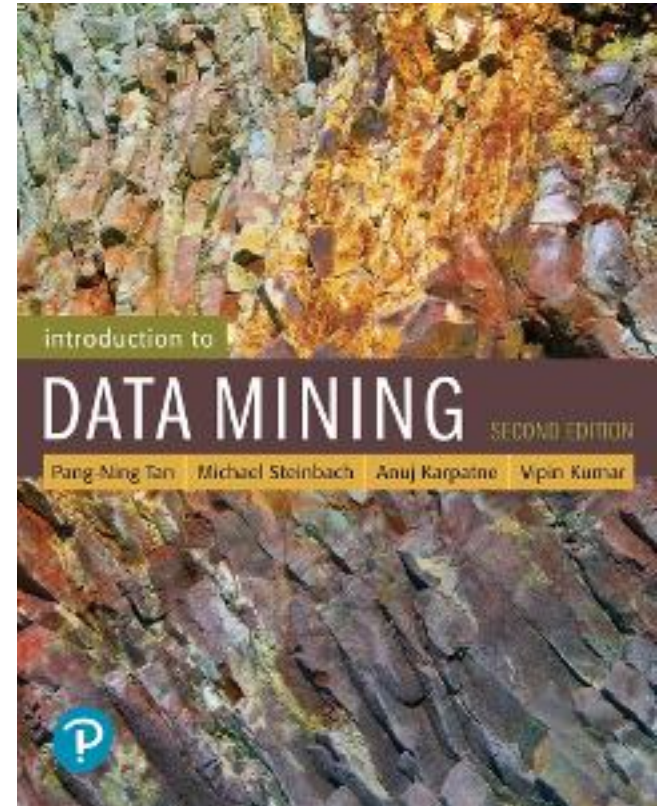
Given a set of training records (memory), and a test record:

1. **Compute the distances** from the records in the training to the test.
2. **Identify the k “nearest” records.**
3. Use target value of nearest neighbors to **determine the value** of unknown record (e.g., by averaging the values).



References

- Nearest Neighbor classifiers. Chapter 5.2. Introduction to Data Mining.

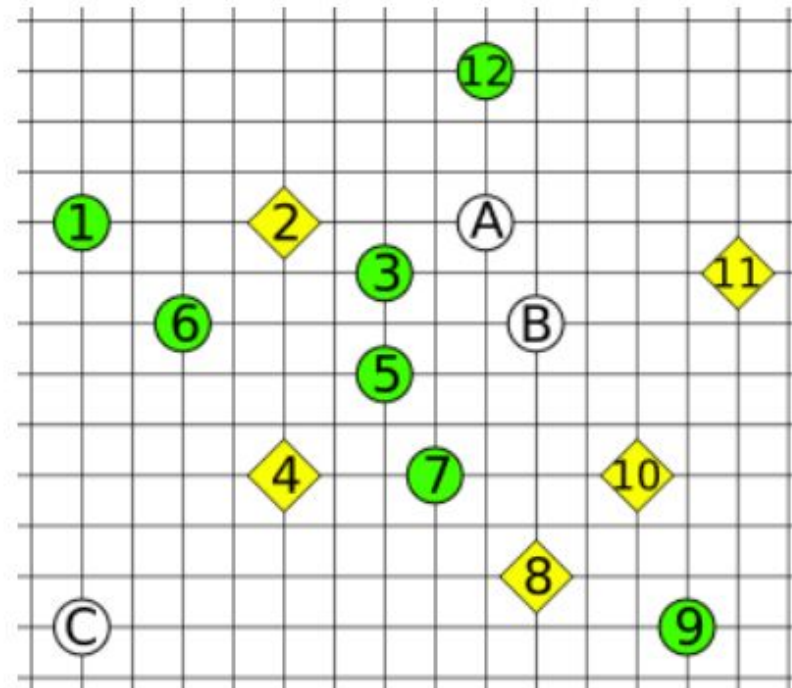


Exercises - kNN

b) k-NN (3 points)

Given the training set on the right, composed of elements numbered from 1 to 12, and labelled as circles and diamonds, use it to classify the remaining 3 elements (letters A, B and C) using a k-NN classifier with $k=3$. For each point to classify, list the points of the dataset that belong to its k-NN set.

Notice: A, B and C belong to the test set, not to the training set. Also, the Euclidean distance should be used.



b) k-NN (3 points)

Given the training set on the right, composed of elements numbered from 1 to 12, and labelled as circles and diamonds, use it to classify the remaining 3 elements (letters A, B and C) using a k-NN classifier with $k=3$. For each point to classify, list the points of the dataset that belong to its k-NN set.

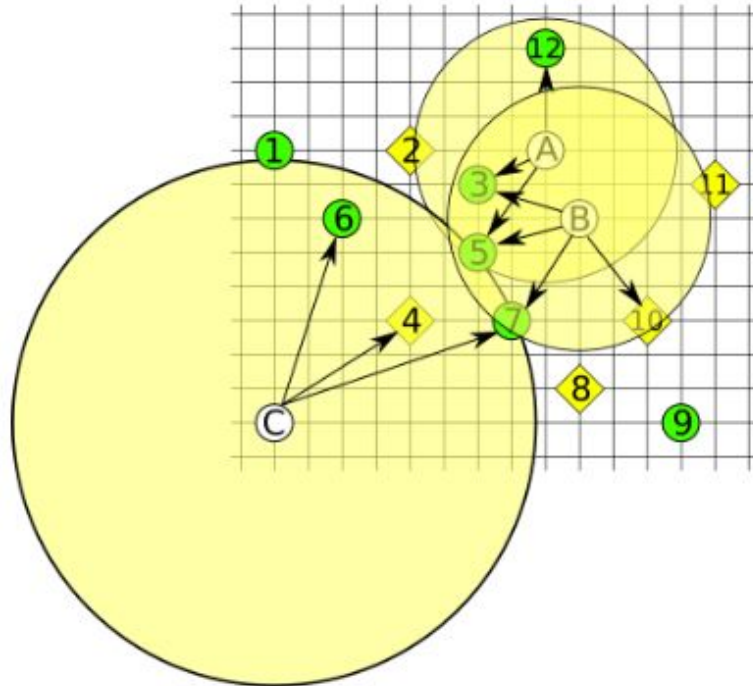
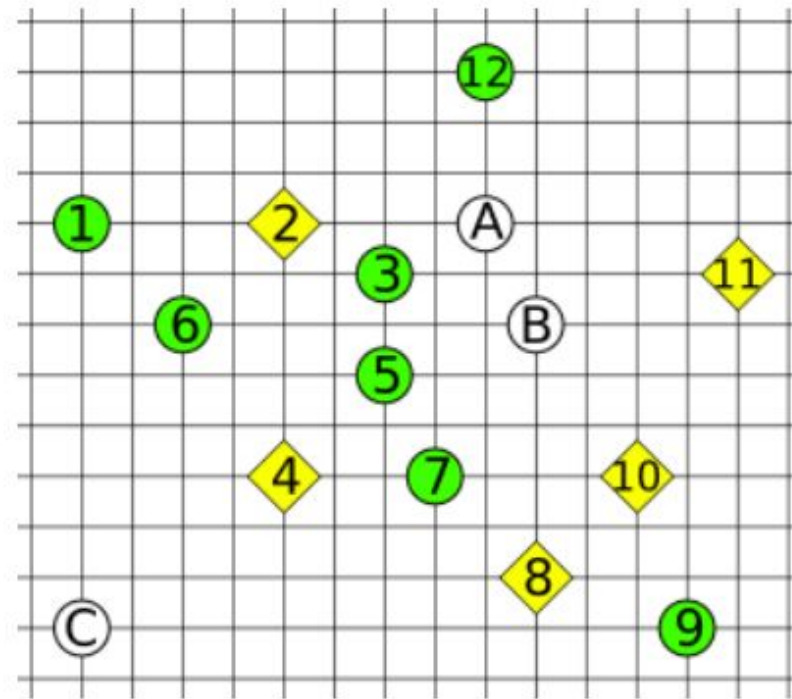
Notice: A, B and C belong to the test set, not to the training set. Also, the Euclidean distance should be used.

Answer:

$kNN(A) = \{3, 5, 12\} \rightarrow \text{CIRCLE}$

$kNN(B) = \{3, 5, 7, 10\} \rightarrow \text{CIRCLE}$

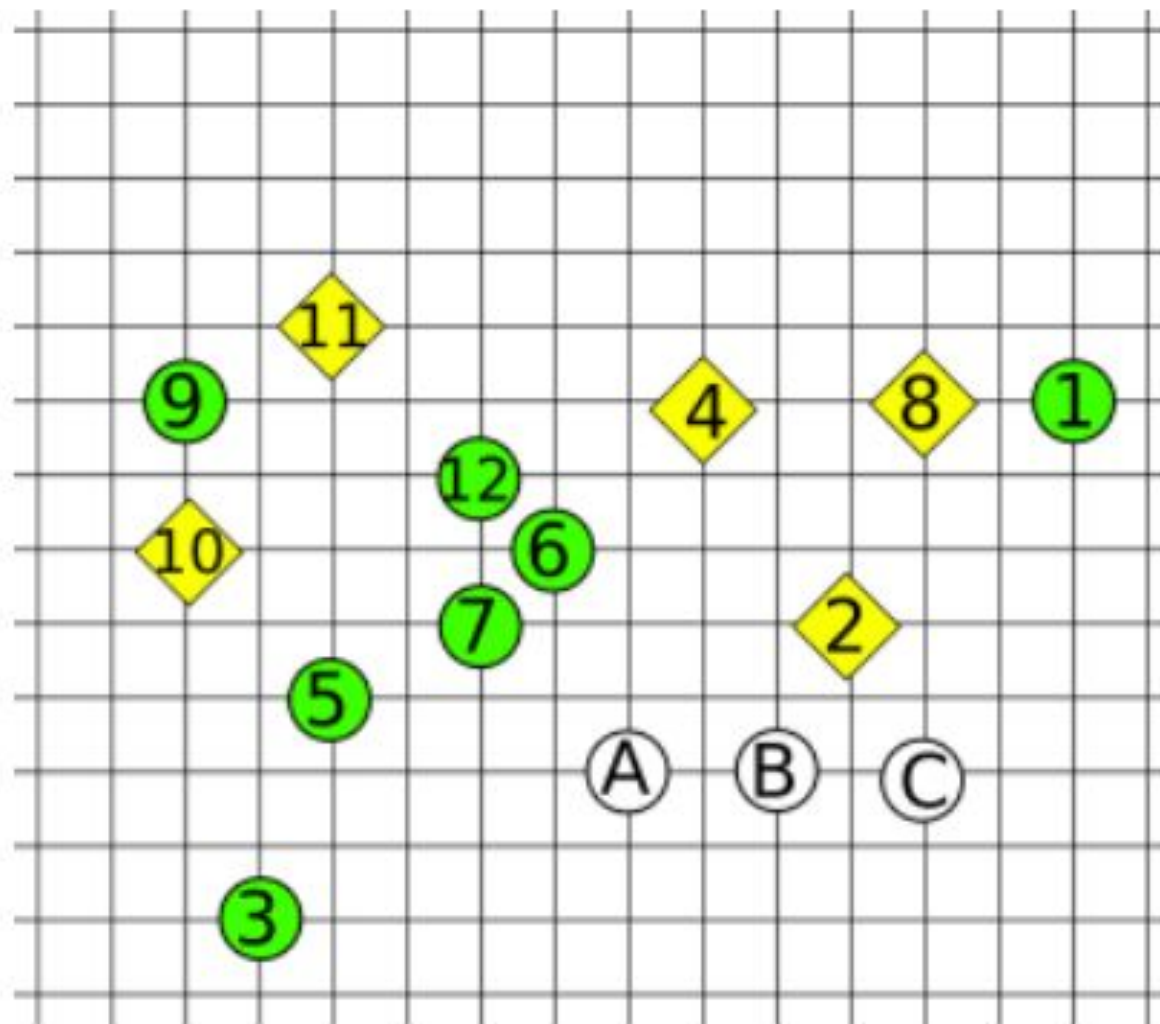
$kNN(C) = \{4, 6, 7\} \rightarrow \text{CIRCLE}$



Given the training set on the right, composed of elements numbered from 1 to 12, and labelled as circles and diamonds, use it to classify the remaining 3 elements (letters A, B and C) using a k-NN classifier with $k=3$.

For each point to classify, list the points of the dataset that belong to its k-NN set.

Notice: A, B and C belong to the test set, not to the training set. Also, the Euclidean distance should be used.



k-Nearest Neighbor Classifier

A medical expert is going to build up a case-based reasoning system for diagnosis tasks. Cases correspond to individual persons where the case problem parts are made up of a number of features describing possible symptoms and the solution parts represent the diagnosis (classification of disease). The case base contains the seven cases provided in the table below.

Training	Fever	Vomiting	Diarrhea	Shivering	Classification
c_1	no	no	no	no	healty (H)
c_2	average	no	no	no	influenza (I)
c_3	high	no	no	yes	influenza (I)
c_4	high	yes	yes	no	salmonella poisoning (S)
c_5	average	no	yes	no	salmonella poisoning (S)
c_6	no	yes	yes	no	bowel inflammation (B)
c_7	average	yes	yes	no	bowel inflammation (B)

Similarity provided by an expert

$$\text{sim}_F$$

q \ c	no	avg	high
no	1.0	0.7	0.2
avg	0.5	1.0	0.8
high	0.0	0.3	1.0

$$\text{sim}_V = \text{sim}_D = \text{sim}_{Sh}$$

q \ c	yes	no
yes	1.0	0.0
no	0.2	1.0

Weights

$$w_F = 0.3$$

$$w_V = 0.2$$

$$w_D = 0.2$$

$$w_{Sh} = 0.3$$

**Classify the new instance $q = (\text{high}; \text{no}; \text{no}; \text{no})$
by applying the KNN algorithm with $K=1,2,3$**

Calculate the similarity between all cases from the case base and the new instance $q = (\text{high}; \text{no}; \text{no}; \text{no})$

c1 = (no; no; no; no):

$$\text{Sim}(q; c1) = 0.3*0.0 + 0.2*1.0 + 0.2*1.0 + 0.3*1.0 = 0.70$$

c2 = (average; no; no; no):

$$\text{Sim}(q; c2) = 0.3*0.3 + 0.2*1.0 + 0.2*1.0 + 0.3*1.0 = 0.79$$

c3 = (high; no; no; yes)

$$\text{Sim}(q; c3) = 0.3*1.0 + 0.2*1.0 + 0.2*1.0 + 0.3*0.2 = 0.76$$

c4 = (high; yes; yes; no):

$$\text{Sim}(q; c4) = 0.3*1.0 + 0.2*0.2 + 0.2*0.2 + 0.3*1.0 = 0.68$$

c5 = (average; no; yes; no):

$$\text{Sim}(q; c5) = 0.3*0.3 + 0.2*1.0 + 0.2*0.2 + 0.3*1.0 = 0.63$$

c6 = (no; yes; yes; no):

$$\text{Sim}(q; c6) = 0.3*0.0 + 0.2*0.2 + 0.2*0.2 + 0.3*1.0 = 0.28$$

c7 = (average; yes; yes; no):

$$\text{Sim}(q; c7) = 0.3*0.3 + 0.2*0.2 + 0.2*0.2 + 0.3*1.0 = 0.47$$

$$\text{sim}_F$$

$q \backslash c$	no	avg	high
no	1.0	0.7	0.2
avg	0.5	1.0	0.8
high	0.0	0.3	1.0

$\text{sim}_V = \text{sim}_D = \text{sim}_{Sh}$

$q \backslash$	yes	no
yes	1.0	0.0
no	0.2	1.0

Weights

$$w_F = 0.3$$

$$w_V = 0.2$$

$$w_D = 0.2$$

$$w_{Sh} = 0.3$$

KNN Classification for K=1

c1 = (no; no; no; no):

$$\text{Sim}(q; c1) = 0.3*0.0 + 0.2*1.0 + 0.2*1.0 + 0.3*1.0 = 0.70$$

c2 = (average; no; no; no):

$$\text{Sim}(q; c2) = 0.3*0.3 + 0.2*1.0 + 0.2*1.0 + 0.3*1.0 = 0.79$$

c3 = (high; no; no; yes)

$$\text{Sim}(q; c3) = 0.3*1.0 + 0.2*1.0 + 0.2*1.0 + 0.3*0.2 = 0.76$$

c4 = (high; yes; yes; no):

$$\text{Sim}(q; c4) = 0.3*1.0 + 0.2*0.2 + 0.2*0.2 + 0.3*1.0 = 0.68$$

c5 = (average; no; yes; no):

$$\text{Sim}(q; c5) = 0.3*0.3 + 0.2*1.0 + 0.2*0.2 + 0.3*1.0 = 0.63$$

c6 = (no; yes; yes; no):

$$\text{Sim}(q; c6) = 0.3*0.0 + 0.2*0.2 + 0.2*0.2 + 0.3*1.0 = 0.28$$

c7 = (average; yes; yes; no):

$$\text{Sim}(q; c7) = 0.3*0.3 + 0.2*0.2 + 0.2*0.2 + 0.3*1.0 = 0.47$$

sim_F

q \ c	no	avg	high
no	1.0	0.7	0.2
avg	0.5	1.0	0.8
high	0.0	0.3	1.0

Weights

$$w_F = 0.3$$

$$w_V = 0.2$$

$$w_D = 0.2$$

$$w_{Sh} = 0.3$$

Class: Influenza

KNN Classification for K=2

c1 = (no; no; no; no):

$$\text{Sim}(q; c1) = 0.3*0.0 + 0.2 *1.0 + 0.2*1.0 + 0.3* 1.0 = 0.70$$

c2 = (average; no; no; no):

$$\text{Sim}(q; c2) = 0.3* 0.3 + 0.2 *1.0 + 0.2*1.0 + 0.3*1.0 = 0.79$$

c3 = (high; no; no; yes):

$$\text{Sim}(q; c3) = 0.3*1.0 + 0.2*1.0 + 0.2*1.0 + 0.3*0.2 = 0.76$$

c4 = (high; yes; yes; no):

$$\text{Sim}(q; c4) = 0.3*1.0 + 0.2*0.2 + 0.2*0.2 + 0.3*1.0 = 0.68$$

c5 = (average; no; yes; no):

$$\text{Sim}(q; c5) = 0.3*0.3 + 0.2*1.0 + 0.2*0.2 + 0.3*1.0 = 0.63$$

c6 = (no; yes; yes; no):

$$\text{Sim}(q; c6) = 0.3*0.0 + 0.2*0.2 + 0.2*0.2 + 0.3*1.0 = 0.28$$

c7 = (average; yes; yes; no):

$$\text{Sim}(q; c7) = 0.3*0.3 + 0.2*0.2 + 0.2*0.2 + 0.3*1.0 = 0.47$$

		sim _F		
q \ c	c	no	avg	high
no		1.0	0.7	0.2
avg		0.5	1.0	0.8
high		0.0	0.3	1.0

Weights

$$w_F=0.3$$

$$w_V=0.2$$

$$w_D=0.2$$

$$w_{Sh}=0.3$$

C2: Influenza

C3: Influenza



Class: Influenza

KNN Classification for K=3

c1 = (no; no; no; no):

$$\text{Sim}(q; c1) = 0.3*0.0 + 0.2 *1.0 + 0.2*1.0 + 0.3* 1.0 = 0.70$$

c2 = (average; no; no; no):

$$\text{Sim}(q; c2) = 0.3* 0.3 + 0.2 *1.0 + 0.2*1.0 + 0.3*1.0 = 0.79$$

c3 = (high; no; no; yes):

$$\text{Sim}(q; c3) = 0.3*1.0 + 0.2*1.0 + 0.2*1.0 + 0.3*0.2 = 0.76$$

c4 = (high; yes; yes; no):

$$\text{Sim}(q; c4) = 0.3*1.0 + 0.2*0.2 + 0.2*0.2 + 0.3*1.0 = 0.68$$

c5 = (average; no; yes; no):

$$\text{Sim}(q; c5) = 0.3*0.3 + 0.2*1.0 + 0.2*0.2 + 0.3*1.0 = 0.63$$

c6 = (no; yes; yes; no):

$$\text{Sim}(q; c6) = 0.3*0.0 + 0.2*0.2 + 0.2*0.2 + 0.3*1.0 = 0.28$$

c7 = (average; yes; yes; no):

$$\text{Sim}(q; c7) = 0.3*0.3 + 0.2*0.2 + 0.2*0.2 + 0.3*1.0 = 0.47$$

sim_F

q \ c	no	avg	high
no	1.0	0.7	0.2
avg	0.5	1.0	0.8
high	0.0	0.3	1.0

Weights

$$w_F=0.3$$

$$w_V=0.2$$

$$W_D=0.2$$

$$w_{Sh}=0.3$$

C1: healthy

C2: Influenza

C3: Influenza



Class: Influenza

DATA MINING 1

Naïve Bayes Classifiers

Dino Pedreschi, Riccardo Guidotti

Slides edited from Tan, Steinbach, Kumar, Introduction to Data Mining



UNIVERSITÀ DI PISA

Probability Notions and Bayes Theorem

- A probabilistic framework for solving classification problems.
- Let P be a probability function that assigns a number between 0 and 1 to events.
- $X = x$ an events is happening.
- $P(X = x)$ is the probability that events $X = x$.
- Joint Probability $P(X = x, Y = y)$
- Conditional Probability $P(Y = y \mid X = x)$
- Relationship: $P(X, Y) = P(Y \mid X) P(X) = P(X \mid Y) P(Y)$
- Bayes Theorem: $P(Y \mid X) = P(X \mid Y) P(Y) / P(X)$
- Another Useful Property: $P(X = x) = P(X = x, Y = 0) + P(X = x, Y = 1)$

Bayes Theorem: Example

- Consider a football game. Team 0 wins 65% of the time, Team 1 the remaining 35%. Among the game won by Team 1, 75% of them are won playing at home. Among the games won by Team 0, 30% of them are won at Team 1's field.
- If Team 1 is hosting the next match, which team will most likely win?
- Team 0 wins: $P(Y = 0) = 0.65$
- Team 1 wins: $P(Y = 1) = 0.35$
- Team 1 hosted the match won by Team 1: $P(X = 1 | Y = 1) = 0.75$
- Team 1 hosted the match won by Team 0: $P(X = 1 | Y = 0) = 0.30$
- Objective $P(Y = 1 | X = 1)$

Bayes Theorem: Example

- $P(Y = 1 | X = 1) = P(X = 1 | Y = 1)P(Y = 1) / P(X = 1) =$
- $= 0.75 \times 0.35 / (P(X = 1, Y = 1) + P(X = 1, Y = 0))$
- $= 0.75 \times 0.35 / (P(X = 1 | Y = 1)P(Y=1) + P(X = 1 | Y = 0)P(Y=0))$
- $= 0.75 \times 0.35 / (0.75 \times 0.35 + 0.30 \times 0.65)$
- $= 0.5738$

- Therefore Team 1 has a better chance to win the match

Bayes Theorem for Classification

- X denotes the attribute sets, $X = \{X_1, X_2, \dots, X_d\}$
- Y denotes the class variable
- We treat the relationship probabilistically using $P(Y/X)$

The diagram shows the formula for Bayes' Theorem with arrows pointing to each part and labels:

- An arrow points from the label "Likelihood" to the term $P(X|Y)$ in the numerator.
- An arrow points from the label "Prior Probability" to the term $P(Y)$ in the numerator.
- An arrow points from the label "Evidence (sum over alternative events)" to the term $P(X)$ in the denominator.
- An arrow points from the label "Posterior Probability" to the entire expression $P(Y|X)$ on the left side of the equation.

$$\bullet P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}$$

Bayes Theorem for Classification

- Learn the posterior $P(Y / X)$ for every combination of X and Y .
- By knowing these probabilities, a test record X' can be classified by finding the class Y' that maximizes the posterior probability $P(Y' / X')$.
- This is equivalent of choosing the value of Y' that maximizes $P(X' / Y')P(Y')$.
- How to estimate it?

Naïve Bayes Classifier

- It estimates the class-conditional probability by ***assuming that the attributes are conditionally independent*** given the class label y .
- The conditional independence is stated as:
- $P(X|Y = y) = \prod_{i=1}^d P(X_i|Y = y)$
- where each attribute set $X = \{X_1, X_2, \dots, X_d\}$

Conditional Independence

- Given three variables Y, X_1, X_2 we can say that Y is independent from X_1 given X_2 if the following condition holds:
- $P(Y | X_1, X_2) = P(Y|X_2)$
- With the conditional independence assumption, instead of computing the class-conditional probability for every combination of X we only have to estimate the conditional probability of each X_i given Y .
- Thus, to classify a record the naive Bayes classifier computes the posterior for each class Y and takes the maximum class as result
- $P(Y|X) = P(Y) \prod_{i=1}^d P(X_i|Y = y) / P(X)$

How to estimate ?

How to Estimate Probability From Data

- Class $P(Y) = N_y / N$
- N_y number of records with outcome y
- N number of records
- Categorical attributes
- $P(X = x \mid Y = y) = N_{xy} / N_y$
- N_{xy} records with value x and outcome y

- $P(\text{Evade} = \text{Yes}) = 3/10$
- $P(\text{Marital Status} = \text{Single} \mid \text{Yes}) = 2/3$

<i>Tid</i>	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

How to Estimate Probability From Data

Continuous attributes

- Discretize the range into bins
 - one ordinal attribute per bin
 - violates independence assumption
- Two-way split: $(X < v)$ or $(X > v)$
 - choose only one of the two splits as new attribute
- Probability density estimation:
 - Assume attribute follows a normal distribution
 - Use data to estimate parameters of distribution (e.g., mean and standard deviation)
 - Once probability distribution is known, can use it to estimate the conditional probability $P(X|y)$

How to Estimate Probability From Data

- Normal distribution

- $P(X_i = x_i \mid Y = y) = \frac{1}{\sqrt{2\pi}\sigma_{ij}} e^{-\frac{(x_i - \mu_{ij})^2}{2\sigma_{ij}^2}}$
- μ_{ij} can be estimated as the mean of X_i for the records that belongs to class y_j .
- Similarly, σ_{ij} as the standard deviation.
- $P(\text{Income} = 120 \mid \text{No}) = 0.0072$
 - mean = 110
 - std dev = 54.54

Tid	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Example

Given $X = \{\text{Refund} = \text{No}, \text{Married}, \text{Income} = 120\text{k}\}$

- $P(\text{Refund}=\text{Yes} | \text{No}) = 3/7$
- $P(\text{Refund}=\text{No} | \text{No}) = 4/7$
- $P(\text{Refund}=\text{Yes} | \text{Yes}) = 0$
- $P(\text{Refund}=\text{No} | \text{Yes}) = 1$
- $P(\text{Marital Status}=\text{Single} | \text{No}) = 2/7$
- $P(\text{Marital Status}=\text{Divorced} | \text{No}) = 1/7$
- $P(\text{Marital Status}=\text{Married} | \text{No}) = 4/7$
- $P(\text{Marital Status}=\text{Single} | \text{Yes}) = 2/3$
- $P(\text{Marital Status}=\text{Divorced} | \text{Yes}) = 1/3$
- $P(\text{Marital Status}=\text{Married} | \text{Yes}) = 0/3$

For taxable income:

- If class=No:
 - mean=110, variance=2975
- If class=Yes:
 - mean=90, variance=25

$$\begin{aligned}
 P(X | \text{Class}=\text{No}) &= P(\text{Refund}=\text{No} | \text{Class}=\text{No}) \\
 &\quad \times P(\text{Married} | \text{Class}=\text{No}) \\
 &\quad \times P(\text{Income}=120\text{K} | \text{Class}=\text{No}) \\
 &= 4/7 \times 4/7 \times 0.0072 \\
 &= 0.0024
 \end{aligned}$$

$$\begin{aligned}
 P(X | \text{Class}=\text{Yes}) &= P(\text{Refund}=\text{No} | \text{Class}=\text{Yes}) \\
 &\quad \times P(\text{Married} | \text{Class}=\text{Yes}) \\
 &\quad \times P(\text{Income}=120\text{K} | \text{Class}=\text{Yes}) \\
 &= 1 \times 0 \times 1.2 \times 10^{-9} \\
 &= 0
 \end{aligned}$$

Since $P(X | \text{No})P(\text{No}) > P(X | \text{Yes})P(\text{Yes})$

Therefore $P(\text{No} | X) > P(\text{Yes} | X)$
 $\Rightarrow \text{Class} = \text{No}$

Tid	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

M-estimate of Conditional Probability

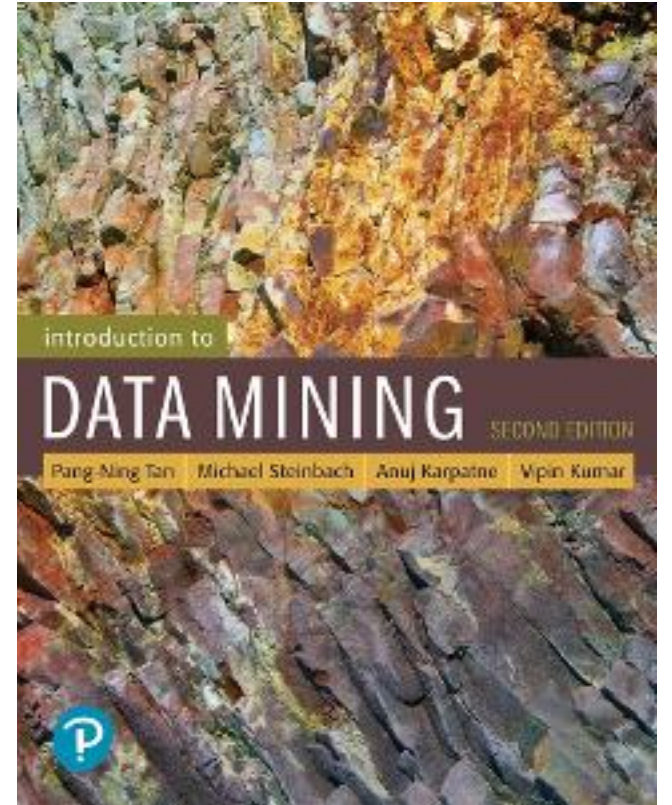
- If one of the conditional probability is zero, then the entire expression becomes zero.
- For example, given $X = \{\text{Refund} = \text{Yes}, \text{Divorced}, \text{Income} = 120\text{k}\}$, if $P(\text{Divorced} | \text{No})$ is zero instead of $1/7$, then
 - $P(X | \text{No}) = 3/7 \times 0 \times 0.00072 = 0$
 - $P(X | \text{Yes}) = 0 \times 1/3 \times 10^{-9} = 0$
- M-estimate $P(X | Y) = \frac{N_{xy} + mp}{N_y + m}$ (if $P(X | Y) = \frac{N_{xy} + 1}{N_y + |Y|}$ is Laplacian estimation)
- m is a parameter, p is a user-specified parameter (e.g. probability of observing x_i among records with class y_j).
- In the example with $m = 2$ and $p = 1/m = 1/2$ (i.e., Laplacian estimation) we have
- $P(\text{Divorced} | \text{Yes}) = (0 + 2 \times 1/2) / (3 + 2) = 1/5$

Naïve Bayes Classifier

- Robust to isolated noise points
- Handle missing values by ignoring the instance during probability estimate calculations
- Robust to irrelevant attributes
- Independence assumption may not hold for some attributes
 - Use other techniques such as Bayesian Belief Networks (BBN, not treated in this course)

References

- Bayesian Classifiers. Chapter 5.3.
Introduction to Data Mining.



Exercises - NBC

Play-tennis example. estimating $P(x_i | C)$

Outlook	Temperature	Humidity	Windy	Class
sunny	hot	high	false	N
sunny	hot	high	true	N
overcast	hot	high	false	P
rain	mild	high	false	P
rain	cool	normal	false	P
rain	cool	normal	true	N
overcast	cool	normal	true	P
sunny	mild	high	false	N
sunny	cool	normal	false	P
rain	mild	normal	false	P
sunny	mild	normal	true	P
overcast	mild	high	true	P
overcast	hot	normal	false	P
rain	mild	high	true	N

$$P(p) = 9/14$$

$$P(n) = 5/14$$

outlook	
$P(\text{sunny} p) =$	$P(\text{sunny} n) =$
$P(\text{overcast} p) =$	$P(\text{overcast} n) =$
$P(\text{rain} p) =$	$P(\text{rain} n) =$
temperature	
$P(\text{hot} p) =$	$P(\text{hot} n) =$
$P(\text{mild} p) =$	$P(\text{mild} n) =$
$P(\text{cool} p) =$	$P(\text{cool} n) =$
humidity	
$P(\text{high} p) =$	$P(\text{high} n) =$
$P(\text{normal} p) =$	$P(\text{normal} n) =$
windy	
$P(\text{true} p) =$	$P(\text{true} n) =$
$P(\text{false} p) =$	$P(\text{false} n) =$

Play-tennis example. estimating $P(x_i | C)$

Outlook	Temperature	Humidity	Windy	Class
sunny	hot	high	false	N
sunny	hot	high	true	N
overcast	hot	high	false	P
rain	mild	high	false	P
rain	cool	normal	false	P
rain	cool	normal	true	N
overcast	cool	normal	true	P
sunny	mild	high	false	N
sunny	cool	normal	false	P
rain	mild	normal	false	P
sunny	mild	normal	true	P
overcast	mild	high	true	P
overcast	hot	normal	false	P
rain	mild	high	true	N

$P(p) =$
$P(n) =$

outlook	
$P(\text{sunny} p) =$	$P(\text{sunny} n) =$
$P(\text{overcast} p) =$	$P(\text{overcast} n) =$
$P(\text{rain} p) =$	$P(\text{rain} n) =$
temperature	
$P(\text{hot} p) =$	$P(\text{hot} n) =$
$P(\text{mild} p) =$	$P(\text{mild} n) =$
$P(\text{cool} p) =$	$P(\text{cool} n) =$
humidity	
$P(\text{high} p) =$	$P(\text{high} n) =$
$P(\text{normal} p) =$	$P(\text{normal} n) =$
windy	
$P(\text{true} p) =$	$P(\text{true} n) =$
$P(\text{false} p) =$	$P(\text{false} n) =$

Play-tennis example. estimating $P(x_i | C)$

Outlook	Temperature	Humidity	Windy	Class
sunny	hot	high	false	N
sunny	hot	high	true	N
overcast	hot	high	false	P
rain	mild	high	false	P
rain	cool	normal	false	P
rain	cool	normal	true	N
overcast	cool	normal	true	P
sunny	mild	high	false	N
sunny	cool	normal	false	P
rain	mild	normal	false	P
sunny	mild	normal	true	P
overcast	mild	high	true	P
overcast	hot	normal	false	P
rain	mild	high	true	N

$$P(p) = 9/14$$

$$P(n) = 5/14$$

outlook	
$P(\text{sunny} p) = 2/9$	$P(\text{sunny} n) = 3/5$
$P(\text{overcast} p) = 4/9$	$P(\text{overcast} n) = 0$
$P(\text{rain} p) = 3/9$	$P(\text{rain} n) = 2/5$
temperature	
$P(\text{hot} p) = 2/9$	$P(\text{hot} n) = 2/5$
$P(\text{mild} p) = 4/9$	$P(\text{mild} n) = 2/5$
$P(\text{cool} p) = 3/9$	$P(\text{cool} n) = 1/5$
humidity	
$P(\text{high} p) = 3/9$	$P(\text{high} n) = 4/5$
$P(\text{normal} p) = 6/9$	$P(\text{normal} n) = 1/5$
windy	
$P(\text{true} p) = 3/9$	$P(\text{true} n) = 3/5$
$P(\text{false} p) = 6/9$	$P(\text{false} n) = 2/5$

Play-tennis example. estimating $P(x_i | C)$

$P(p) = 9/14$
$P(n) = 5/14$

Outlook	Tempeprature	Humidity	Windy	Class
rain	hot	high	false	?

outlook	
$P(\text{sunny} p) = 2/9$	$P(\text{sunny} n) = 3/5$
$P(\text{overcast} p) = 4/9$	$P(\text{overcast} n) = 0$
$P(\text{rain} p) = 3/9$	$P(\text{rain} n) = 2/5$
temperature	
$P(\text{hot} p) = 2/9$	$P(\text{hot} n) = 2/5$
$P(\text{mild} p) = 4/9$	$P(\text{mild} n) = 2/5$
$P(\text{cool} p) = 3/9$	$P(\text{cool} n) = 1/5$
humidity	
$P(\text{high} p) = 3/9$	$P(\text{high} n) = 4/5$
$P(\text{normal} p) = 6/9$	$P(\text{normal} n) = 1/5$
windy	
$P(\text{true} p) = 3/9$	$P(\text{true} n) = 3/5$
$P(\text{false} p) = 6/9$	$P(\text{false} n) = 2/5$

$$P(X|p) \cdot P(p) =$$

$$P(X|n) \cdot P(n) =$$

Play-tennis example. estimating $P(x_i | C)$

$P(p) = 9/14$
$P(n) = 5/14$

Outlook	Temperature	Humidity	Windy	Class
rain	hot	high	false	N

outlook	
$P(\text{sunny} p) = 2/9$	$P(\text{sunny} n) = 3/5$
$P(\text{overcast} p) = 4/9$	$P(\text{overcast} n) = 0$
$P(\text{rain} p) = 3/9$	$P(\text{rain} n) = 2/5$
temperature	
$P(\text{hot} p) = 2/9$	$P(\text{hot} n) = 2/5$
$P(\text{mild} p) = 4/9$	$P(\text{mild} n) = 2/5$
$P(\text{cool} p) = 3/9$	$P(\text{cool} n) = 1/5$
humidity	
$P(\text{high} p) = 3/9$	$P(\text{high} n) = 4/5$
$P(\text{normal} p) = 6/9$	$P(\text{normal} n) = 1/5$
windy	
$P(\text{true} p) = 3/9$	$P(\text{true} n) = 3/5$
$P(\text{false} p) = 6/9$	$P(\text{false} n) = 2/5$

$$P(X|p) \cdot P(p) = P(\text{rain}|p) \cdot P(\text{hot}|p) \cdot P(\text{high}|p) \cdot P(\text{false}|p) \cdot P(p) =$$

$$P(X|n) \cdot P(n) = P(\text{rain}|n) \cdot P(\text{hot}|n) \cdot P(\text{high}|n) \cdot P(\text{false}|n) \cdot P(n) =$$

Play-tennis example. estimating $P(x_i | C)$

$P(p) = 9/14$
$P(n) = 5/14$

Outlook	Temperature	Humidity	Windy	Class
rain	hot	high	false	N

outlook	
$P(\text{sunny} p) = 2/9$	$P(\text{sunny} n) = 3/5$
$P(\text{overcast} p) = 4/9$	$P(\text{overcast} n) = 0$
$P(\text{rain} p) = 3/9$	$P(\text{rain} n) = 2/5$
temperature	
$P(\text{hot} p) = 2/9$	$P(\text{hot} n) = 2/5$
$P(\text{mild} p) = 4/9$	$P(\text{mild} n) = 2/5$
$P(\text{cool} p) = 3/9$	$P(\text{cool} n) = 1/5$
humidity	
$P(\text{high} p) = 3/9$	$P(\text{high} n) = 4/5$
$P(\text{normal} p) = 6/9$	$P(\text{normal} n) = 1/5$
windy	
$P(\text{true} p) = 3/9$	$P(\text{true} n) = 3/5$
$P(\text{false} p) = 6/9$	$P(\text{false} n) = 2/5$

$$P(X|p) \cdot P(p) = P(\text{rain}|p) \cdot P(\text{hot}|p) \cdot P(\text{high}|p) \cdot P(\text{false}|p) \cdot P(p) = 3/9 \cdot 2/9 \cdot 3/9 \cdot 6/9 \cdot 9/14 = 0.010582$$

$$P(X|n) \cdot P(n) = P(\text{rain}|n) \cdot P(\text{hot}|n) \cdot P(\text{high}|n) \cdot P(\text{false}|n) \cdot P(n) = 2/5 \cdot 2/5 \cdot 4/5 \cdot 2/5 \cdot 5/14 = 0.018286$$

Example of Naïve Bayes Classifier

Name	Give Birth	Can Fly	Live in Water	Have Legs	Class
human	yes	no	no	yes	mammals
python	no	no	no	no	non-mammals
salmon	no	no	yes	no	non-mammals
whale	yes	no	yes	no	mammals
frog	no	no	sometimes	yes	non-mammals
komodo	no	no	no	yes	non-mammals
bat	yes	yes	no	yes	mammals
pigeon	no	yes	no	yes	non-mammals
cat	yes	no	no	yes	mammals
leopard shark	yes	no	yes	no	non-mammals
turtle	no	no	sometimes	yes	non-mammals
penguin	no	no	sometimes	yes	non-mammals
porcupine	yes	no	no	yes	mammals
eel	no	no	yes	no	non-mammals
salamander	no	no	sometimes	yes	non-mammals
gila monster	no	no	no	yes	non-mammals
platypus	no	no	no	yes	mammals
owl	no	yes	no	yes	non-mammals
dolphin	yes	no	yes	no	mammals
eagle	no	yes	no	yes	non-mammals

A: attributes

M: mammals

N: non-mammals

Give Birth	Can Fly	Live in Water	Have Legs	Class
yes	no	yes	no	?

a) Naive Bayes (3 points)

Given the training set below, build a Naive Bayes classification model (i.e. the corresponding table of probabilities) using (i) the normal formula and (ii) using Laplace formula. What are the main effects of Laplace on the models?

A	B	class
no	green	N
no	red	Y
yes	green	N
no	red	N
no	red	Y
no	green	Y
yes	green	N

Answer:

Normal

	Y	N		Y	N
	3	4		0.43	0.57
	A Y	A N		A Y	A N
yes	0	2	yes	0.00	0.50
no	3	2	no	1.00	0.50
	B Y	B N		B Y	B N
green	1	3	green	0.33	0.75
red	2	1	red	0.67	0.25

Laplace

	Y	N		Y	N
	3	4		0.43	0.57
	A Y	A N		A Y	A N
yes	0	2	yes	0.20	0.50
no	3	2	no	0.80	0.50
	B Y	B N		B Y	B N
green	1	3	green	0.40	0.67
red	2	1	red	0.60	0.33

a) Naive Bayes (3 points)

Given the training set on the left, build a Naive Bayes classification model and apply it to the test set on the right.

SCORE	FIRST-TRY	FACULTY	class
good	no	science	Y
medium	yes	science	N
bad	yes	science	N
bad	yes	humanities	Y
good	no	humanities	N
good	no	science	Y
medium	no	humanities	Y

SCORE	FIRST-TRY	FACULTY	class
bad	no	humanities	
good	yes	science	
medium	yes	humanities	

DATA MINING 1

Classification Model Evaluation

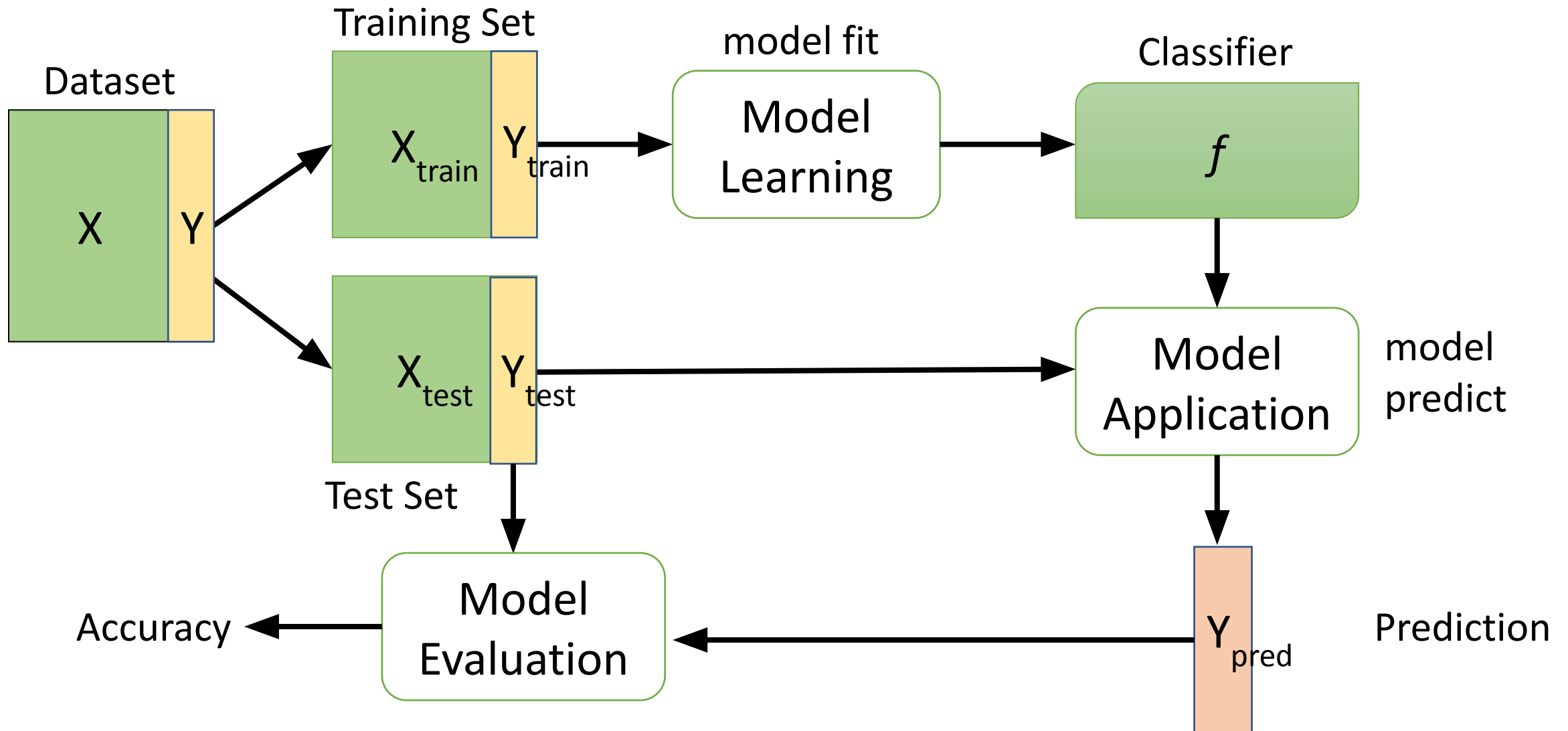
Dino Pedreschi, Riccardo Guidotti

Revisited slides from Lecture Notes for Chapter 3 “Introduction to Data Mining”, 2nd Edition by Tan, Steinbach, Karpatne, Kumar



UNIVERSITÀ DI PISA

What is Classification?



Model Evaluation

Model Evaluation

- Metrics for Performance Evaluation
 - How to evaluate the performance of a model?
- Methods for Performance Evaluation
 - How to obtain reliable estimates?
- Methods for Model Comparison
 - How to compare the relative performance among competing models?

Model Evaluation

- Metrics for Performance Evaluation
 - How to evaluate the performance of a model?
- Methods for Performance Evaluation
 - How to obtain reliable estimates?
- Methods for Model Comparison
 - How to compare the relative performance among competing models?

Problem Setting

- Let suppose we have a vector y of actual/real class labels, i.e.,
- $y = [0\ 0\ 0\ 1\ 1\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 1\ 1\ 0\ 0]$
- Let name y' the vector returned by a trained model f , i.e.,
- $y' = [0\ 0\ 1\ 1\ 1\ 0\ 0\ 1\ 0\ 1\ 1\ 1\ 0\ 0\ 0\ 0]$

Metrics for Performance Evaluation

- Focus on the predictive capability of a model
 - Rather than how fast it takes to classify or build models, scalability, etc.
- **Confusion Matrix:**

ACTUAL CLASS	PREDICTED CLASS		
		Class=Yes	Class=No
	Class=Yes	a	b
	Class=No	c	d

a: TP (true positive)

b: FN (false negative)

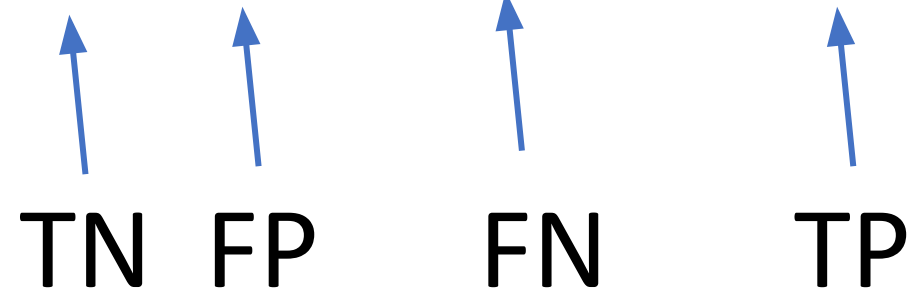
c: FP (false positive)

d: TN (true negative)

Metrics for Performance Evaluation

• $y = [0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0]$

• $y' = [0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0 \ 1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0]$

• 
TN FP FN TP

Metrics for Performance Evaluation...

ACTUAL CLASS	PREDICTED CLASS	
	Class=Yes	Class=No
Class=Yes	a (TP)	b (FN)
	c (FP)	d (TN)

Most widely-used metric:

$$\text{Accuracy} = \frac{a + d}{a + b + c + d} = \frac{TP + TN}{TP + TN + FP + FN}$$

Limitation of Accuracy

- Consider a 2-class problem
 - Number of Class 0 examples = 9990
 - Number of Class 1 examples = 10
- If model predicts everything to be class 0, accuracy is $9990/10000 = 99.9\%$
- Accuracy is misleading because model does not detect any class 1 example

Cost-Sensitive Measures

$$\text{Precision (p)} = \frac{TP}{TP + FP}$$

$$\text{Recall (r)} = \frac{TP}{TP + FN}$$

$$\text{F-measure (F)} = \frac{2rp}{r + p} = \frac{2TP}{2TP + FN + FP}$$

- Precision is biased towards $C(\text{Yes}|\text{Yes})$ & $C(\text{Yes}|\text{No})$
- Recall is biased towards $C(\text{Yes}|\text{Yes})$ & $C(\text{No}|\text{Yes})$
- F-measure is biased towards all except $C(\text{No}|\text{No})$

$$\text{Weighted Accuracy} = \frac{w_1 a + w_4 d}{w_1 a + w_2 b + w_3 c + w_4 d}$$

Cost Matrix

	PREDICTED CLASS		
	$C(i j)$	Class=Yes	Class=No
ACTUAL CLASS	Class=Yes	$C(\text{Yes} \text{Yes})$	$C(\text{No} \text{Yes})$
	Class=No	$C(\text{Yes} \text{No})$	$C(\text{No} \text{No})$

$C(i|j)$: Cost of misclassifying class j example as class i

Computing Cost of Classification

Cost Matrix	PREDICTED CLASS		
	$C(i j)$	+	-
	+	-1	100
	-	1	0

Model M_1	PREDICTED CLASS		
ACTUAL CLASS		+	-
	+	150	40
	-	60	250

Accuracy = 80%
Cost = 3910

Model M_2	PREDICTED CLASS		
ACTUAL CLASS		+	-
	+	250	45
	-	5	200

Accuracy = 90%
Cost = 4255

Cost vs Accuracy

Count	PREDICTED CLASS		
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	a	b
	Class=No	c	d

Cost	PREDICTED CLASS		
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	p	q
	Class=No	q	p

Accuracy is proportional to cost if

$$1. C(\text{Yes}|\text{No})=C(\text{No}|\text{Yes}) = q$$

$$2. C(\text{Yes}|\text{Yes})=C(\text{No}|\text{No}) = p$$

$$N = a + b + c + d$$

$$\text{Accuracy} = (a + d)/N$$

$$\text{Cost} = p (a + d) + q (b + c)$$

$$= p (a + d) + q (N - a - d)$$

$$= q N - (q - p)(a + d)$$

$$= N [q - (q-p) \times \text{Accuracy}]$$

Binary vs Multiclass Evaluation

	PREDICTED CLASS		
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	TP	FN
	Class=No	FP	TN

	PREDICTED CLASS			
ACTUAL CLASS		Class=A	Class=B	Class=C
	Class=A	TP-A		
	Class=B		TP-B	
	Class=C			TP-C

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FN+FP} = \frac{\# \text{ correct}}{N}$$

$$\text{Accuracy} = \# \text{ correct} / N = (\text{TP-A} + \text{TP-B} + \text{TP-C}) / N$$

Multiclass Evaluation

	PREDICTED CLASS			
		Class=A	Class=B	Class=C
ACTUAL CLASS	Class=A	TP-A	a	b
	Class=B	c	TP-B	d
	Class=C	e	f	TP-C

$$\text{Precision (p)} = \frac{TP}{TP + FP}$$

$$\text{Recall (r)} = \frac{TP}{TP + FN}$$

$$\text{F-measure (F)} = \frac{2rp}{r + p} = \frac{2TP}{2TP + FN + FP}$$

A	PREDICTED CLASS		
ACTUAL CLASS		Class=A	Class=Not A
	Class=A	TP-A	a + b
	Class=Not A	c + e	TP-B + TP-C + d + f

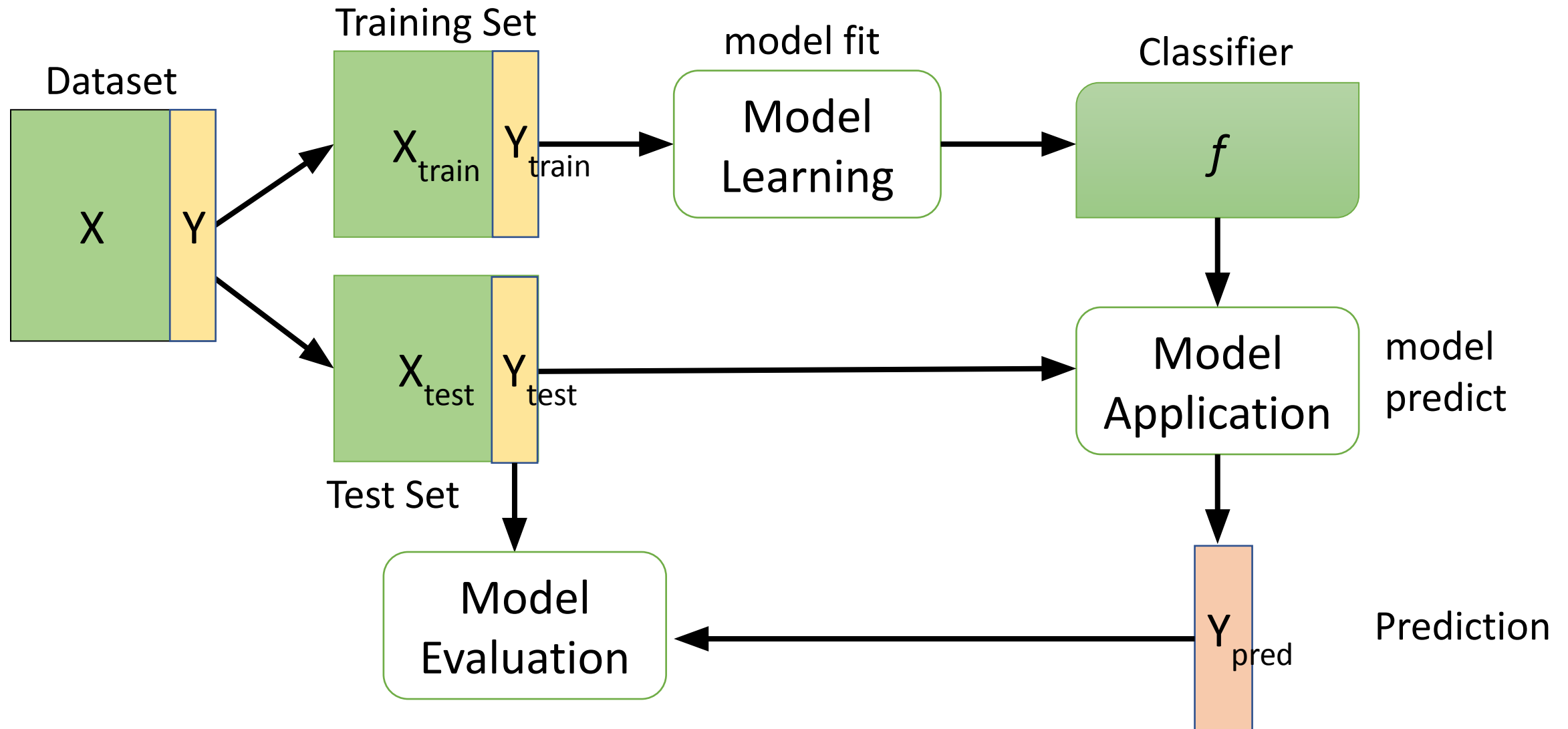
B	PREDICTED CLASS		
ACTUAL CLASS		Class=B	Class=Not B
	Class=B	TP-B	c + d
	Class=Not B	a + f	TP-A + TP-C + b + e

C	PREDICTED CLASS		
ACTUAL CLASS		Class=C	Class=Not C
	Class=C	TP-C	e + f
	Class=Not C	b + d	TP-A + TP-B + a + c

Model Evaluation

- Metrics for Performance Evaluation
 - How to evaluate the performance of a model?
- Methods for Performance Evaluation
 - How to obtain reliable estimates?
- Methods for Model Comparison
 - How to compare the relative performance among competing models?

Methods for Evaluation



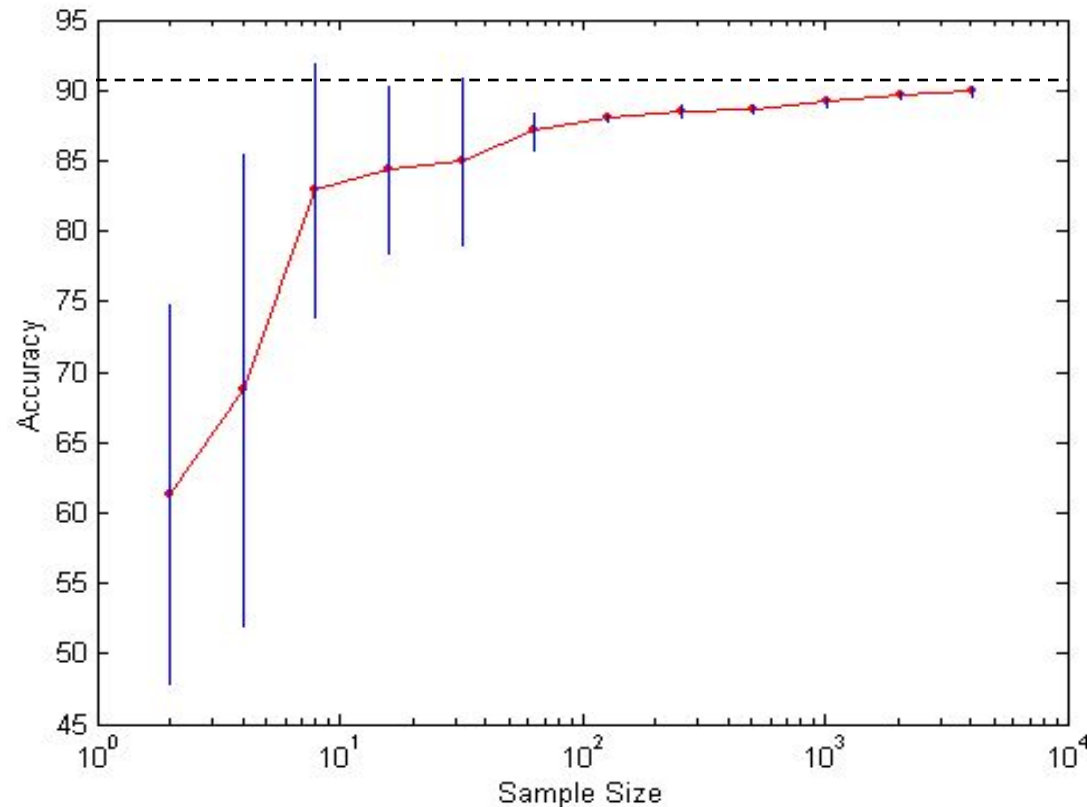
Parameter Tuning

- It is important that the test data is not used in any way to create the classifier
- Some learning schemes operate in two stages:
 - **Stage 1:** builds the basic structure
 - **Stage 2:** optimizes parameter settings
 - **The test data can't be used for parameter tuning!**
 - Proper procedure uses three sets:
 - training data,
 - validation data,
 - test data
 - **Validation data is used to optimize parameters**
- Once evaluation is complete, all the data can be used to build the final classifier
- Generally, the larger the training data the better the classifier
- The larger the test data the more accurate the error estimate

Methods for Performance Evaluation

- How to obtain a reliable estimate of performance?
- Performance of a model may depend on other factors besides the learning algorithm:
 - Class distribution
 - Cost of misclassification
 - Size of training and test sets

Learning Curve



- Learning curve shows how accuracy changes with varying sample size
- Requires a sampling schedule for creating learning curve:

Effect of small sample size:

- Bias in the estimate
- Variance of estimate

1. How much a classification model benefits from adding more training data?
2. Does the model suffer from a variance error or a bias error?

Methods of Estimation

- Holdout
 - Reserve $2/3$ for training and $1/3$ for testing
- Random subsampling
 - Repeated holdout
- Cross validation
 - Partition data into k disjoint subsets
 - k -fold: train on $k-1$ partitions, test on the remaining one
 - Leave-one-out: $k=n$
- Stratified sampling
 - oversampling vs undersampling
- Bootstrap
 - Sampling with replacement

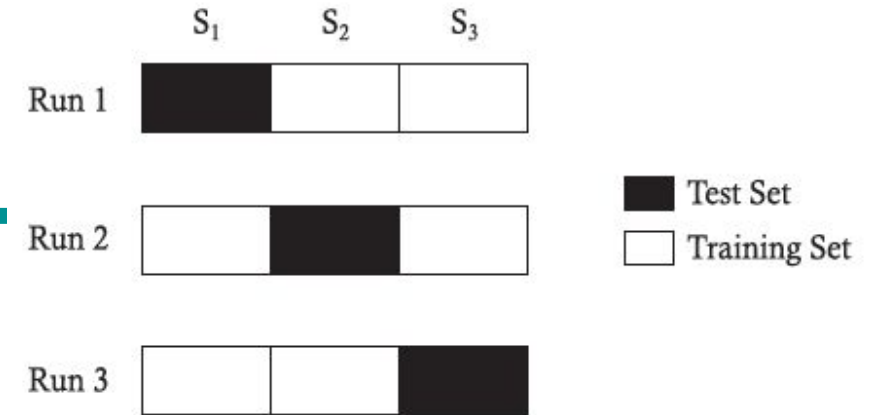
Holdout

- The holdout method reserves a certain amount for **testing** and uses the remainder for **training**
- Usually, **one third for testing**, the rest for training.
- Typical quantities are 60%-40%, 66%-34%, 70%-30%.
- For small or “unbalanced” datasets, **samples might not be representative**
 - For instance, few or none instances of some classes
- Stratified sample
 - **Balancing the data**
 - Make sure that each class is represented with approximately equal proportions in both subsets

Repeated Holdout

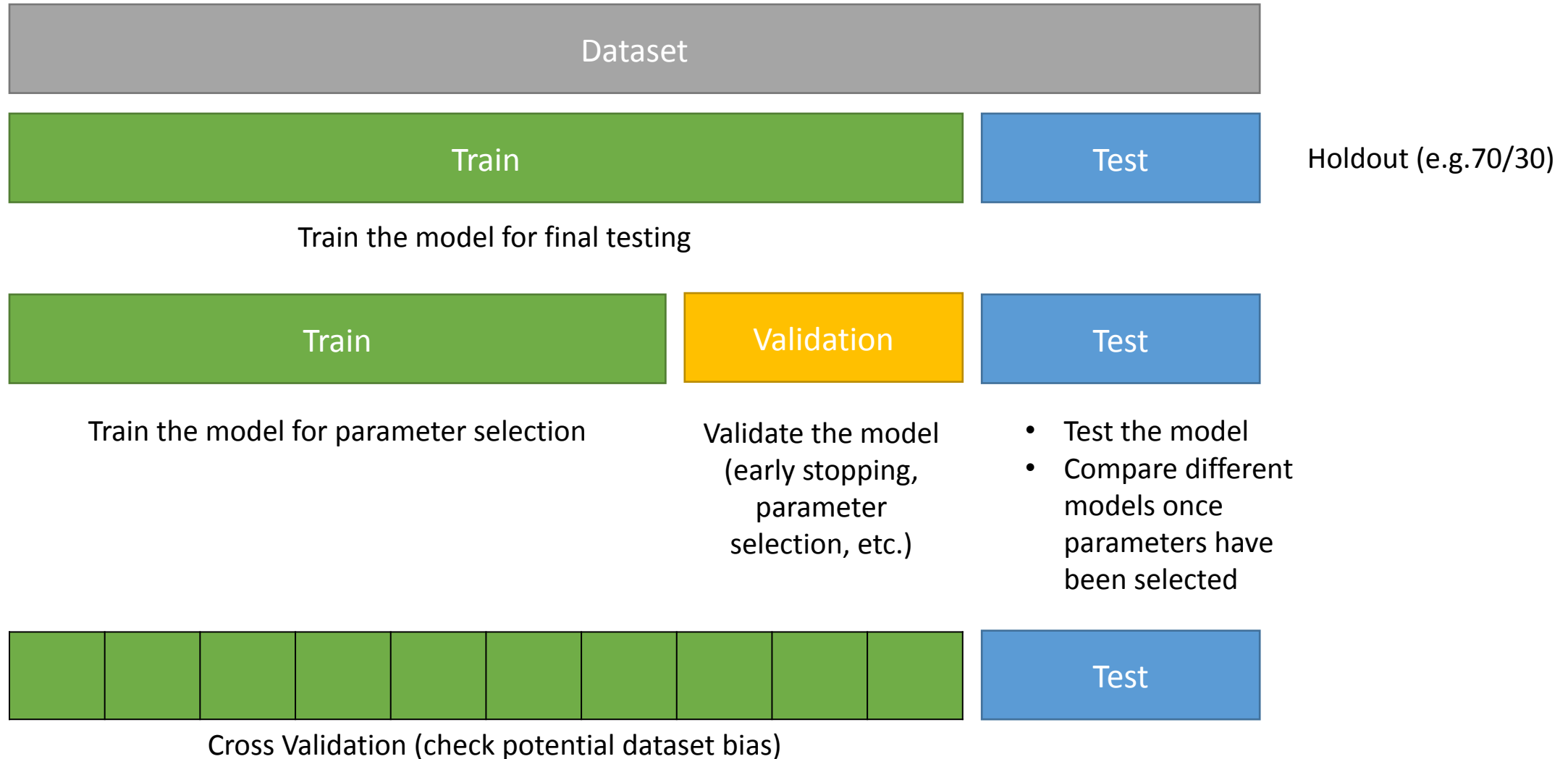
- Holdout estimate can be made more reliable by **repeating the process with different subsamples**
 - In each iteration, a certain proportion is **randomly selected for training** (possibly with stratification)
 - The error rates on the different iterations are **averaged** to yield an overall error rate
- This is called the **repeated holdout method**
- Still not optimum: the different test sets overlap

Cross Validation

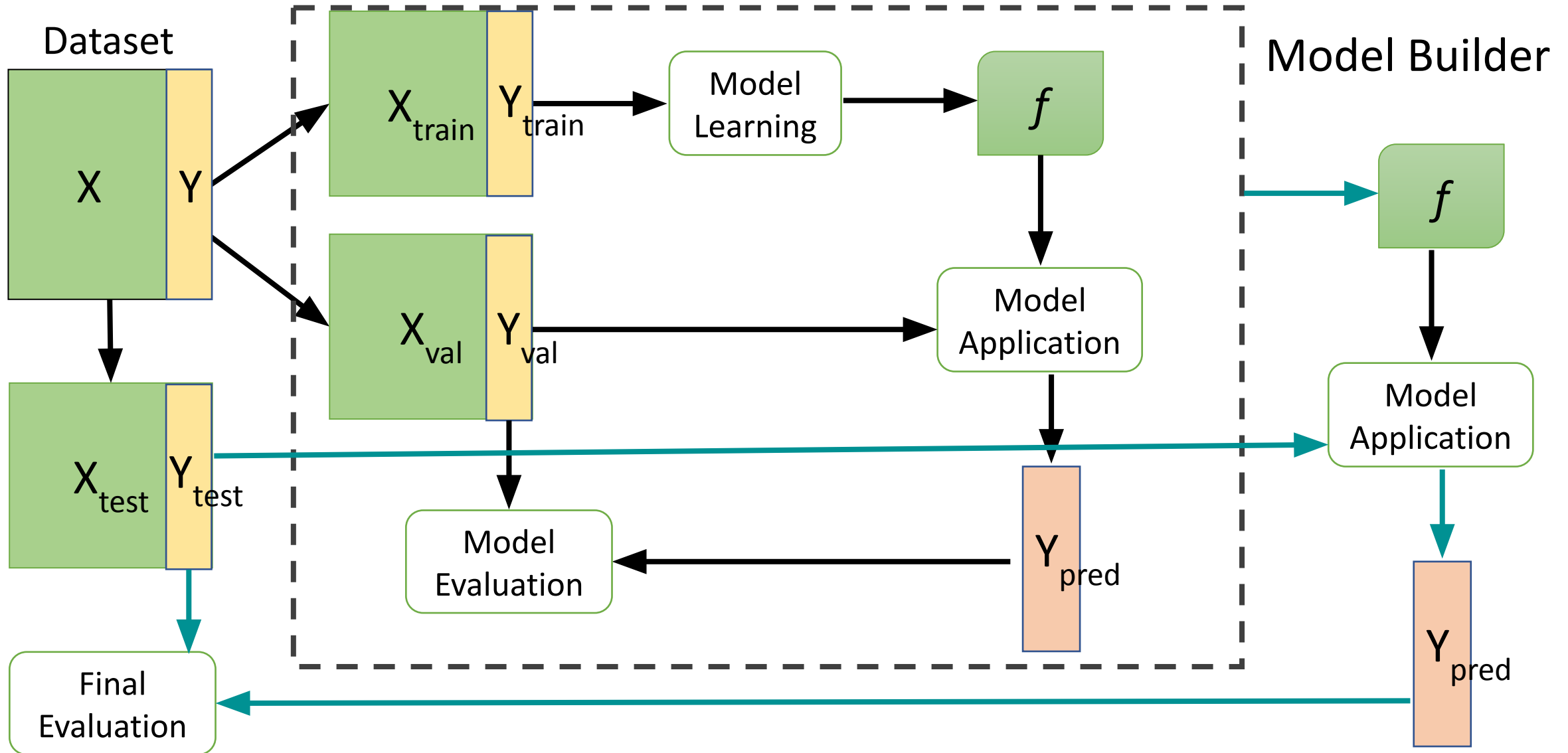


- Avoids overlapping test sets
 - **First step:** data is split into k subsets of equal size
 - **Second step:** each subset in turn is used for testing and the remainder for training
- This is called **k-fold cross-validation**
- Often the subsets are stratified before cross-validation is performed
- The **error estimates** are **averaged** to yield an overall error estimate
- **Even better:** repeated stratified cross-validation E.g. ten-fold cross-validation is repeated ten times and results are averaged (reduces the variance)

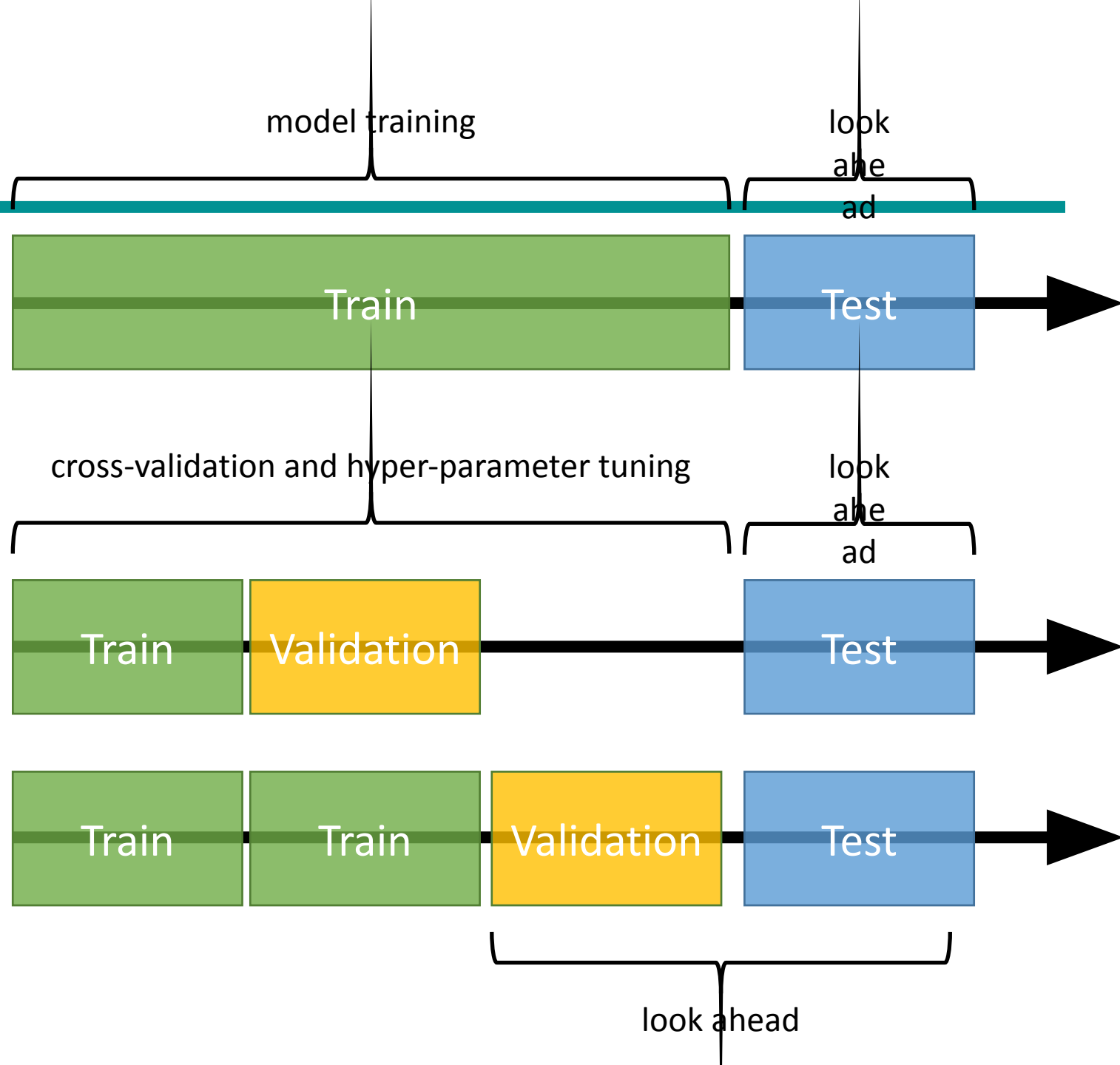
Data Partitioning



Evaluation: Training, Validation, Tests



Cross Validation with Time



Model Evaluation

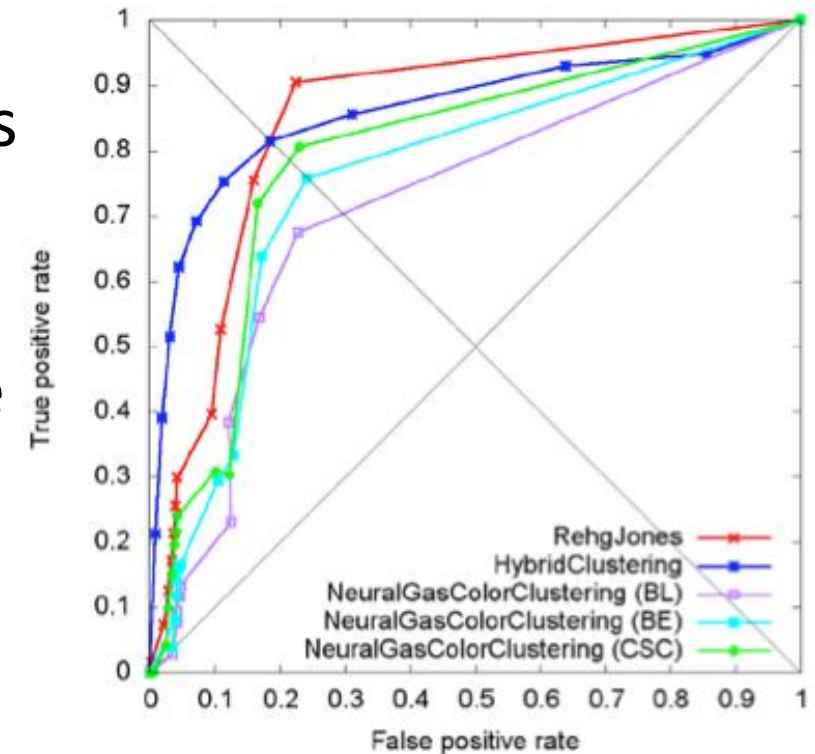
- Metrics for Performance Evaluation
 - How to evaluate the performance of a model?
- Methods for Performance Evaluation
 - How to obtain reliable estimates?
- **Methods for Model Comparison**
 - How to compare the relative performance among competing models?

ROC (Receiver Operating Characteristic)

- Developed in 1950s for signal detection theory to analyze noisy signals
 - Characterize the trade-off between positive hits and false alarms
- ROC curve plots TPR (on the y-axis) against FPR (on the x-axis)
- **Performance of each classifier represented as a point on the ROC curve**
 - changing the threshold of algorithm, sample distribution or cost matrix changes the location of the point

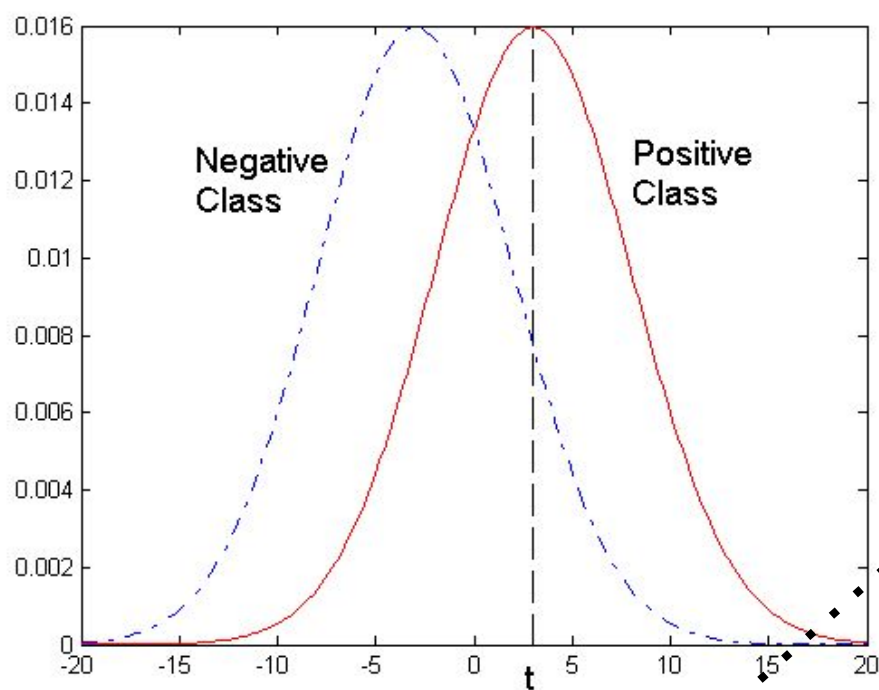
Receiver Operating Characteristic Curve

- It illustrates the ability of a binary classifier as its discrimination threshold THR is varied.
- The **ROC** curve is created by plotting the true positive rate (TPR) against the false positive rate (FPR) at various THR.
- The $TPR = TP / (TP + FN)$ is also known as **sensitivity**, **recall** or probability of detection.
- The $FPR = FP / (TN + FP)$ is also known as probability of **false alarm** and can be calculated as $(1 - \text{specificity})$.



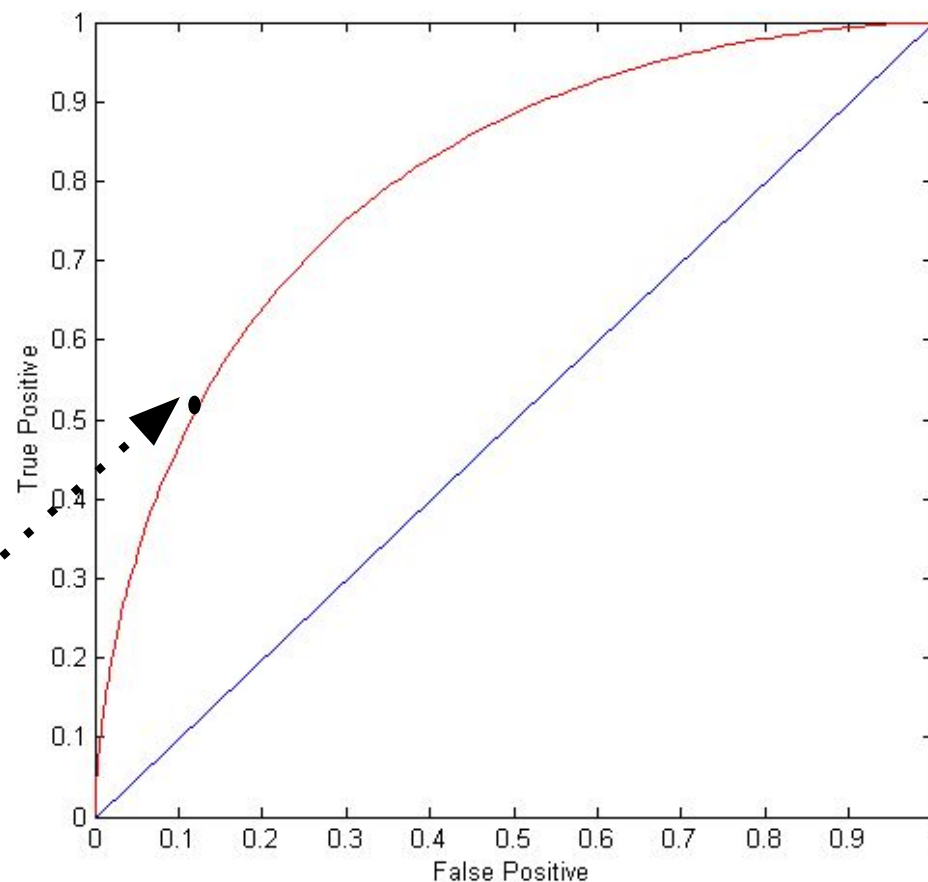
ROC Curve

- 1-dimensional data set containing 2 classes (positive and negative)
- any points located at $x > t$ is classified as positive



At threshold t :

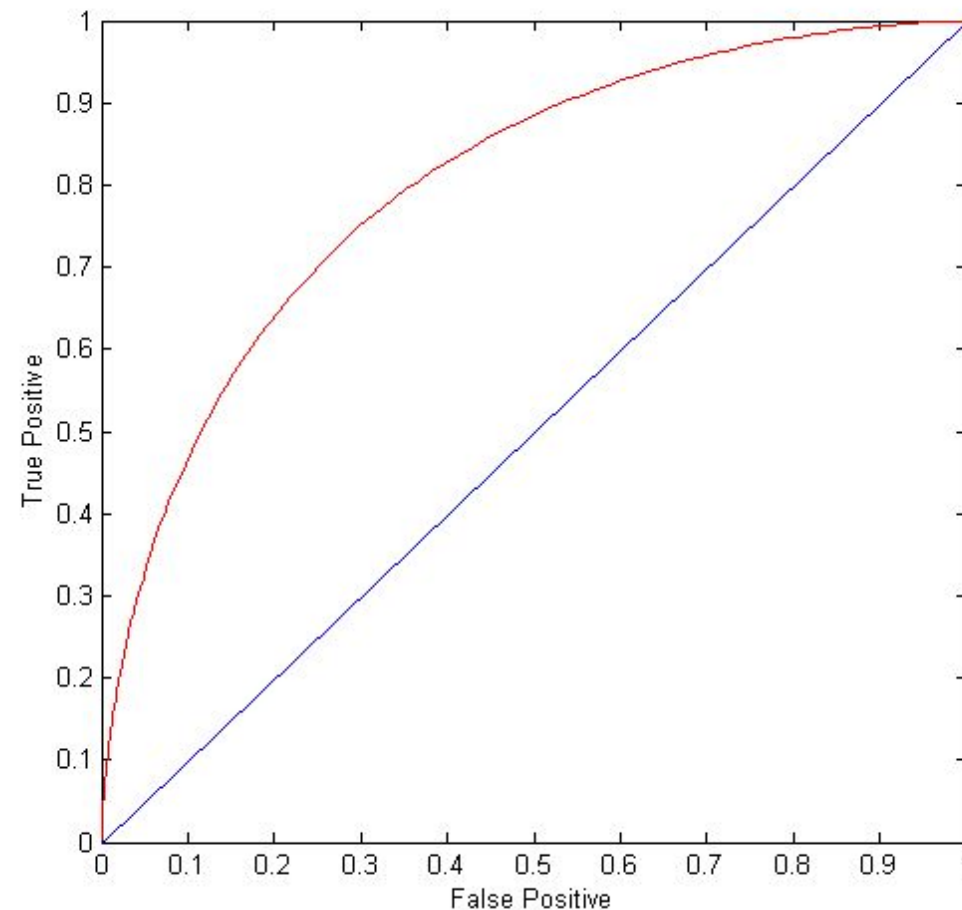
TP=0.5, FN=0.5, FP=0.12, FN=0.88



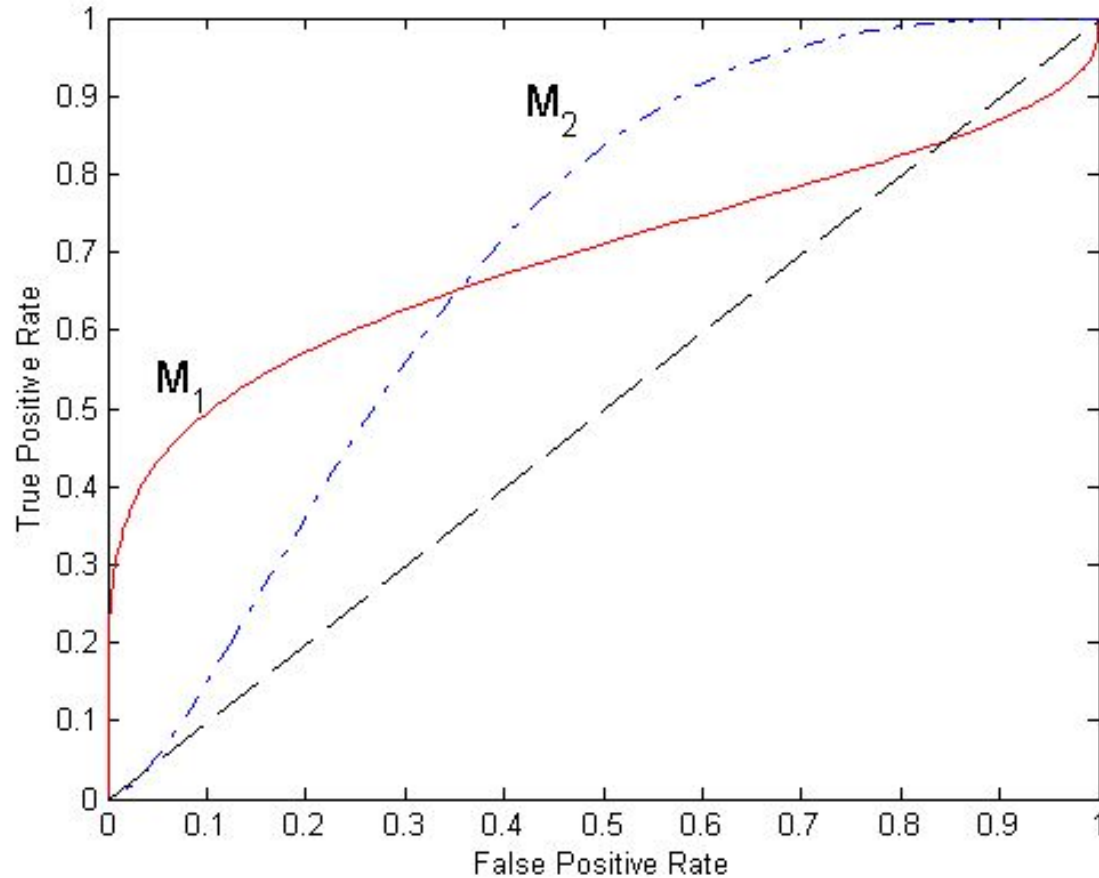
ROC Curve

(TP,FP):

- (0,0): declare everything to be negative class
- (1,1): declare everything to be positive class
- (0,1): ideal
- Diagonal line:
 - Random guessing
 - Below diagonal line:
 - prediction is opposite of the true class



Using ROC for Model Comparison



- No model consistently outperform the other
 - M_1 is better for small FPR
 - M_2 is better for large FPR
- Area Under the ROC curve
 - Ideal: Area = 1
 - Random: Area = 0.5

How to Construct the ROC curve

Instance	$P(+ A)$	True Class
1	0.95	+
2	0.93	+
3	0.87	-
4	0.85	-
5	0.85	-
6	0.85	+
7	0.76	-
8	0.53	+
9	0.43	-
10	0.25	+

- Use classifier that produces posterior probability for each test instance $P(+|A)$
- Sort the instances according to $P(+|A)$ in decreasing order
- Apply threshold at each unique value of $P(+|A)$
- Count the number of TP, FP, TN, FN at each threshold
- TP rate, $TPR = TP/(TP+FN)$
- FP rate, $FPR = FP/(FP + TN)$

TPR = TP / (TP + FN)

FPR = FP / (TN + FP)

How to Construct the ROC curve

Class												Inst.		
												P(+ A)		
												True Class		
Threshold >=														
TP												1		
FP												2		
TN												3		
FN												4		
TPR												5		
FPR												6		
												7		
												8		
												9		
												10		

TPR = TP / (TP + FN)

FPR = FP / (TN + FP)

How to Construct the ROC curve

Threshold >=

→
→

Class	+	-	+	-	-	-	+	-	+	+	
	0.25	0.43	0.53	0.76	0.85	0.85	0.85	0.87	0.93	0.95	1.00
TP	5										
FP	5										
TN	0										
FN	0										
TPR	1										
FPR	1										

Inst.	P(+ A)	True Class
1	0.95	+
2	0.93	+
3	0.87	-
4	0.85	-
5	0.85	-
6	0.85	+
7	0.76	-
8	0.53	+
9	0.43	-
10	0.25	+

$TPR = TP / (TP + FN)$
 $FPR = FP / (TN + FP)$

How to Construct the ROC curve

Class	+	-	+	-	-	-	+	-	+	+	
Threshold >=	0.25	0.43	0.53	0.76	0.85	0.85	0.85	0.87	0.93	0.95	1.00
TP	5	4									
FP	5	5									
TN	0	0									
FN	0	1									
→ TPR	1	0.8									
→ FPR	1	1									

Inst.	P(+ A)	True Class
1	0.95	+
2	0.93	+
3	0.87	-
4	0.85	-
5	0.85	-
6	0.85	+
7	0.76	-
8	0.53	+
9	0.43	-
10	0.25	+

$TPR = TP / (TP + FN)$
 $FPR = FP / (TN + FP)$

How to Construct the ROC curve

Class	+	-	+	-	-	-	+	-	+	+	
Threshold >=	0.25	0.43	0.53	0.76	0.85	0.85	0.85	0.87	0.93	0.95	1.00
TP	5	4	4								
FP	5	5	4								
TN	0	0	1								
FN	0	1	1								
→ TPR	1	0.8	0.8								
→ FPR	1	1	0.8								

Inst.	P(+ A)	True Class
1	0.95	+
2	0.93	+
3	0.87	-
4	0.85	-
5	0.85	-
6	0.85	+
7	0.76	-
8	0.53	+
9	0.43	-
10	0.25	+

$TPR = TP / (TP + FN)$
 $FPR = FP / (TN + FP)$

How to Construct the ROC curve

Threshold >=

→
→

Class	+	-	+	-	-	-	+	-	+	+	
	0.25	0.43	0.53	0.76	0.85	0.85	0.85	0.87	0.93	0.95	1.00
TP	5	4	4	3							
FP	5	5	4	4							
TN	0	0	1	1							
FN	0	1	1	2							
TPR	1	0.8	0.8	0.6							
FPR	1	1	0.8	0.8							

Inst.	P(+ A)	True Class
1	0.95	+
2	0.93	+
3	0.87	-
4	0.85	-
5	0.85	-
6	0.85	+
7	0.76	-
8	0.53	+
9	0.43	-
10	0.25	+

$$TPR = TP / (TP + FN)$$

$$FPR = FP / (TN + FP)$$

How to Construct the ROC curve

												Inst.	P(+ A)	True Class
												1	0.95	+
												2	0.93	+
												3	0.87	-
												4	0.85	-
												5	0.85	-
												6	0.85	+
												7	0.76	-
												8	0.53	+
												9	0.43	-
												10	0.25	+

Class		+	-	+	-	-	-	+	-	+	+	
Threshold >=		0.25	0.43	0.53	0.76	0.85	0.85	0.85	0.87	0.93	0.95	1.00
TP		5	4	4	3	3						
FP		5	5	4	4	3						
TN		0	0	1	1	2						
FN		0	1	1	2	2						
→	TPR	1	0.8	0.8	0.6	0.6						
→	FPR	1	1	0.8	0.8	0.6						

$$TPR = TP / (TP + FN)$$

$$FPR = FP / (TN + FP)$$

How to Construct the ROC curve

Threshold >=

Class	+	-	+	-	-	-	+	-	+	+	
	0.25	0.43	0.53	0.76	0.85	0.85	0.85	0.87	0.93	0.95	1.00
TP	5	4	4	3	3	3	3	2			
FP	5	5	4	4	3	2	1	1			
TN	0	0	1	1	2	3	4	4			
FN	0	1	1	2	2	2	2	3			
TPR	1	0.8	0.8	0.6	0.6	0.6	0.6	0.4			
FPR	1	1	0.8	0.8	0.6	0.4	0.2	0.2			

Inst.	P(+ A)	True Class
1	0.95	+
2	0.93	+
3	0.87	-
4	0.85	-
5	0.85	-
6	0.85	+
7	0.76	-
8	0.53	+
9	0.43	-
10	0.25	+

$$TPR = TP / (TP + FN)$$

$$FPR = FP / (TN + FP)$$

How to Construct the ROC curve

Threshold >=

Class	+	-	+	-	-	-	+	-	+	+	
	0.25	0.43	0.53	0.76	0.85	0.85	0.85	0.87	0.93	0.95	1.00
TP	5	4	4	3	3	3	3	2	2		
FP	5	5	4	4	3	2	1	1	0		
TN	0	0	1	1	2	3	4	4	5		
FN	0	1	1	2	2	2	2	3	3		
TPR	1	0.8	0.8	0.6	0.6	0.6	0.6	0.4	0.4		
FPR	1	1	0.8	0.8	0.6	0.4	0.2	0.2	0		

→

→

Inst.	P(+ A)	True Class
1	0.95	+
2	0.93	+
3	0.87	-
4	0.85	-
5	0.85	-
6	0.85	+
7	0.76	-
8	0.53	+
9	0.43	-
10	0.25	+

$$TPR = TP / (TP + FN)$$

$$FPR = FP / (TN + FP)$$

How to Construct the ROC curve

Threshold >=

Class	+	-	+	-	-	-	+	-	+	+	
	0.25	0.43	0.53	0.76	0.85	0.85	0.85	0.87	0.93	0.95	1.00
TP	5	4	4	3	3	3	3	2	2	1	
FP	5	5	4	4	3	2	1	1	0	0	
TN	0	0	1	1	2	3	4	4	5	5	
FN	0	1	1	2	2	2	2	3	3	4	
TPR	1	0.8	0.8	0.6	0.6	0.6	0.6	0.4	0.4	0.2	
FPR	1	1	0.8	0.8	0.6	0.4	0.2	0.2	0	0	

Inst.	P(+ A)	True Class
1	0.95	+
2	0.93	+
3	0.87	-
4	0.85	-
5	0.85	-
6	0.85	+
7	0.76	-
8	0.53	+
9	0.43	-
10	0.25	+

$$TPR = TP / (TP + FN)$$

$$FPR = FP / (TN + FP)$$

How to Construct the ROC curve

Threshold >=

Class	+	-	+	-	-	-	+	-	+	+	
	0.25	0.43	0.53	0.76	0.85	0.85	0.85	0.87	0.93	0.95	1.00
TP	5	4	4	3	3	3	3	2	2	1	0
FP	5	5	4	4	3	2	1	1	0	0	0
TN	0	0	1	1	2	3	4	4	5	5	5
FN	0	1	1	2	2	2	2	3	3	4	5
TPR	1	0.8	0.8	0.6	0.6	0.6	0.6	0.4	0.4	0.2	0
FPR	1	1	0.8	0.8	0.6	0.4	0.2	0.2	0	0	0

Inst.	P(+ A)	True Class
1	0.95	+
2	0.93	+
3	0.87	-
4	0.85	-
5	0.85	-
6	0.85	+
7	0.76	-
8	0.53	+
9	0.43	-
10	0.25	+

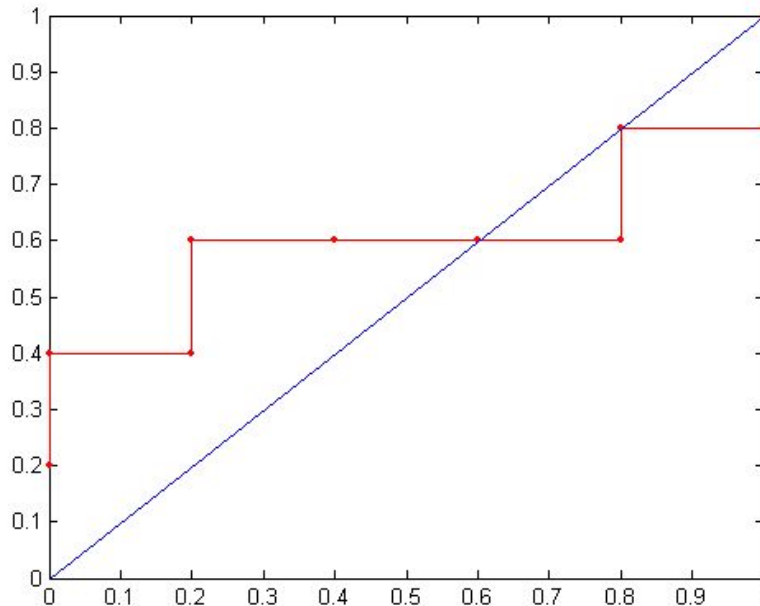
How to Construct the ROC curve

Class	+	-	+	-	-	-	+	-	+	+	
Threshold >=	0.25	0.43	0.53	0.76	0.85	0.85	0.85	0.87	0.93	0.95	1.00
TP	5	4	4	3	3	3	3	2	2	1	0
FP	5	5	4	4	3	2	1	1	0	0	0
TN	0	0	1	1	2	3	4	4	5	5	5
FN	0	1	1	2	2	2	2	3	3	4	5
TPR	1	0.8	0.8	0.6	0.6	0.6	0.6	0.4	0.4	0.2	0
FPR	1	1	0.8	0.8	0.6	0.4	0.2	0.2	0	0	0

$$\text{TPR} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{FPR} = \text{FP} / (\text{TN} + \text{FP})$$

ROC Curve:



Inst.	P(+ A)	True Class
1	0.95	+
2	0.93	+
3	0.87	-
4	0.85	-
5	0.85	-
6	0.85	+
7	0.76	-
8	0.53	+
9	0.43	-
10	0.25	+

Test of Significance

- Given two models:
 - Model M1: accuracy = 85%, tested on 30 instances
 - Model M2: accuracy = 75%, tested on 5000 instances
- Can we say M1 is better than M2?
 - How much confidence can we place on accuracy of M1 and M2?
 - Can the difference in performance measure be explained as a result of **random fluctuations** in the test set?

Confidence Interval for Accuracy

- Prediction can be regarded as a Bernoulli trial (binomial random experiment)
 - A Bernoulli trial has 2 possible outcomes
 - Possible outcomes for prediction: correct or wrong
 - Probability of success is constant
 - Collection of Bernoulli trials has a Binomial distribution:
 - $x \sim \text{Bin}(N, p)$ **x**: # of correct predictions, **N** trials, **p** constant prob.
 - e.g: Toss a fair coin 50 times, how many heads would turn up?
Expected number of heads = $N \times p = 50 \times 0.5 = 25$

Given x (# of correct predictions) or equivalently, $\text{acc} = x/N$, and N (# of test instances)

Can we predict p (true accuracy of model)?

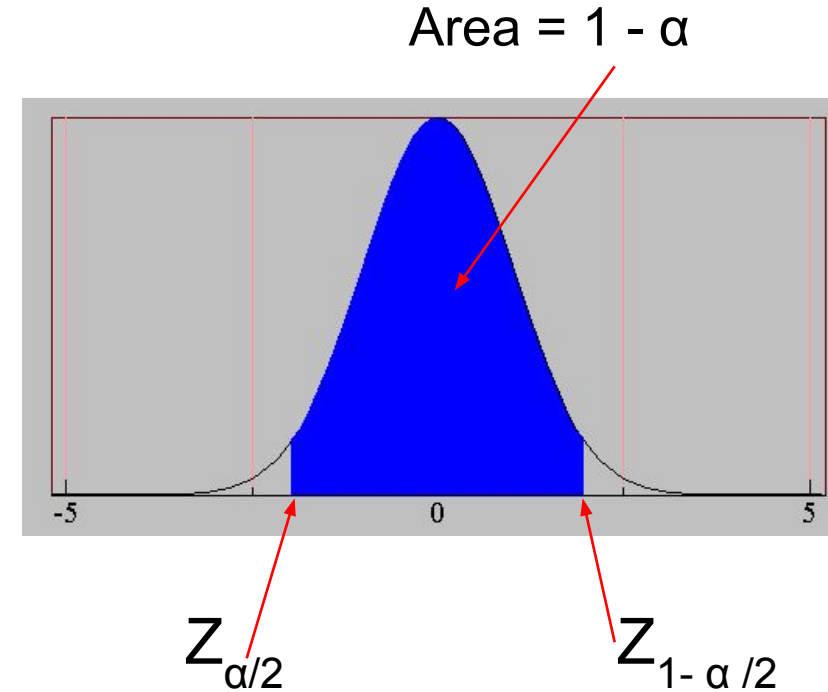
Confidence Interval for Accuracy

- For large test sets ($N > 30$),
 - acc has a normal distribution with mean p and variance $p(1-p)/N$

$$P(Z_{\alpha/2} < \frac{acc - p}{\sqrt{p(1-p)/N}} < Z_{1-\alpha/2})$$

- Confidence Interval for p :

$$p = \frac{2 \times N \times acc + Z_{\alpha/2}^2 \pm \sqrt{Z_{\alpha/2}^2 + 4 \times N \times acc - 4 \times N \times acc^2}}{2(N + Z_{\alpha/2}^2)}$$

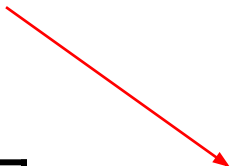


Confidence Interval for Accuracy

- Consider a model that produces an accuracy of 80% when evaluated on 100 test instances:
 - N=100, acc = 0.8
 - Let $1-\alpha = 0.95$ (95% confidence)
 - Which is the confidence interval?**
 - From probability table, $Z_{\alpha/2} = 1.96$

N	50	100	500	1000	5000
p(lower)	0.670	0.711	0.763	0.774	0.789
p(upper)	0.888	0.866	0.833	0.824	0.811

$1-\alpha$	Z
0.99	2.58
0.98	2.33
0.95	1.96
0.90	1.65



Comparing Performance of 2 Models

- Given two models, say M1 and M2, which is better?
 - M1 is tested on D1 (size= n_1), found error rate = e_1
 - M2 is tested on D2 (size= n_2), found error rate = e_2
 - Assume D1 and D2 are independent
 - If n_1 and n_2 are sufficiently large, then

$$e_1 \sim N(\mu_1, \sigma_1)$$

$$e_2 \sim N(\mu_2, \sigma_2)$$

- Approximate variance of error rates:

$$\hat{\sigma}_i = \frac{e_i(1-e_i)}{n_i}$$

Comparing Performance of 2 Models

- To test if performance difference is statistically significant: $d = e_1 - e_2$
 - $d \sim N(d_t, \sigma_t)$ where d_t is the true difference
 - Since D1 and D2 are independent, their variance adds up:

$$\begin{aligned}\sigma_t^2 &= \sigma_1^2 + \sigma_2^2 \cong \hat{\sigma}_1^2 + \hat{\sigma}_2^2 \\ &= \frac{e1(1-e1)}{n1} + \frac{e2(1-e2)}{n2}\end{aligned}$$

- It can be shown at $(1-\alpha)$ confidence level,

$$d_t = d \pm Z_{\alpha/2} \hat{\sigma}_t$$

An Illustrative Example

- Given: M1: $n_1 = 30$, $e_1 = 0.15$
M2: $n_2 = 5000$, $e_2 = 0.25$

- $d = |e_2 - e_1| = 0.1$ (2-sided test to check: $d_t = 0$ or $d_t \neq 0$)

$$\hat{\sigma}_d^2 = \frac{0.15(1-0.15)}{30} + \frac{0.25(1-0.25)}{5000} = 0.0043$$

- At 95% confidence level, $Z_{\alpha/2} = 1.96$

$$d_t = 0.100 \pm 1.96 \times \sqrt{0.0043} = 0.100 \pm 0.128$$

=> Interval contains 0 => difference may not be statistically significant

Comparing Performance of 2 Algorithms

- Each learning algorithm may produce k models:
 - L1 may produce $M11, M12, \dots, M1k$
 - L2 may produce $M21, M22, \dots, M2k$
- If models are generated on the same test sets $D1, D2, \dots, Dk$ (e.g., via cross-validation)

- For each set: compute $d_j = e_{1j} - e_{2j}$
- d_j has mean d_t and variance $\sigma_t^2 = \frac{1}{k} \sum_{j=1}^k (d_j - \bar{d})^2$
- Estimate:

$$\hat{\sigma}_t^2 = \frac{\sum_{j=1}^k (d_j - \bar{d})^2}{k(k-1)}$$

$$d_t = \bar{d} \pm t_{1-\alpha, k-1} \hat{\sigma}_t$$

Lift Chart

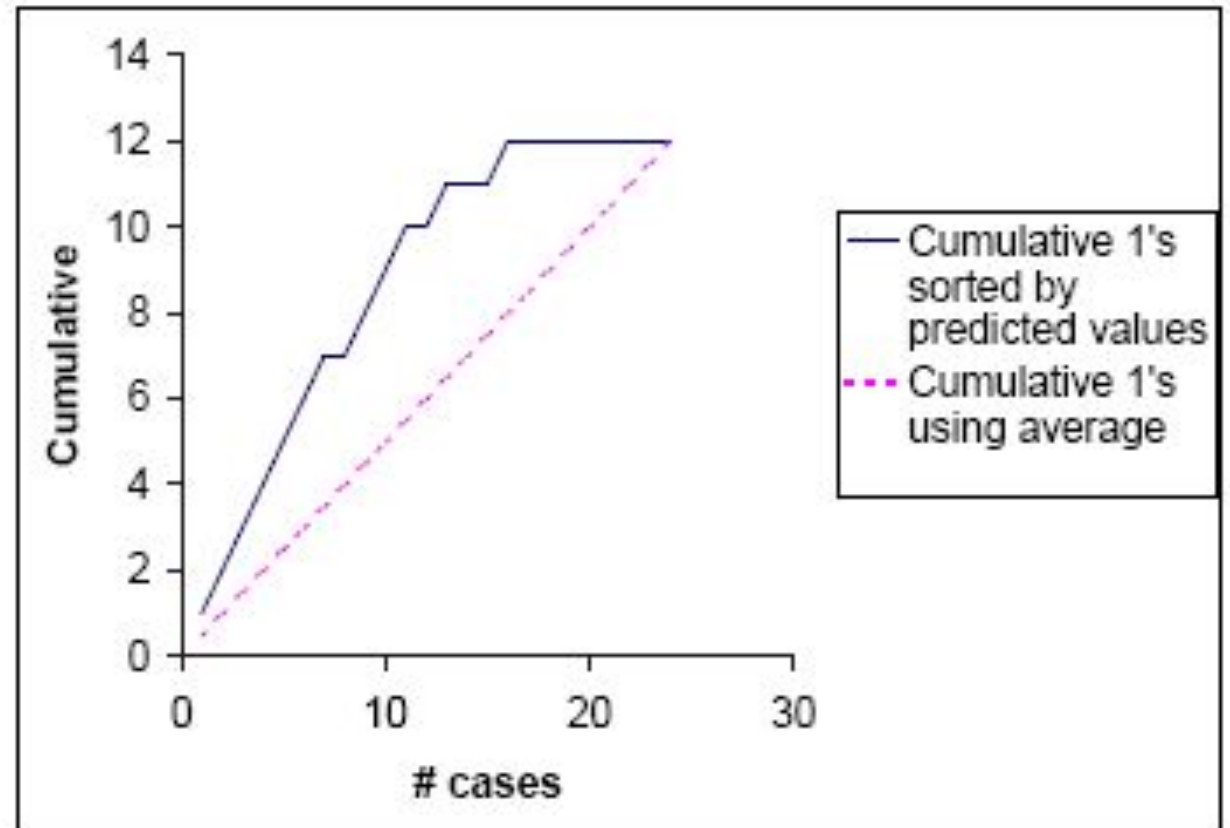
http://www2.cs.uregina.ca/~dbd/cs831/notes/lift_chart/lift_chart.html
http://mlwiki.org/index.php/Cumulative_Gain_Chart

- The lift curve is a popular technique in direct marketing.
- The input is a dataset that has been “scored” by appending to each case the estimated probability that it will belong to a given class.
- The cumulative **lift chart** (also called **gains chart**) is constructed with the cumulative number of cases (descending order of probability) on the x-axis and the cumulative number of true positives on the y-axis.
- The dashed line is a reference line. For any given number of cases (the x-axis value), it represents the expected number of positives we would predict if we did not have a model but simply selected cases at random. It provides a benchmark against which we can see performance of the model.

Notice: “Lift chart” is a rather general term, often used to identify also other kinds of plots. Don’t get confused!

Lift Chart – Example

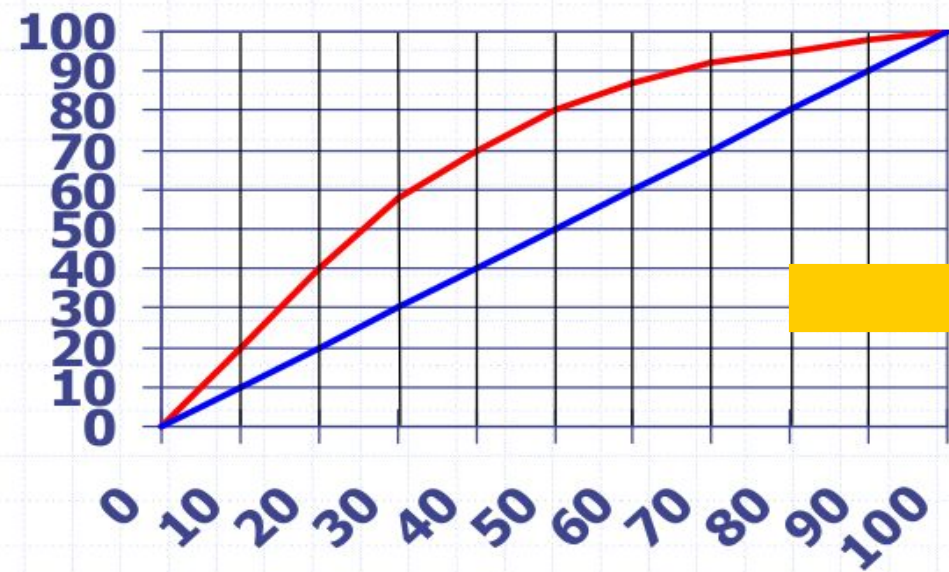
Serial no.	Predicted prob of 1	Actual Class	Cumulative Actual class
1	0.995976726	1	1
2	0.987533139	1	2
3	0.984456382	1	3
4	0.980439587	1	4
5	0.948110638	1	5
6	0.889297203	1	6
7	0.847631864	1	7
8	0.762806287	0	7
9	0.706991915	1	8
10	0.680754087	1	9
11	0.656343749	1	10
12	0.622419543	0	10
13	0.505506928	1	11
14	0.47134045	0	11
15	0.337117362	0	11
16	0.21796781	1	12
17	0.199240432	0	12
18	0.149482655	0	12
19	0.047962588	0	12
20	0.038341401	0	12
21	0.024850999	0	12
22	0.021806029	0	12
23	0.016129906	0	12
24	0.003559986	0	12



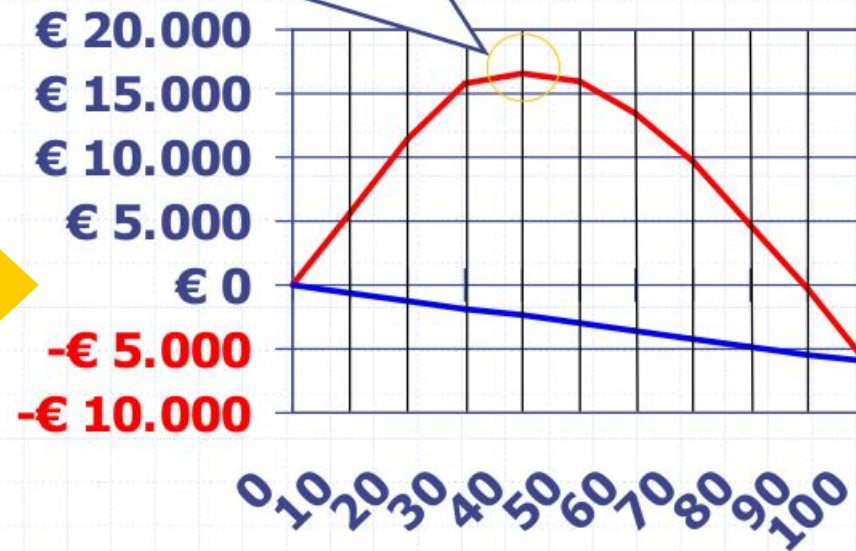
Lift Chart – Application Example

- From Lift chart we can easily derive an “economical value” plot, e.g. in target marketing.
- Given our predictive model, how many customers should we target to maximize income?
- $\text{Profit} = \text{UnitB} * \text{MaxR} * \text{Lift}(X) - \text{UnitCost} * N * X / 100$
- UnitB = unit benefit, UnitCost = unit postal cost
- N = total customers
- MaxR = expected potential respondents in all population (N)
- $\text{Lift}(X)$ = lift chart value for X, in $[0, \dots, 1]$

Lift Chart – Application Example

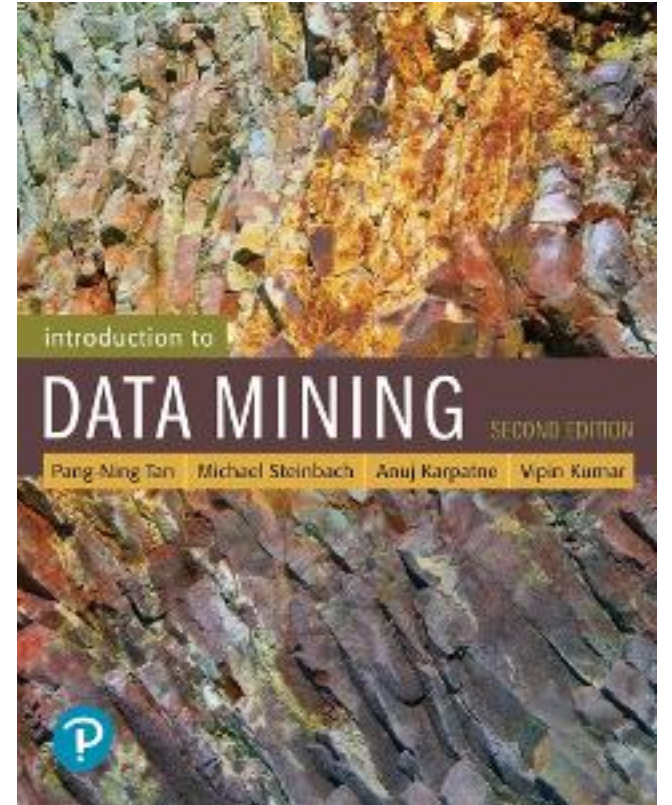


UnitB = 6€ N=30000
MaxR = 10500 UnitCost = 2.30€



References

- Chapter 3. Classification: Basic Concepts and Techniques.



DATA MINING 1

Decision Tree Classifiers

Dino Pedreschi, Riccardo Guidotti

Revisited slides from Lecture Notes for Chapter 3 “Introduction to Data Mining”, 2nd Edition by Tan, Steinbach, Karpatne, Kumar



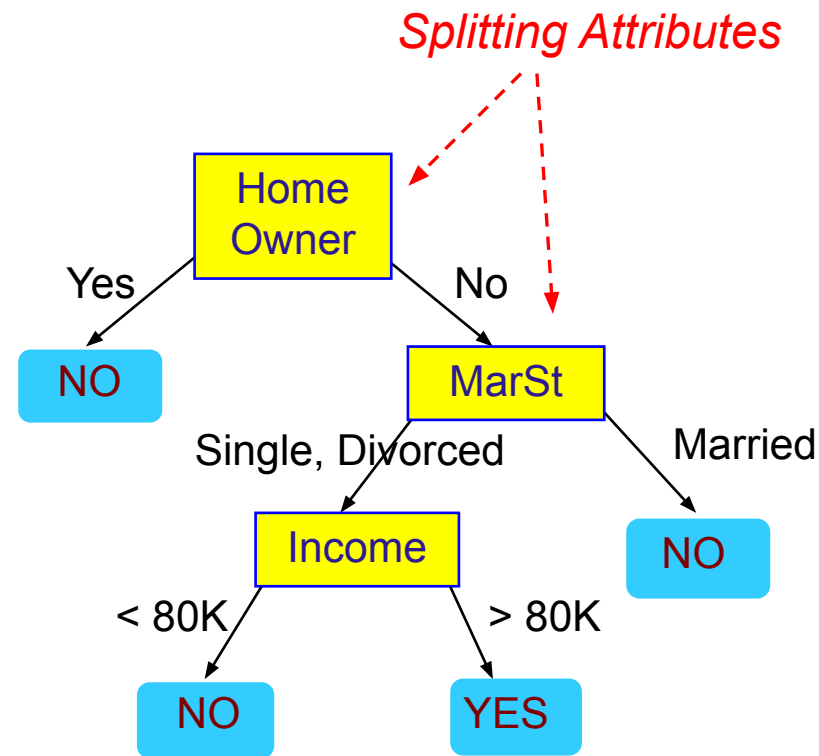
UNIVERSITÀ DI PISA

Example of a Decision Tree

Consider the problem of predicting whether a loan borrower will repay the loan or default on the loan payments.

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

categorical
categorical
continuous
class



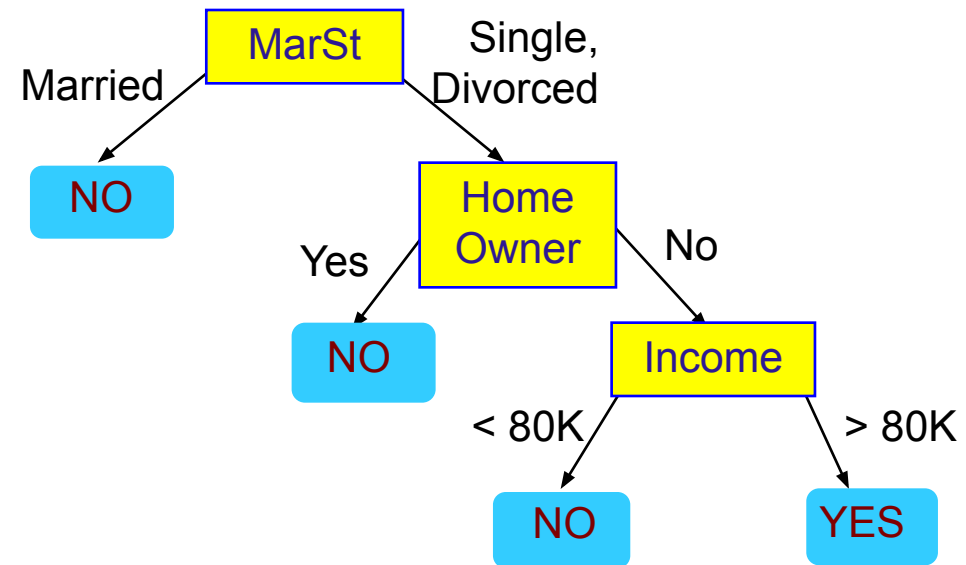
Training Data

Model: Decision Tree

Another Example of Decision Tree

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

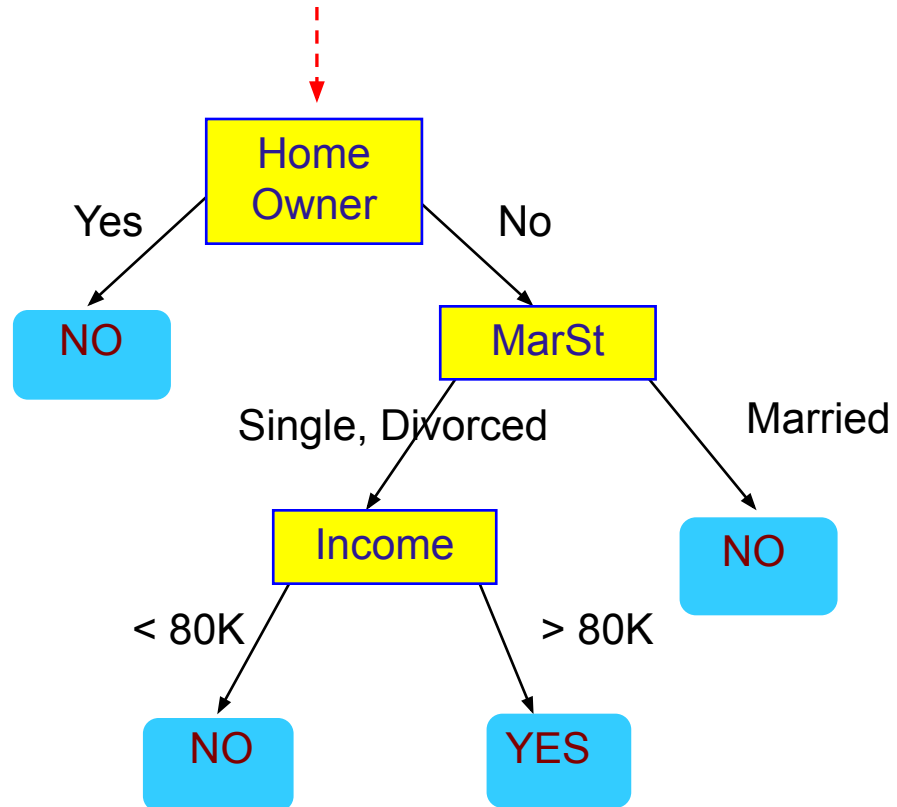
categorical
categorical
continuous
class



There could be more than one tree that fits the same data!

Apply Model to Test Data

Start from the root of tree.



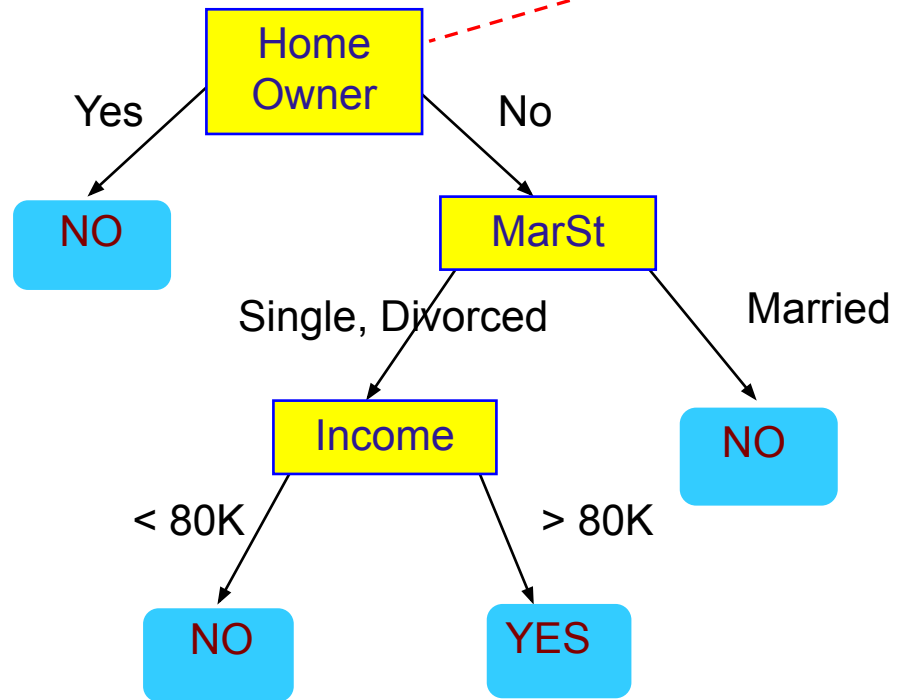
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

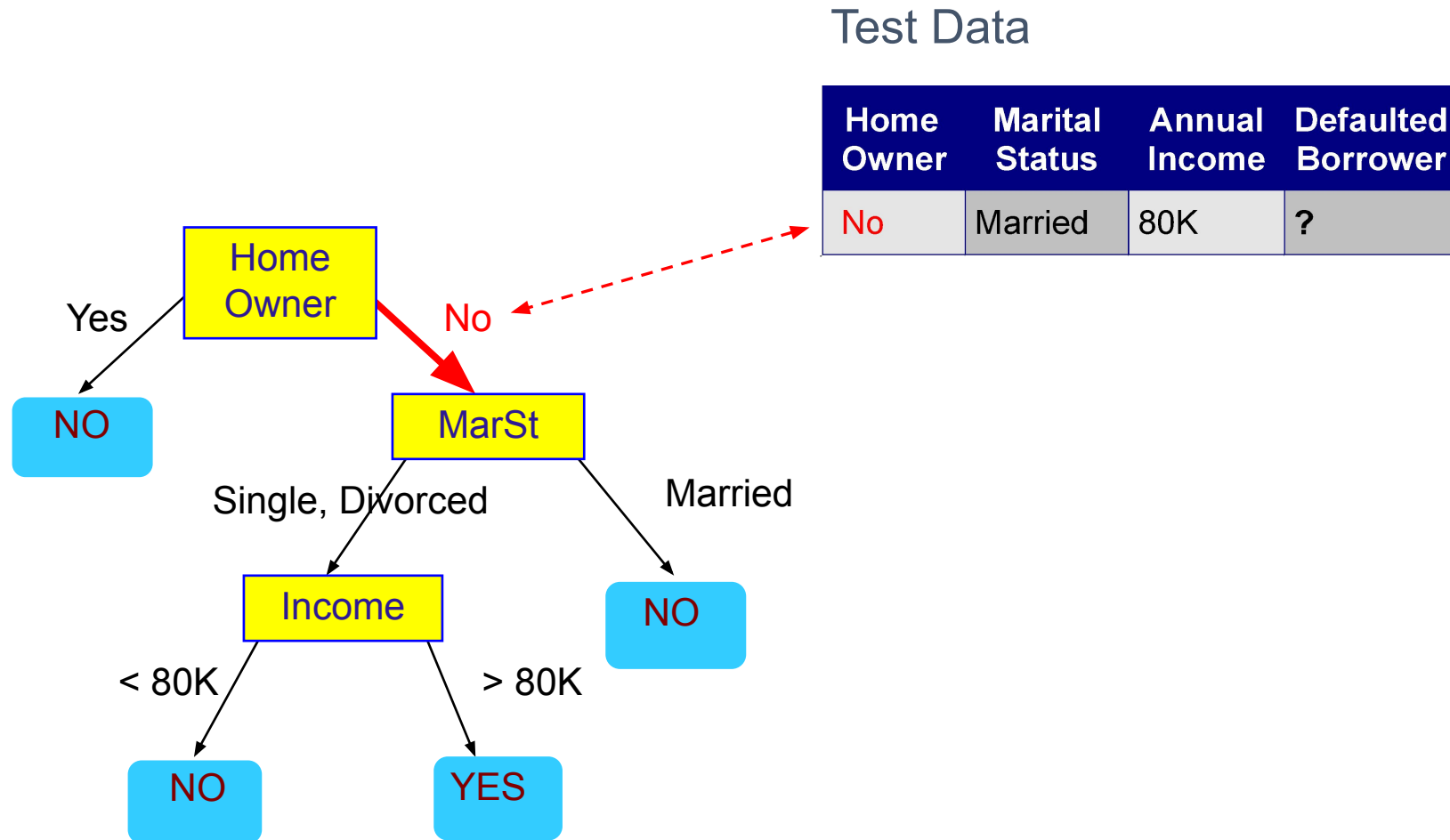
Apply Model to Test Data

Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



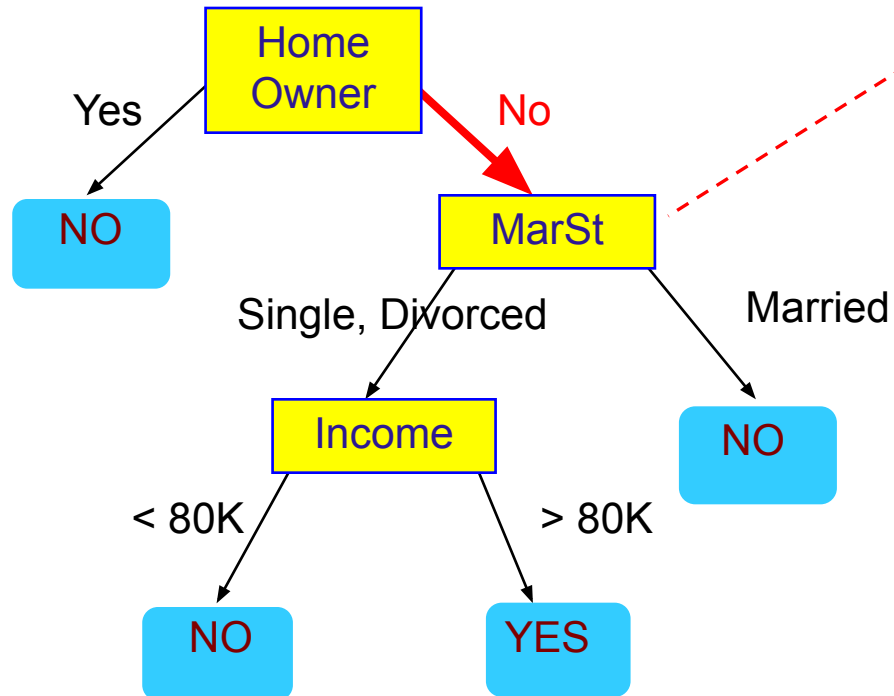
Apply Model to Test Data



Apply Model to Test Data

Test Data

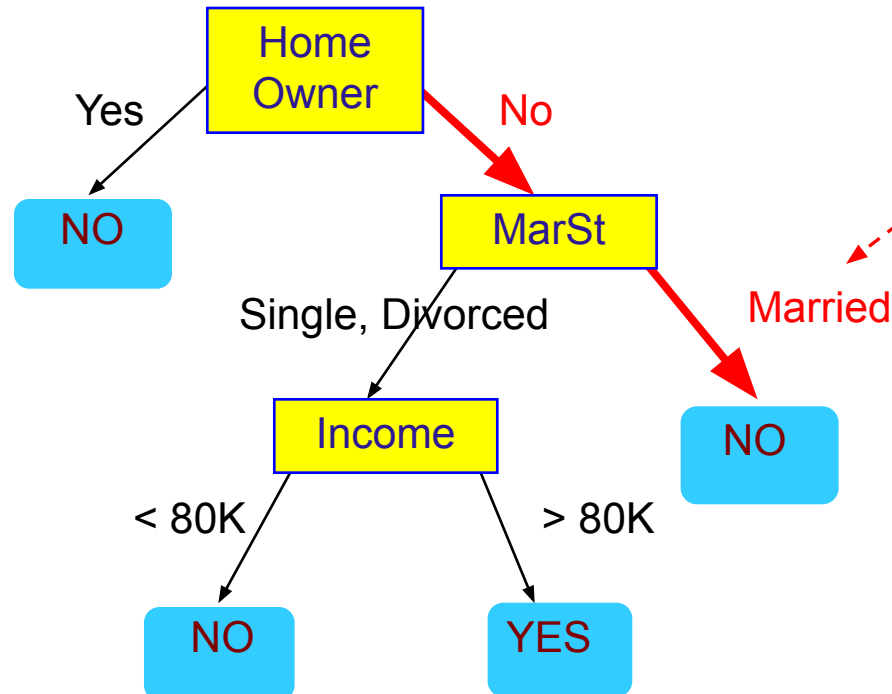
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Apply Model to Test Data

Test Data

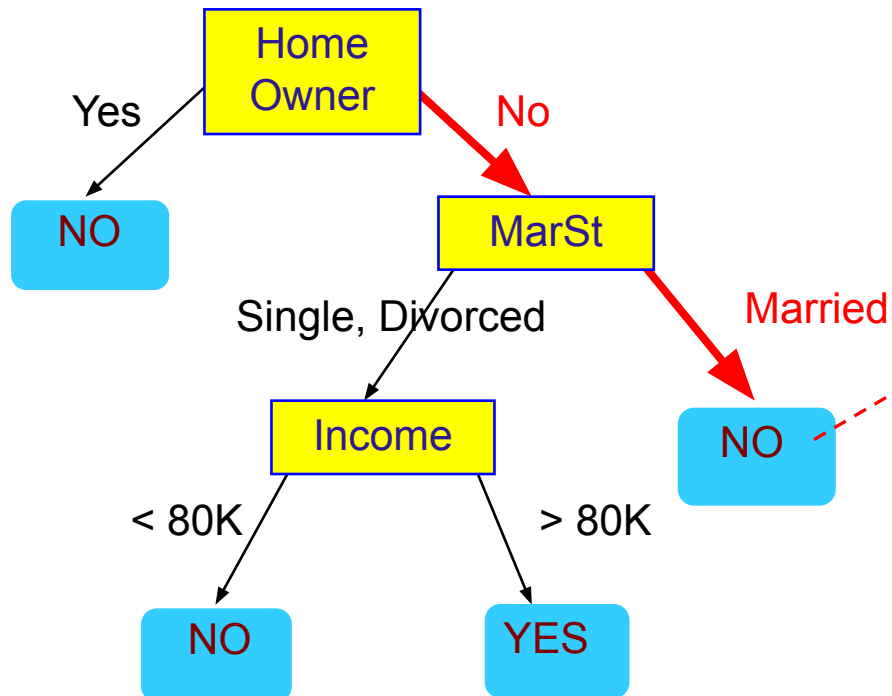
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Apply Model to Test Data

Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Assign Defaulted to
"No"

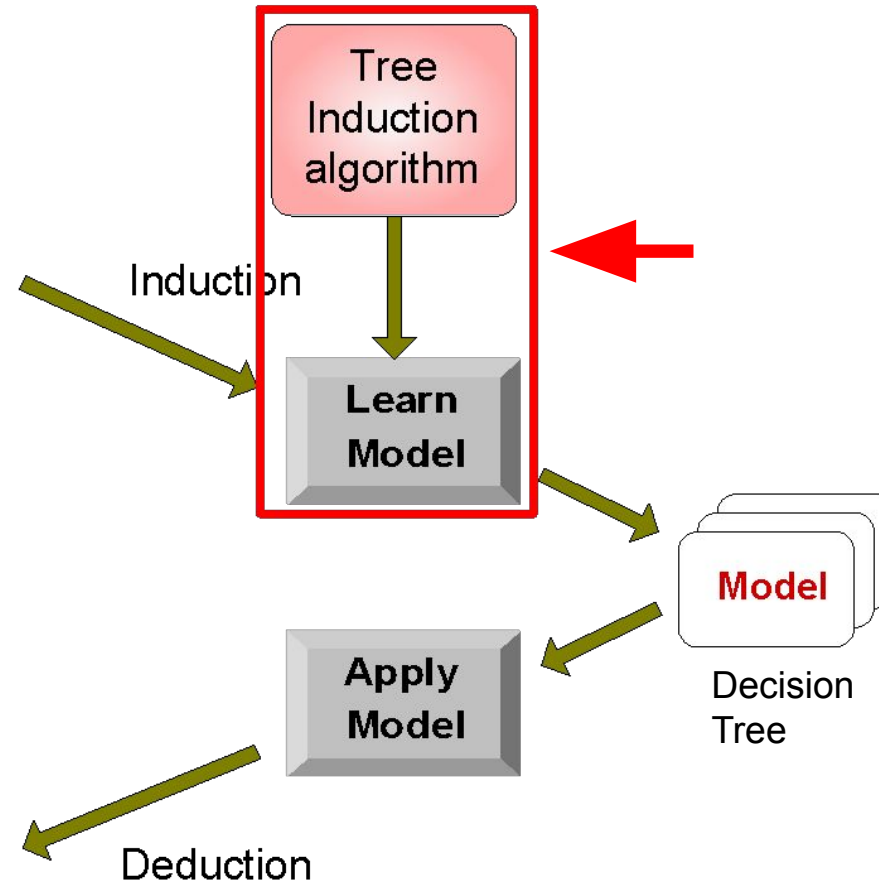
Decision Tree Classification Task

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Test Set



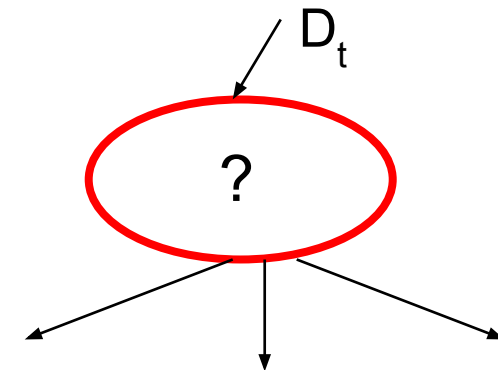
Decision Tree Induction

- Many Algorithms:
 - Hunt's Algorithm (one of the earliest)
 - CART
 - ID3, C4.5
 - SLIQ,SPRINT

General Structure of Hunt's Algorithm

- Let D_t be the set of training records that reach a node t
- General Procedure:
 - If D_t contains records that belong the **same class** y_t , then t is a **leaf node** labeled as y_t
 - If D_t contains records that belong to **more than one class**, use an attribute test to **split** the data into smaller subsets. Recursively apply the procedure to each subset.

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



Hunt's Algorithm

Defaulted = No

(7,3)

(a)

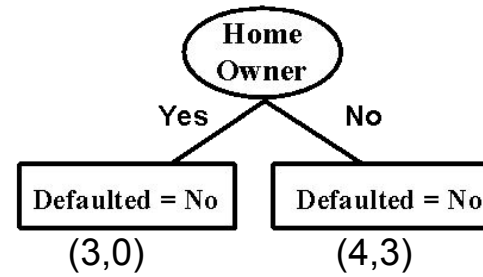
ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Hunt's Algorithm

Defaulted = No

(7,3)

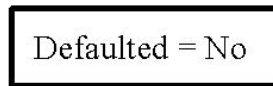
(a)



(b)

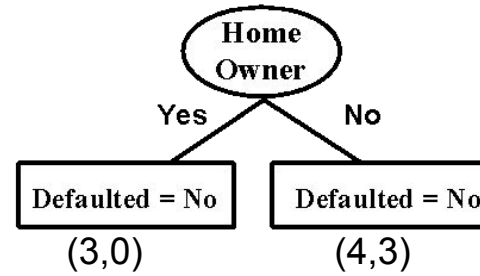
ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Hunt's Algorithm

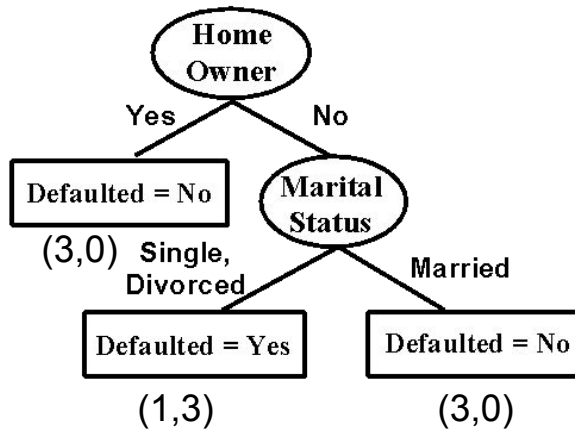


(7,3)

(a)



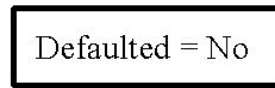
(b)



(c)

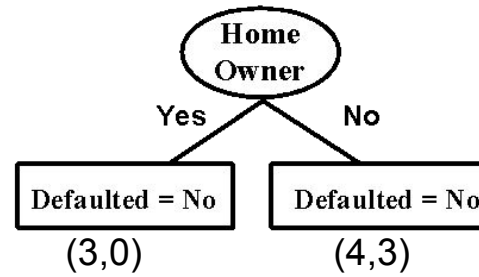
ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Hunt's Algorithm

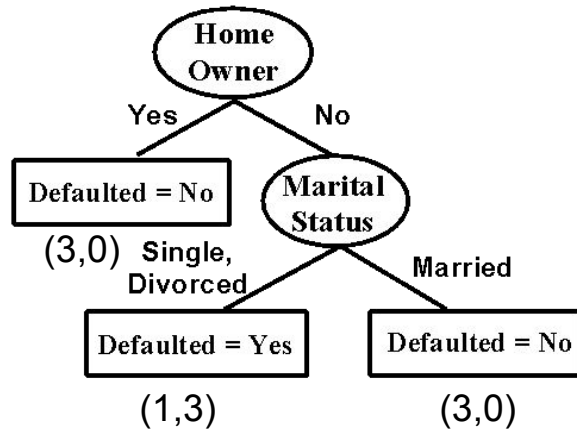


(7,3)

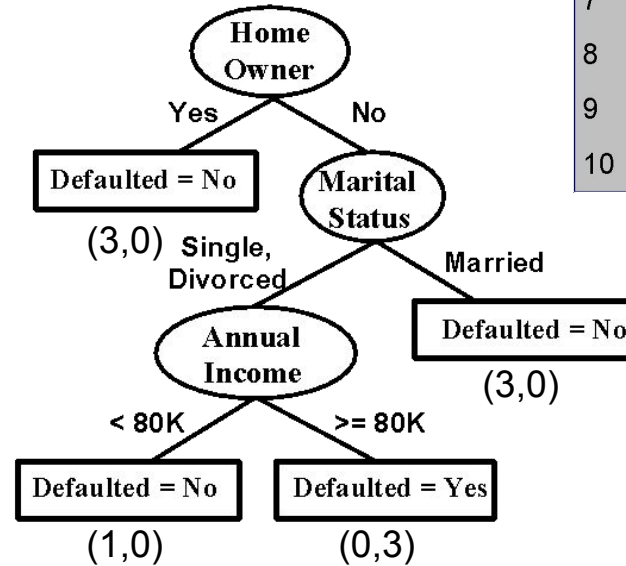
(a)



(b)



(c)



(d)

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Design Issues of Decision Tree Induction

- **Greedy strategy:**
 - the number of possible decision trees can be very large, many decision tree algorithms employ a heuristic-based approach to guide their search in the vast hypothesis space.
- **Split the records based on an attribute test that optimizes certain criterion.**

Tree Induction

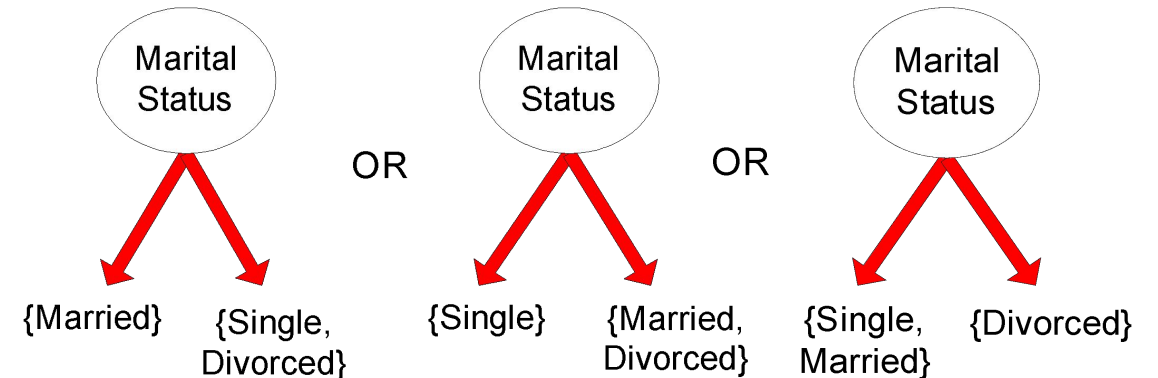
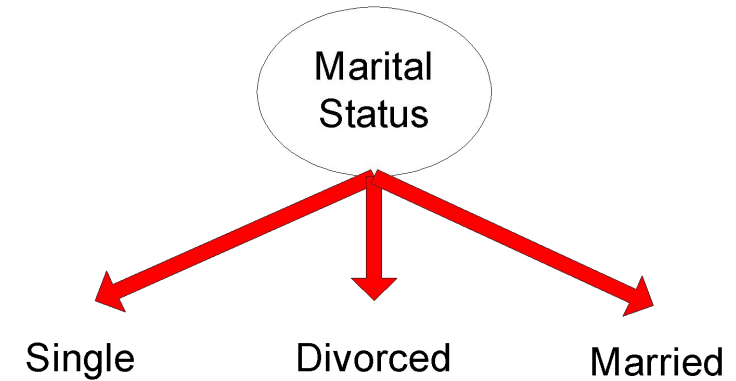
- How should training records be split?
 - Method for specifying test condition depending on attribute types
 - Measure for evaluating the goodness of a test condition
- How should the splitting procedure stop?
 - Stop splitting if all the records belong to the same class or have identical attribute values
 - Early termination

Methods for Expressing Test Conditions

- Depends on attribute types
 - Binary
 - Nominal
 - Ordinal
 - Continuous
- Depends on number of ways to split
 - 2-way split
 - Multi-way split

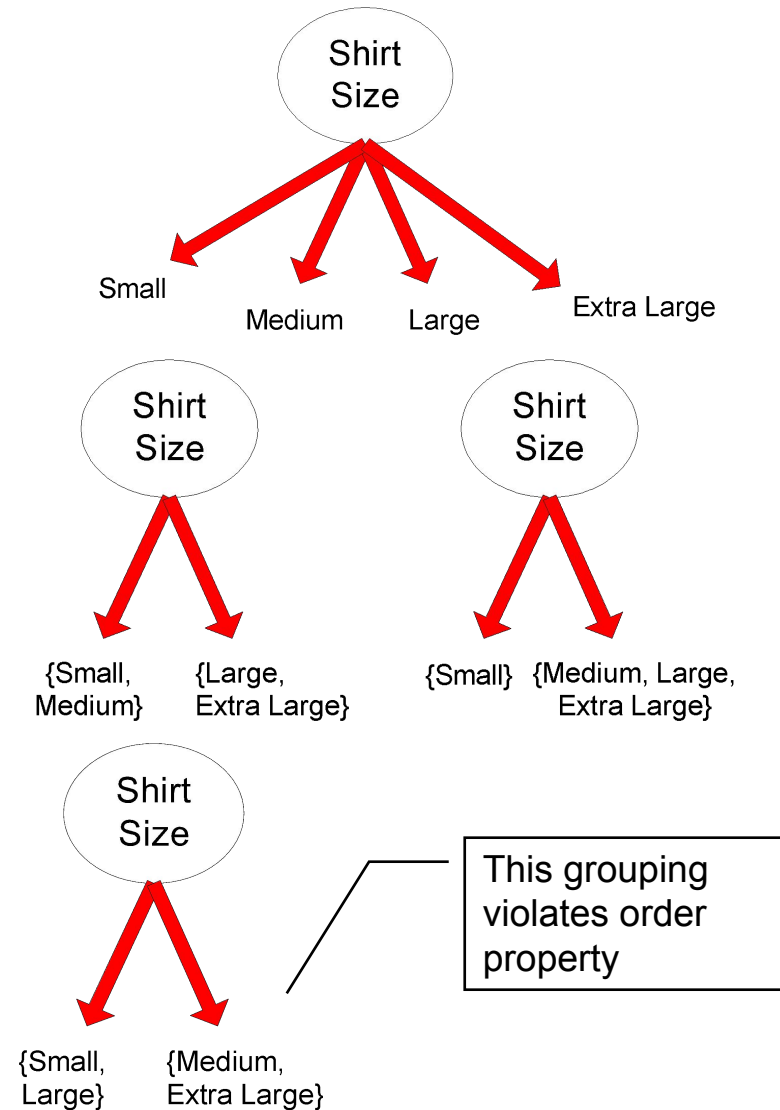
Test Condition for Nominal Attributes

- **Multi-way split:**
 - Use as many partitions as distinct values.
- **Binary split:**
 - Divides values into two subsets

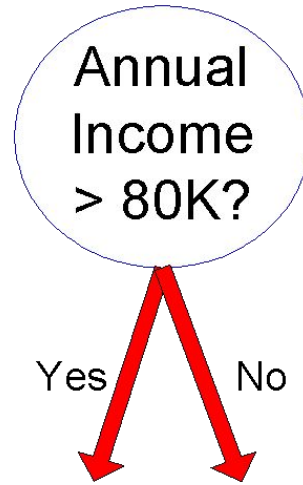


Test Condition for Ordinal Attributes

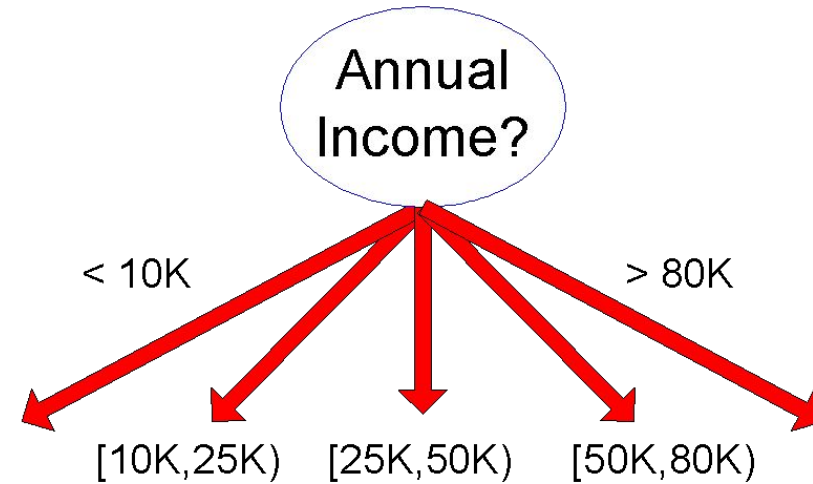
- **Multi-way split:**
 - Use as many partitions as distinct values
- **Binary split:**
 - Divides values into two subsets
 - Preserve order property among attribute values



Test Condition for Continuous Attributes



(i) Binary split



(ii) Multi-way split

Splitting Based on Continuous Attributes

- Different ways of handling
 - **Discretization** to form an ordinal categorical attribute

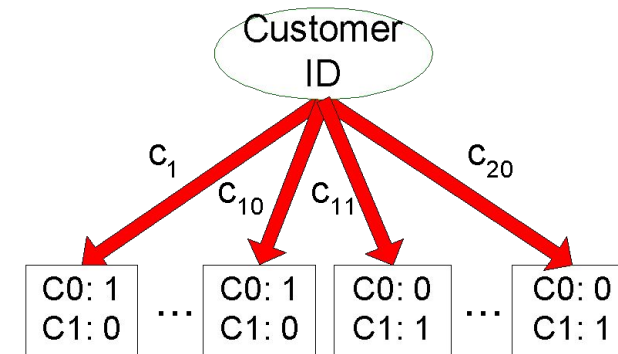
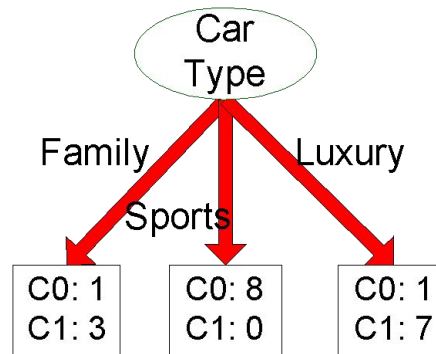
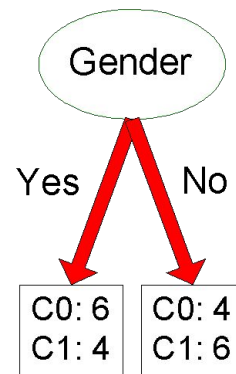
Ranges can be found by equal interval bucketing, equal frequency bucketing (percentiles), or clustering.

 - Static – discretize once at the beginning
 - Dynamic – repeat at each node
 - **Binary Decision**: $(A < v)$ or $(A \geq v)$
 - consider all possible splits and finds the best cut
 - can be more compute intensive

How to determine the Best Split

Before Splitting: 10 records of class 0,
10 records of class 1

Customer Id	Gender	Car Type	Shirt Size	Class
1	M	Family	Small	C0
2	M	Sports	Medium	C0
3	M	Sports	Medium	C0
4	M	Sports	Large	C0
5	M	Sports	Extra Large	C0
6	M	Sports	Extra Large	C0
7	F	Sports	Small	C0
8	F	Sports	Small	C0
9	F	Sports	Medium	C0
10	F	Luxury	Large	C0
11	M	Family	Large	C1
12	M	Family	Extra Large	C1
13	M	Family	Medium	C1
14	M	Luxury	Extra Large	C1
15	F	Luxury	Small	C1
16	F	Luxury	Small	C1
17	F	Luxury	Medium	C1
18	F	Luxury	Medium	C1
19	F	Luxury	Medium	C1
20	F	Luxury	Large	C1



Which test condition is the best?

How to determine the Best Split

- Greedy approach:
 - Nodes with **purser / homogeneous** class distribution are preferred
- Need a measure of node impurity:

C0: 5
C1: 5

High degree of impurity,
Non-homogeneous

C0: 9
C1: 1

Low degree of impurity,
Homogeneous

Measures of Node Impurity

- Gini Index

$$GINI(t) = 1 - \sum_j [p(j | t)]^2$$

- Entropy

$$Entropy(t) = -\sum_j p(j | t) \log p(j | t)$$

- Misclassification Error

$$Error(t) = 1 - \max_i P(i | t)$$

Finding the Best Split

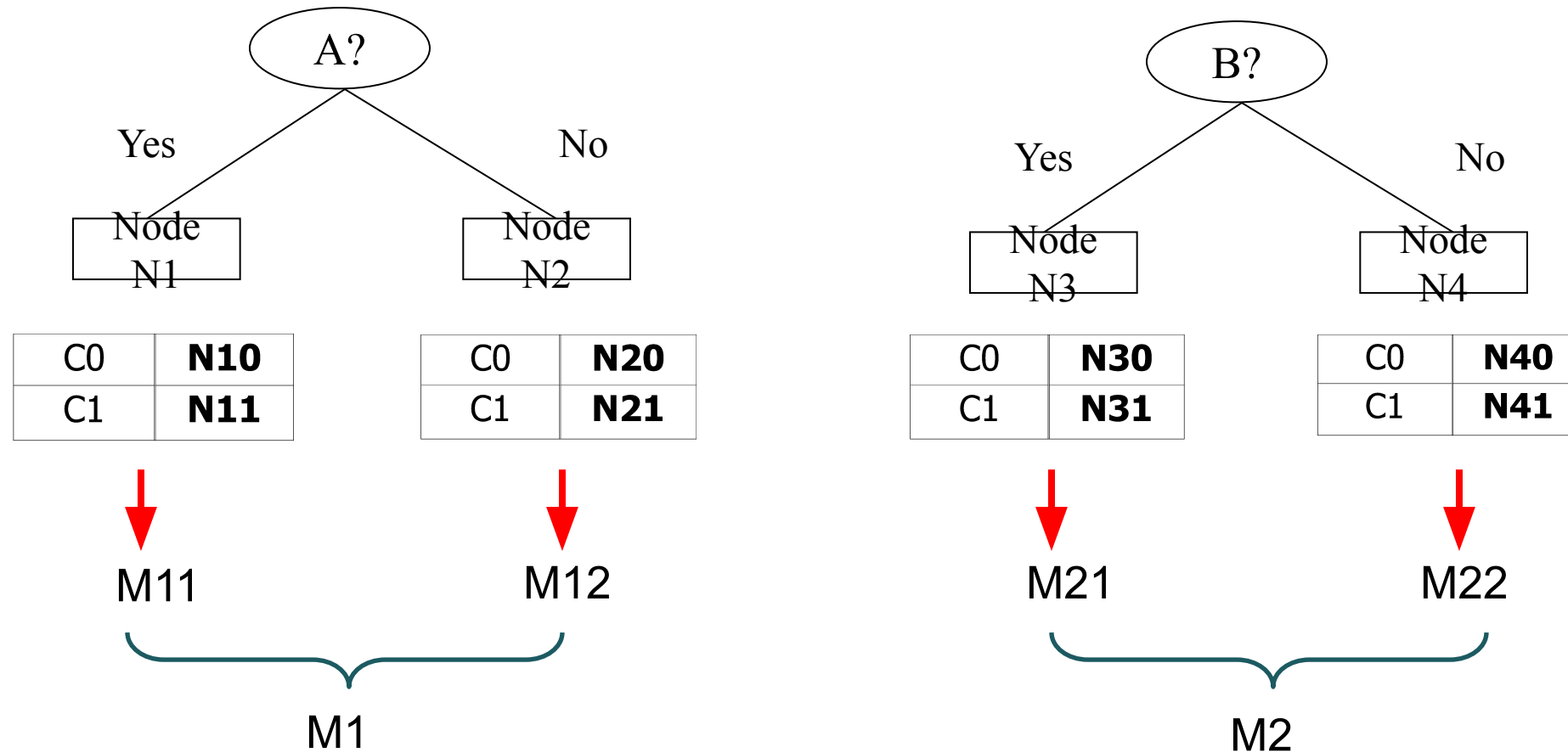
1. Compute impurity measure (P) before splitting
2. Compute impurity measure (M) after splitting
 - Compute impurity measure of each child node
 - M is the weighted impurity of children
3. Choose the attribute test condition that produces the highest gain ($\text{Gain} = P - M$) or equivalently, lowest impurity measure after splitting (M)

Finding the Best Split

Before Splitting:

C0	N00
C1	N01

→ P



$$\text{Gain} = P - M1 \quad \text{vs} \quad P - M2$$

Measure of Impurity: GINI

- Gini Index for a given node t :

$$GINI(t) = 1 - \sum_j [p(j | t)]^2$$

(NOTE: $p(j | t)$ is the relative frequency of class j at node t).

- Maximum $(1 - 1/n_c)$ when records are **equally distributed** among all classes, implying **least interesting information**
- Minimum (0.0) when all records belong to one class, implying **most interesting information**

Measure of Impurity: GINI

- Gini Index for a given node t :

$$GINI(t) = 1 - \sum_j [p(j | t)]^2$$

(NOTE: $p(j | t)$ is the relative frequency of class j at node t).

- For 2-class problem ($p, 1 - p$):
 - $GINI = 1 - p^2 - (1 - p)^2 = 2p(1-p)$

C1	0
C2	6
Gini=0.000	

C1	1
C2	5
Gini=0.278	

C1	2
C2	4
Gini=0.444	

C1	3
C2	3
Gini=0.500	

Computing Gini Index of a Single Node

$$GINI(t) = 1 - \sum_j [p(j | t)]^2$$

C1	0
C2	6

$$P(C1) = 0/6 = 0 \quad P(C2) = 6/6 = 1$$

$$Gini = 1 - P(C1)^2 - P(C2)^2 = 1 - 0 - 1 = 0$$

C1	1
C2	5

$$P(C1) = 1/6 \quad P(C2) = 5/6$$

$$Gini = 1 - (1/6)^2 - (5/6)^2 = 0.278$$

C1	2
C2	4

$$P(C1) = 2/6 \quad P(C2) = 4/6$$

$$Gini = 1 - (2/6)^2 - (4/6)^2 = 0.444$$

Gini Index for a Collection of Nodes

- When a node p is split into k partitions (children)

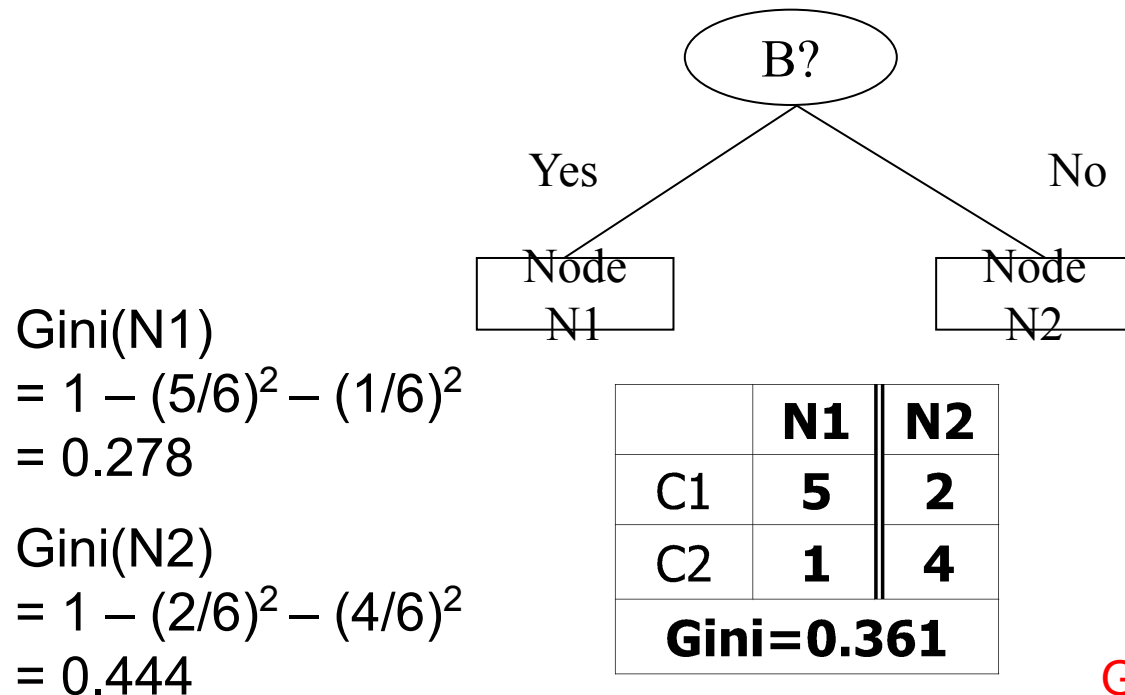
$$GINI_{split} = \sum_{i=1}^k \frac{n_i}{n} GINI(i)$$

where, n_i = number of records at child i ,
 n = number of records at parent node p .

- Choose the attribute that minimizes weighted average Gini index of the children
- Gini index is used in decision tree algorithms such as CART, SLIQ, SPRINT

Binary Attributes: Computing GINI Index

- Splits into two partitions
- Effect of Weighing partitions:
 - Larger and Purer Partitions are sought for.



	Parent
C1	7
C2	5
Gini = 0.486	

$$\text{Gain} = 0.486 - 0.361 = 0.125$$

Categorical Attributes: Computing Gini Index

- For each distinct value, gather counts for each class in the dataset
- Use the count matrix to make decisions

Multi-way split

	CarType		
	Family	Sports	Luxury
C1	1	8	1
C2	3	0	7
Gini	0.163		

Two-way split
(find best partition of values)

	CarType	
	{Sports, Luxury}	{Family}
C1	9	1
C2	7	3
Gini	0.468	

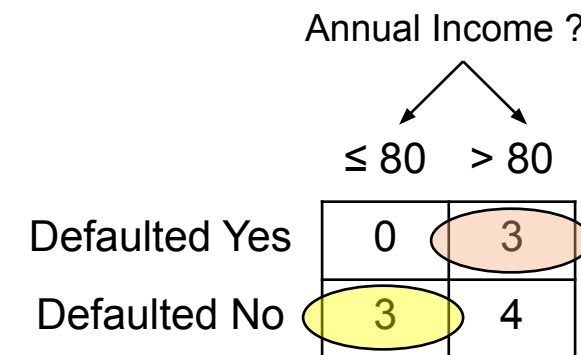
	CarType	
	{Sports}	{Family, Luxury}
C1	8	2
C2	0	10
Gini	0.167	

Which of these is the best?

Continuous Attributes: Computing Gini Index

- Use Binary Decisions based on one value
- Several Choices for the splitting value
 - Number of possible splitting values = Number of distinct values
- Each splitting value has a count matrix associated with it
 - Class counts in each of the partitions, $A < v$ and $A \geq v$
- Simple method to choose best v
 - For each v , scan the database to gather count matrix and compute its Gini index
 - Computationally Inefficient! ($O(N^2)$) Repetition of work.

ID	Home Owner	Marital Status	Annual Income	Defaulted
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



Continuous Attributes: Computing Gini Index...

- For efficient computation $O(N \log N)$: for each attribute,
 - Sort the attribute on values
 - Linearly scan these values, each time updating the count matrix and computing gini index
 - Choose the split position that has the least Gini index

Sorted Values →	Cheat	No	No	No	Yes	Yes	Yes	No	No	No	No
	Annual Income										
		60	70	75	85	90	95	100	120	125	220

Continuous Attributes: Computing Gini Index...

- For efficient computation: for each attribute,
 - Sort the attribute on values
 - Linearly scan these values, each time updating the count matrix and computing gini index
 - Choose the split position that has the least Gini index

Sorted Values Split Positions	Cheat	No		No		No		Yes		Yes		Yes		No		No		No		No		
	Annual Income																					
	60		70		75		85		90		95		100		120		125		220			
	55		65		72		80		87		92		97		110		122		172		230	
	<= >		<= >		<= >		<= >		<= >		<= >		<= >		<= >		<= >		<= >		<= >	

Continuous Attributes: Computing Gini Index...

- For efficient computation: for each attribute,
 - Sort the attribute on values
 - Linearly scan these values, each time updating the count matrix and computing gini index
 - Choose the split position that has the least Gini index

		Cheat																					
		No		No		No		Yes		Yes		Yes		No		No		No		No			
		Annual Income																					
Sorted Values	→	60		70		75		85		90		95		100		120		125		220			
Split Positions	→	55		65		72		80		87		92		97		110		122		172		230	
		<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>
Yes								0	3														
No								3	4														
Gini								0.343															

Continuous Attributes: Computing Gini Index...

- For efficient computation: for each attribute,
 - Sort the attribute on values
 - Linearly scan these values, each time updating the count matrix and computing gini index
 - Choose the split position that has the least Gini index

Choose the split position that has the least Gini index

Cheat	No	No	No	Yes	Yes	Yes	No	No	No	No													
		Annual Income																					
Sorted Values		60		70		75		85		90		95		100		120		125		220			
Split Positions		55		65		72		80		87		92		97		110		122		172		230	
		<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>
Yes		0	3	0	3	0	3	0	3	1	2	2	1	3	0	3	0	3	0	3	0	3	0
No		0	7	1	6	2	5	3	4	3	4	3	4	3	4	4	3	5	2	6	1	7	0
Gini		0.420		0.400		0.375		0.343		0.417		0.400		<u>0.300</u>		0.343		0.375		0.400		0.420	

Continuous Attributes: Computing Gini Index...

- For efficient computation: for each attribute,
 - Sort the attribute on values
 - Linearly scan these values, each time updating the count matrix and computing gini index
 - Choose the split position that has the least Gini index

Cheat		No		No		No		Yes		Yes		Yes		No		No		No		No			
Sorted Values Split Positions	→	Annual Income																					
		60		70		75		85		90		95		100		120		125		220			
	→	55		65		72		80		87		92		97		110		122		172		230	
		<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>
	Yes	0	3	0	3	0	3	0	3	1	2	2	1	3	0	3	0	3	0	3	0	3	0
	No	0	7	1	6	2	5	3	4	3	4	3	4	3	4	4	3	5	2	6	1	7	0
	Gini	0.420		0.400		0.375		0.343		0.417		0.400		<u>0.300</u>		0.343		0.375		0.400		0.420	

Measure of Impurity: Entropy

- Entropy at a given node t :

$$Entropy(t) = -\sum_j p(j | t) \log p(j | t)$$

(NOTE: $p(j | t)$ is the relative frequency of class j at node t).

- Maximum ($\log n_c$) when records are equally distributed among all classes implying least information
- Minimum (0.0) when all records belong to one class, implying most information
- Entropy based computations are quite similar to the GINI index computations

Computing Entropy of a Single Node

$$Entropy(t) = -\sum_j p(j | t) \log_2 p(j | t)$$

C1	0
C2	6

$$P(C1) = 0/6 = 0 \quad P(C2) = 6/6 = 1$$

$$Entropy = -0 \log 0 - 1 \log 1 = -0 - 0 = 0$$

C1	1
C2	5

$$P(C1) = 1/6 \quad P(C2) = 5/6$$

$$Entropy = - (1/6) \log_2 (1/6) - (5/6) \log_2 (5/6) = 0.65$$

C1	2
C2	4

$$P(C1) = 2/6 \quad P(C2) = 4/6$$

$$Entropy = - (2/6) \log_2 (2/6) - (4/6) \log_2 (4/6) = 0.92$$

Computing Information Gain After Splitting

- Information Gain:

$$GAIN_{split} = Entropy(p) - \left(\sum_{i=1}^k \frac{n_i}{n} Entropy(i) \right)$$

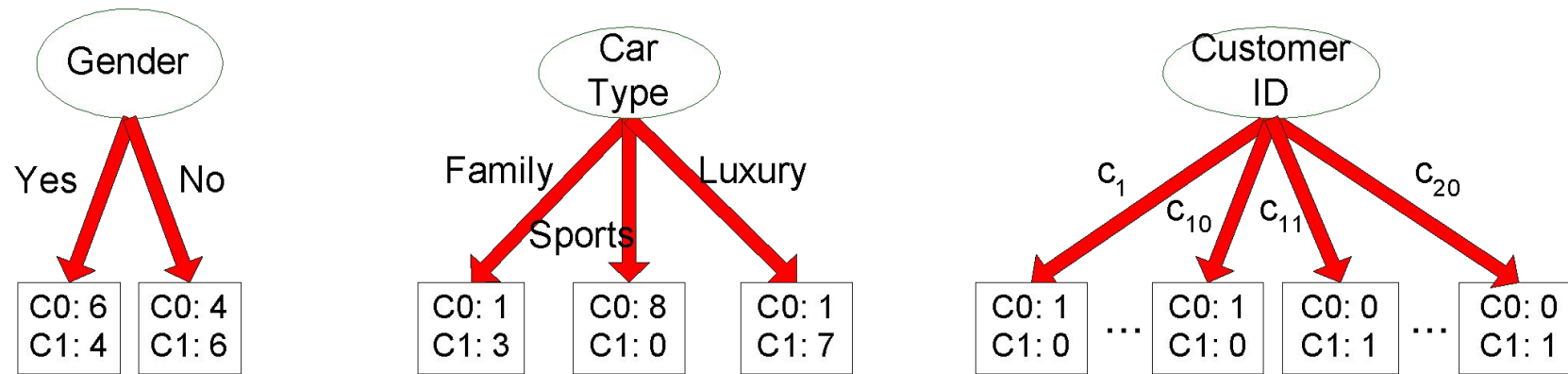
Parent Node, p is split into k partitions;

n_i is number of records in partition i

- Measures **Reduction in Entropy** achieved because of the split. Choose the split that achieves most reduction (maximizes GAIN)
- Used in the ID3 and C4.5 decision tree algorithms
- Disadvantage:** Tends to prefer splits that result in large number of partitions, each being small but pure.

Problem with large number of partitions

- Node impurity measures tend to prefer splits that result in large number of partitions, each being small but pure



- Customer ID has **highest information gain** because entropy for all the children is zero
- Can we use such a test condition on new test instances?**

Solution

- A **low impurity value alone is insufficient** to find a good attribute test condition for a node
- **Solution:** Consider the **number of children** produced by the splitting attribute in the identification of the best split
- High number of child nodes implies more complexity
- **Method 1:** Generate only binary decision trees
 - This strategy is employed by decision tree classifiers such as CART
- **Method 2:** Modify the splitting criterion to take into account the number of partitions produced by the attribute

Gain Ratio

- Gain Ratio:

$$GainRATIO_{split} = \frac{GAIN_{Split}}{SplitINFO}$$

$$SplitINFO = -\sum_{i=1}^k \frac{n_i}{n} \log \frac{n_i}{n}$$

Parent Node, p is split into k partitions

n_i is the number of records in partition i

- Adjusts Information Gain by the entropy of the partitioning (SplitINFO).
 - **Higher entropy partitioning (large number of small partitions) is penalized!**
- Used in C4.5 algorithm
- Designed to overcome the disadvantage of Information Gain

Gain Ratio

- Gain Ratio:

$$GainRatio_{split} = \frac{GAIN_{split}}{SplitINFO} \quad SplitINFO = -\sum_{i=1}^k \frac{n_i}{n} \log \frac{n_i}{n}$$

Parent Node, p is split into k partitions

n_i is the number of records in partition i

	CarType		
	Family	Sports	Luxury
C1	1	8	1
C2	3	0	7
Gini	0.163		

SplitINFO = 1.52

	CarType	
	{Sports, Luxury}	{Family}
C1	9	1
C2	7	3
Gini	0.468	

SplitINFO = 0.72

	CarType	
	{Sports}	{Family, Luxury}
C1	8	2
C2	0	10
Gini	0.167	

SplitINFO = 0.97

Measure of Impurity: Classification Error

- Classification error at a node t :

$$Error(t) = 1 - \max_i P(i | t)$$

- Maximum $(1 - 1/n_c)$ when records are equally distributed among all classes, implying least interesting information
- Minimum (0) when all records belong to one class, implying most interesting information

Computing Error of a Single Node

$$Error(t) = 1 - \max_i P(i | t)$$

C1	0
C2	6

$$P(C1) = 0/6 = 0 \quad P(C2) = 6/6 = 1$$

$$Error = 1 - \max(0, 1) = 1 - 1 = 0$$

C1	1
C2	5

$$P(C1) = 1/6 \quad P(C2) = 5/6$$

$$Error = 1 - \max(1/6, 5/6) = 1 - 5/6 = 1/6$$

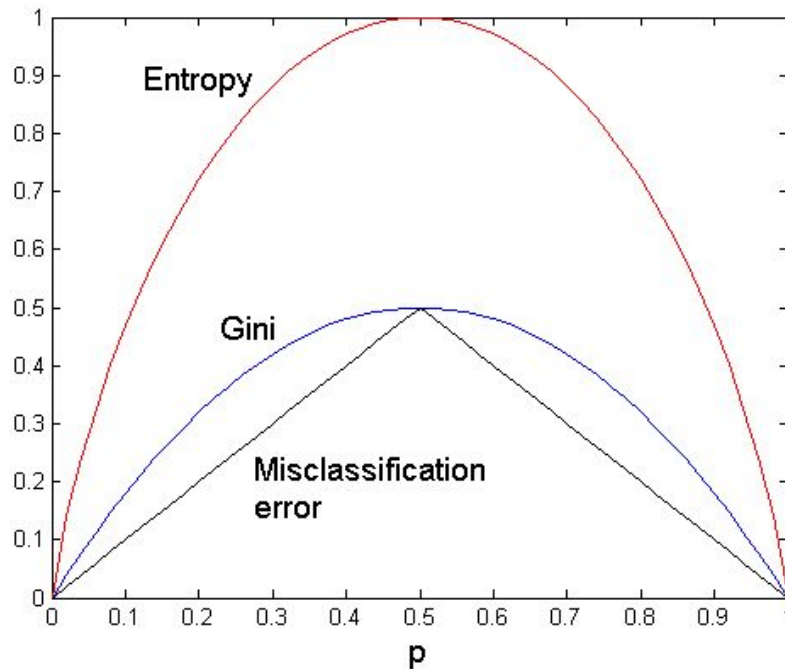
C1	2
C2	4

$$P(C1) = 2/6 \quad P(C2) = 4/6$$

$$Error = 1 - \max(2/6, 4/6) = 1 - 4/6 = 1/3$$

Comparison among Impurity Measures

For a 2-class problem:

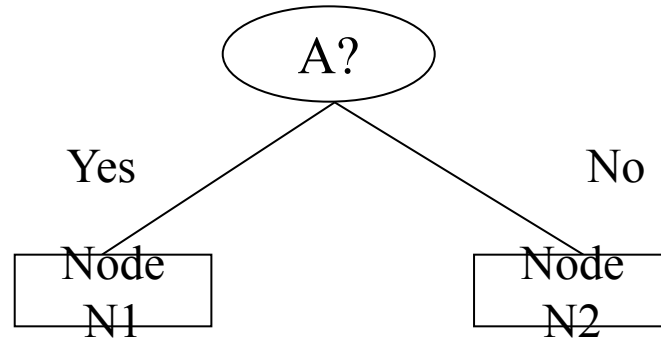


Consistency among the impurity measures

- if a node N1 has lower entropy than node N2, then the Gini index and error rate of N1 will also be lower than that of N2

The attribute chosen as splitting criterion by the impurity measures can still be different!

Misclassification Error vs Gini Index



	Parent
C1	7
C2	3
Gini = 0.42	

$$\begin{aligned}
 &\text{Gini}(N1) \\
 &= 1 - (3/3)^2 - (0/3)^2 \\
 &= 0
 \end{aligned}$$

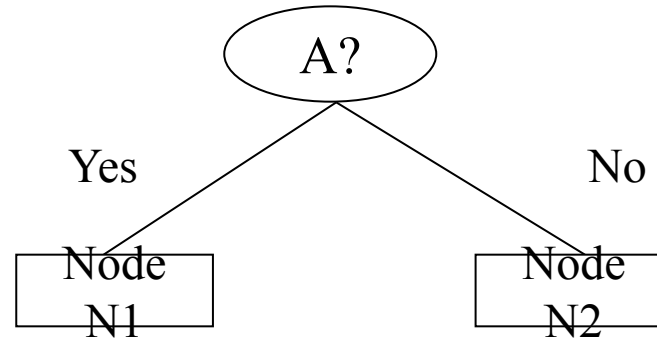
$$\begin{aligned}
 &\text{Gini}(N2) \\
 &= 1 - (4/7)^2 - (3/7)^2 \\
 &= 0.489
 \end{aligned}$$

	N1	N2
C1	3	4
C2	0	3
Gini=0.342		

$$\begin{aligned}
 &\text{Gini(Children)} \\
 &= 3/10 * 0 \\
 &+ 7/10 * 0.489 \\
 &= 0.342
 \end{aligned}$$

Gini improves but
error remains the
same!!

Misclassification Error vs Gini Index



	Parent
C1	7
C2	3
Gini = 0.42	

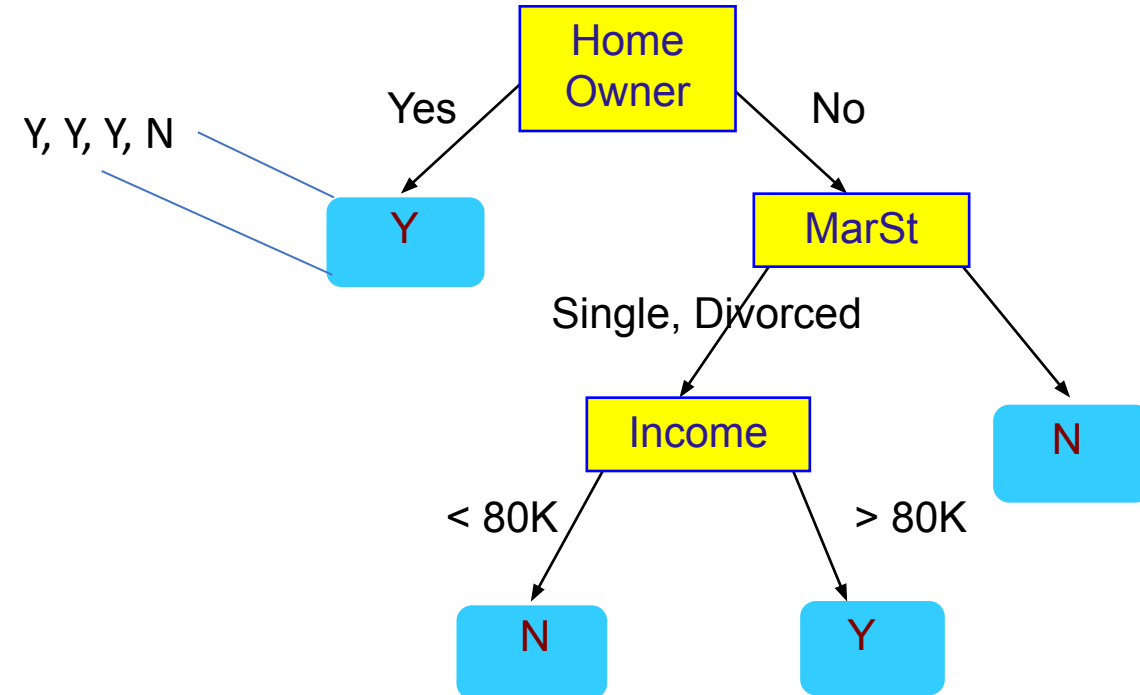
	N1	N2
C1	3	4
C2	0	3
Gini=0.342		

	N1	N2
C1	3	4
C2	1	2
Gini=0.416		

Misclassification error for all three cases = 0.3 !

Stopping Criteria for Tree Induction

- Stop expanding a node when all the records belong to the same class
- Stop expanding a node when all the records have similar attribute values
- Early termination (discussed later)



Algorithms: ID3, C4.5, C5.0, CART

- **ID3** uses the Hunt's algorithm with information gain criterion and gain ratio
- **C4.5** improves **ID3**
 - Needs entire data to fit in memory
 - Handles missing attributes and continuous attributes
 - Performs tree post-pruning
 - **C5.0** is the current commercial successor of **C4.5**
 - Unsuitable for Large Datasets
- **CART** builds multivariate decision (binary) trees

Advantages of Decision Tree

- Easy to interpret for small-sized trees
- Accuracy is comparable to other classification techniques for many simple data sets
- Robust to noise (especially when methods to avoid overfitting are employed)
- Can easily handle redundant or irrelevant attributes
- Inexpensive to construct
- Extremely fast at classifying unknown record
- Handle Missing Values

Irrelevant Attributes

- **Irrelevant** attributes are poorly associated with the target class labels, so they have little or no gain in purity
- In case of a **large number** of irrelevant attributes, some of them may be accidentally chosen during the tree-growing process
- Feature selection techniques can help to eliminate the irrelevant attributes during preprocessing

Redundant Attributes

- Decision trees can handle the presence of redundant attributes
- An attribute is **redundant** if it is strongly **correlated** with another attribute in the data
- Since redundant attributes show **similar gains** in purity if they are selected for splitting, **only one** of them will be selected as an attribute test condition in the decision tree algorithm.

Advantages of Decision Tree

- Easy to interpret for small-sized trees
- Accuracy is comparable to other classification techniques for many simple data sets
- Robust to noise (especially when methods to avoid overfitting are employed)
- Can easily handle redundant or irrelevant attributes
- Inexpensive to construct
- Extremely fast at classifying unknown record
- Handle Missing Values

Computational Complexity

- Finding an optimal decision tree is NP-hard
- Hunt's Algorithm uses a greedy, top-down, recursive partitioning strategy for growing a decision tree
- Such techniques quickly construct a reasonably good decision tree even when the training set size is very large.
- **Construction DT Complexity:** $O(M N \log N)$ where $M=n$. attributes, $N=n$. instances
- Once a decision tree has been built, **classifying** a test record is **extremely fast**, with a worst-case complexity of $O(w)$, where w is the **maximum depth of the tree**.

Handling Missing Attribute Values

- Missing values affect decision tree construction in three different ways:
 - Affects how impurity measures are computed
 - Affects how to distribute instance with missing value to child nodes
 - Affects how a test instance with missing value is classified

Computing Impurity Measure

Tid	Refund	Marital Status	Taxable Income	Class
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	?	Single	90K	Yes

Missing
value

Before Splitting:

Entropy(Parent)

$$= -0.3 \log(0.3) - (0.7) \log(0.7) = 0.8813$$

	Class = Yes	Class = No
Refund=Yes	0	3
Refund=No	2	4
Refund=?	1	0

Split on Refund:

Entropy(Refund=Yes) = 0

Entropy(Refund=No)

$$= -(2/6) \log(2/6) - (4/6) \log(4/6) = 0.9183$$

Entropy(Children)

$$= 0.3 (0) + 0.6 (0.9183) = 0.551$$

$$\text{Gain} = 0.9 \times (0.8813 - 0.551) = 0.3303$$

Distribute Instances

<i>Tid</i>	Refund	Marital Status	Taxable Income	Class
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No

Refund			
Yes			
Class=Yes	0	Cheat=Yes	2
Class=No	3	Cheat=No	4

<i>Tid</i>	Refund	Marital Status	Taxable Income	Class
10	?	Single	90K	Yes

Refund			
Yes			
Class=Yes	0 + 3/9	Class=Yes	2 + 6/9
Class=No	3	Class=No	4

Probability that Refund=Yes is 3/9

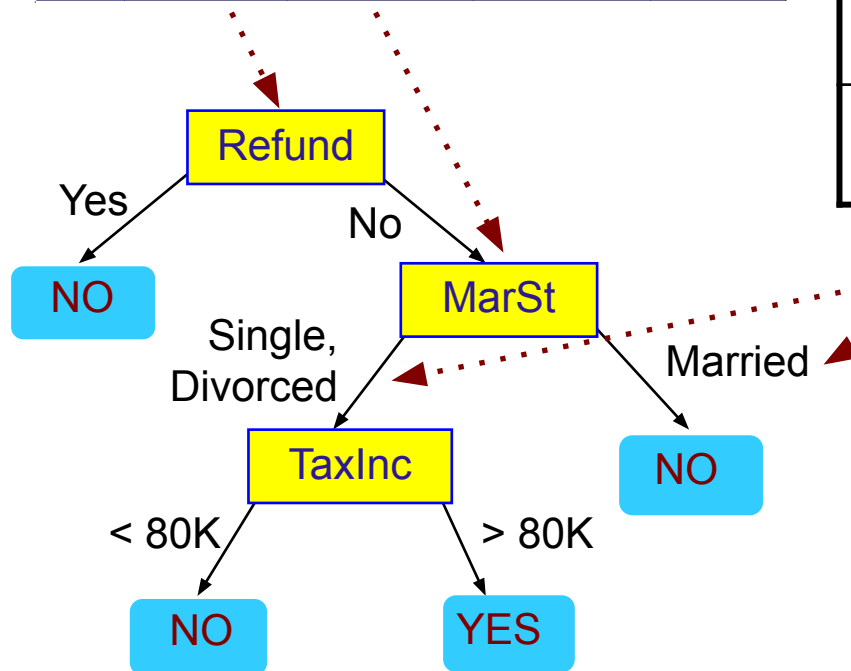
Probability that Refund=No is 6/9

Assign record to the left child with weight = 3/9 and to the right child with weight = 6/9

Classify Instances

New record:

Tid	Refund	Marital Status	Taxable Income	Class
11	No	?	85K	?



	Married	Single	Divorced	Total
Class=No	3	1	0	4
Class=Yes	6/9	1	1	2.67
Total	3.67	2	1	6.67

Probabilistic split method (C4.5)

Probability that Marital Status = Married is $3.67/6.67$

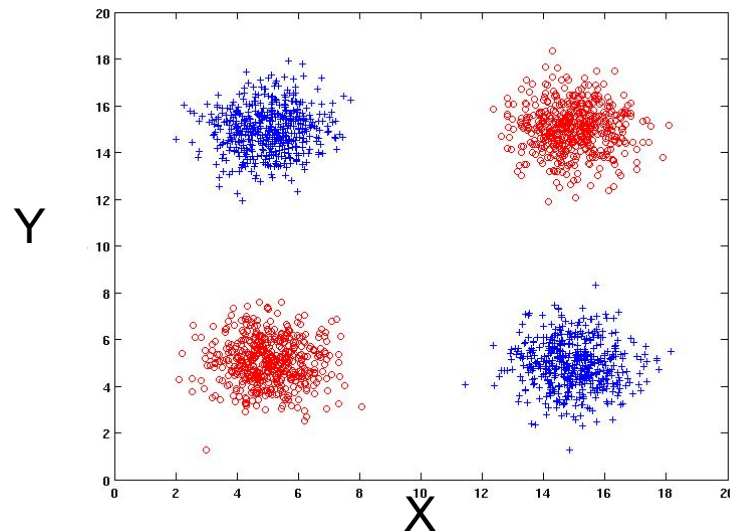
Probability that Marital Status = {Single, Divorced} is $3/6.67$

Disadvantages

- Space of possible decision trees is exponentially large. Greedy approaches are often unable to find the best tree.
- Does not take into account **interactions** between attributes
- Each decision boundary involves **only a single attribute**

Handling interactions

Interacting attributes: able to distinguish between classes when used together, but individually they provide little or no information.



+ : 1000 instances

o : 1000 instances

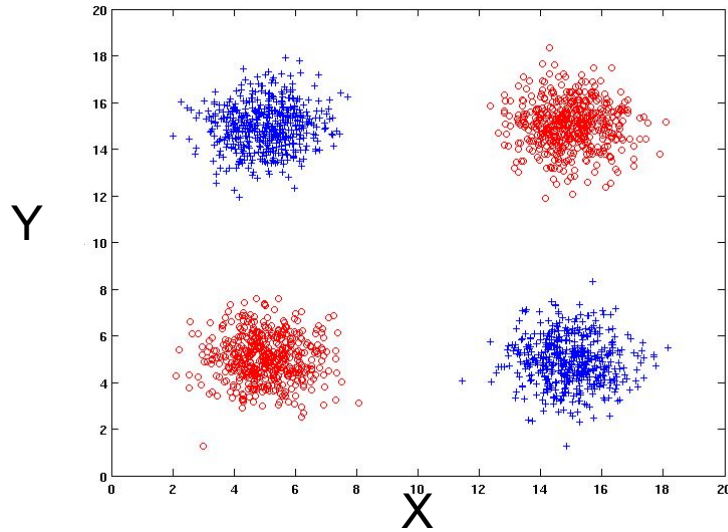
Test Condition:
 $X \leq 10$ and $Y \leq 10$

Entropy (X) : 0.99
Entropy (Y) : 0.99



No reduction in the
impurity measure when
used individually

Handling interactions



+ : 1000 instances

o : 1000 instances

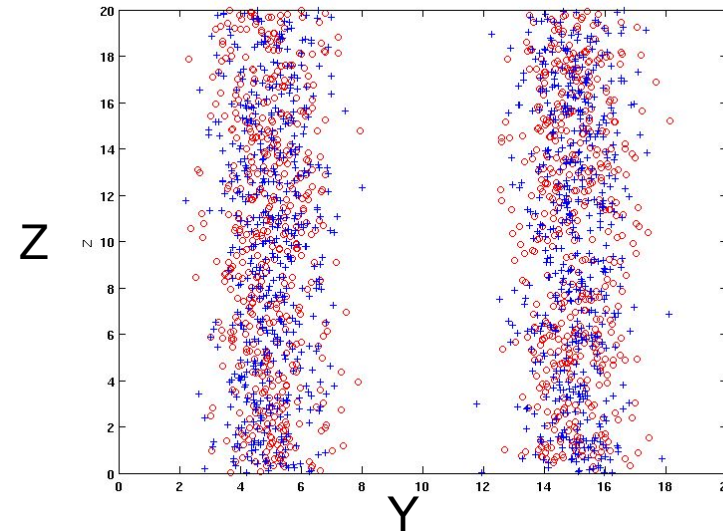
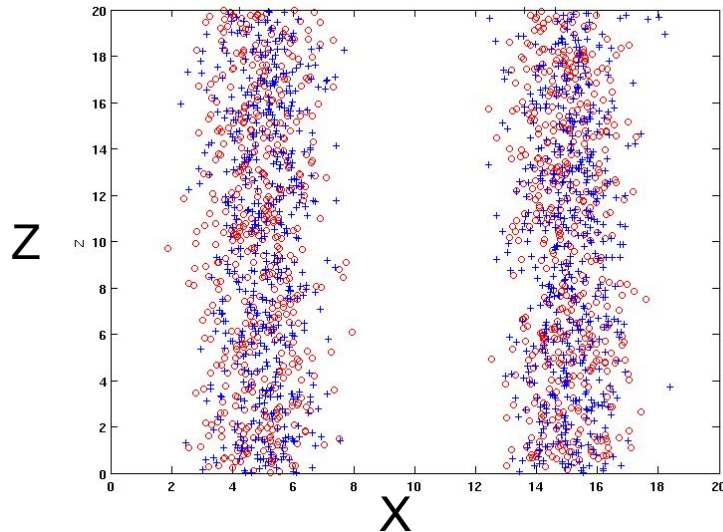
Entropy (X) : 0.99

Entropy (Y) : 0.99

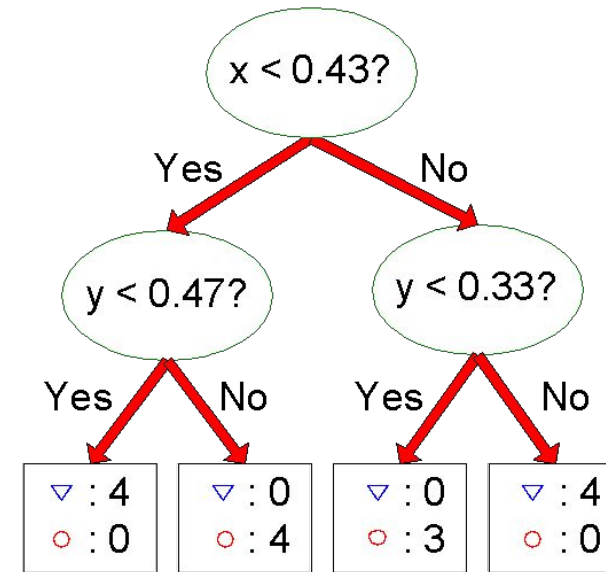
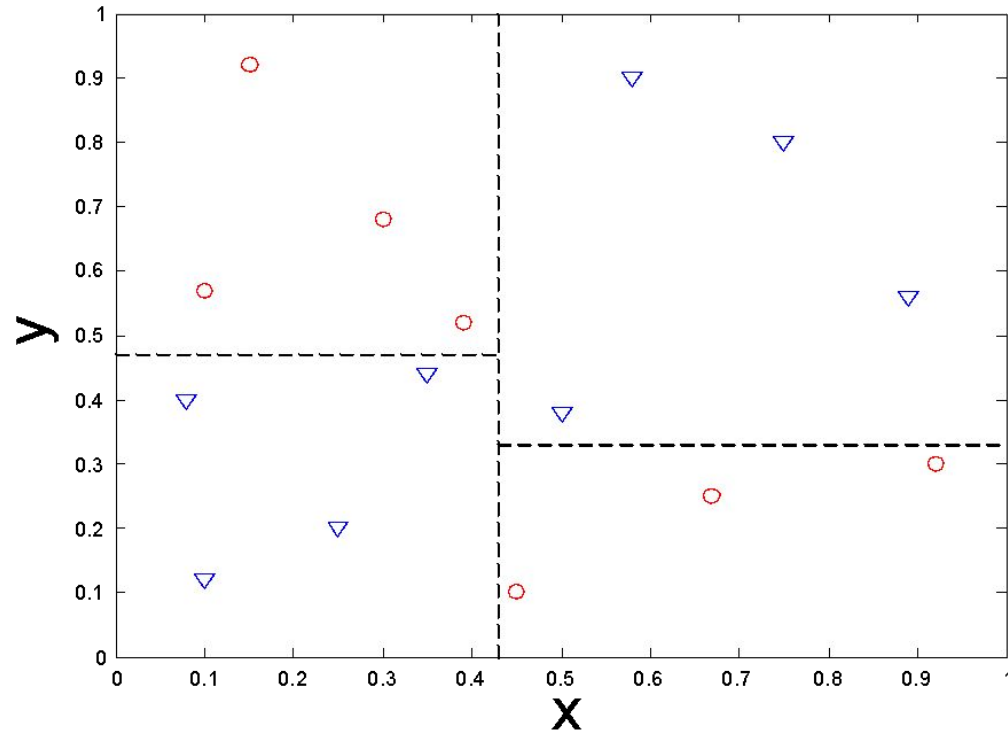
Entropy (Z) : 0.98

Adding Z as a noisy
attribute generated
from a uniform
distribution

Attribute Z will be
chosen for splitting!

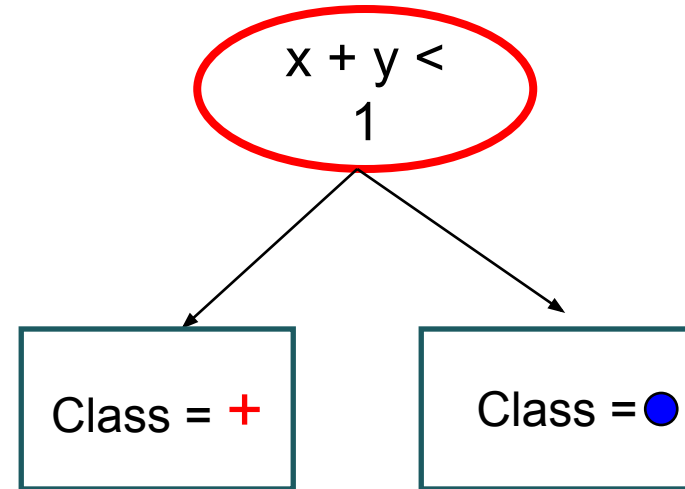
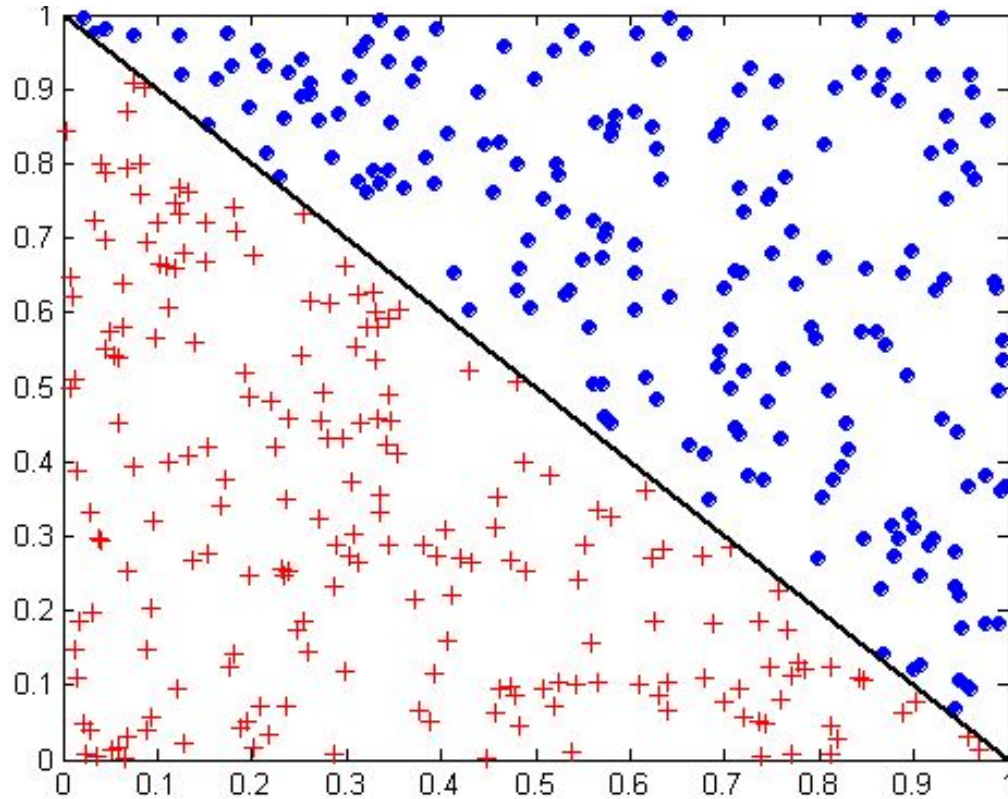


Decision Boundary



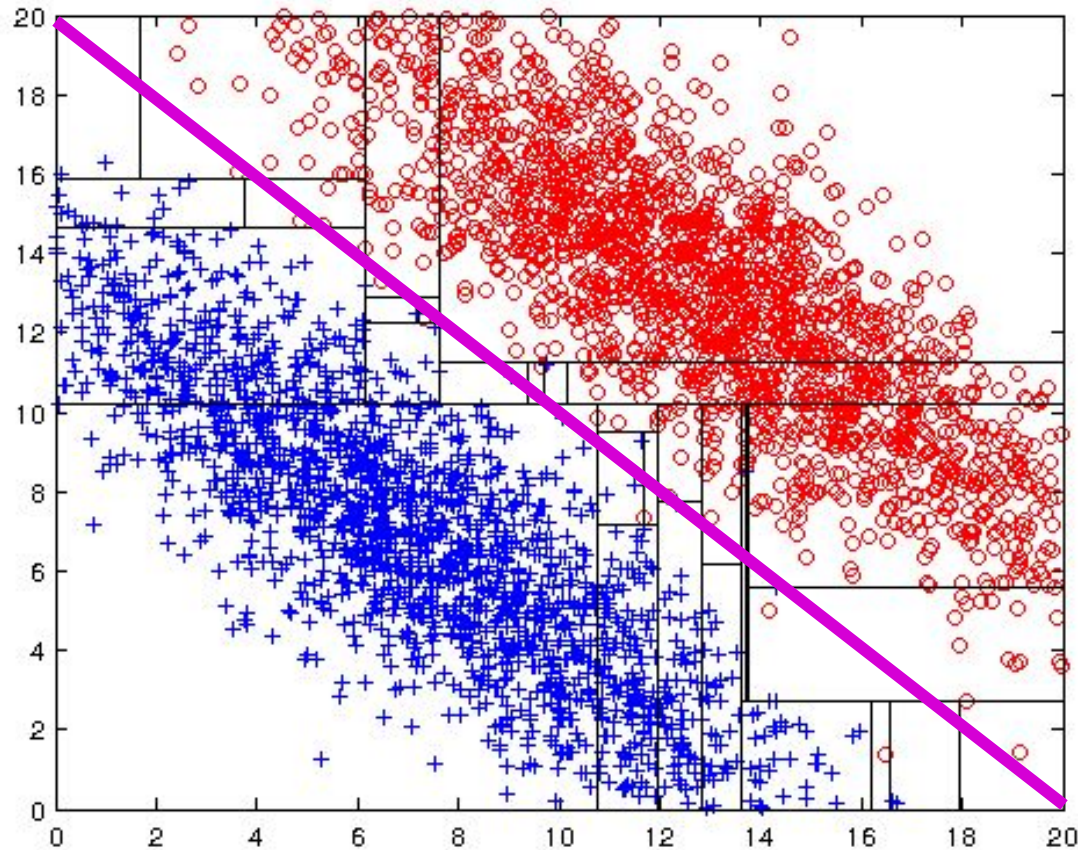
- Border line between two neighboring regions of different classes is known as decision boundary
- Decision boundary is parallel to axes because test condition involves a single attribute at-a-time

Oblique Decision Trees



- Test condition may involve multiple attributes
- More expressive representation
- Finding optimal test condition is computationally expensive

Limitations of single attribute-based decision boundaries



both **positive (+)** and **negative (o)** classes generated from skewed Gaussians with centers at (8,8) and (12,12) respectively.

Test Condition
 $x + y < 20$

Other Issues

- Data Fragmentation
- Tree Replication

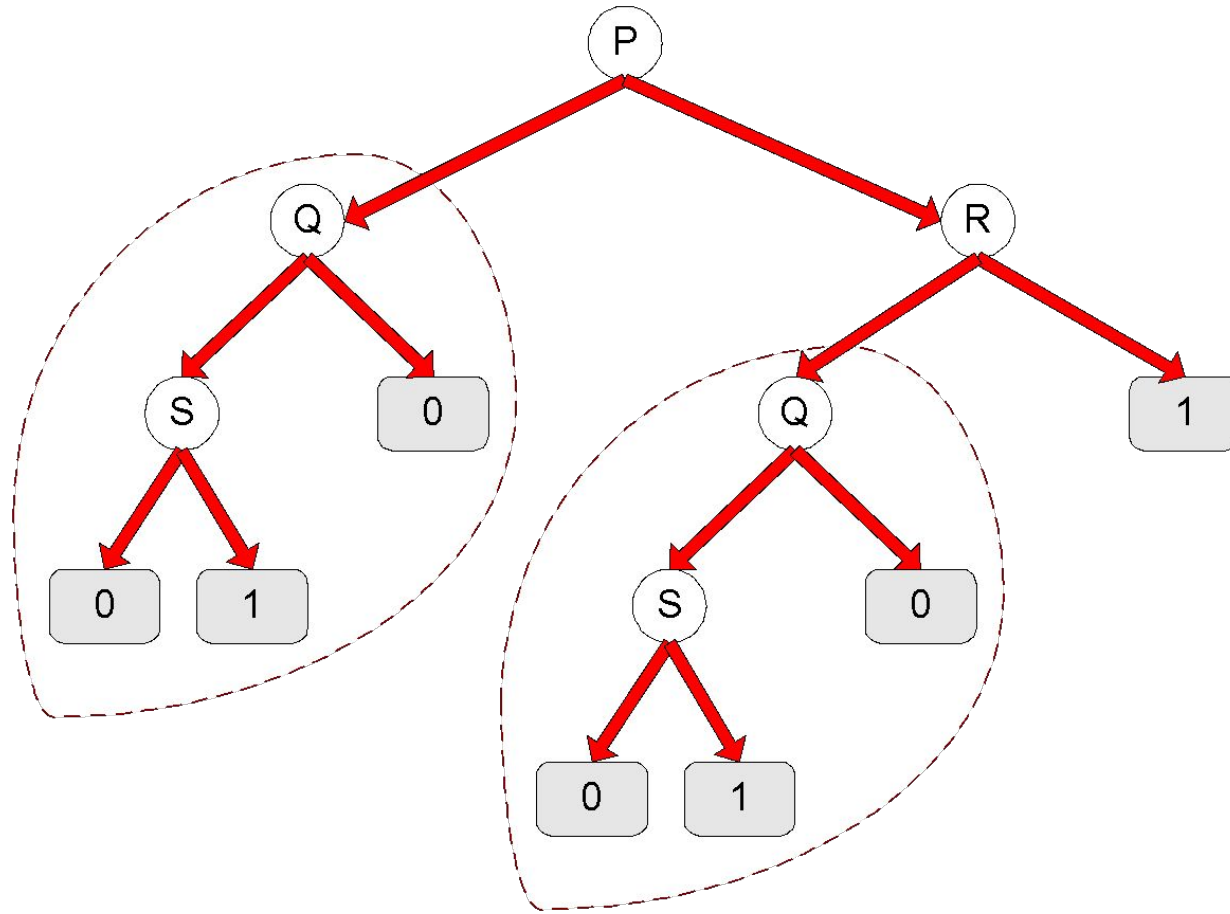
Data Fragmentation

- Number of instances gets smaller as you traverse down the tree
- Number of instances at the leaf nodes could be too small to make any statistically significant decision

Expressiveness

- Decision tree provides expressive representation for learning discrete-valued function
 - Every discrete-valued function can be represented as an assignment table, where every unique combination of discrete attributes is assigned a class label.
 - But they do not generalize well to certain types of Boolean functions
 - Example: parity function:
 - Class = 1 if there is an even number of Boolean attributes with truth value = True
 - Class = 0 if there is an odd number of Boolean attributes with truth value = True
 - For accurate modeling, must have a complete tree
- Not expressive enough for modeling continuous variables
 - Particularly when test condition involves only a single attribute at-a-time

Tree Replication



Same subtree appears in multiple branches

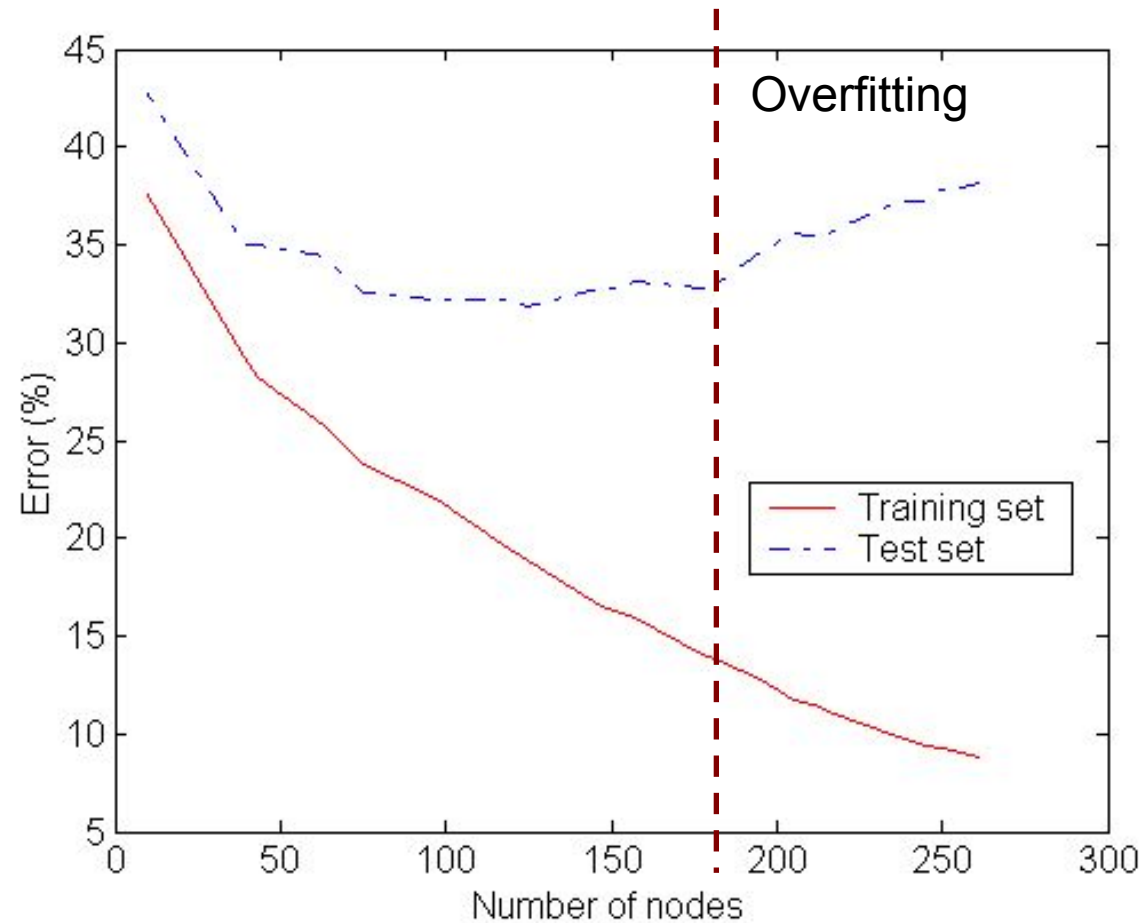
Practical Issues of Classification

- Underfitting and Overfitting
- Costs of Classification

Classification Errors

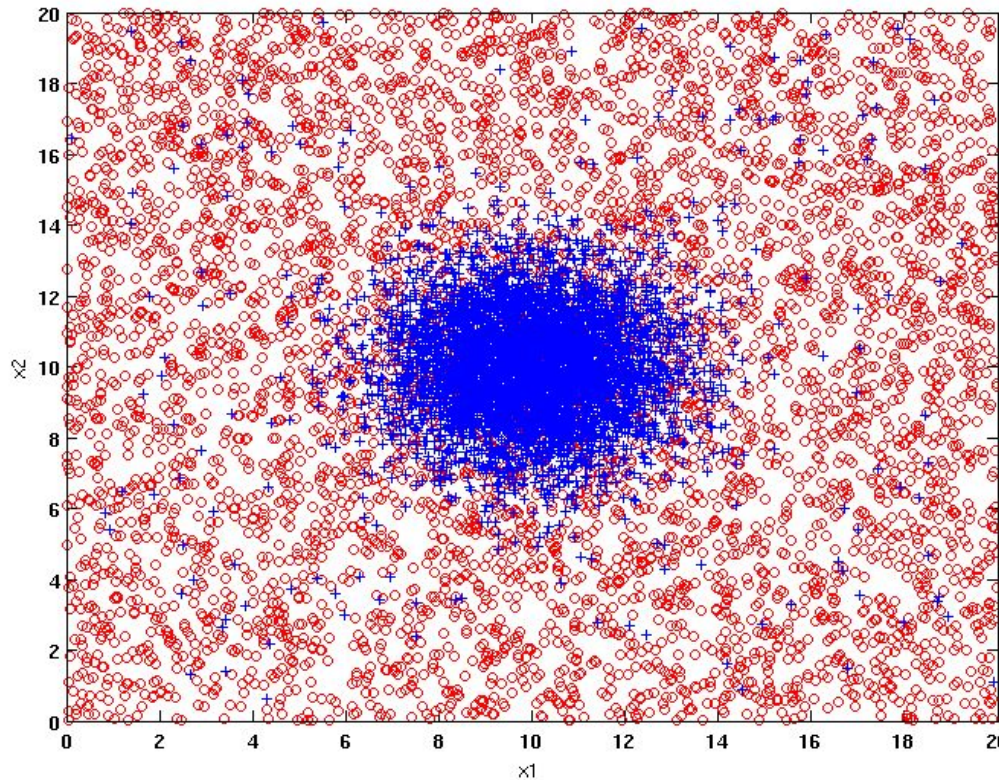
- Training errors (apparent errors)
 - Errors committed on the training set
- Test errors
 - Errors committed on the test set
- Generalization errors
 - Expected error of a model over random selection of records from same distribution

Underfitting and Overfitting



Underfitting: when model is too simple, both training and test errors are large

Example Data Set



Two class problem:

+ : 5200 instances

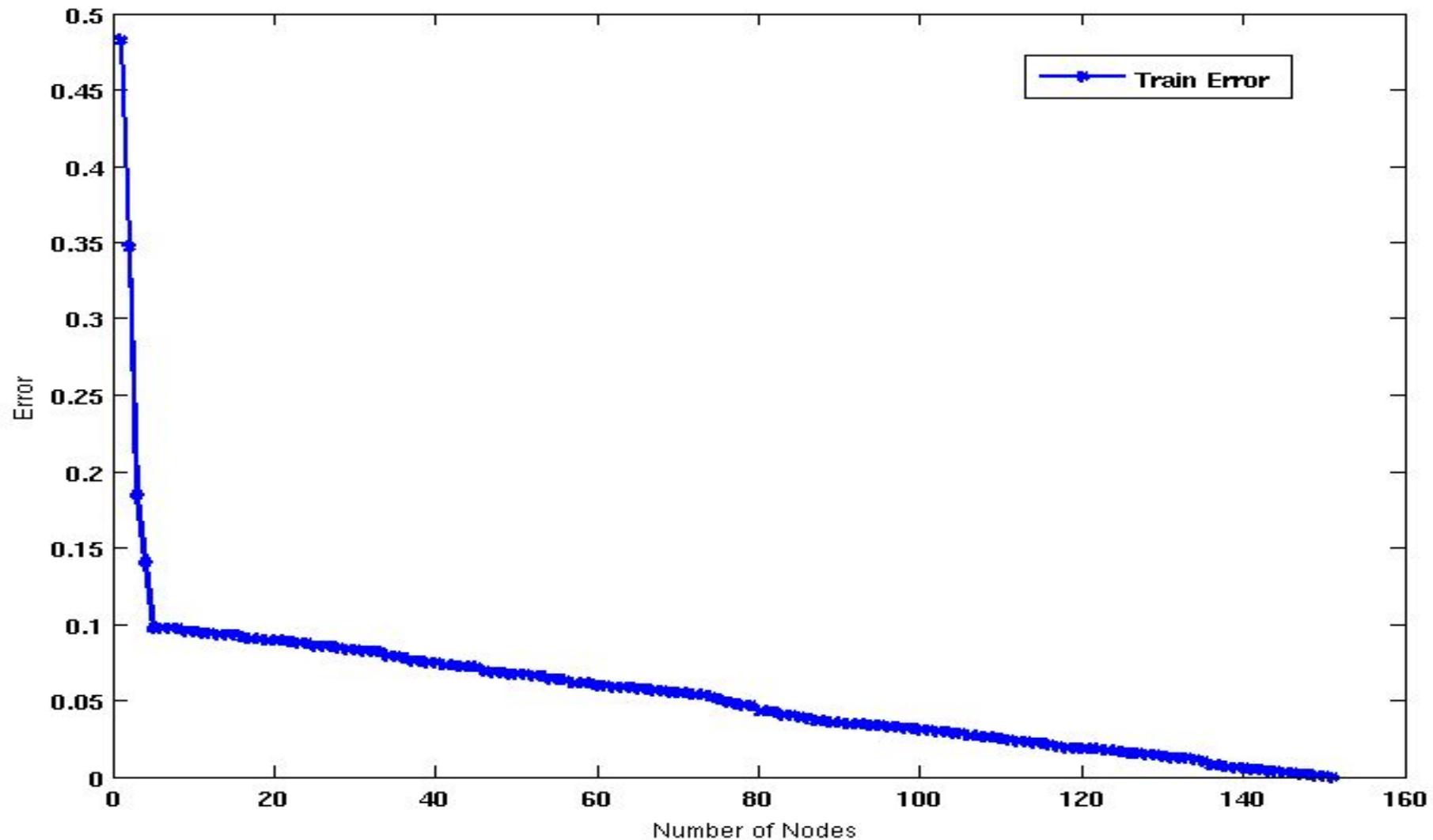
- 5000 instances generated from a Gaussian centered at (10,10)
- 200 noisy instances added

o : 5200 instances

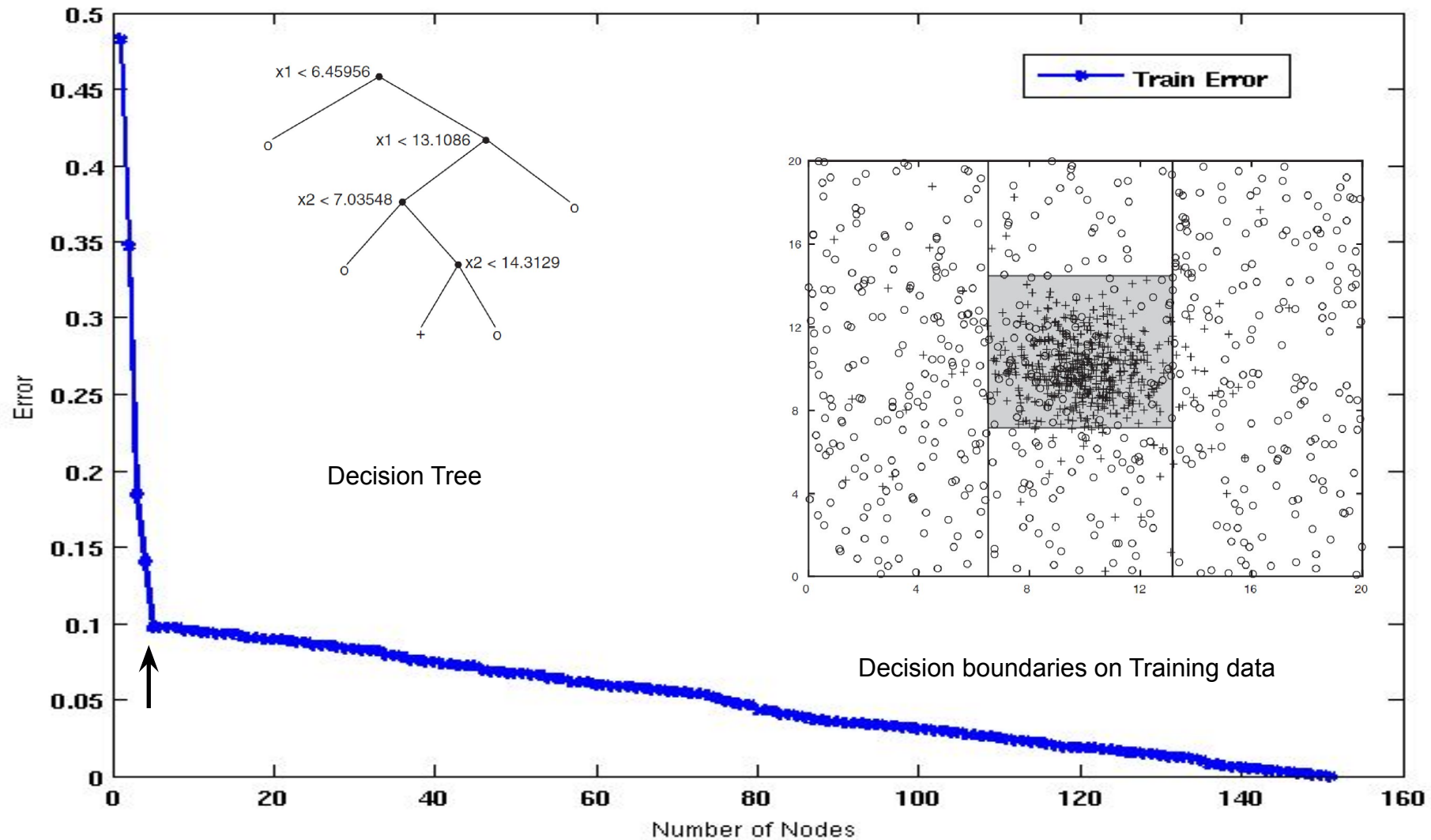
- Generated from a uniform distribution

10 % of the data used for training and 90% of the data used for testing

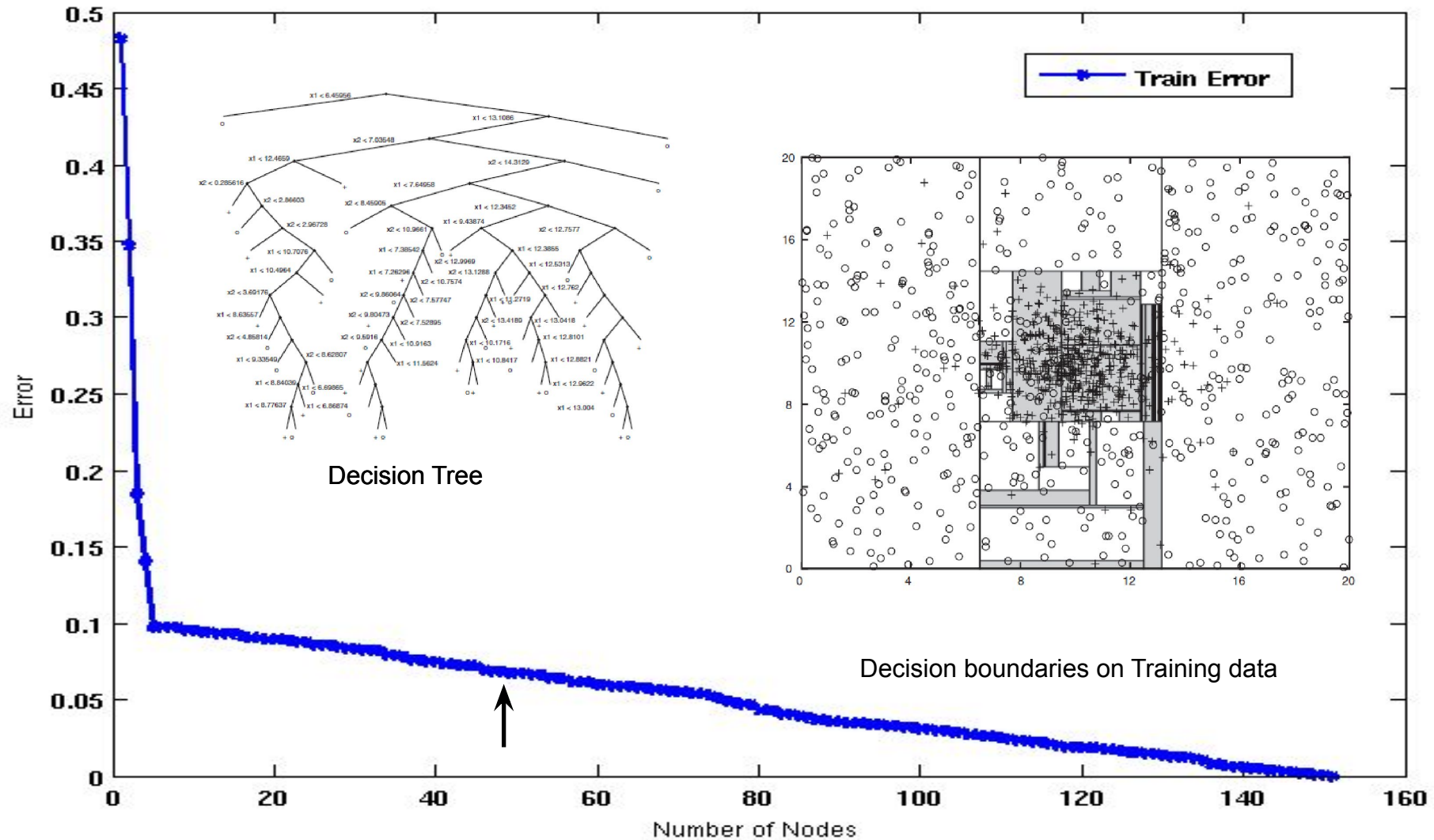
Increasing number of nodes in Decision Trees



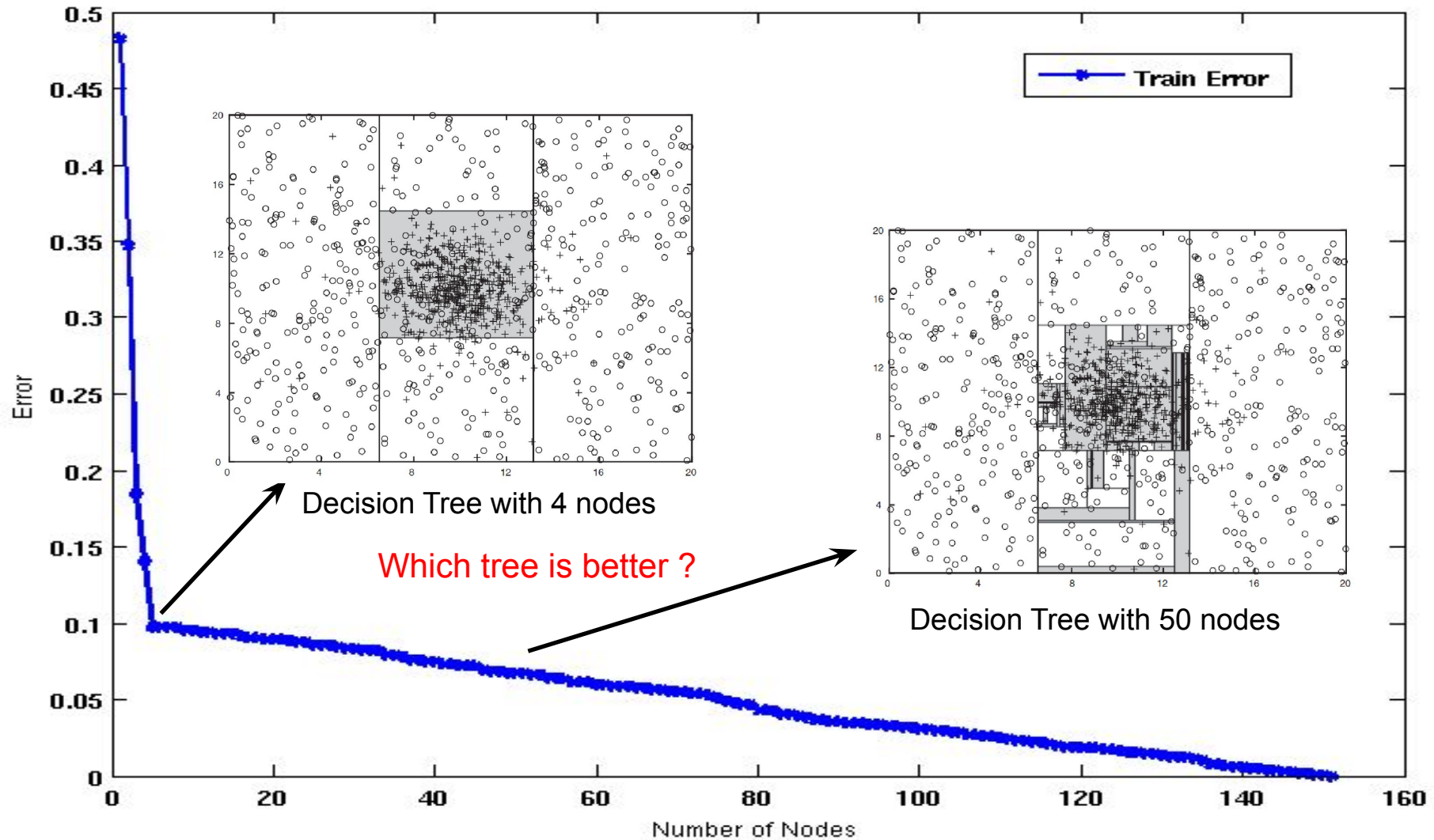
Decision Tree with 4 nodes



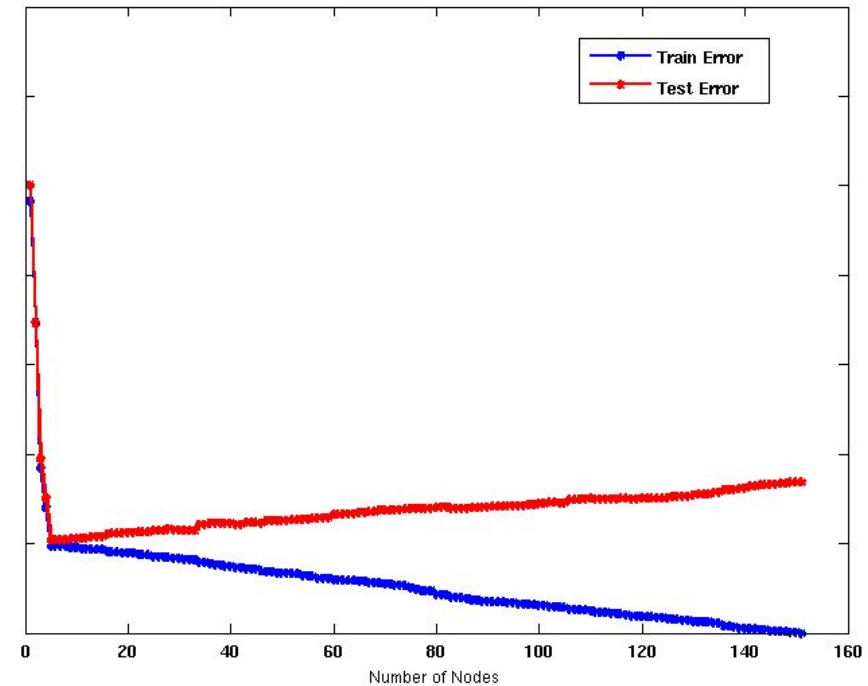
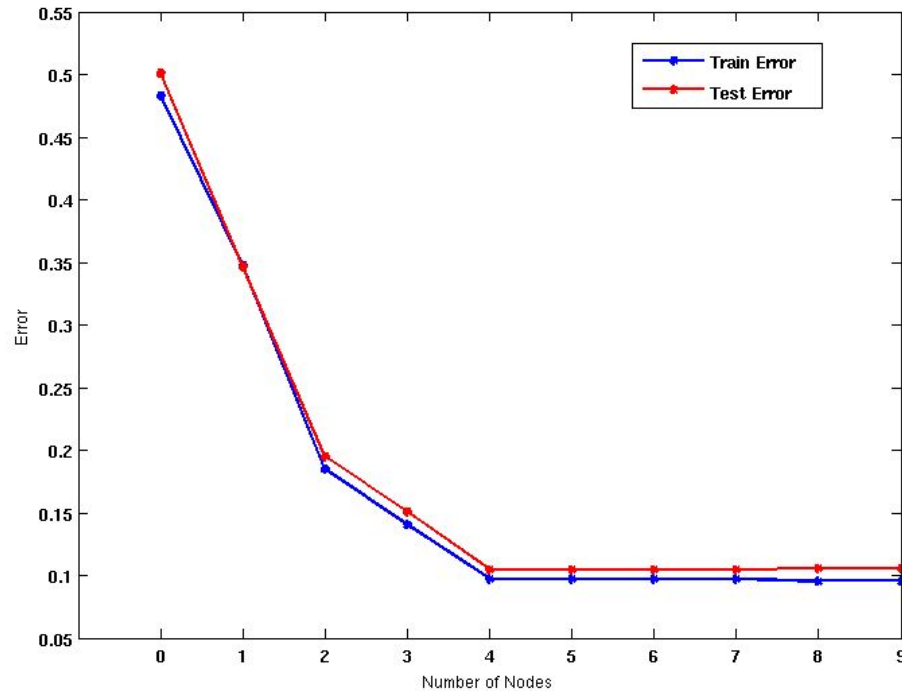
Decision Tree with 50 nodes



Which tree is better?



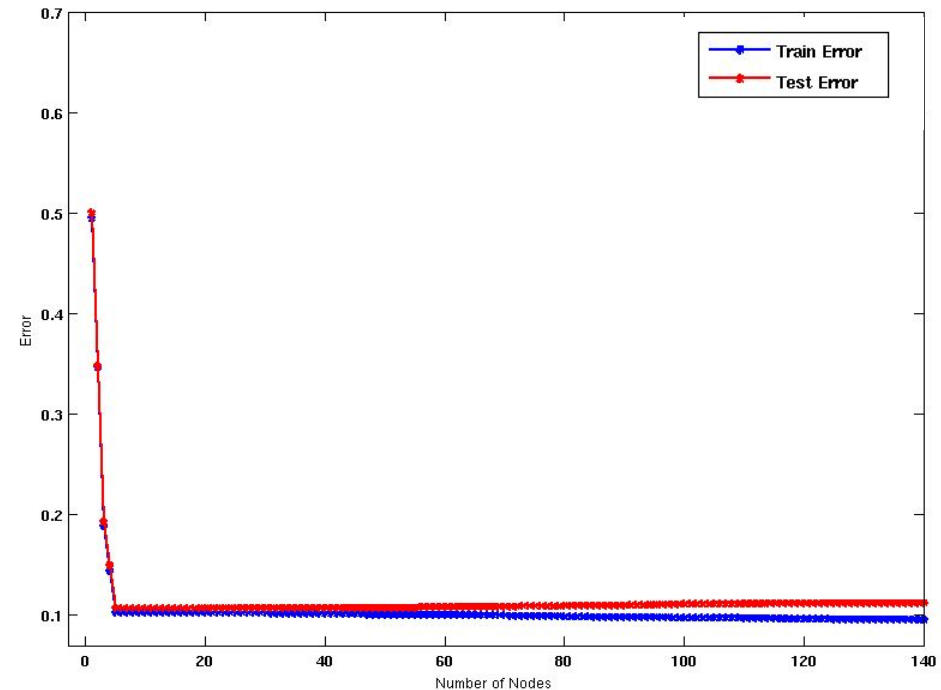
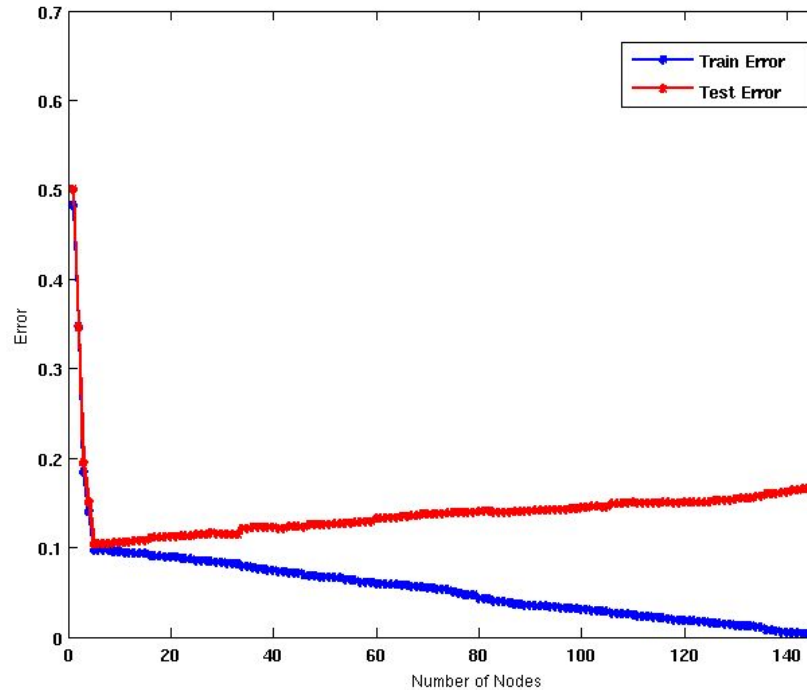
Model Overfitting



Underfitting: when model is too simple, both training and test errors are large

Overfitting: when model is too complex, training error is small but test error is large

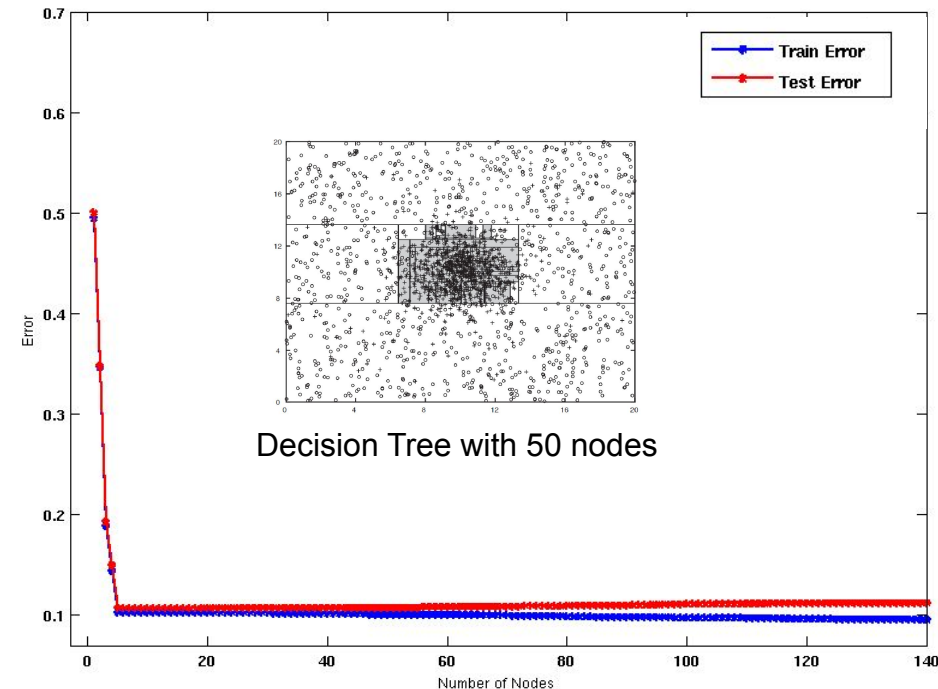
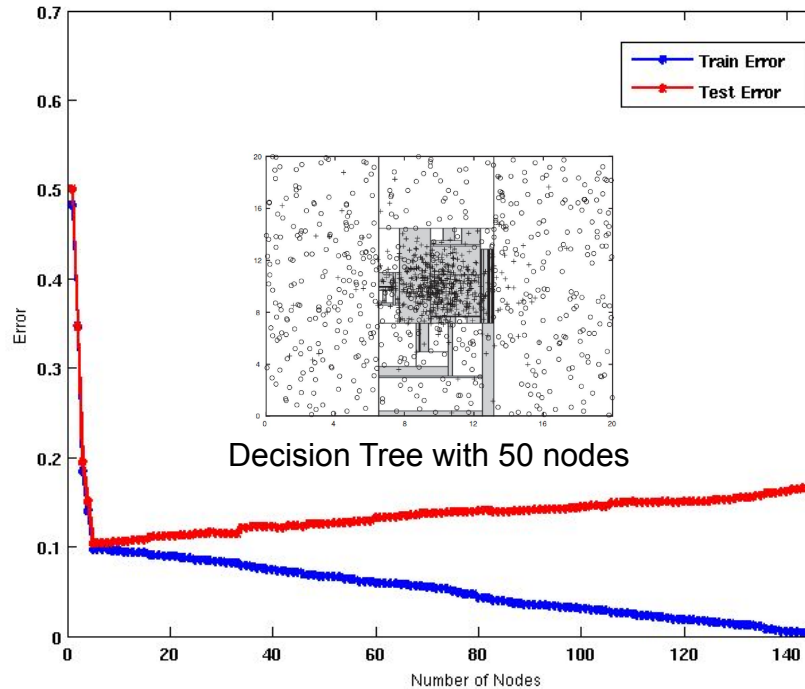
Model Overfitting



Using twice the number of data instances

- If training data is under-representative, testing errors increase and training errors decrease on increasing number of nodes
- Increasing the size of training data reduces the difference between training and testing errors at a given number of nodes

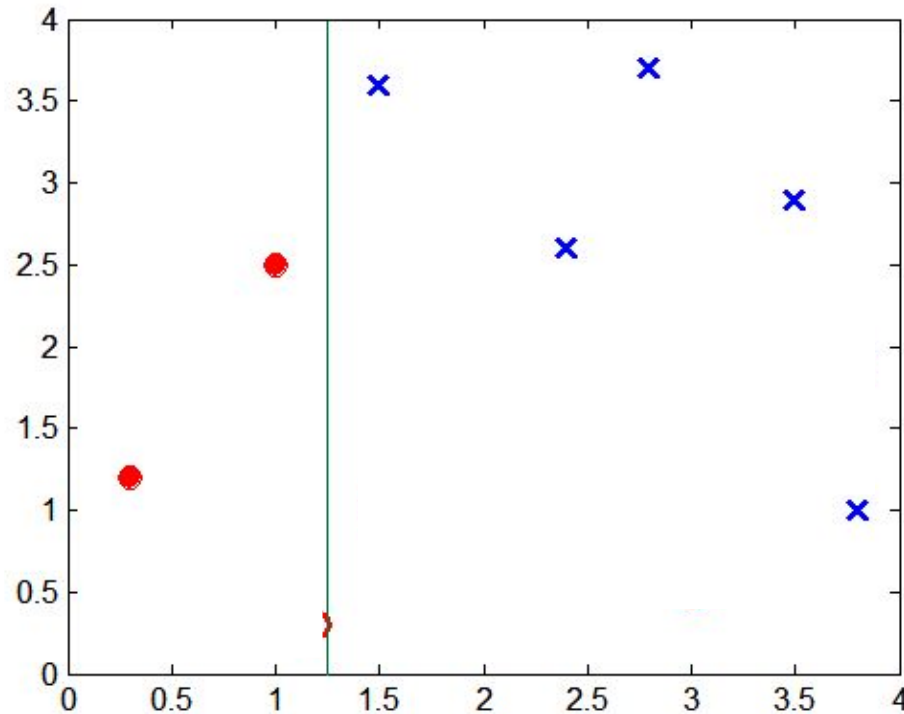
Model Overfitting



Using twice the number of data instances

- If training data is under-representative, testing errors increase and training errors decrease on increasing number of nodes
- Increasing the size of training data reduces the difference between training and testing errors at a given number of nodes

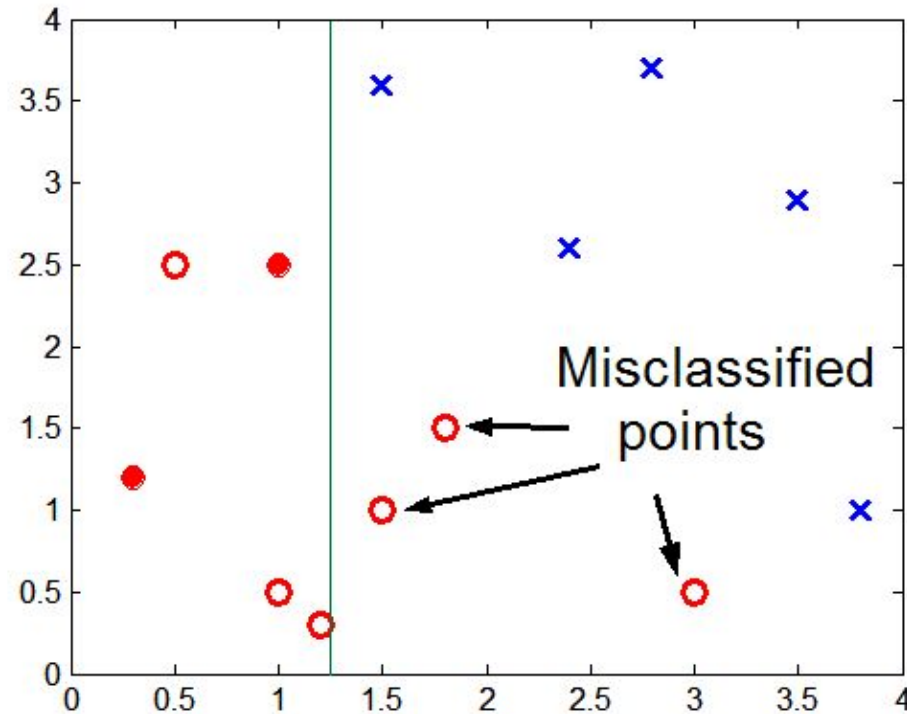
Overfitting due to Insufficient Examples



Lack of data points in the lower half of the diagram makes it difficult to predict correctly the class labels of that region

- Insufficient number of training records in the region causes the decision tree to predict the test examples using other training records that are irrelevant to the classification task

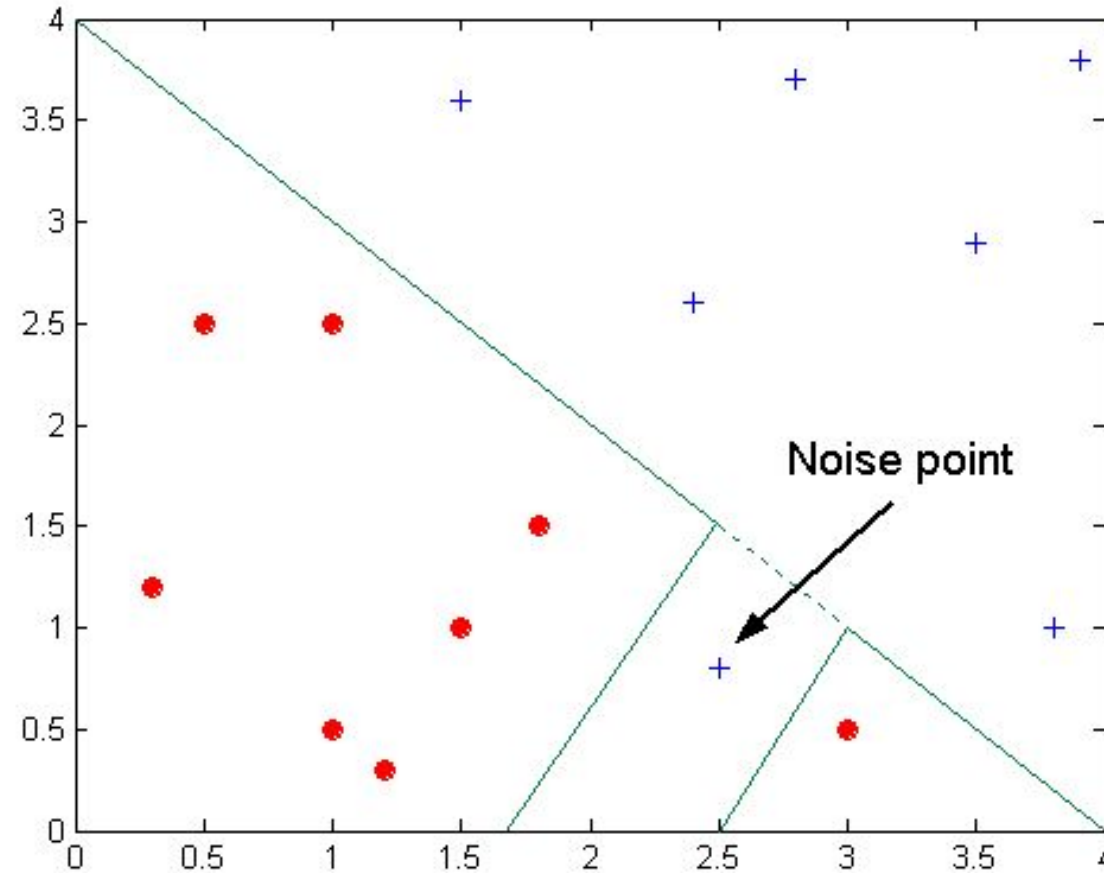
Overfitting due to Insufficient Examples



Lack of data points in the lower half of the diagram makes it difficult to predict correctly the class labels of that region

- Insufficient number of training records in the region causes the decision tree to predict the test examples using other training records that are irrelevant to the classification task

Overfitting due to Noise



Decision boundary is distorted by noise point

Notes on Overfitting

- Overfitting results in decision trees that are more complex than necessary
- Training error no longer provides a good estimate of how well the tree will perform on previously unseen records
- Need new ways for estimating errors

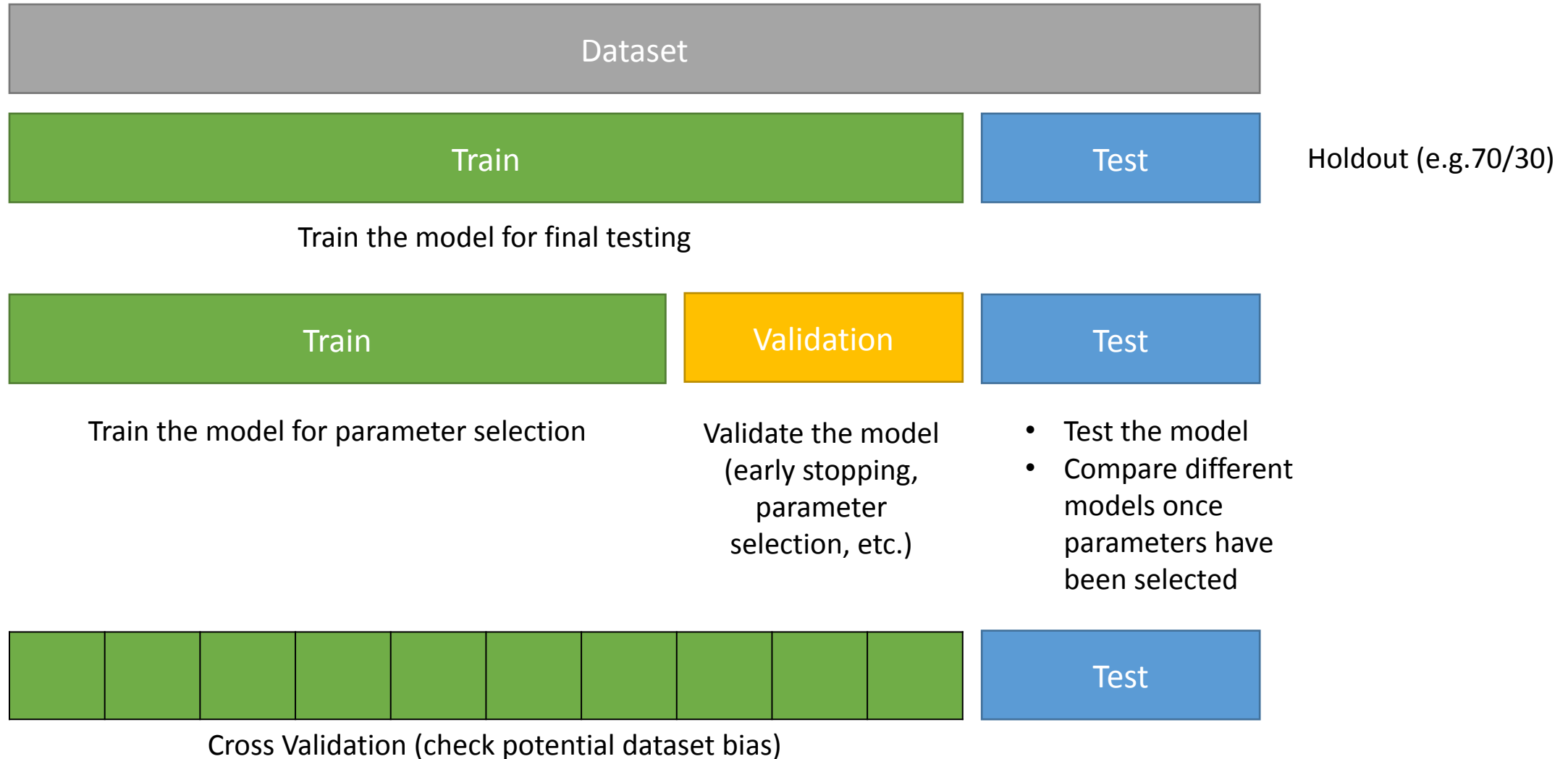
Model Selection

- Performed during model building
- Purpose is to ensure that model is not overly complex (to avoid overfitting)
- Need to estimate generalization error
 - Using Validation Set
 - Incorporating Model Complexity
 - Estimating Statistical Bounds

Model Selection Using Validation Set

- Divide training data into two parts:
 - Training set:
 - use for model building
 - Validation set:
 - use for estimating generalization error
 - Note: validation set is not the same as test set
- Drawback:
 - Less data available for training

Data Partitioning



Occam's Razor

- Given two models of similar generalization errors, **one should prefer the simpler model** over the more complex model
- For complex models, there is a greater chance that it was fitted accidentally by errors in data
- Therefore, one should include model complexity when evaluating a model

Model Selection Incorporating Model Complexity

- Given two models of similar generalization errors, **one should prefer the simpler model** over the more complex model
- For complex models, there is a greater chance that it was fitted accidentally by errors in data
- Therefore, one should include model complexity when evaluating a model

$$\text{Gen. Error}(\text{Model}) = \text{Train. Error}(\text{Model}, \text{Train. Data}) + \text{Complexity}(\text{Model})$$

Estimating Generalization Errors

- **Re-substitution errors:** error on training ($\sum \text{err}(t)$)
- **Generalization errors:** error on testing ($\sum \text{err}'(t)$)
- Methods for estimating generalization errors:
 - **Pessimistic approach**
 - **Optimistic approach**
 - **Reduced error pruning (REP):**
 - uses validation data set to estimate generalization error

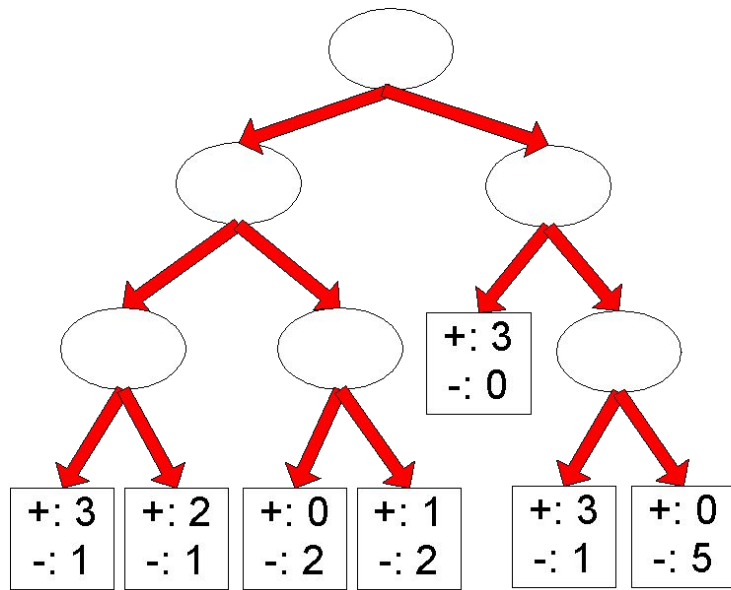
Estimating the Complexity of Decision Trees

- **Pessimistic Error Estimate** of decision tree T with k leaf nodes:

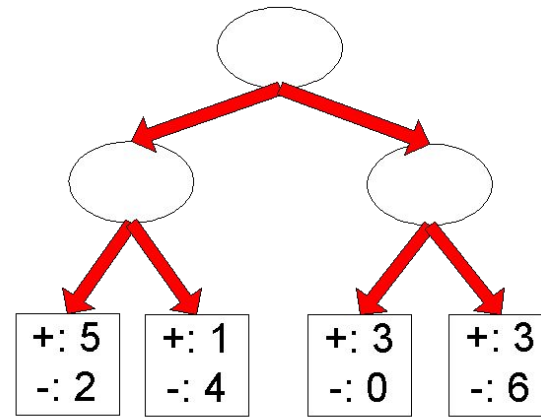
$$err_{gen}(T) = err(T) + \Omega \times \frac{k}{N_{train}},$$

- $err(T)$: error rate on all training records
- Ω : Relative cost of adding a leaf node
- k : number of leaf nodes
- N_{train} : total number of training records

Estimating the Complexity of Decision Trees: Example



Decision Tree, T_L



Decision Tree, T_R

$$e(T_L) = 4/24$$

$$e(T_R) = 6/24$$

$$\Omega = 1$$

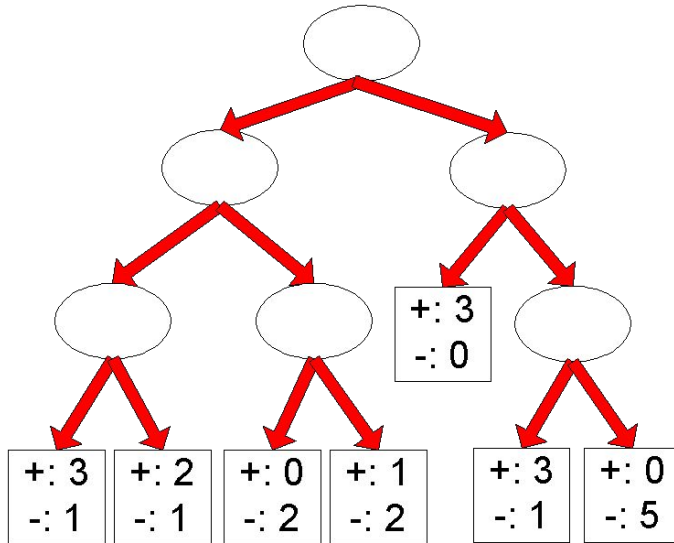
$$e_{\text{gen}}(T_L) = 4/24 + 1 \cdot 7/24 = 11/24 = 0.458$$

$$e_{\text{gen}}(T_R) = 6/24 + 1 \cdot 4/24 = 10/24 = 0.417$$

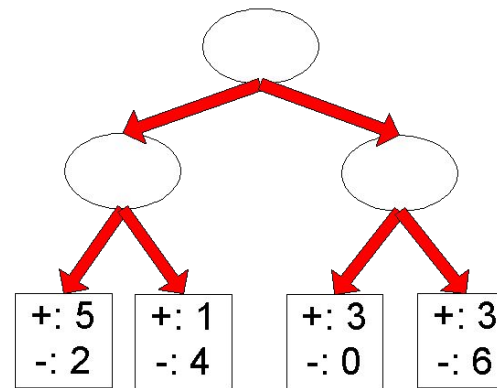
$$err_{\text{gen}}(T) = err(T) + \Omega \times \frac{k}{N_{\text{train}}},$$

Estimating the Complexity of Decision Trees

- Re-substitution Estimate:
 - Using training error as an **optimistic** estimate of generalization error
 - Referred to as optimistic error estimate



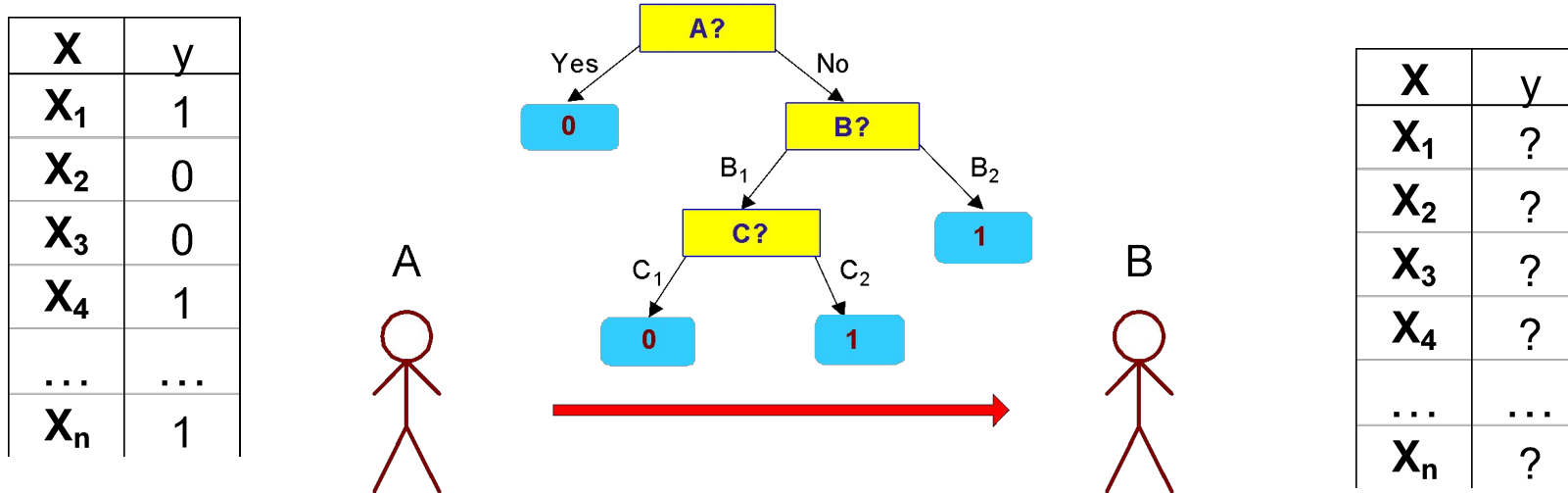
Decision Tree, T_L

Decision Tree, T_R

$$e(T_L) = 4/24$$

$$e(T_R) = 6/24$$

Minimum Description Length (MDL)

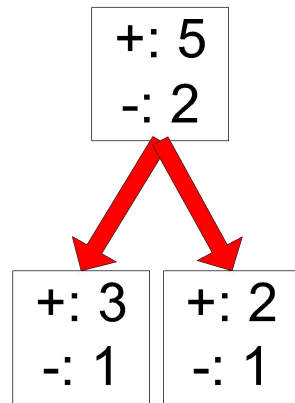


- $\text{Cost}(\text{Model}, \text{Data}) = \text{Cost}(\text{Data} | \text{Model}) + \text{Cost}(\text{Model})$
 - Cost is the number of bits needed for encoding.
 - Search for the least costly model.
- $\text{Cost}(\text{Data} | \text{Model})$ encodes the misclassification errors.
- $\text{Cost}(\text{Model})$ uses node encoding (number of children) plus splitting condition encoding.

Estimating Statistical Bounds

Apply a statistical correction to the training error rate of the model that is indicative of its model complexity.

- Need probability distribution of training error: available or assumed.
- The number of errors committed by a leaf node in a decision tree can be assumed to follow a binomial distribution.



Before splitting: $e = 2/7$, $e'(7, 2/7, 0.25) = 0.503$
 $e'(T) = 7 \times 0.503 = 3.521$

After splitting:

$e(T_L) = 1/4$, $e'(4, 1/4, 0.25) = 0.537$
 $e(T_R) = 1/3$, $e'(3, 1/3, 0.25) = 0.650$
 $e'(T) = 4 \times 0.537 + 3 \times 0.650 = 4.098$

$$e'(N, e, \alpha) = \frac{e + \frac{z_{\alpha/2}^2}{2N} + z_{\alpha/2} \sqrt{\frac{e(1-e)}{N} + \frac{z_{\alpha/2}^2}{4N^2}}}{1 + \frac{z_{\alpha/2}^2}{N}}$$

Therefore, do not split

How to Address Overfitting...

- Pre-Pruning (Early Stopping Rule)
 - Stop the algorithm before it becomes a fully-grown tree
 - Typical stopping conditions for a node:
 - Stop if all instances belong to the same class
 - Stop if all the attribute values are the same
 - More restrictive conditions:
 - Stop if number of instances is less than some user-specified threshold
 - Stop if class distribution of instances are independent of the available features (e.g., using χ^2 test)
 - Stop if expanding the current node does not improve impurity measures (e.g., Gini or information gain).
 - Stop if estimated generalization error falls below certain threshold

How to Address Overfitting...

- Post-pruning

- Grow decision tree to its entirety
- Trim the nodes of the decision tree in a bottom-up fashion
- If generalization error improves after trimming, replace sub-tree by a leaf node.
- Class label of leaf node is determined from majority class of instances in the sub-tree
- Can use MDL for post-pruning

Example of Post-Pruning

Class = Yes	20
Class = No	10
Error = 10/30	

Training Error (Before splitting) = 10/30

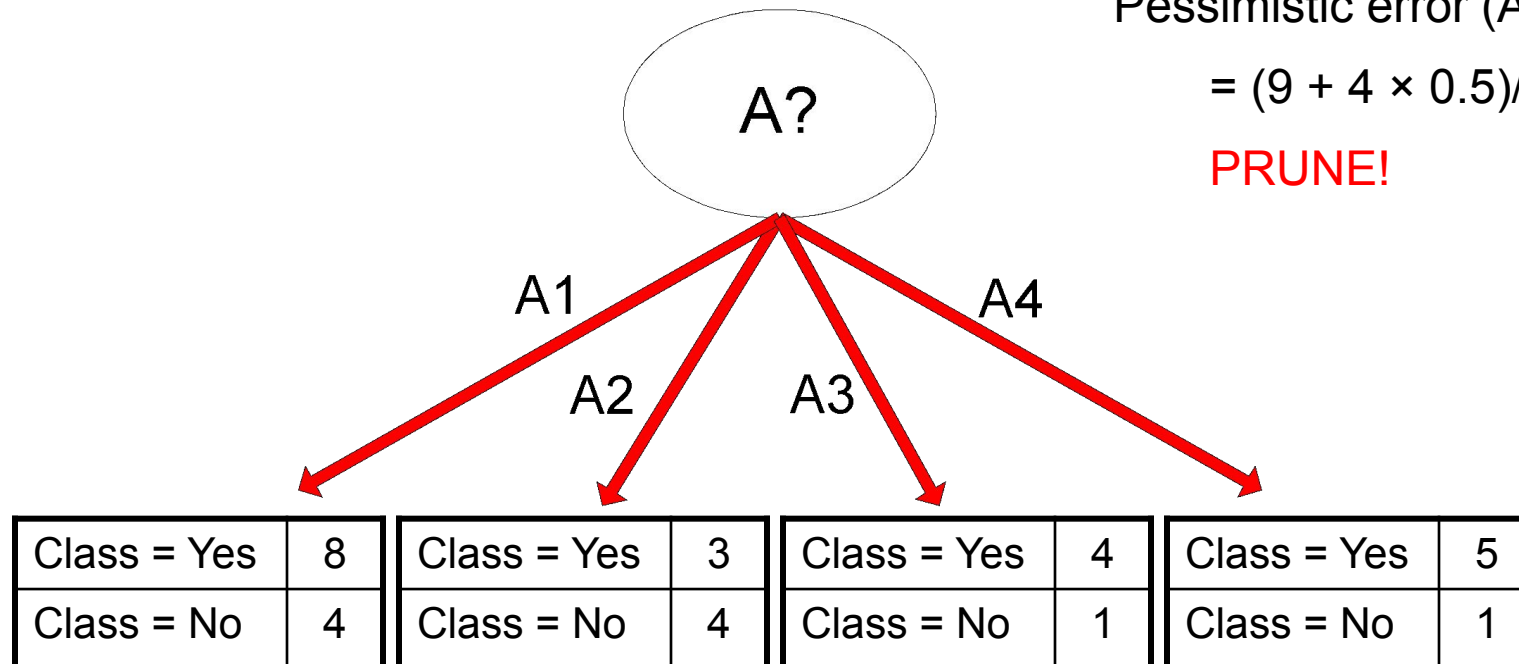
Pessimistic error = $(10 + 0.5)/30 = 10.5/30$

Training Error (After splitting) = 9/30

Pessimistic error (After splitting)

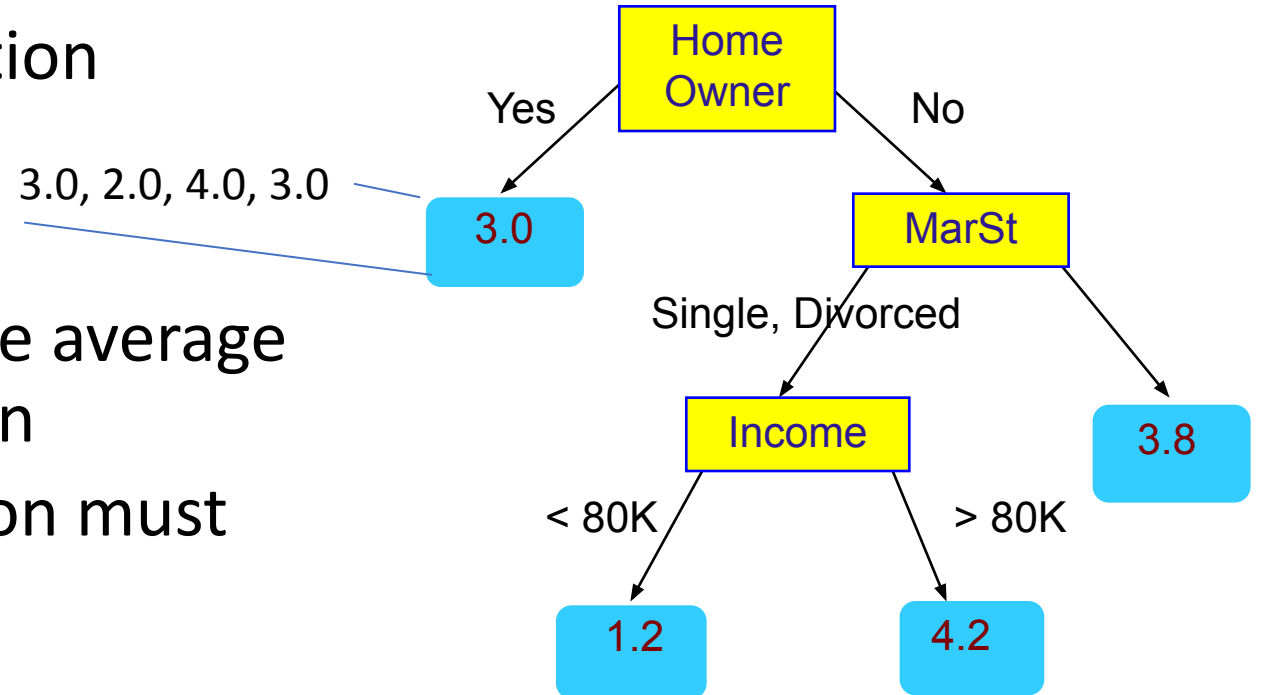
= $(9 + 4 \times 0.5)/30 = 11/30$

PRUNE!



Decision Trees for Regression

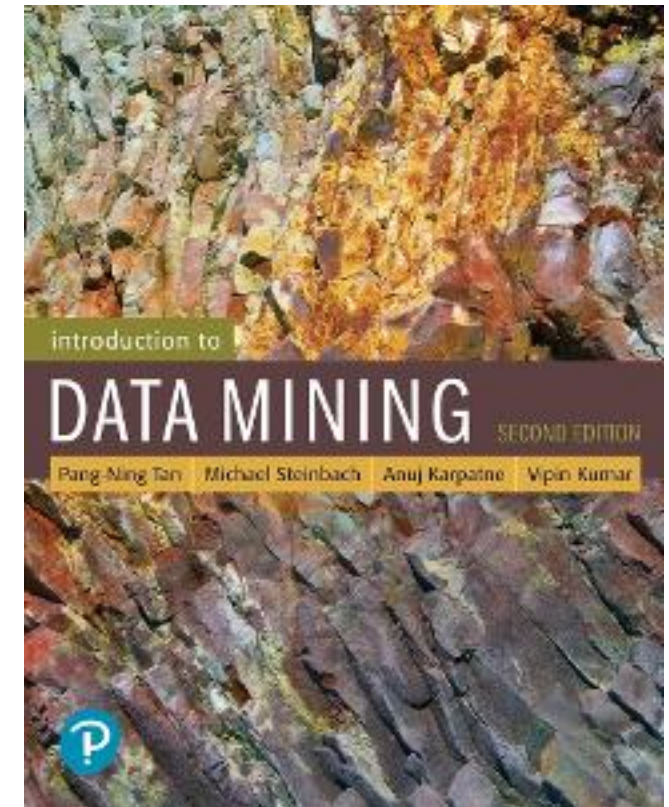
- The same induction and application procedures can be used.
- The only differences are:
 - When leaves are not pure, the average value is returned as prediction
 - Different optimization criterion must be used such as
 - MSE
 - MAE



$$\text{MSE}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} (y_i - \hat{y}_i)^2 \quad \text{MAE}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} |y_i - \hat{y}_i|$$

References

- Classification: Basic Concepts and Techniques . Chapter 3. Introduction to Data Mining.



	Outlook	Temperature	Wind	Play Tennis
1	Sunny	25	Weak	No
2	Sunny	26	Strong	No
3	Overcast	27	Weak	Yes
4	Rain	19	Weak	Yes
5	Rain	8	Weak	Yes
6	Rain	7	Strong	No
7	Overcast	10	Strong	Yes
8	Sunny	15	Weak	No
9	Sunny	8	Weak	Yes
10	Rain	15	Weak	Yes

	Outlook	Temperature	Wind	Play Tennis
11	Sunny	14	Strong	Yes
12	Overcast	13	Strong	Yes
13	Overcast	26	Weak	Yes
14	Rain	16	Strong	No

Split Wind		
	Weak	Strong
Yes	5	1
No	2	2
	7	3
ME	0,28571429	0,33333333
ME	0,3	
Gain	0,1	

Split Outlook			
	Sunny	Outlook	Rain
Yes	1	2	3
No	3	0	1
	4	2	4
ME	0,25	0	0,25
ME	0,2		
Gain	0,2		
	Sunny	Not Sunny	
Yes	1	5	
No	3	1	
	4	6	
ME	0,25	0,16666667	
ME	0,2		
Gain	0,2		

	Outlook	Not Outlook	
Yes	2	4	
No	0	4	
	2	8	
ME	0	0,5	
ME	0,4		
Gain	0		
	Rain	Not Rain	
Yes	3	5	
No	1	1	
	4	6	
ME	0,25	0,16666667	
ME	0,2		
Gain	0,2		

Yes	6		0,4
No	4		

Split Temperature															
	<=		<=		<=		<=		<=		<=		<=		
	7		8		10		15		19		25		26		
Yes	0	6	2	4	3	3	4	2	5	1	5	1	5	1	
No	1	3	1	3	1	3	2	2	2	2	3	1	4	0	
	1	9	3	7	4	6	6	4	7	3	8	2	9	1	
ME	0	0,3333	0,3333	0,4286	0,25	0,5	0,3333	0,5	0,2857	0,3333	0,375	0,5	0,4444	0	
ME	0,3		0,4		0,4		0,4		0,3		0,4		0,4		
Gain	0,1		0		0		0		0,1		0		0		

	Outlook	Temperature	Wind	Play Tennis
1	Sunny	25	Weak	No
2	Sunny	26	Strong	No
8	Sunny	15	Weak	No
9	Sunny	8	Weak	Yes

P	
Yes	1
No	3
ME	0,25

Split Wind		
	Weak	Strong
Yes	1	0
No	2	1
	3	1
ME	0,3333	0
ME	0,25	
Gain	0	

Split Temperature						
	<=8	>8	<=15	>15	<=25	>25
Yes	1	0	1	0	1	0
No	0	3	1	2	2	1
	1	3	2	2	3	1
ME	0	0	0,5	0	0,3333	0
ME	0		0,25		0,25	
Gain	0,25		0		0	

	Outlook	Temperature	Wind	Play Tennis
3	Overcast	27	Weak	Yes
7	Overcast	10	Strong	Yes
algorithm ends				

	Outlook	Temperature	Wind	Play Tennis
4	Rain	19	Weak	Yes
5	Rain	8	Weak	Yes
6	Rain	7	Strong	No
10	Rain	15	Weak	Yes

P	
Yes	3
No	1
ME	0,25

Split Wind		
	Weak	Strong
Yes	3	0
No	0	1
	3	1
ME	0	0
ME	0	
Gain	0,25	

Split Temperature						
	<=7	>7	<=8	>8	<=15	>15
Yes	0	3	1	2	2	1
No	1	0	1	0	1	0
	1	3	2	2	3	1
ME	0	0	0,5	0	0,3333	0
ME	0		0,25		0,25	
Gain	0,25		0		0	

if Outlook = Overcast		
	- Play Tennis yes	
elif Outlook = Sunny		
	if Temperature <=8	
		- Play Tennis yes
	else Temperature > 8	
		- Play Tennis no
else Outlook = Rain		
	if Wind = Weak	
		- Play Tennis yes
	else Wind = Strong	
		- Play Tennis no

	Outlook	Temp	Wind	Play Tennis	Predicted
11	Sunny	14	Strong	Yes	No
12	Overcast	13	Strong	Yes	Yes
13	Overcast	26	Weak	Yes	Yes
14	Rain	16	Strong	No	No

TP: 2

FP: 0

FN: 1

TN: 1

Accuracy = 3/4

Precision = $TP / (TP + FP) = 2/2$

Recall = $TP / (TP + FN) = 2/3$

Sex	Lies	Cookies	Present
F	10	Milk	Y
M	5	Dark	N
F	2	Milk	Y
F	3	Dark	Y
M	8	Milk	N
M	3	No	Y
M	10	Dark	N
F	2	No	N
M	1	No	Y

Classification_Knn_NaiveBayes

November 22, 2025

```
[1]: !pip install scikit-plot  
      !pip install scipy==1.11.4
```

```
Requirement already satisfied: scikit-plot in /usr/local/lib/python3.12/dist-packages (0.3.7)  
Requirement already satisfied: matplotlib>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from scikit-plot) (3.10.0)  
Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.12/dist-packages (from scikit-plot) (1.6.1)  
Requirement already satisfied: scipy>=0.9 in /usr/local/lib/python3.12/dist-packages (from scikit-plot) (1.11.4)  
Requirement already satisfied: joblib>=0.10 in /usr/local/lib/python3.12/dist-packages (from scikit-plot) (1.5.2)  
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=1.4.0->scikit-plot) (1.3.3)  
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=1.4.0->scikit-plot) (0.12.1)  
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=1.4.0->scikit-plot) (4.60.1)  
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=1.4.0->scikit-plot) (1.4.9)  
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=1.4.0->scikit-plot) (1.26.4)  
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=1.4.0->scikit-plot) (25.0)  
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=1.4.0->scikit-plot) (11.3.0)  
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=1.4.0->scikit-plot) (3.2.5)  
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=1.4.0->scikit-plot) (2.9.0.post0)  
Requirement already satisfied: threadpoolctl>=3.1.0 in
```

/usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->scikit-plot)
(3.6.0)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
packages (from python-dateutil>=2.7->matplotlib>=1.4.0->scikit-plot) (1.17.0)

Requirement already satisfied: scipy==1.11.4 in /usr/local/lib/python3.12/dist-
packages (1.11.4)

Requirement already satisfied: numpy<1.28.0,>=1.21.6 in
/usr/local/lib/python3.12/dist-packages (from scipy==1.11.4) (1.26.4)

```
[2]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
```

```
[3]: import scipy
scipy.__version__
```

```
[3]: '1.11.4'
```

```
[4]: from sklearn.metrics import (
    accuracy_score,
    f1_score,
    classification_report,
    confusion_matrix,
    roc_auc_score,
)
from scikitplot.metrics import plot_roc
from scikitplot.metrics import plot_precision_recall
```

```
[5]: from sklearn.datasets import load_iris

frame = load_iris(as_frame=True)
df_orig = frame['data']
X = df_orig.values
y = np.array(frame['target'])
```

```
[6]: df_orig.head()
```

```
[6]:   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0              5.1             3.5             1.4             0.2
1              4.9             3.0             1.4             0.2
2              4.7             3.2             1.3             0.2
3              4.6             3.1             1.5             0.2
4              5.0             3.6             1.4             0.2
```

```
[7]: np.unique(y, return_counts=True)
```

```
[7]: (array([0, 1, 2]), array([50, 50, 50]))
```

```
[8]: df_orig.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  -
0   sepal length (cm)      150 non-null   float64
1   sepal width (cm)       150 non-null   float64
2   petal length (cm)      150 non-null   float64
3   petal width (cm)       150 non-null   float64
dtypes: float64(4)
memory usage: 4.8 KB
```

0.1 Partitioning

```
[9]: from sklearn.model_selection import train_test_split
```

```
[10]: random_state = 0
```

```
[11]: X_train, X_test, y_train, y_test = train_test_split(
      X, y, test_size=0.4, random_state=random_state
    )
```

```
[12]: # without stratify
print(np.unique(y, return_counts=True)[1] / len(y))
print(np.unique(y_train, return_counts=True)[1] / len(y_train))
print(np.unique(y_test, return_counts=True)[1] / len(y_test))
```

```
[0.33333333 0.33333333 0.33333333]
[0.37777778 0.3         0.32222222]
[0.26666667 0.38333333 0.35       ]
```

```
[13]: X_train, X_test, y_train, y_test = train_test_split(
      X, y, test_size=0.4, stratify=y, random_state=random_state
    )
```

```
[14]: # with stratify
print(np.unique(y, return_counts=True)[1] / len(y))
print(np.unique(y_train, return_counts=True)[1] / len(y_train))
print(np.unique(y_test, return_counts=True)[1] / len(y_test))
```

```
[0.33333333 0.33333333 0.33333333]
[0.33333333 0.33333333 0.33333333]
[0.33333333 0.33333333 0.33333333]
```



```
[15]: print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(90, 4) (60, 4) (90,) (60,)
```

normalization

```
[16]: from sklearn.preprocessing import StandardScaler
```

```
[17]: norm = StandardScaler()
      norm.fit(X_train)

      X_train_norm = norm.transform(X_train)
      X_test_norm = norm.transform(X_test)
```

0.2 Classifiers

Classifiers usually have a: - fit method to train them on training data - predict method to validate/test them on validation/test data

0.3 KNN

```
[18]: from sklearn.neighbors import KNeighborsClassifier
```

```
[19]: clf = KNeighborsClassifier(n_neighbors=5, metric="euclidean", weights="uniform")
      clf.fit(X_train_norm, y_train)
```

```
[19]: KNeighborsClassifier(metric='euclidean')
```

```
[20]: # predict: Predict the class labels for the provided data.
      y_test_pred = clf.predict(X_test_norm)
      y_test_pred
```

```
[20]: array([2, 2, 0, 2, 0, 0, 0, 2, 1, 1, 0, 2, 2, 0, 1, 2, 1, 0, 2, 2, 1, 0,
          2, 0, 1, 1, 2, 0, 1, 2, 0, 2, 1, 1, 2, 0, 2, 1, 1, 2, 1, 0, 1, 2,
          2, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 2, 0, 1, 1])
```

```
[21]: # y_test contains the target labels
      y_test
```

```
[21]: array([2, 2, 0, 2, 0, 0, 0, 2, 1, 1, 0, 2, 2, 0, 1, 2, 1, 0, 2, 2, 1, 0,
          2, 0, 1, 1, 2, 0, 1, 2, 0, 2, 1, 1, 2, 0, 2, 1, 1, 2, 1, 0, 1, 2,
          2, 1, 1, 0, 0, 0, 0, 2, 1, 0, 0, 1, 2, 0, 1, 1])
```

```
[22]: print("Accuracy:", accuracy_score(y_test, y_test_pred))
```

```
Accuracy: 0.9833333333333333
```

```
[23]: # score: Return the mean accuracy on the given test data and labels.
      clf.score(X_test_norm, y_test)
```

```
[23]: 0.9833333333333333
```

```
[24]: # KNeighborsClassifier.score is doing this
      (y_test_pred == y_test).sum() / len(y_test)
```

```
[24]: 0.9833333333333333
```

Performance evaluation

```
[25]: # print("F1:", f1_score(y_test, y_test_pred))
```

```
[26]: print("F1:", f1_score(y_test, y_test_pred, average="macro"))
```

```
F1: 0.9833229101521784
```

```
[27]: print("F1:", f1_score(y_test, y_test_pred, average="micro"))
```

```
F1: 0.9833333333333333
```

```
[28]: print("F1:", f1_score(y_test, y_test_pred, labels=[1], average="micro"))
```

```
F1: 0.975609756097561
```

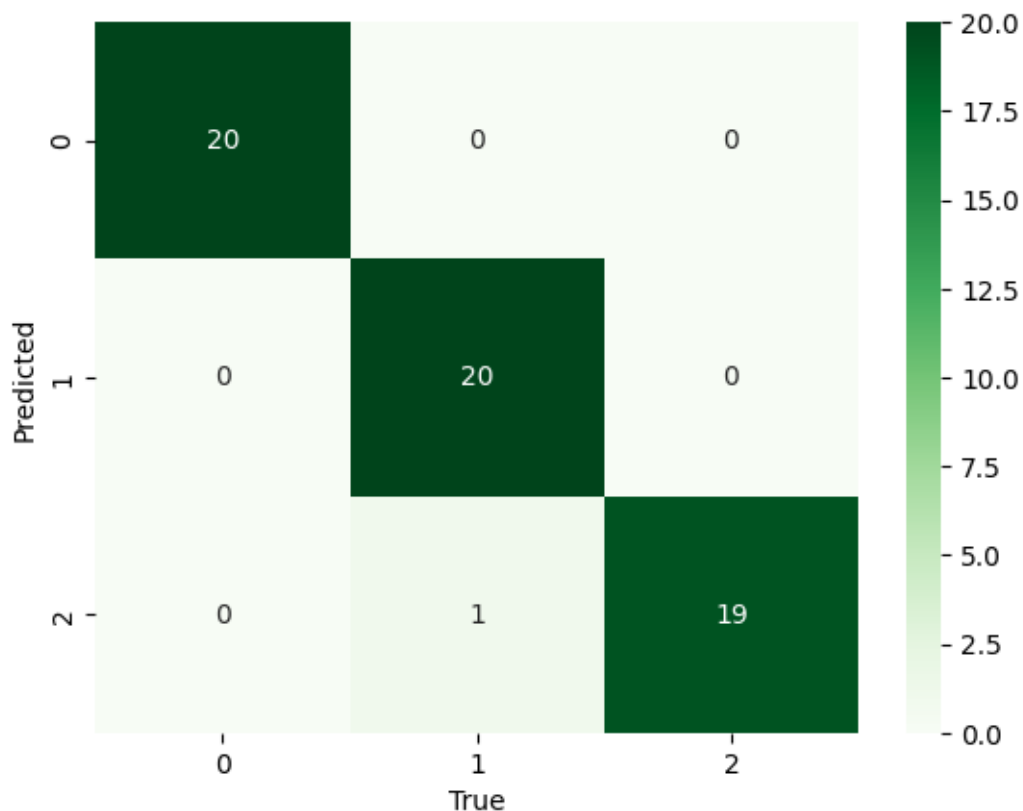
```
[29]: print("F1:", f1_score(y_test, y_test_pred, labels=[1], average="macro"))
```

```
F1: 0.975609756097561
```

```
[30]: print(classification_report(y_test, y_test_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	20
1	0.95	1.00	0.98	20
2	1.00	0.95	0.97	20
accuracy			0.98	60
macro avg	0.98	0.98	0.98	60
weighted avg	0.98	0.98	0.98	60

```
[31]: cf = confusion_matrix(y_test, y_test_pred)
      sns.heatmap(cf, annot=True, cmap="Greens")
      plt.xlabel("True")
      plt.ylabel("Predicted")
      plt.show()
```



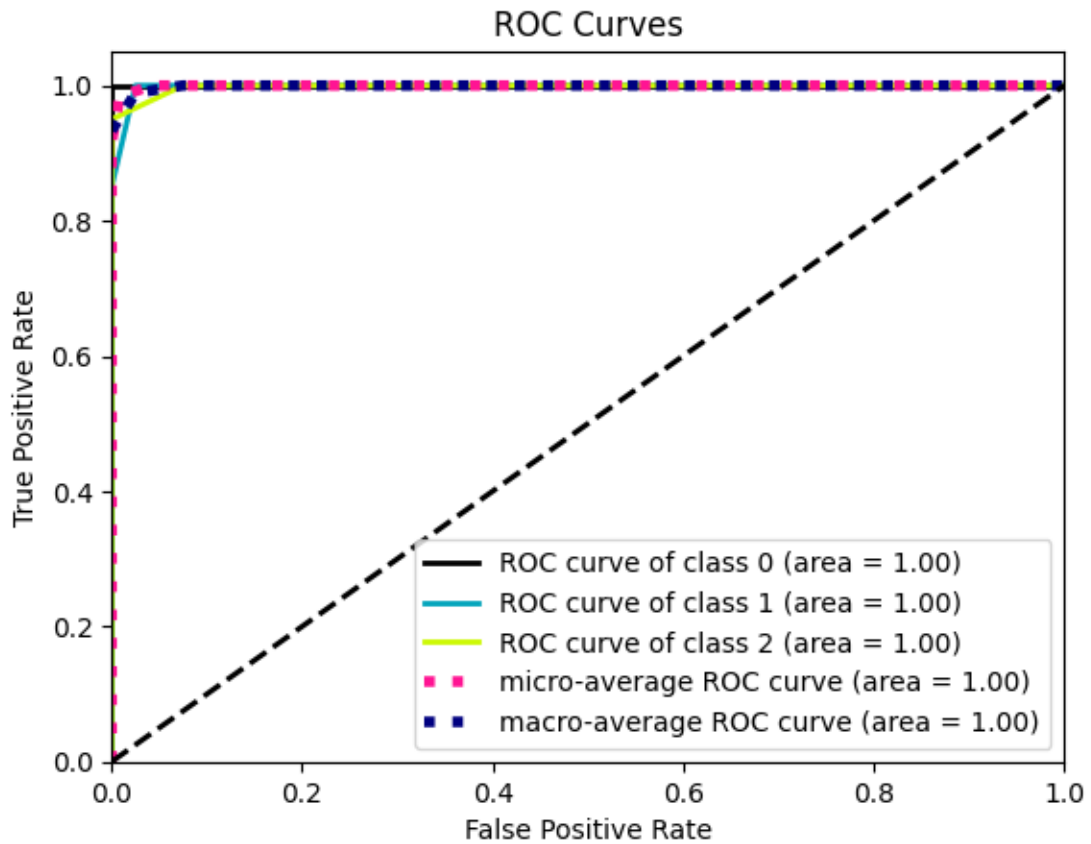
```
[32]: # Return probability estimates for the test data X.
y_test_pred_proba = clf.predict_proba(X_test_norm)
y_test_pred_proba[0:10]
```

```
[32]: array([[0. , 0.2, 0.8],
          [0. , 0. , 1. ],
          [1. , 0. , 0. ],
          [0. , 0. , 1. ],
          [1. , 0. , 0. ],
          [1. , 0. , 0. ],
          [1. , 0. , 0. ],
          [0. , 0. , 1. ],
          [0. , 1. , 0. ],
          [0. , 1. , 0. ]])
```

```
[33]: y_test_pred[0:10]
```

```
[33]: array([2, 2, 0, 2, 0, 0, 0, 2, 1, 1])
```

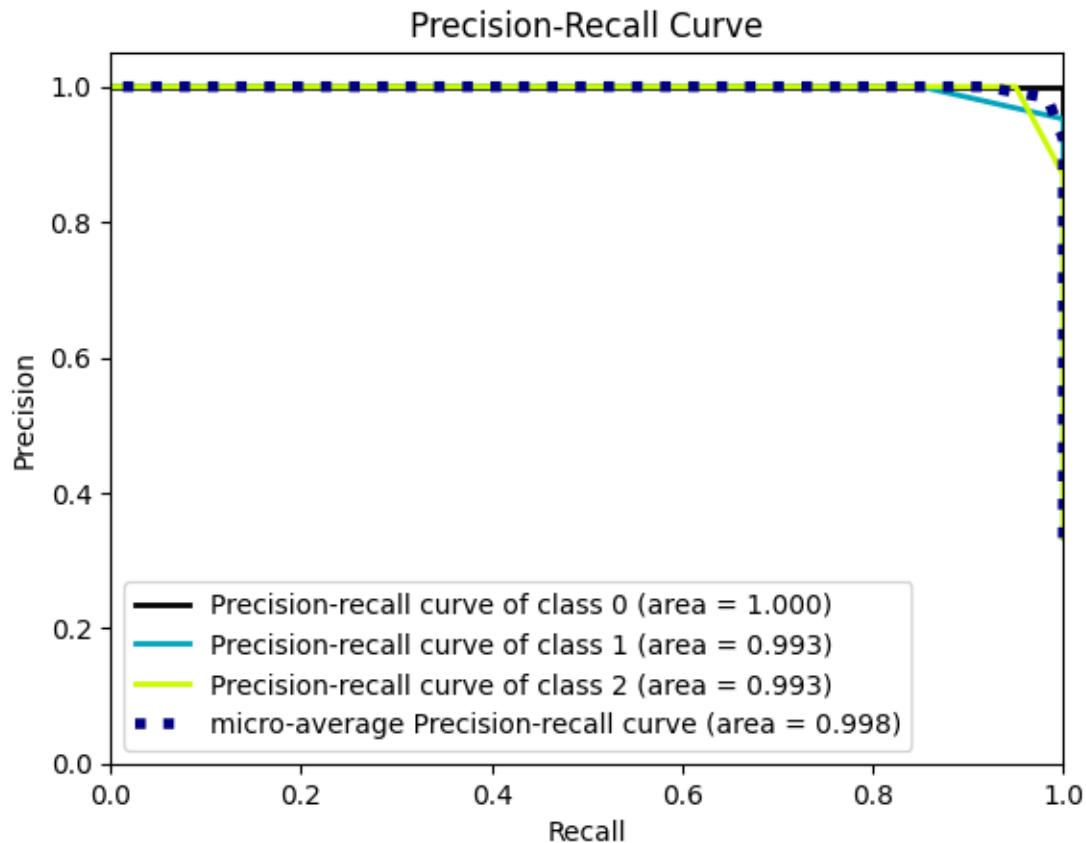
```
[34]: # https://scikit-learn.org/stable/auto\_examples/model\_selection/plot\_roc.html  
plot_roc(y_test, y_test_pred_proba)  
plt.show()
```



```
[35]: # https://scikit-learn.org/stable/modules/generated/sklearn.metrics.  
      ↪roc\_auc\_score.html  
roc_auc_score(y_test, y_test_pred_proba, multi_class="ovr", average="macro")
```

```
[35]: 0.9987499999999999
```

```
[36]: plot_precision_recall(y_test, y_test_pred_proba)  
plt.show()
```



Repeated Holdout

```
[37]: N = 50
      err = 0

      for i in range(N):
          # stratified holdout
          X_rh_train, X_rh_test, y_rh_train, y_rh_test = train_test_split(X, y,
          ↪test_size=0.4, stratify=y)

          # normalize train set
          norm.fit(X_rh_train)
          X_rh_train_norm = norm.transform(X_rh_train)
          X_rh_test_norm = norm.transform(X_rh_test)

          # initialize and fit classifier
          clf = KNeighborsClassifier(n_neighbors=5, metric="euclidean",
          ↪weights="uniform")
          clf.fit(X_rh_train_norm, y_rh_train)
```

```

# computing error
acc = clf.score(X_rh_test_norm, y_rh_test)
err += 1 - acc

print("Overall error estimate:", err/N)

```

Overall error estimate: 0.04866666666666667

[37]:

Cross-validation https://scikit-learn.org/stable/modules/cross_validation.html

```

[38]: from sklearn.model_selection import cross_val_score
k = 10

```

```

[39]: # initialize classifier
clf = KNeighborsClassifier(n_neighbors=5, metric="euclidean", weights="uniform")

scores = cross_val_score(clf, X_train_norm, y_train, cv=k)
scores

```

```

[39]: array([0.88888889, 1.          , 1.          , 1.          , 0.77777778,
           1.          , 0.55555556, 0.88888889, 1.          , 1.          ])

```

```

[40]: print("Overall error estimate:", 1 - scores.mean())

```

Overall error estimate: 0.08888888888888889

```

[41]: print('Accuracy: %0.4f (+/- %0.2f)' % (scores.mean(), scores.std()))

```

Accuracy: 0.9111 (+/- 0.14)

```

[42]: # scoring default is accuracy
cross_val_score(clf, X_train_norm, y_train, cv=k, scoring='f1_macro')

```

```

[42]: array([0.88571429, 1.          , 1.          , 1.          , 0.77777778,
           1.          , 0.55555556, 0.88571429, 1.          , 1.          ])

```

[42]:

0.3.1 Hyperparameters Tuning

```

[43]: n_neighbors = range(1,20)
scores = list()

for n in n_neighbors:

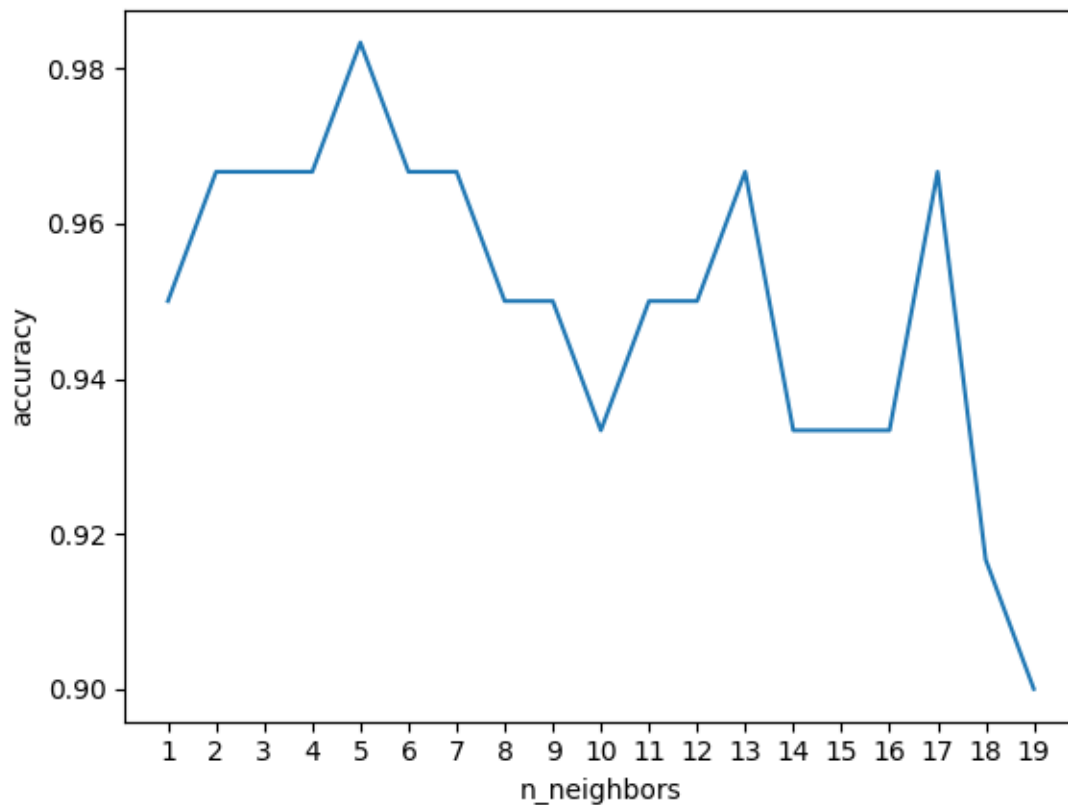
```

```

    clf = KNeighborsClassifier(n_neighbors=n, metric="euclidean",
↪weights="uniform")
    clf.fit(X_train_norm, y_train)
    scores.append(clf.score(X_test_norm, y_test))

plt.plot(scores)
plt.xticks(range(len(n_neighbors)), n_neighbors)
plt.xlabel("n_neighbors")
plt.ylabel("accuracy")
plt.show()

```



```

[44]: n_neighbors = range(1,20)
      avg_scores = list()
      std_scores = list()

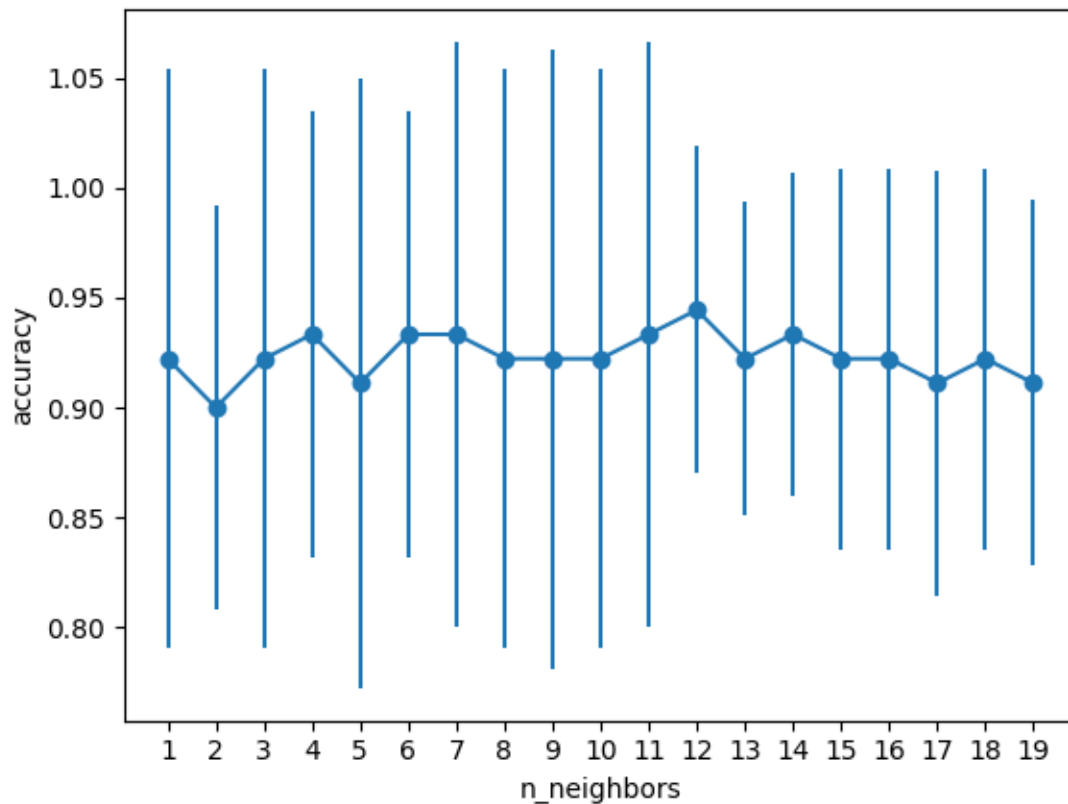
      for n in n_neighbors:
          clf = KNeighborsClassifier(n_neighbors=n, metric="euclidean",
↪weights="uniform")
          scores = cross_val_score(clf, X_train_norm, y_train, cv=k)
          avg_scores.append(np.mean(scores))
          std_scores.append(np.std(scores))

```

```

#plt.plot(avg_scores)
plt.errorbar(range(len(n_neighbors)), y=avg_scores, yerr=std_scores, marker='o')
plt.xticks(range(len(n_neighbors)), n_neighbors)
plt.xlabel("n_neighbors")
plt.ylabel("accuracy")
plt.show()

```



```

[45]: clf = KNeighborsClassifier(n_neighbors=12, metric="euclidean",
    ↪weights="uniform")
clf.fit(X_train_norm, y_train)
y_test_pred = clf.predict(X_test_norm)
print("Accuracy:", accuracy_score(y_test, y_test_pred))

```

Accuracy: 0.95

Grid Search

```

[46]: from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.model_selection import RandomizedSearchCV, GridSearchCV

```



```
[47]: %%time
param_grid = {
    "n_neighbors": (np.arange(1, X_train.shape[0]//2)),
    "weights": ["uniform", "distance"],
    "metric": ["euclidean", "cityblock"],
}

grid = GridSearchCV(
    KNeighborsClassifier(),
    param_grid=param_grid,
    cv=RepeatedStratifiedKFold(random_state=0),
    n_jobs=-1,
    refit=True,
    # verbose=2
)

#param_distributions=param_grid,

grid.fit(X_train_norm, y_train)
clf = grid.best_estimator_
```

CPU times: user 2.72 s, sys: 73.8 ms, total: 2.79 s
Wall time: 41.9 s

```
[48]: print(grid.best_params_, grid.best_score_)

{'metric': 'euclidean', 'n_neighbors': 14, 'weights': 'distance'}
0.9466666666666665
```

```
[49]: y_test_pred = clf.predict(X_test_norm)
print("Accuracy:", accuracy_score(y_test, y_test_pred))
```

Accuracy: 0.9666666666666667

```
[50]: clf.score(X_test_norm, y_test)
```

```
[50]: 0.9666666666666667
```

```
[51]: # grid.cv_results_
```

```
[52]: results = pd.DataFrame(grid.cv_results_)
results
```

```
[52]:
```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	\
0	0.004583	0.003372	0.013991	0.009098	
1	0.003988	0.002896	0.005595	0.002973	
2	0.002931	0.002279	0.009794	0.005777	
3	0.002762	0.001809	0.003857	0.001688	

4	0.002036	0.001417	0.008432	0.004344
..	...	...	...	...
171	0.001093	0.000549	0.001989	0.001017
172	0.001003	0.000344	0.001336	0.000160
173	0.001200	0.000989	0.001999	0.000790
174	0.001051	0.000410	0.001465	0.000550
175	0.001002	0.000256	0.001812	0.000439

	param_metric	param_n_neighbors	param_weights	\
0	euclidean	1	uniform	
1	euclidean	1	distance	
2	euclidean	2	uniform	
3	euclidean	2	distance	
4	euclidean	3	uniform	
..	...	...	...	
171	cityblock	42	distance	
172	cityblock	43	uniform	
173	cityblock	43	distance	
174	cityblock	44	uniform	
175	cityblock	44	distance	

	params	split0_test_score	\
0	{'metric': 'euclidean', 'n_neighbors': 1, 'wei...	0.888889	
1	{'metric': 'euclidean', 'n_neighbors': 1, 'wei...	0.888889	
2	{'metric': 'euclidean', 'n_neighbors': 2, 'wei...	0.888889	
3	{'metric': 'euclidean', 'n_neighbors': 2, 'wei...	0.888889	
4	{'metric': 'euclidean', 'n_neighbors': 3, 'wei...	0.888889	
..	...	...	
171	{'metric': 'cityblock', 'n_neighbors': 42, 'we...	0.833333	
172	{'metric': 'cityblock', 'n_neighbors': 43, 'we...	0.888889	
173	{'metric': 'cityblock', 'n_neighbors': 43, 'we...	0.833333	
174	{'metric': 'cityblock', 'n_neighbors': 44, 'we...	0.833333	
175	{'metric': 'cityblock', 'n_neighbors': 44, 'we...	0.833333	

	split1_test_score	...	split43_test_score	split44_test_score	\
0	0.944444	...	0.944444	0.833333	
1	0.944444	...	0.944444	0.833333	
2	0.888889	...	0.944444	0.888889	
3	0.944444	...	0.944444	0.833333	
4	0.888889	...	0.944444	0.888889	
..	...	...	...	...	
171	0.944444	...	0.944444	0.833333	
172	0.888889	...	0.888889	0.777778	
173	0.944444	...	0.944444	0.833333	
174	0.833333	...	0.888889	0.666667	
175	0.944444	...	0.944444	0.833333	

	split45_test_score	split46_test_score	split47_test_score	\
0	1.000000	0.944444	0.944444	
1	1.000000	0.944444	0.944444	
2	0.888889	0.944444	0.944444	
3	1.000000	0.944444	0.944444	
4	0.944444	0.944444	0.944444	
..	...	...	...	
171	0.944444	0.833333	0.944444	
172	1.000000	0.833333	0.777778	
173	1.000000	0.833333	0.944444	
174	0.888889	0.833333	0.944444	
175	0.944444	0.833333	0.944444	

	split48_test_score	split49_test_score	mean_test_score	std_test_score	\
0	0.833333	0.944444	0.911111	0.060858	
1	0.833333	0.944444	0.911111	0.060858	
2	0.833333	0.944444	0.900000	0.056656	
3	0.833333	0.944444	0.911111	0.060858	
4	0.888889	1.000000	0.925556	0.056229	
..	...	...	...	...	
171	0.944444	0.944444	0.926667	0.063285	
172	0.888889	0.888889	0.897778	0.058119	
173	0.944444	0.944444	0.930000	0.062667	
174	0.888889	0.833333	0.873333	0.069424	
175	0.944444	0.944444	0.927778	0.061111	

	rank_test_score
0	113
1	113
2	140
3	113
4	59
..	...
171	57
172	147
173	34
174	174
175	48

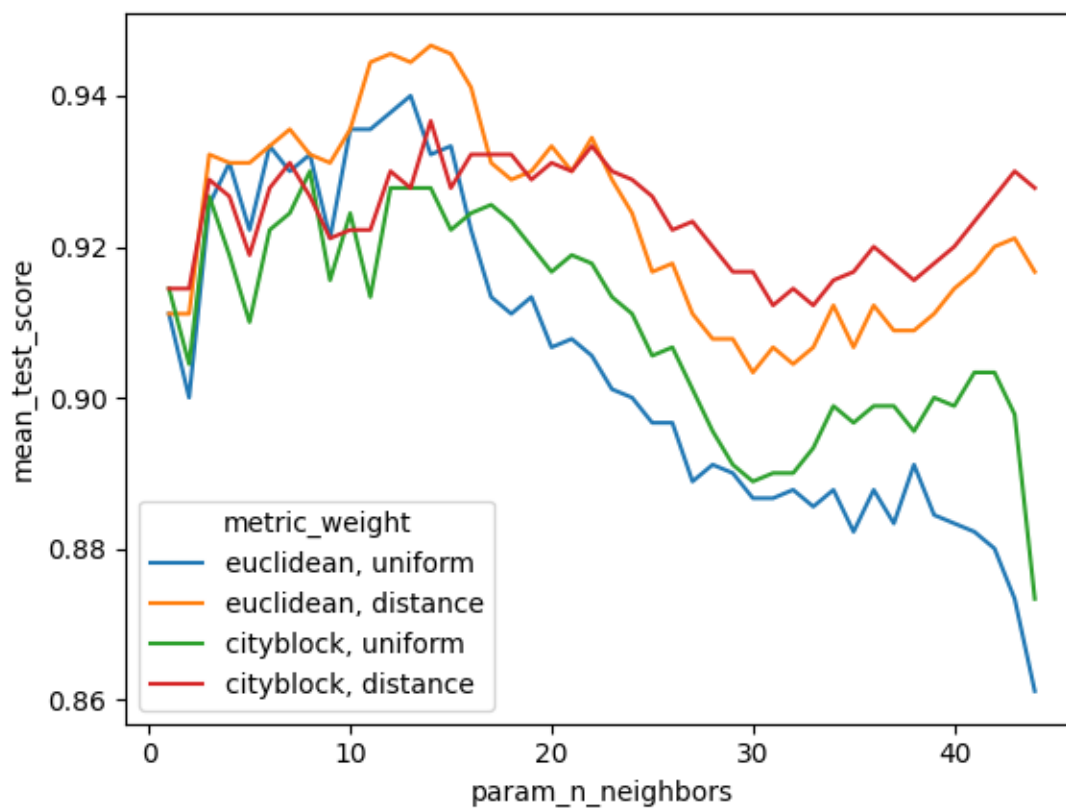
[176 rows x 61 columns]

```
[53]: results["metric_weight"] = results["param_metric"] + ", " +
      ↪results["param_weights"]
```

```
[54]: sns.lineplot(
      data=results, x="param_n_neighbors", y="mean_test_score",
      ↪hue="metric_weight"
```

```
)
```

```
[54]: <Axes: xlabel='param_n_neighbors', ylabel='mean_test_score'>
```



```
[54]:
```

0.3.2 Naive Bayes

```
[55]: from sklearn.naive_bayes import GaussianNB, CategoricalNB
```

Gaussian

```
[56]: clf = GaussianNB()
```

```
[57]: %%time
      clf.fit(X_train, y_train)
```

CPU times: user 2.6 ms, sys: 0 ns, total: 2.6 ms

Wall time: 2.5 ms

```
[57]: GaussianNB()
```

```
[58]: y_pred = clf.predict(X_test)
      y_pred
```

```
[58]: array([2, 2, 0, 2, 0, 0, 0, 2, 1, 1, 0, 2, 2, 0, 1, 2, 1, 0, 2, 2, 1, 0,
           2, 0, 1, 1, 2, 0, 1, 2, 0, 2, 1, 1, 2, 0, 2, 1, 1, 2, 1, 0, 1, 2,
           2, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 2, 0, 1, 2])
```

```
[59]: print(classification_report(y_test, y_pred))
```

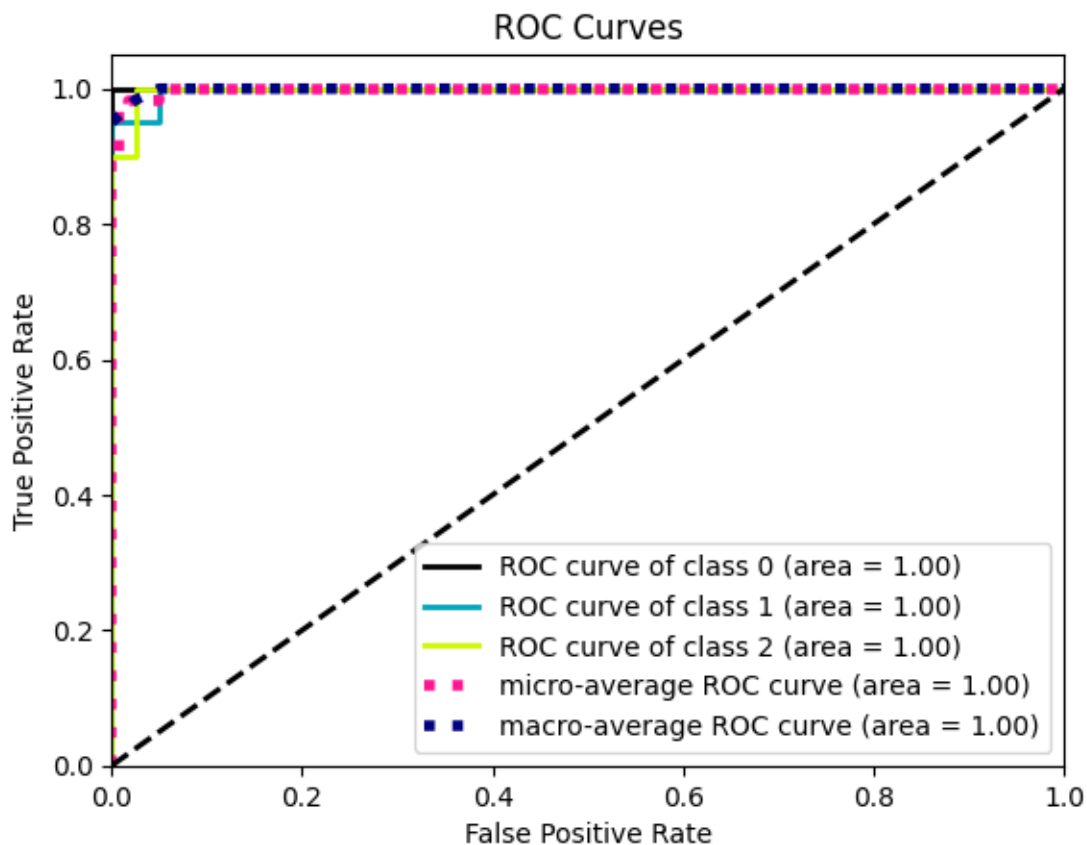
	precision	recall	f1-score	support
0	1.00	1.00	1.00	20
1	0.95	0.95	0.95	20
2	0.95	0.95	0.95	20
accuracy			0.97	60
macro avg	0.97	0.97	0.97	60
weighted avg	0.97	0.97	0.97	60

```
[60]: clf.predict_proba(X_test)
```

```
[60]: array([[3.05055218e-156, 1.98469602e-002, 9.80153040e-001],
             [2.50142195e-277, 3.75932821e-012, 1.00000000e+000],
             [1.00000000e+000, 2.74048199e-017, 2.82825801e-026],
             [3.77047257e-248, 6.87918788e-010, 9.99999999e-001],
             [1.00000000e+000, 1.81924966e-016, 2.52343415e-026],
             [1.00000000e+000, 5.84024273e-017, 8.14494337e-027],
             [1.00000000e+000, 1.12344252e-016, 5.92512274e-026],
             [6.19173845e-176, 4.67941390e-004, 9.99532059e-001],
             [1.10238158e-032, 9.99999857e-001, 1.43256972e-007],
             [7.56032487e-075, 9.99968118e-001, 3.18821921e-005],
             [1.00000000e+000, 1.66546269e-010, 2.54675324e-019],
             [3.73265040e-154, 6.58305961e-002, 9.34169404e-001],
             [0.00000000e+000, 1.70481203e-012, 1.00000000e+000],
             [1.00000000e+000, 6.04531187e-012, 4.37830158e-020],
             [1.35690609e-092, 9.96128400e-001, 3.87159969e-003],
             [7.97250163e-267, 3.29344440e-010, 1.00000000e+000],
             [1.86513942e-098, 9.78842746e-001, 2.11572540e-002],
             [1.00000000e+000, 4.88720961e-019, 3.68247362e-028],
             [3.59336710e-162, 1.86623554e-002, 9.81337645e-001],
             [7.11242895e-161, 3.42090433e-002, 9.65790957e-001],
             [1.60527365e-059, 9.99978974e-001, 2.10262412e-005],
             [1.00000000e+000, 1.64906577e-013, 9.88604933e-023],
             [3.34150260e-200, 1.31058957e-006, 9.99998689e-001],
             [1.00000000e+000, 1.98789412e-017, 5.32896313e-026],
             [6.47625569e-079, 9.99895656e-001, 1.04343789e-004],
```

```
[5.29919058e-075, 9.99975461e-001, 2.45393824e-005],
[1.83093152e-230, 2.54506289e-007, 9.99999745e-001],
[1.00000000e+000, 5.44711014e-017, 2.03232679e-025],
[1.01049595e-112, 9.55075200e-001, 4.49247999e-002],
[5.37442739e-197, 6.84956056e-006, 9.99993150e-001],
[1.00000000e+000, 5.19559717e-017, 5.25698390e-026],
[1.49048596e-231, 1.04650690e-006, 9.9998953e-001],
[9.09721319e-076, 9.99853438e-001, 1.46562275e-004],
[1.18521797e-117, 7.83718534e-001, 2.16281466e-001],
[6.39795698e-170, 4.39554222e-004, 9.99560446e-001],
[1.00000000e+000, 2.48282507e-017, 6.08647478e-027],
[4.53144050e-231, 1.86017095e-008, 9.9999981e-001],
[7.95618521e-107, 9.36508854e-001, 6.34911462e-002],
[9.78193383e-052, 9.9999547e-001, 4.52961673e-007],
[1.14590516e-144, 1.41113424e-001, 8.58886576e-001],
[1.74966107e-113, 7.82539255e-001, 2.17460745e-001],
[1.00000000e+000, 4.34259424e-017, 1.51767006e-026],
[9.46439464e-108, 9.94182260e-001, 5.81774047e-003],
[7.27856849e-186, 2.70725704e-003, 9.97292743e-001],
[2.69165369e-190, 7.44082653e-004, 9.99255917e-001],
[1.93165252e-110, 8.30626172e-001, 1.69373828e-001],
[2.57768066e-078, 9.99852564e-001, 1.47435570e-004],
[1.00000000e+000, 1.44977761e-017, 1.07574272e-026],
[1.00000000e+000, 1.97922847e-016, 2.11881014e-025],
[1.00000000e+000, 8.87084357e-017, 6.08037091e-026],
[1.00000000e+000, 1.63263330e-016, 3.04083318e-026],
[4.03680708e-162, 6.49790168e-001, 3.50209832e-001],
[1.39614388e-087, 9.99941983e-001, 5.80174634e-005],
[1.00000000e+000, 3.80451594e-015, 1.43324210e-023],
[1.00000000e+000, 6.37708578e-014, 7.98165218e-022],
[1.51696008e-096, 9.99487070e-001, 5.12930491e-004],
[7.60262226e-200, 4.10479572e-006, 9.99995895e-001],
[1.00000000e+000, 1.99771588e-017, 2.03445104e-027],
[1.71249185e-067, 9.99995530e-001, 4.46979378e-006],
[4.82476701e-145, 7.05726601e-002, 9.29427340e-001]]])
```

```
[61]: # https://scikit-learn.org/stable/auto\_examples/model\_selection/plot\_roc.html
plot_roc(y_test, clf.predict_proba(X_test))
plt.show()
print(roc_auc_score(y_test, clf.predict_proba(X_test), multi_class="ovr",
↪average="macro"))
```



0.9983333333333334

0.3.3 Wine dataset

These data are the results of a chemical analysis of wines grown in the same region in Italy but derived from 3 different cultivars. The analysis determined the quantities of 13 constituents found in each of the three types of wines.

```
[62]: from sklearn.datasets import load_wine

d = load_wine(as_frame=True)
df_orig = d["data"]
y = np.array(d["target"])

df_orig.head()
```

```
[62]:   alcohol  malic_acid  ash  alcalinity_of_ash  magnesium  total_phenols  \
0    14.23         1.71  2.43              15.6         127.0           2.80
1    13.20         1.78  2.14              11.2          100.0           2.65
2    13.16         2.36  2.67              18.6          101.0           2.80
```

3	14.37	1.95	2.50	16.8	113.0	3.85
4	13.24	2.59	2.87	21.0	118.0	2.80

	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	\
0	3.06	0.28	2.29	5.64	1.04	
1	2.76	0.26	1.28	4.38	1.05	
2	3.24	0.30	2.81	5.68	1.03	
3	3.49	0.24	2.18	7.80	0.86	
4	2.69	0.39	1.82	4.32	1.04	

	od280/od315_of_diluted_wines	proline
0	3.92	1065.0
1	3.40	1050.0
2	3.17	1185.0
3	3.45	1480.0
4	2.93	735.0

```
[63]: # adding a fake categorical variable to see how to deal with it
cat_variable_1_values = ["red", "blue", "green"]
df_orig["color"] = [
    cat_variable_1_values[np.random.randint(0, len(cat_variable_1_values))]
    for _ in range(len(df_orig))
]

# creating an ordinal variable
df_orig["alkalinity_of_ash_binned"] = pd.qcut(
    df_orig["alkalinity_of_ash"], q=4, labels=False
)

X_orig = df_orig.values
df_orig.head()

# one-hot encoding of categorical data
categorical_cols = ["color"]

df = pd.get_dummies(df_orig, columns=categorical_cols)
X = df.values
df.head()
```

	alcohol	malic_acid	ash	alkalinity_of_ash	magnesium	total_phenols	\
0	14.23	1.71	2.43	15.6	127.0	2.80	
1	13.20	1.78	2.14	11.2	100.0	2.65	
2	13.16	2.36	2.67	18.6	101.0	2.80	
3	14.37	1.95	2.50	16.8	113.0	3.85	
4	13.24	2.59	2.87	21.0	118.0	2.80	

	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	\
--	------------	----------------------	-----------------	-----------------	-----	---

0	3.06	0.28	2.29	5.64	1.04
1	2.76	0.26	1.28	4.38	1.05
2	3.24	0.30	2.81	5.68	1.03
3	3.49	0.24	2.18	7.80	0.86
4	2.69	0.39	1.82	4.32	1.04

	od280/od315_of_diluted_wines	proline	alcalinity_of_ash_binned	\
0	3.92	1065.0		0
1	3.40	1050.0		0
2	3.17	1185.0		1
3	3.45	1480.0		0
4	2.93	735.0		2

	color_blue	color_green	color_red
0	False	True	False
1	False	True	False
2	False	False	True
3	False	True	False
4	False	True	False

```
[64]: X_train, X_test, y_train, y_test = train_test_split(
      X, y, test_size=0.3, stratify=y, random_state=random_state
    )
```

```
[65]: print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(124, 17) (54, 17) (124,) (54,)
```

Gaussian NB

```
[66]: clf = GaussianNB()
```

```
[67]: %%time
      clf.fit(X_train, y_train)
```

```
CPU times: user 2.28 ms, sys: 758 µs, total: 3.04 ms
```

```
Wall time: 3.03 ms
```

```
[67]: GaussianNB()
```

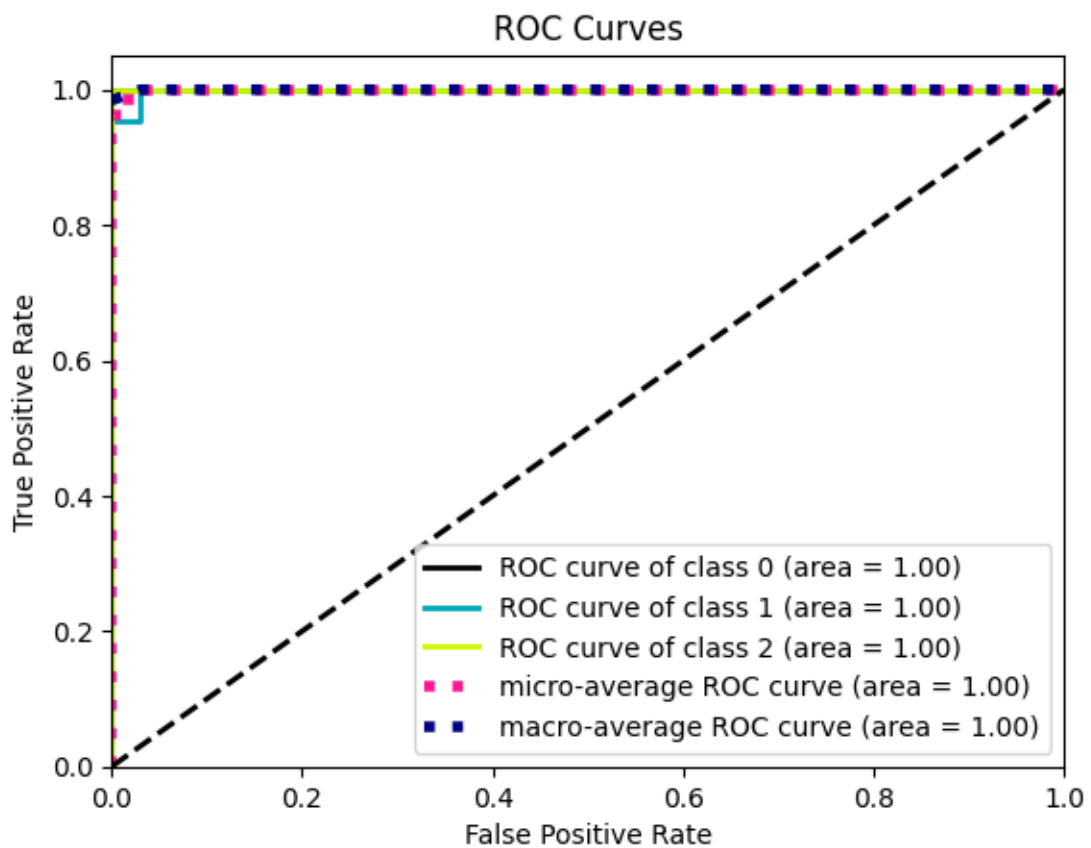
```
[68]: y_pred = clf.predict(X_test)
```

```
[69]: print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	0.94	0.97	18
1	0.95	0.95	0.95	21

	2	0.94	1.00	0.97	15
accuracy				0.96	54
macro avg		0.96	0.97	0.96	54
weighted avg		0.96	0.96	0.96	54

```
[70]: # https://scikit-learn.org/stable/auto\_examples/model\_selection/plot\_roc.html
plot_roc(y_test, clf.predict_proba(X_test))
plt.show()
print(roc_auc_score(y_test, clf.predict_proba(X_test), multi_class="ovr",
↪average="macro"))
```



0.9995189995189996

[70]:

Categorical

```

[71]: non_cat_columns = [
        "alcohol",
        "malic_acid",
        "ash",
        "alcalinity_of_ash",
        "magnesium",
        "total_phenols",
        "flavanoids",
        "nonflavanoid_phenols",
        "proanthocyanins",
        "color_intensity",
        "hue",
        "od280/od315_of_diluted_wines",
        "proline",
    ]

X_noncat = df[non_cat_columns].values

X_train_noncat, X_test_noncat, y_train_noncat, y_test_noncat = train_test_split(
    X_noncat, y, test_size=0.3, stratify=y, random_state=random_state
)

# train and test set should be binned separately
X_train_cat = list()
for column_idx in range(X_train_noncat.shape[1]):
    X_train_cat.append(pd.qcut(X_train_noncat[:, column_idx], q=4,
        labels=False))
X_train_cat = np.array(X_train_cat).T

X_test_cat = list()
for column_idx in range(X_test_noncat.shape[1]):
    X_test_cat.append(pd.qcut(X_test_noncat[:, column_idx], q=4, labels=False))
X_test_cat = np.array(X_test_cat).T

print(X_train_cat.shape, X_test_cat.shape)

```

(124, 13) (54, 13)

```

[72]: X_train_cat

```

```

[72]: array([[2, 3, 1, ..., 1, 1, 1],
        [3, 0, 3, ..., 3, 2, 3],
        [3, 0, 3, ..., 3, 1, 3],
        ...,
        [3, 3, 0, ..., 0, 0, 0],
        [0, 0, 1, ..., 2, 2, 2],
        [1, 3, 2, ..., 3, 2, 0]])

```

```
[73]: clf = CategoricalNB()
      clf.fit(X_train_cat, y_train_noncat)
```

```
[73]: CategoricalNB()
```

```
[74]: X_train_cat
```

```
[74]: array([[2, 3, 1, ..., 1, 1, 1],
          [3, 0, 3, ..., 3, 2, 3],
          [3, 0, 3, ..., 3, 1, 3],
          ...,
          [3, 3, 0, ..., 0, 0, 0],
          [0, 0, 1, ..., 2, 2, 2],
          [1, 3, 2, ..., 3, 2, 0]])
```

```
[75]: y_pred = clf.predict(X_test_cat)
```

```
[76]: print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	18
1	1.00	0.90	0.95	21
2	0.88	1.00	0.94	15
accuracy			0.96	54
macro avg	0.96	0.97	0.96	54
weighted avg	0.97	0.96	0.96	54

1 Binary Classification Example

```
[77]: from scikitplot.metrics import plot_cumulative_gain, plot_lift_curve
      # plot_cumulative_gain and plot_lift_curve only work in a binary classification_
      ↪ case
```

```
[78]: from sklearn.datasets import load_breast_cancer
```

```
[79]: X, y = load_breast_cancer(return_X_y=True)
```

```
[79]:
```

```
[80]: X_train, X_test, y_train, y_test = train_test_split(
      X, y, test_size=0.3, stratify=y, random_state=random_state
      )
```

```
[81]: clf = GaussianNB()
```

```
[82]: clf.fit(X_train, y_train)
```

```
[82]: GaussianNB()
```

```
[83]: y_test_pred_proba = clf.predict_proba(X_test)
      y_test_pred_proba
```

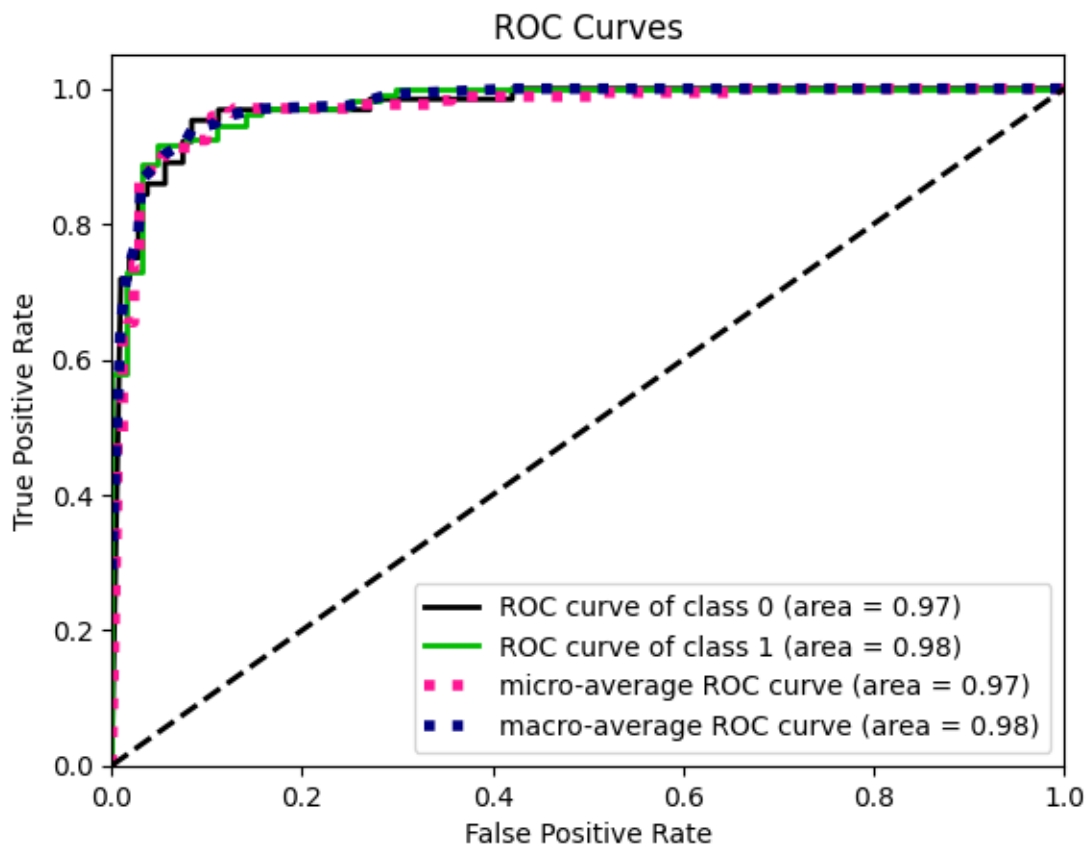
```
[83]: array([[1.00000000e+000, 9.60798304e-067],
             [3.28452382e-012, 1.00000000e+000],
             [1.00000000e+000, 1.37358518e-029],
             [9.07175634e-001, 9.28243663e-002],
             [1.00000000e+000, 1.13066573e-026],
             [4.86470493e-011, 1.00000000e+000],
             [1.00000000e+000, 1.85354720e-229],
             [1.00000000e+000, 2.24061507e-034],
             [1.00000000e+000, 7.64991022e-020],
             [1.00000000e+000, 5.98723466e-025],
             [1.55676568e-010, 1.00000000e+000],
             [4.87665583e-012, 1.00000000e+000],
             [2.51652415e-014, 1.00000000e+000],
             [3.35003877e-015, 1.00000000e+000],
             [1.00000000e+000, 7.27215795e-019],
             [1.00000000e+000, 5.01138848e-047],
             [1.00000000e+000, 3.84869046e-038],
             [5.11581566e-005, 9.99948842e-001],
             [1.00000000e+000, 5.99928618e-030],
             [1.00000000e+000, 8.93234543e-263],
             [9.23899517e-016, 1.00000000e+000],
             [1.22947094e-013, 1.00000000e+000],
             [4.60035794e-013, 1.00000000e+000],
             [1.00000000e+000, 2.83284900e-302],
             [1.00000000e+000, 1.92821921e-054],
             [4.94411999e-005, 9.99950559e-001],
             [6.63793002e-012, 1.00000000e+000],
             [1.00000000e+000, 4.93986527e-097],
             [5.84717888e-014, 1.00000000e+000],
             [1.00000000e+000, 1.59716799e-098],
             [8.87809741e-001, 1.12190259e-001],
             [4.04806236e-013, 1.00000000e+000],
             [9.99999683e-001, 3.16530138e-007],
             [3.93236381e-014, 1.00000000e+000],
             [1.25847936e-014, 1.00000000e+000],
             [1.00000000e+000, 1.61535881e-014],
             [6.30346164e-008, 9.99999937e-001],
             [1.00000000e+000, 9.79137325e-237],
```

[3.98832803e-019, 1.00000000e+000],
[9.96202524e-001, 3.79747602e-003],
[6.37533647e-001, 3.62466353e-001],
[1.00000000e+000, 3.47386136e-073],
[8.65947829e-011, 1.00000000e+000],
[1.00000000e+000, 2.16768230e-012],
[4.79603200e-010, 1.00000000e+000],
[1.00000000e+000, 8.70229084e-065],
[1.00000000e+000, 1.49902651e-028],
[2.13772341e-016, 1.00000000e+000],
[2.42737760e-019, 1.00000000e+000],
[1.27400030e-016, 1.00000000e+000],
[4.08428604e-010, 1.00000000e+000],
[1.00000000e+000, 3.06395512e-201],
[6.37932236e-018, 1.00000000e+000],
[1.00000000e+000, 8.43935225e-045],
[2.87486870e-005, 9.99971251e-001],
[1.68740960e-015, 1.00000000e+000],
[9.79395040e-001, 2.06049605e-002],
[6.30316863e-008, 9.9999937e-001],
[1.95612618e-014, 1.00000000e+000],
[2.47412098e-014, 1.00000000e+000],
[6.41925512e-007, 9.9999358e-001],
[8.87706008e-015, 1.00000000e+000],
[1.00000000e+000, 7.27668630e-020],
[1.00000000e+000, 5.75617721e-022],
[1.00000000e+000, 4.83161832e-017],
[9.99571463e-001, 4.28537207e-004],
[1.00000000e+000, 2.76760752e-014],
[1.00000000e+000, 5.84136695e-100],
[1.00000000e+000, 4.19305275e-026],
[9.96580698e-019, 1.00000000e+000],
[7.84186796e-020, 1.00000000e+000],
[1.00000000e+000, 1.62440933e-055],
[5.11529722e-017, 1.00000000e+000],
[6.10023118e-010, 9.9999999e-001],
[3.53918511e-010, 1.00000000e+000],
[3.89701907e-014, 1.00000000e+000],
[2.76850277e-010, 1.00000000e+000],
[9.99999984e-001, 1.58812530e-008],
[1.00000000e+000, 6.62179158e-015],
[6.07278434e-009, 9.9999994e-001],
[1.86299027e-013, 1.00000000e+000],
[2.57800545e-015, 1.00000000e+000],
[8.67048632e-009, 9.9999991e-001],
[1.00000000e+000, 8.20268595e-021],
[3.06330200e-001, 6.93669800e-001],

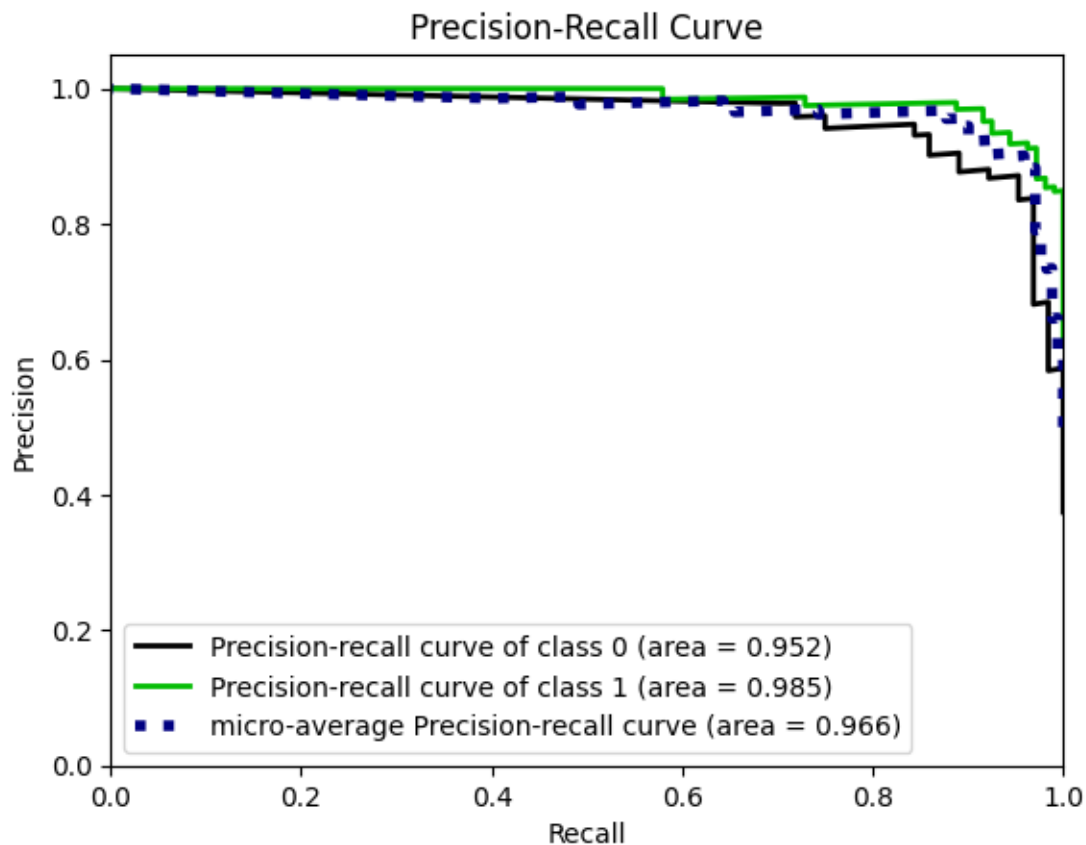
[2.76904635e-012, 1.00000000e+000],
 [1.00000000e+000, 1.07090316e-030],
 [1.00000000e+000, 5.88202001e-025],
 [1.00000000e+000, 1.12952567e-049],
 [1.00000000e+000, 1.37101315e-090],
 [1.00000000e+000, 5.06841370e-012],
 [6.87390643e-004, 9.99312609e-001],
 [2.32024148e-014, 1.00000000e+000],
 [7.98351890e-007, 9.99999202e-001],
 [2.45345747e-010, 1.00000000e+000],
 [1.00000000e+000, 4.96544130e-047],
 [1.00000000e+000, 2.73594696e-026],
 [3.34701403e-008, 9.9999967e-001],
 [1.00000000e+000, 2.08754217e-057],
 [9.99999998e-001, 2.43932083e-009],
 [9.64990534e-003, 9.90350095e-001],
 [8.02821659e-014, 1.00000000e+000],
 [2.26325644e-011, 1.00000000e+000],
 [1.00000000e+000, 2.69374767e-011],
 [7.90120703e-012, 1.00000000e+000],
 [1.13204641e-002, 9.88679536e-001],
 [4.93482495e-011, 1.00000000e+000],
 [9.99999878e-001, 1.22307191e-007],
 [4.50662369e-012, 1.00000000e+000],
 [8.25753085e-011, 1.00000000e+000],
 [3.07725427e-015, 1.00000000e+000],
 [1.00000000e+000, 2.85640504e-044],
 [2.51559739e-002, 9.74844026e-001],
 [4.61383390e-011, 1.00000000e+000],
 [1.82186521e-016, 1.00000000e+000],
 [5.28308302e-006, 9.99994717e-001],
 [1.65228830e-011, 1.00000000e+000],
 [1.00000000e+000, 4.56498688e-022],
 [1.23132066e-017, 1.00000000e+000],
 [1.22800876e-015, 1.00000000e+000],
 [3.56992997e-014, 1.00000000e+000],
 [2.03488382e-013, 1.00000000e+000],
 [2.81915044e-016, 1.00000000e+000],
 [2.36902189e-019, 1.00000000e+000],
 [9.99995090e-001, 4.91045622e-006],
 [4.83312761e-013, 1.00000000e+000],
 [2.31509954e-014, 1.00000000e+000],
 [9.32648265e-014, 1.00000000e+000],
 [7.72754547e-016, 1.00000000e+000],
 [8.43325608e-015, 1.00000000e+000],
 [1.25510491e-017, 1.00000000e+000],
 [1.29458768e-012, 1.00000000e+000],

```
[6.16172274e-016, 1.00000000e+000],
[5.64369777e-012, 1.00000000e+000],
[1.00000000e+000, 2.20873654e-061],
[7.71743562e-014, 1.00000000e+000],
[2.00824407e-011, 1.00000000e+000],
[2.77403674e-001, 7.22596326e-001],
[1.00000000e+000, 2.19534740e-044],
[1.00000000e+000, 2.50277461e-043],
[6.29379257e-018, 1.00000000e+000],
[1.00000000e+000, 2.98256222e-110],
[1.06730721e-019, 1.00000000e+000],
[1.00000000e+000, 2.50446769e-112],
[6.39069091e-013, 1.00000000e+000],
[1.00000000e+000, 1.47862500e-054],
[5.65173547e-018, 1.00000000e+000],
[8.37303002e-016, 1.00000000e+000],
[5.84731461e-015, 1.00000000e+000],
[2.86749633e-015, 1.00000000e+000],
[7.62722007e-016, 1.00000000e+000],
[7.16277436e-015, 1.00000000e+000],
[6.90868938e-014, 1.00000000e+000],
[5.36775578e-006, 9.99994632e-001],
[1.00000000e+000, 1.09686447e-082],
[2.21035391e-015, 1.00000000e+000],
[5.08074190e-004, 9.99491926e-001],
[8.51190961e-018, 1.00000000e+000],
[1.30925836e-016, 1.00000000e+000],
[1.24317330e-009, 9.99999999e-001],
[1.00000000e+000, 2.47391080e-111],
[1.87794810e-016, 1.00000000e+000],
[2.36997051e-011, 1.00000000e+000],
[8.62804098e-014, 1.00000000e+000],
[5.10489927e-003, 9.94895101e-001],
[2.25348614e-017, 1.00000000e+000],
[5.95648811e-011, 1.00000000e+000],
[7.42836733e-003, 9.92571633e-001],
[2.03031410e-011, 1.00000000e+000],
[2.39826926e-012, 1.00000000e+000],
[1.02708885e-022, 1.00000000e+000]]])
```

```
[84]: plot_roc(y_test,y_test_pred_proba)
plt.show()
```

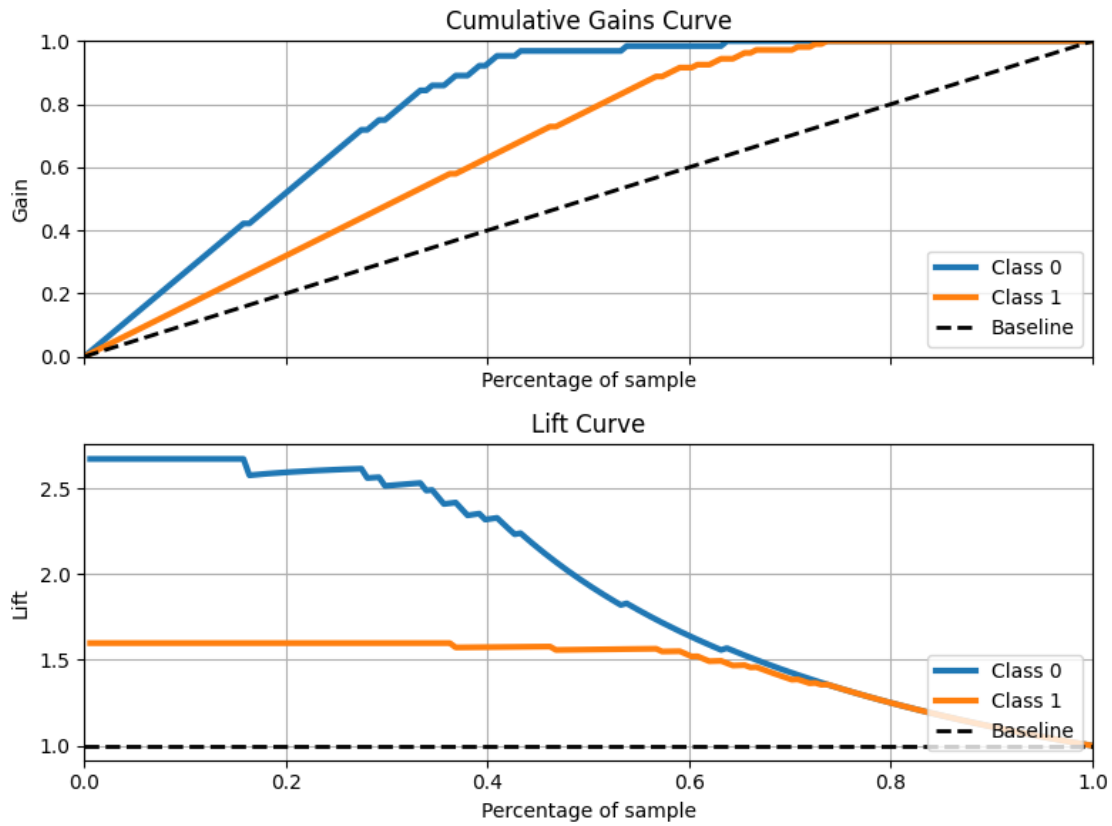



```
[85]: plot_precision_recall(y_test, y_test_pred_proba)
plt.show()
```



For binary classification tasks you can also try to use cumulative gains curve and lift curve as implemented here

```
[86]: fig, axs = plt.subplots(2, 1, sharex=True, figsize=(8,6))
      plot_cumulative_gain(y_test, y_test_pred_proba, ax=axs[0])
      plot_lift_curve(y_test, y_test_pred_proba, ax=axs[1])
      plt.tight_layout()
      plt.show()
```



[86] :

[86] :

Classification_Decision_Tree

November 22, 2025

```
[1]: # In COLAB, we will now use sklearn.metrics for plotting ROC and
      ↳ Precision-Recall curves instead of scikit-plot for compatibility.
      # No specific installations are needed for scikit-learn's plotting functions as
      ↳ it's typically pre-installed.
```

```
[2]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

Titanic Dataset

```
[3]: df = pd.read_csv("data/titanic.csv")
df.head()
```

```
[3]: PassengerId  Survived  Pclass  \
0             1         0         3
1             2         1         1
2             3         1         3
3             4         1         1
4             5         0         3
```

```

                                Name      Sex  Age  SibSp  \
0                        Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                        Heikkinen, Miss. Laina    female  26.0      0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)    female  35.0      1
4                        Allen, Mr. William Henry    male  35.0      0
```

```

Parch      Ticket      Fare Cabin Embarked
0      0      A/5 21171   7.2500   NaN      S
1      0      PC 17599  71.2833   C85      C
2      0  STON/O2. 3101282   7.9250   NaN      S
3      0      113803  53.1000  C123      S
4      0      373450   8.0500   NaN      S
```

Data Cleaning

```
[4]: df.isnull().sum()
```

```
[4]: PassengerId      0
      Survived        0
      Pclass         0
      Name           0
      Sex            0
      Age            177
      SibSp          0
      Parch          0
      Ticket         0
      Fare           0
      Cabin         687
      Embarked       2
      dtype: int64
```

```
[5]: df['Age'] = df.groupby(['Sex', 'Pclass'])['Age'].transform(lambda x: x.fillna(x.
      ↪median()))
```

```
[6]: df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
```

Removing useless variables

```
[7]: column2drop = ['PassengerId', 'Name', 'Cabin']
      df.drop(column2drop, axis=1, inplace=True)
```

```
[8]: df.head()
```

```
[8]:   Survived  Pclass   Sex   Age  SibSp  Parch            Ticket     Fare \
0         0      3  male  22.0     1     0        A/5 21171     7.2500
1         1      1 female  38.0     1     0         PC 17599    71.2833
2         1      3 female  26.0     0     0  STON/O2. 3101282     7.9250
3         1      1 female  35.0     1     0         113803    53.1000
4         0      3  male  35.0     0     0        373450     8.0500
```

```
      Embarked
0           S
1           C
2           S
3           S
4           S
```

0.0.1 Feature Engineering

```
[9]: # number of people a passanger is travelling with
      df['FamilySize'] = df['SibSp'] + df['Parch'] + 1
```

```
[10]: # if he/she is travelling alone or not
df['IsAlone'] = 1 # initialize to yes/1 is alone
df.loc[df.FamilySize > 1, 'IsAlone'] = 0 # update to no/0 if family size is
↳ greater than 1
```

Features SibSp and Parch are now redundant with respect to FamilySize.

```
[11]: column2drop = ['SibSp', 'Parch']
df.drop(column2drop, axis=1, inplace=True)
```

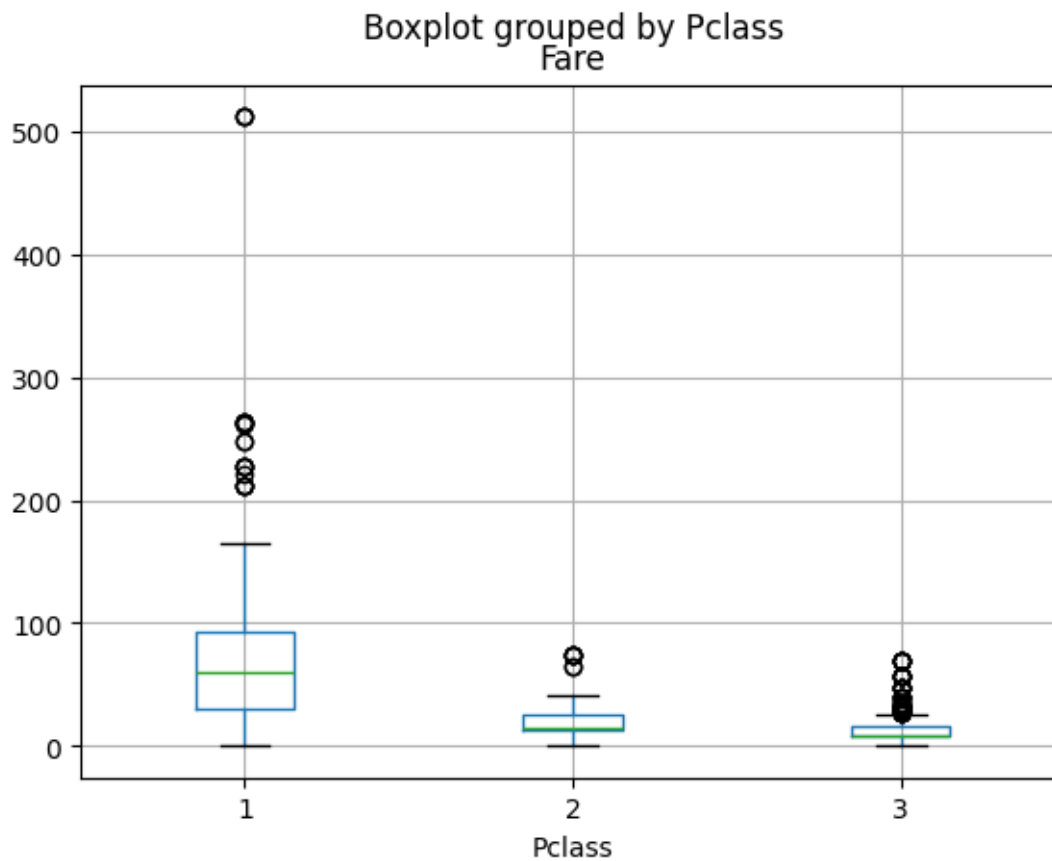
```
[12]: df.head()
```

```
[12]:
```

	Survived	Pclass	Sex	Age	Ticket	Fare	Embarked	\
0	0	3	male	22.0	A/5 21171	7.2500	S	
1	1	1	female	38.0	PC 17599	71.2833	C	
2	1	3	female	26.0	STON/O2. 3101282	7.9250	S	
3	1	1	female	35.0	113803	53.1000	S	
4	0	3	male	35.0	373450	8.0500	S	

	FamilySize	IsAlone
0	2	0
1	2	0
2	1	1
3	2	0
4	1	1

```
[13]: # examining Fare feature
df.boxplot(['Fare'], by='Pclass', showfliers=True)
plt.show()
```



```
[14]: dupl_df = df[df.duplicated('Ticket', keep=False)]
      dupl_df
```

```
[14]:
```

	Survived	Pclass	Sex	Age	Ticket	Fare	Embarked	FamilySize	\
3	1	1	female	35.0	113803	53.1000	S	2	
7	0	3	male	2.0	349909	21.0750	S	5	
8	1	3	female	27.0	347742	11.1333	S	3	
9	1	2	female	14.0	237736	30.0708	C	2	
10	1	3	female	4.0	PP 9549	16.7000	S	3	
..	...	...	...	...	...	...	...		
876	0	3	male	20.0	7534	9.8458	S	1	
879	1	1	female	56.0	11767	83.1583	C	2	
880	1	2	female	25.0	230433	26.0000	S	2	
885	0	3	female	39.0	382652	29.1250	Q	6	
888	0	3	female	21.5	W./C. 6607	23.4500	S	4	

	IsAlone
3	0
7	0

```

8          0
9          0
10         0
..         ...
876        1
879        0
880        0
885        0
888        0

```

[344 rows x 9 columns]

```
[15]: std = dupl_df[['Fare', 'Ticket']].groupby(['Ticket']).std()
std
```

```
[15]:
Ticket
110152    0.0
110413    0.0
110465    0.0
111361    0.0
113505    0.0
...
SC/Paris 2123    0.0
STON/O2. 3101279    0.0
W./C. 6607    0.0
W./C. 6608    0.0
WE/P 5735    0.0

```

[134 rows x 1 columns]

```
[16]: std[std['Fare'] > 0]
```

```
[16]:
Ticket
7534    0.444841

```

```
[17]: df[df['Ticket'] == '7534']
```

```
[17]:
Survived  Pclass  Sex  Age  Ticket  Fare  Embarked  FamilySize  IsAlone
138      0      3  male  16.0  7534  9.2167      S      1      1
876      0      3  male  20.0  7534  9.8458      S      1      1

```

```
[18]: df[df['Ticket'] == '110152']
```

```
[18]:
Survived  Pclass  Sex  Age  Ticket  Fare  Embarked  FamilySize  \
257      1      1  female  30.0  110152  86.5      S      1

```


504	1	1	female	16.0	110152	86.5	S	1
759	1	1	female	33.0	110152	86.5	S	1

IsAlone	
257	1
504	1
759	1

It looks like Fare is cumulative with respect to various Tickets with the same identifier. Consequently we proceed in correcting the Fare of each passenger by dividing it with the number of Tickets with the same identifier.

```
[19]: ticket_map = dict(df['Ticket'].value_counts()) # n. of occurrences by ticket
      df['NumTickets'] = df['Ticket'].map(ticket_map) # creates new column based on
      ↪ the value counts map
```

```
[20]: df['Fare'] = df['Fare']/df['NumTickets']
```

```
[21]: df[df['Ticket'] == '110152']
```

```
[21]:
```

	Survived	Pclass	Sex	Age	Ticket	Fare	Embarked	FamilySize	\
257	1	1	female	30.0	110152	28.833333	S	1	
504	1	1	female	16.0	110152	28.833333	S	1	
759	1	1	female	33.0	110152	28.833333	S	1	

	IsAlone	NumTickets
257	1	3
504	1	3
759	1	3

The features NumTickets and Ticket helped in correcting Fare and we do not need them any more.

```
[22]: column2drop = ['NumTickets', 'Ticket']
      df.drop(column2drop, axis=1, inplace=True)
```

```
[23]: df.head()
```

```
[23]:
```

	Survived	Pclass	Sex	Age	Fare	Embarked	FamilySize	IsAlone
0	0	3	male	22.0	7.2500	S	2	0
1	1	1	female	38.0	71.2833	C	2	0
2	1	3	female	26.0	7.9250	S	1	1
3	1	1	female	35.0	26.5500	S	2	0
4	0	3	male	35.0	8.0500	S	1	1

```
[24]: from sklearn.preprocessing import LabelEncoder
```

```
[25]: # label encoding Sex feature
      le = LabelEncoder()
```

```
df['Sex'] = le.fit_transform(df['Sex'])
```

```
[26]: # one-hot encoding of categorical data
categorical_cols = ["Embarked"]
df = pd.get_dummies(df, columns=categorical_cols, dtype=int)
```

```
[27]: df.head()
```

```
[27]:
```

	Survived	Pclass	Sex	Age	Fare	FamilySize	IsAlone	Embarked_C	\
0	0	3	1	22.0	7.2500	2	0	0	
1	1	1	0	38.0	71.2833	2	0	1	
2	1	3	0	26.0	7.9250	1	1	0	
3	1	1	0	35.0	26.5500	2	0	0	
4	0	3	1	35.0	8.0500	1	1	0	

	Embarked_Q	Embarked_S
0	0	1
1	0	0
2	0	1
3	0	1
4	0	1

0.0.2 Partitioning

```
[28]: from sklearn.model_selection import train_test_split
```

```
[29]: attributes = [col for col in df.columns if col != 'Survived']
X = df[attributes].values
y = df['Survived']
```

```
[30]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
↳ random_state=1, stratify=y)
```

0.0.3 Decision Tree

```
[31]: from sklearn.tree import DecisionTreeClassifier, plot_tree
```

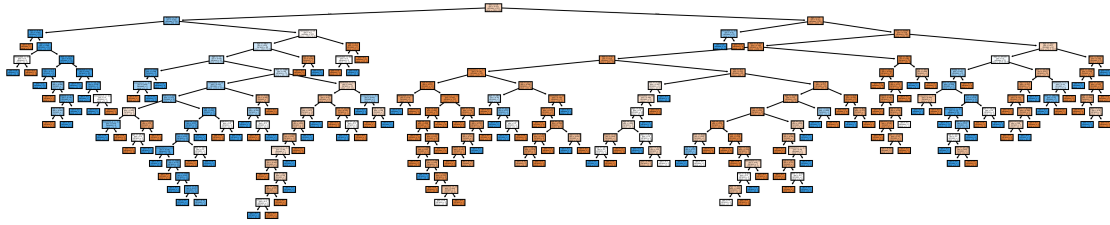
```
[32]: dt = DecisionTreeClassifier()
```

```
[33]: %%time
dt.fit(X_train, y_train)
```

```
CPU times: user 6.3 ms, sys: 0 ns, total: 6.3 ms
Wall time: 10.1 ms
```

```
[33]: DecisionTreeClassifier()
```

```
[34]: plt.figure(figsize=(20, 4), dpi=300)
      plot_tree(dt, feature_names=attributes, filled=True)
      plt.show()
```



```
[35]: from sklearn.metrics import (
      accuracy_score,
      f1_score,
      classification_report,
      confusion_matrix,
      roc_auc_score,
      )
```

```
[36]: y_train_pred = dt.predict(X_train)
      y_test_pred = dt.predict(X_test)
```

```
[37]: print('Train Accuracy %s' % accuracy_score(y_train, y_train_pred))
      print('Train F1-score %s' % f1_score(y_train, y_train_pred, average=None))
      print()

      print('Test Accuracy %s' % accuracy_score(y_test, y_test_pred))
      print('Test F1-score %s' % f1_score(y_test, y_test_pred, average=None))
```

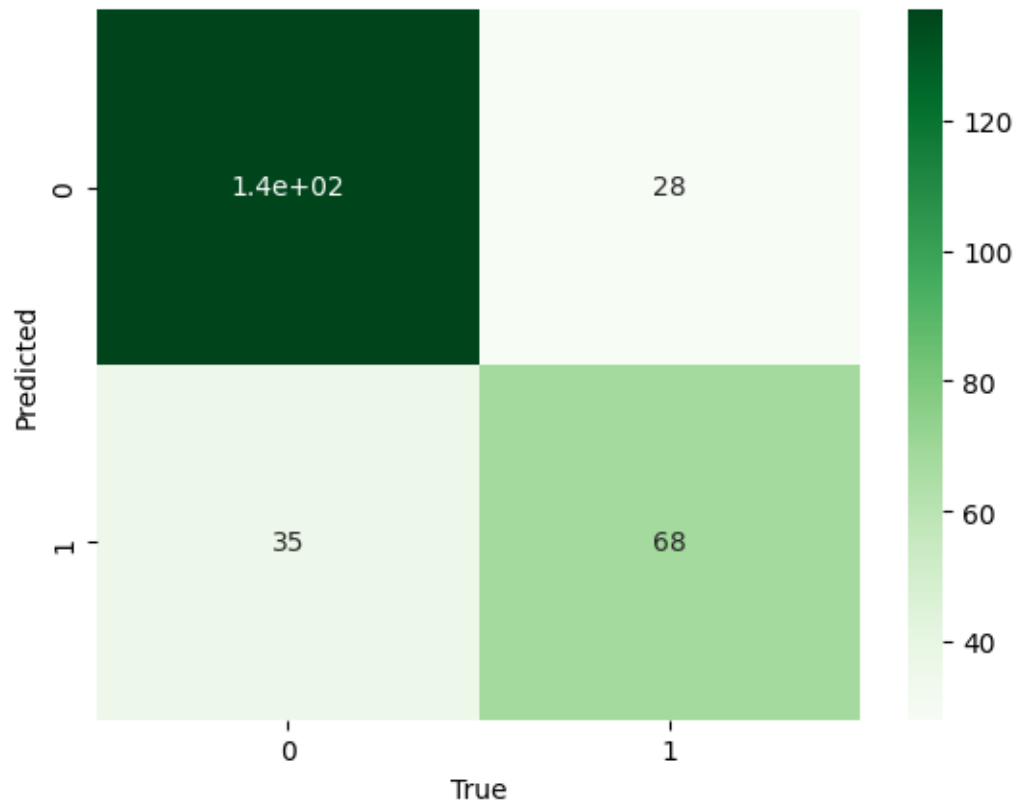
Train Accuracy 0.9839486356340289
 Train F1-score [0.9871134 0.9787234]

Test Accuracy 0.7649253731343284
 Test F1-score [0.81305638 0.68341709]

```
[38]: print(classification_report(y_test, y_test_pred))
```

	precision	recall	f1-score	support
0	0.80	0.83	0.81	165
1	0.71	0.66	0.68	103
accuracy			0.76	268
macro avg	0.75	0.75	0.75	268
weighted avg	0.76	0.76	0.76	268

```
[39]: cf = confusion_matrix(y_test, y_test_pred)
sns.heatmap(cf, annot=True, cmap="Greens")
plt.xlabel("True")
plt.ylabel("Predicted")
plt.show()
```



Feature Importance

```
[40]: zipped = zip(attributes, dt.feature_importances_)
zipped = sorted(zipped, key=lambda x: x[1], reverse=True)
for col, imp in zipped:
    print(col, imp)
```

```
Sex 0.2925058533225051
Fare 0.23954770251754395
Age 0.21692664424863126
FamilySize 0.10357080023756302
Pclass 0.10125791393654211
Embarked_C 0.015174506631692064
Embarked_S 0.01395165145656919
```

```
Embarked_Q 0.009541782631775114
IsAlone 0.007523145017178179
```

Let us discuss overfitting

```
[41]: nodes = []
      train_err = []
      test_err = []

      for max_depth in list(np.arange(1, 20)):
          dt = DecisionTreeClassifier(max_depth=max_depth)
          dt.fit(X_train, y_train)
          y_train_pred = dt.predict(X_train)
          y_test_pred = dt.predict(X_test)

          train_err.append(1 - accuracy_score(y_train, y_train_pred))
          test_err.append(1 - accuracy_score(y_test, y_test_pred))
          nodes.append(dt.tree_.node_count)
```

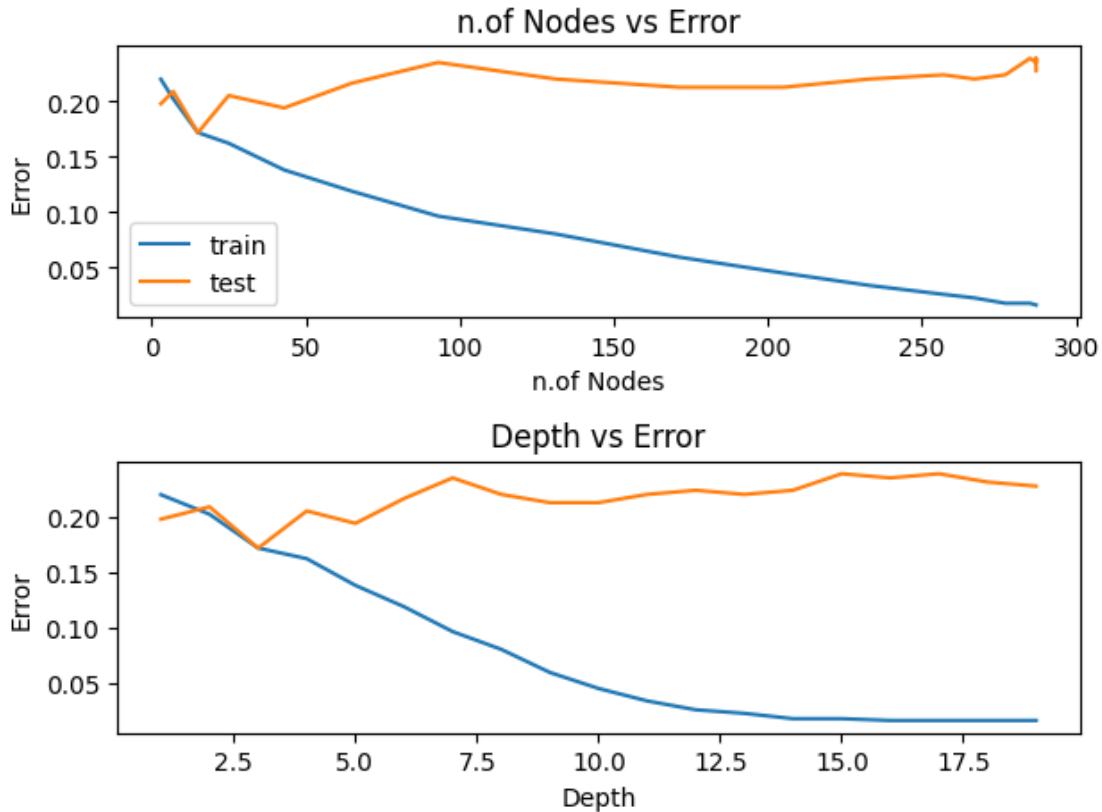
```
[42]: fig, ax = plt.subplots(2,1)
      ax[0].plot(nodes, train_err, label='train')
      ax[0].plot(nodes, test_err, label='test')
      ax[0].legend()

      ax[0].set_xlabel("n.of Nodes")
      ax[0].set_ylabel("Error")
      ax[0].set_title("n.of Nodes vs Error")

      ax[1].plot(list(np.arange(1, 20)), train_err, label='train')
      ax[1].plot(list(np.arange(1, 20)), test_err, label='test')

      ax[1].set_xlabel("Depth")
      ax[1].set_ylabel("Error")
      ax[1].set_title("Depth vs Error")

      plt.tight_layout()
      plt.show()
```



[42] :

Decision Tree hyperparameters `max_depth` : int, default=None The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than `min_samples_split` samples.

`min_samples_split` : int or float, default=2 The minimum number of samples required to split an internal node:

- If int, then consider ``min_samples_split`` as the minimum number.
- If float, then ``min_samples_split`` is a fraction and ``ceil(min_samples_split * n_samples)`` are the minimum number of samples for each split.

`min_samples_leaf` : int or float, default=1 The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least `min_samples_leaf` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression.

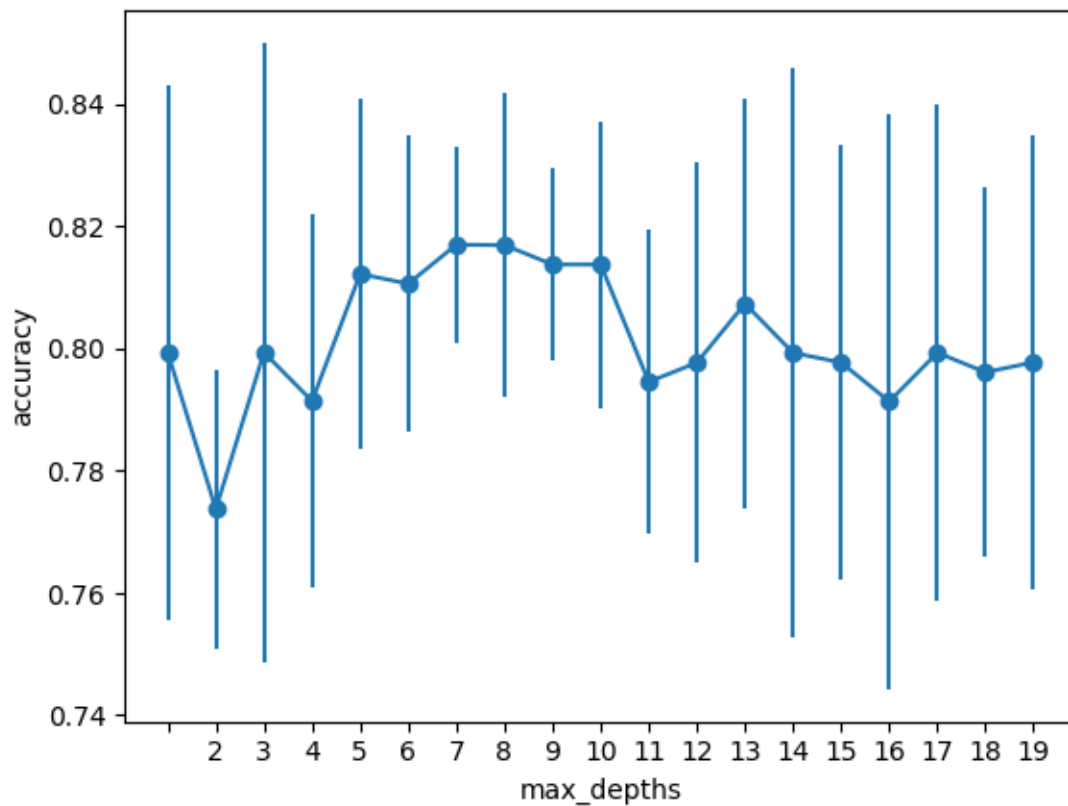
- If int, then consider ``min_samples_leaf`` as the minimum number.
- If float, then ``min_samples_leaf`` is a fraction and ``ceil(min_samples_leaf * n_samples)`` are the minimum number of samples for each node.

```
[43]: from sklearn.model_selection import cross_val_score
```

```
[44]: max_depths = [None] + list(np.arange(2, 20))
avg_scores = list()
std_scores = list()

for max_depth in max_depths:
    dt = DecisionTreeClassifier(max_depth=max_depth)
    scores = cross_val_score(dt, X_train, y_train, cv=5)
    avg_scores.append(np.mean(scores))
    std_scores.append(np.std(scores))

plt.errorbar(range(len(max_depths)), y=avg_scores, yerr=std_scores, marker='o')
plt.xticks(range(len(max_depths)), max_depths)
plt.xlabel("max_depths")
plt.ylabel("accuracy")
plt.show()
```



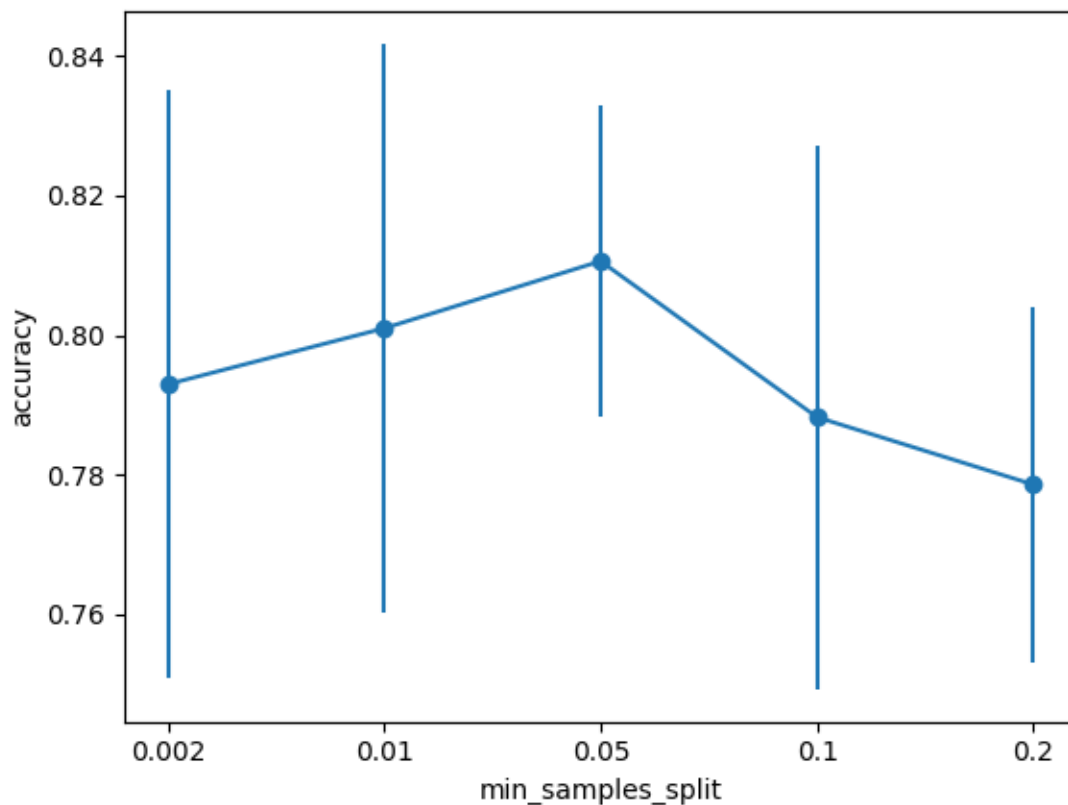
```
[45]: min_samples_splits = [0.002, 0.01, 0.05, 0.1, 0.2]
avg_scores = list()
std_scores = list()
```

```

for min_samples_split in min_samples_splits:
    dt = DecisionTreeClassifier(min_samples_split=min_samples_split)
    scores = cross_val_score(dt, X_train, y_train, cv=5)
    avg_scores.append(np.mean(scores))
    std_scores.append(np.std(scores))

plt.errorbar(range(len(min_samples_splits)), y=avg_scores, yerr=std_scores,
            marker='o')
plt.xticks(range(len(min_samples_splits)), min_samples_splits)
plt.xlabel("min_samples_split")
plt.ylabel("accuracy")
plt.show()

```



```

[46]: min_samples_leafs = [1, 5, 10, 20, 30, 50, 100]
avg_scores = list()
std_scores = list()

for min_samples_leaf in min_samples_leafs:
    dt = DecisionTreeClassifier(min_samples_leaf=min_samples_leaf)
    scores = cross_val_score(dt, X_train, y_train, cv=5)

```

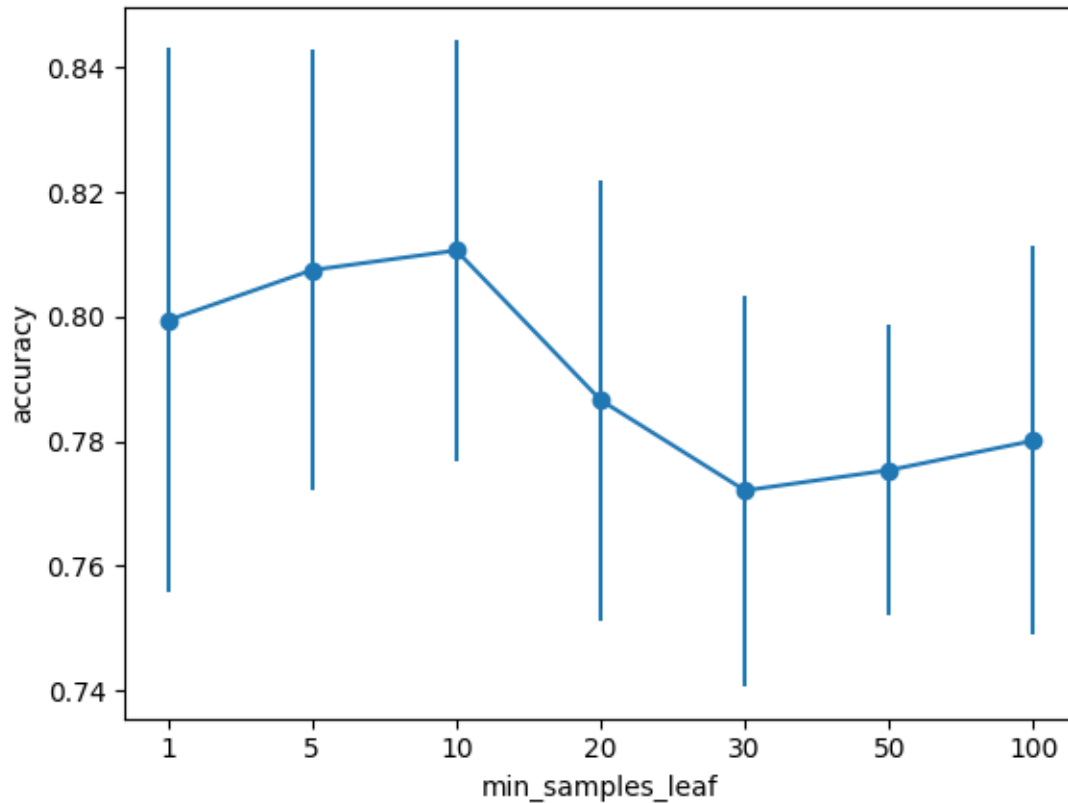


```

avg_scores.append(np.mean(scores))
std_scores.append(np.std(scores))

plt.errorbar(range(len(min_samples_leafs)), y=avg_scores, yerr=std_scores,
             marker='o')
plt.xticks(range(len(min_samples_leafs)), min_samples_leafs)
plt.xlabel("min_samples_leaf")
plt.ylabel("accuracy")
plt.show()

```



0.0.4 Randomized Search

```

[47]: from sklearn.model_selection import RandomizedSearchCV
      from sklearn.model_selection import RepeatedStratifiedKFold

```

```

[48]: %%time
      param_list = {
          'max_depth': [None] + list(np.arange(2, 20)),
          'min_samples_split': [2, 5, 10, 20, 30, 50, 100],
          'min_samples_leaf': [1, 5, 10, 20, 30, 50, 100],
          'criterion': ['gini', 'entropy']
      }

```

```

}

clf = DecisionTreeClassifier(criterion='gini', max_depth=None,
    ↪min_samples_split=2, min_samples_leaf=1)

random_search = RandomizedSearchCV(
    DecisionTreeClassifier(),
    param_distributions=param_list,
    cv=RepeatedStratifiedKFold(random_state=0),
    n_jobs=1, # Changed from -1 to 1 to avoid multiprocessing issues
    refit=True,
    n_iter=200,
    #verbose=2
)

random_search.fit(X_train, y_train)
clf = random_search.best_estimator_

```

CPU times: user 35.7 s, sys: 63.1 ms, total: 35.7 s
 Wall time: 36.1 s

```
[49]: print(random_search.best_params_, random_search.best_score_)
```

```
{'min_samples_split': 10, 'min_samples_leaf': 5, 'max_depth': 6, 'criterion':
'entropy'} 0.8200645161290322
```

```
[50]: y_test_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_test_pred))
```

Accuracy: 0.7985074626865671

```
[51]: clf.score(X_test, y_test)
```

```
[51]: 0.7985074626865671
```

```
[52]: print('Train Accuracy %s' % accuracy_score(y_train, y_train_pred))
print('Train F1-score %s' % f1_score(y_train, y_train_pred, average=None))
print()

print('Test Accuracy %s' % accuracy_score(y_test, y_test_pred))
print('Test F1-score %s' % f1_score(y_test, y_test_pred, average=None))
```

Train Accuracy 0.9839486356340289
 Train F1-score [0.9871134 0.9787234]

Test Accuracy 0.7985074626865671
 Test F1-score [0.84393064 0.71578947]

```
[53]: print(classification_report(y_test, y_test_pred))
```

	precision	recall	f1-score	support
0	0.81	0.88	0.84	165
1	0.78	0.66	0.72	103
accuracy			0.80	268
macro avg	0.79	0.77	0.78	268
weighted avg	0.80	0.80	0.79	268

```
[54]: results = pd.DataFrame(random_search.cv_results_)
```

```
[55]: results
```

```
[55]:
```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	\
0	0.002263	0.000234	0.001173	0.000163	
1	0.001996	0.000159	0.001081	0.000175	
2	0.002199	0.000431	0.001174	0.000261	
3	0.001748	0.000163	0.001115	0.000155	
4	0.001905	0.000201	0.001083	0.000187	
..	...	...	...	...	
195	0.001808	0.000142	0.001021	0.000096	
196	0.001453	0.000134	0.001007	0.000104	
197	0.001852	0.000125	0.000991	0.000103	
198	0.001599	0.000113	0.000963	0.000043	
199	0.001799	0.000101	0.000997	0.000074	

	param_min_samples_split	param_min_samples_leaf	param_max_depth	\
0	50	1	17	
1	5	20	12	
2	30	20	17	
3	2	1	2	
4	5	30	17	
..	...	...	...	
195	30	30	7	
196	5	100	16	
197	30	20	None	
198	100	50	13	
199	20	20	5	

	param_criterion	params	\
0	gini	{'min_samples_split': 50, 'min_samples_leaf': ...	
1	entropy	{'min_samples_split': 5, 'min_samples_leaf': 2...	
2	entropy	{'min_samples_split': 30, 'min_samples_leaf': ...	
3	entropy	{'min_samples_split': 2, 'min_samples_leaf': 1...	

```

4          entropy {'min_samples_split': 5, 'min_samples_leaf': 3...
..          ...
195        entropy {'min_samples_split': 30, 'min_samples_leaf': ...
196          gini {'min_samples_split': 5, 'min_samples_leaf': 1...
197        entropy {'min_samples_split': 30, 'min_samples_leaf': ...
198        entropy {'min_samples_split': 100, 'min_samples_leaf':...
199        entropy {'min_samples_split': 20, 'min_samples_leaf': ...

```

	split0_test_score	...	split43_test_score	split44_test_score	\
0	0.800	...	0.782258	0.741935	
1	0.848	...	0.782258	0.806452	
2	0.848	...	0.782258	0.806452	
3	0.768	...	0.782258	0.782258	
4	0.800	...	0.790323	0.766129	
..	...	...	...	...	
195	0.800	...	0.790323	0.766129	
196	0.760	...	0.790323	0.790323	
197	0.848	...	0.782258	0.806452	
198	0.752	...	0.798387	0.782258	
199	0.848	...	0.782258	0.806452	

	split45_test_score	split46_test_score	split47_test_score	\
0	0.760	0.784	0.752	
1	0.848	0.776	0.752	
2	0.848	0.768	0.752	
3	0.816	0.760	0.752	
4	0.816	0.776	0.752	
..	...	...	...	
195	0.816	0.776	0.752	
196	0.808	0.752	0.784	
197	0.848	0.768	0.752	
198	0.816	0.776	0.752	
199	0.848	0.768	0.752	

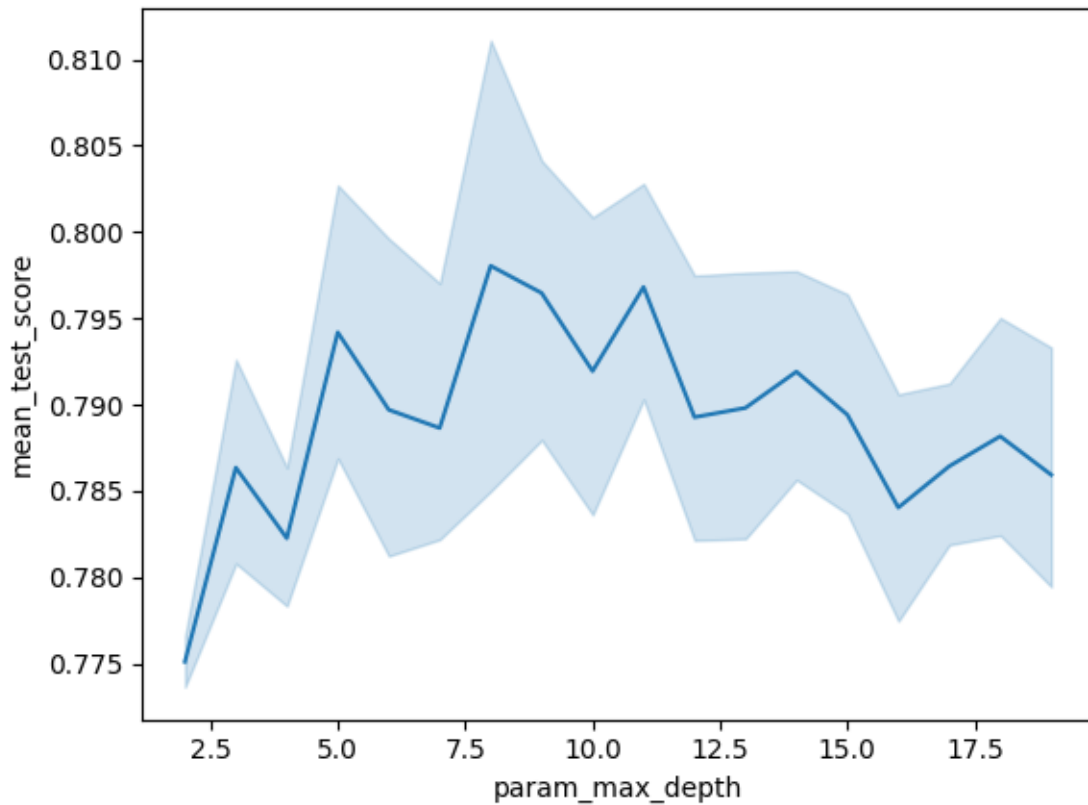
	split48_test_score	split49_test_score	mean_test_score	std_test_score	\
0	0.782258	0.854839	0.784262	0.035370	
1	0.758065	0.838710	0.793409	0.033424	
2	0.758065	0.838710	0.792769	0.033711	
3	0.782258	0.798387	0.774338	0.026770	
4	0.750000	0.806452	0.783330	0.029422	
..	...	...	...	...	
195	0.750000	0.806452	0.783650	0.029859	
196	0.766129	0.790323	0.780124	0.027891	
197	0.758065	0.838710	0.793090	0.033427	
198	0.782258	0.798387	0.775634	0.025992	
199	0.758065	0.838710	0.792609	0.033885	

	rank_test_score
0	102
1	56
2	62
3	194
4	110
..	...
195	108
196	123
197	58
198	161
199	64

[200 rows x 62 columns]

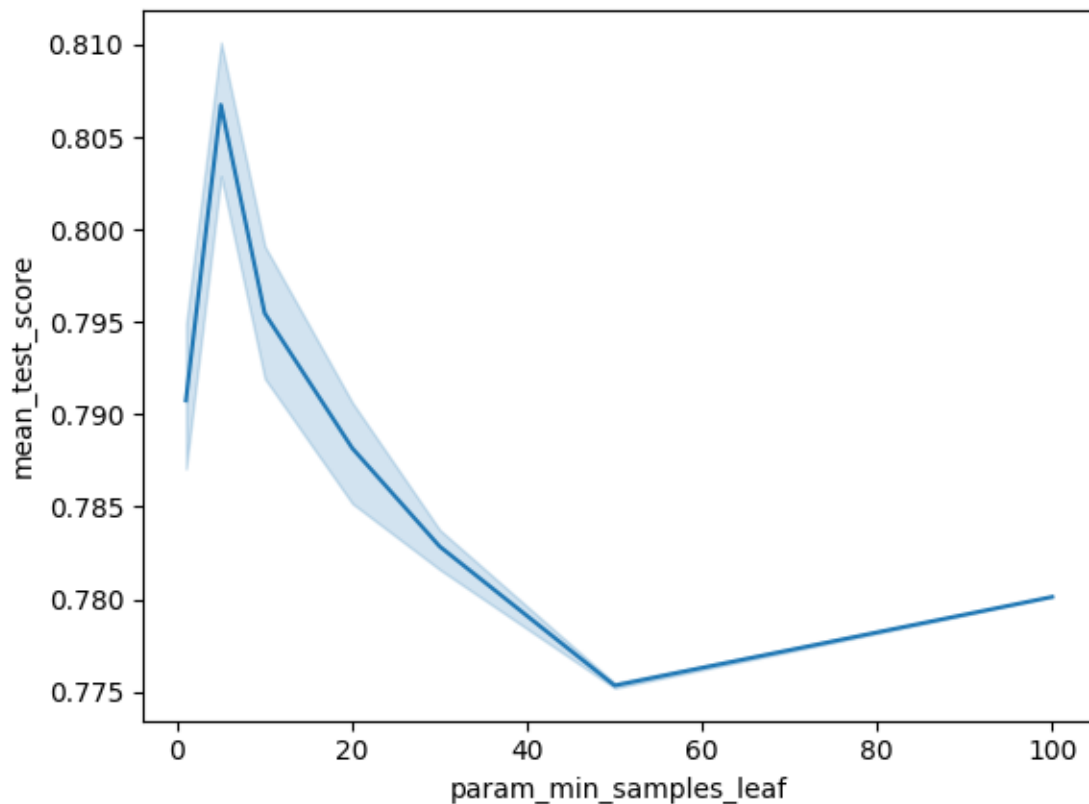
```
[56]: sns.lineplot(data=results, x="param_max_depth", y="mean_test_score")
```

```
[56]: <Axes: xlabel='param_max_depth', ylabel='mean_test_score'>
```



```
[57]: sns.lineplot(data=results, x="param_min_samples_leaf", y="mean_test_score")
```

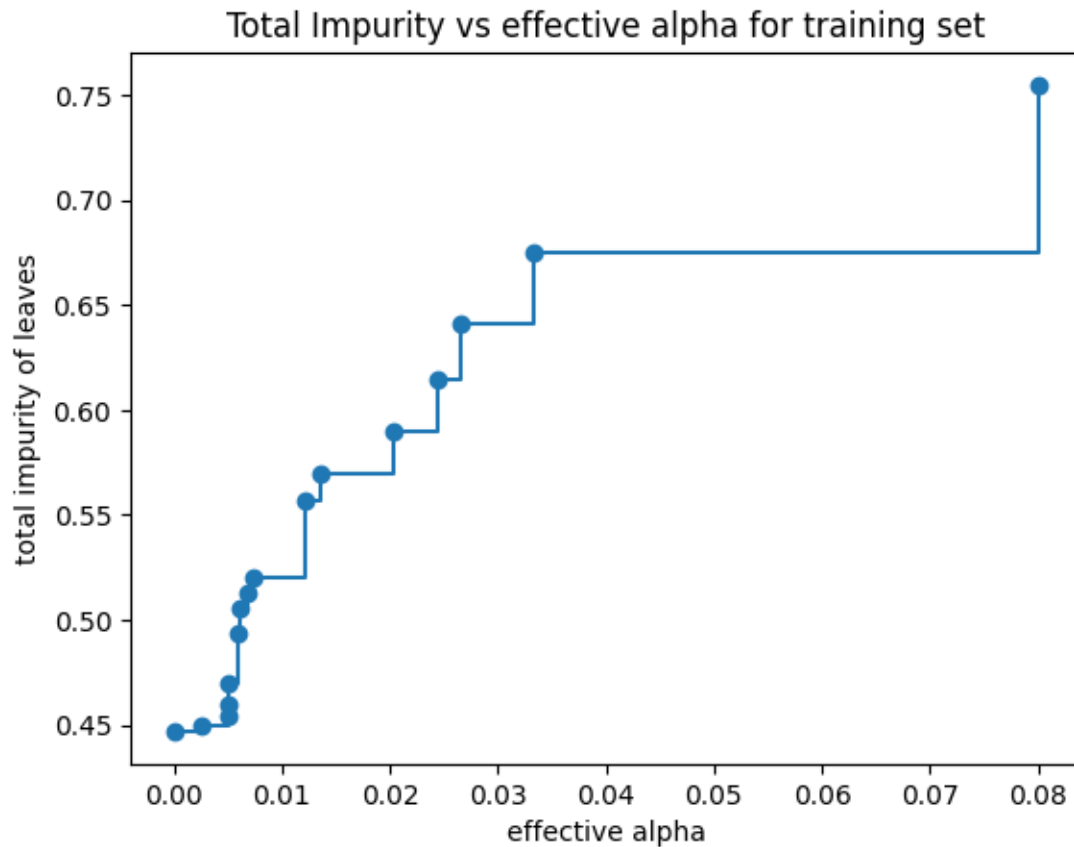
```
[57]: <Axes: xlabel='param_min_samples_leaf', ylabel='mean_test_score'>
```



```
[58]: zipped = zip(attributes, clf.feature_importances_)
zipped = sorted(zipped, key=lambda x: x[1], reverse=True)
for col, imp in zipped:
    print(col, imp)
```

```
Sex 0.40072068942597955
Pclass 0.17751028393940269
Age 0.16415838878707298
Fare 0.1566477687838109
FamilySize 0.09341600971472502
Embarked_S 0.0075468593490089
IsAlone 0.0
Embarked_C 0.0
Embarked_Q 0.0
```

```
[59]: plt.figure(figsize=(20, 4), dpi=300)
plot_tree(clf, feature_names=attributes, filled=True)
plt.show()
```

```
[63]: random_search.best_params_
```

```
[63]: {'min_samples_split': 10,
      'min_samples_leaf': 5,
      'max_depth': 6,
      'criterion': 'entropy'}
```

```
[64]: clfs = []
      for ccp_alpha in ccp_alphas:
          clf = DecisionTreeClassifier(random_state=0, ccp_alpha=ccp_alpha,
          ↳**random_search.best_params_)
          clf.fit(X_train, y_train)
          clfs.append(clf)

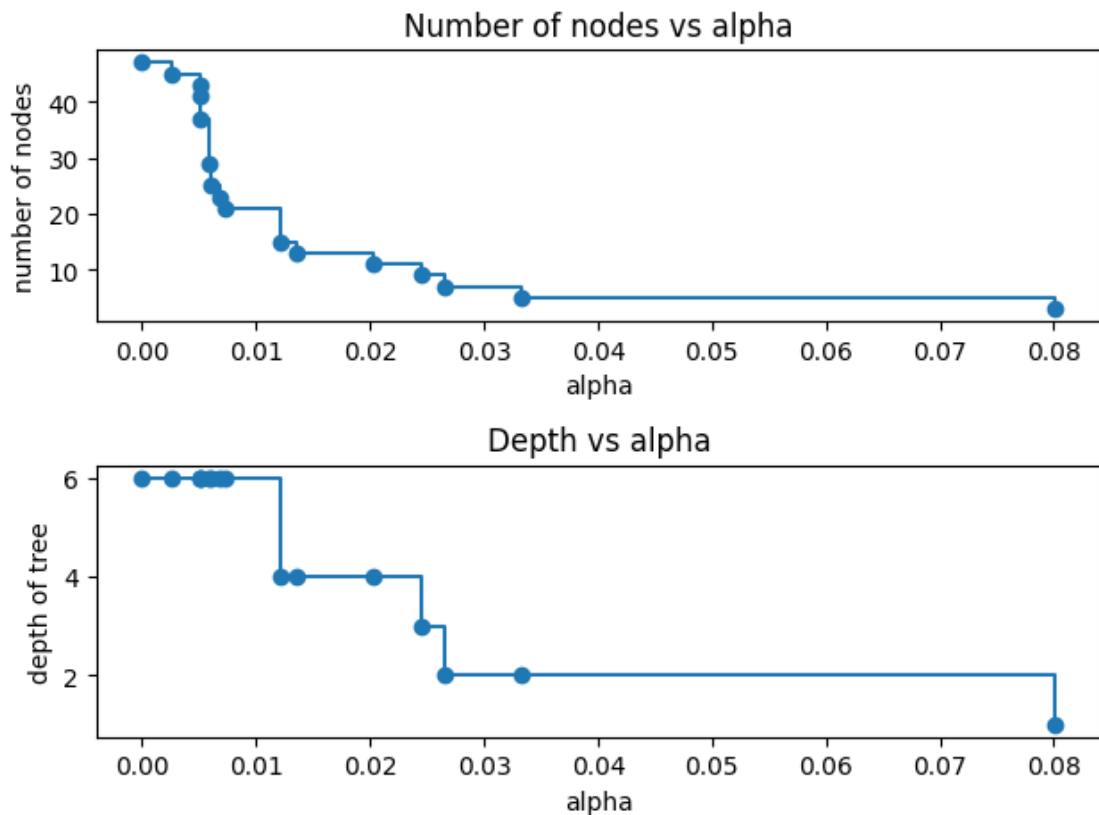
      print("Number of nodes in the last tree is: {} with ccp_alpha: {}".format(
          clfs[-1].tree_.node_count, ccp_alphas[-1])
      )
```

Number of nodes in the last tree is: 1 with ccp_alpha: 0.20581849199058122


```
[65]: clfs = clfs[:-1]
      ccp_alphas = ccp_alphas[:-1]

      node_counts = [clf.tree_.node_count for clf in clfs]
      depth = [clf.tree_.max_depth for clf in clfs]

      fig, ax = plt.subplots(2, 1)
      ax[0].plot(ccp_alphas, node_counts, marker="o", drawstyle="steps-post")
      ax[0].set_xlabel("alpha")
      ax[0].set_ylabel("number of nodes")
      ax[0].set_title("Number of nodes vs alpha")
      ax[1].plot(ccp_alphas, depth, marker="o", drawstyle="steps-post")
      ax[1].set_xlabel("alpha")
      ax[1].set_ylabel("depth of tree")
      ax[1].set_title("Depth vs alpha")
      fig.tight_layout()
```



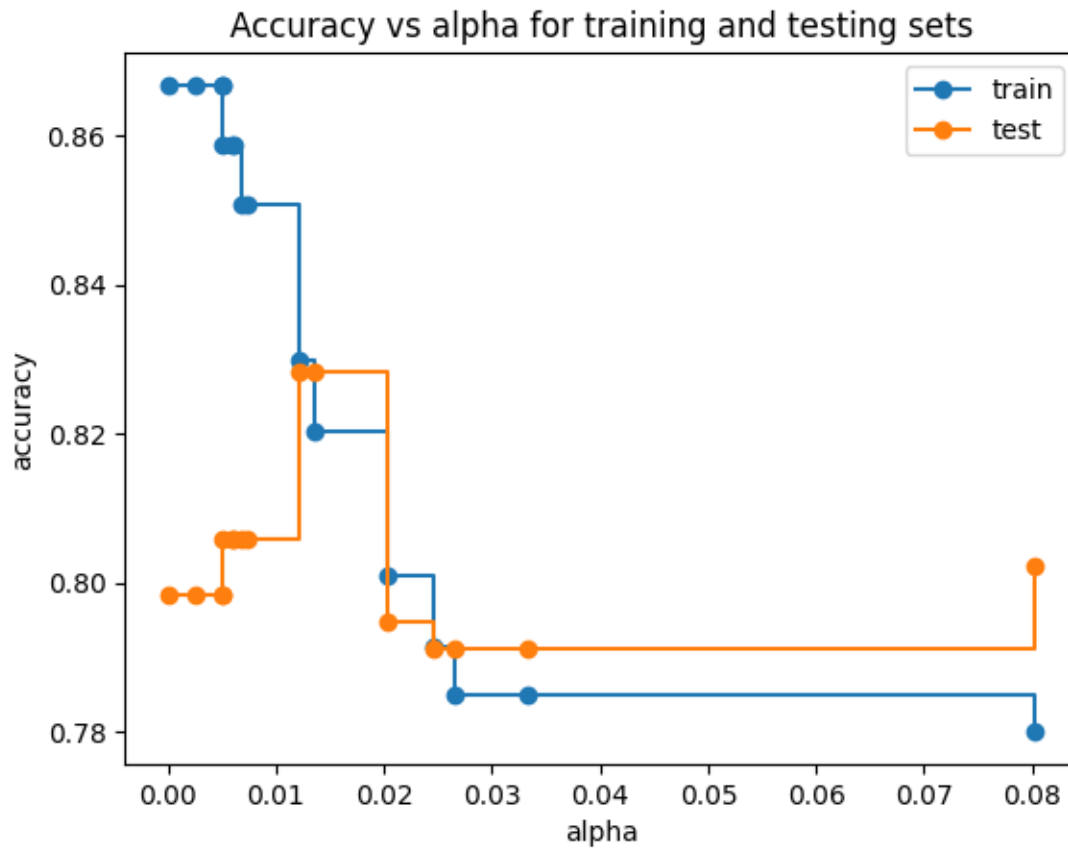
```
[66]: train_scores = [clf.score(X_train, y_train) for clf in clfs]
      test_scores = [clf.score(X_test, y_test) for clf in clfs]

      fig, ax = plt.subplots()
```

```

ax.set_xlabel("alpha")
ax.set_ylabel("accuracy")
ax.set_title("Accuracy vs alpha for training and testing sets")
ax.plot(ccp_alphas, train_scores, marker="o", label="train",
        drawstyle="steps-post")
ax.plot(ccp_alphas, test_scores, marker="o", label="test",
        drawstyle="steps-post")
ax.legend()
plt.show()

```



```

[67]: dtp = DecisionTreeClassifier()
      dtp.fit(X_train, y_train)

```

```

[67]: DecisionTreeClassifier()

```

```

[68]: y_test_pred = dtp.predict(X_test)

```

```

[69]: print('Train Accuracy %s' % accuracy_score(y_train, y_train_pred))
      print('Train F1-score %s' % f1_score(y_train, y_train_pred, average=None))

```

```
print()

print('Test Accuracy %s' % accuracy_score(y_test, y_test_pred))
print('Test F1-score %s' % f1_score(y_test, y_test_pred, average=None))
```

Train Accuracy 0.9839486356340289
Train F1-score [0.9871134 0.9787234]

Test Accuracy 0.7723880597014925
Test F1-score [0.820059 0.69035533]

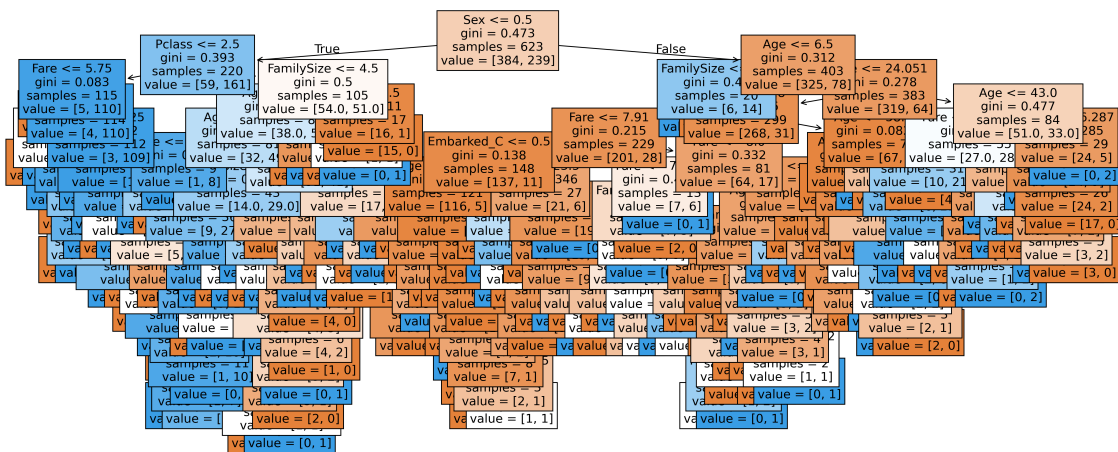
```
[70]: y_test_pred = dtp.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_test_pred))
```

Accuracy: 0.7723880597014925

```
[71]: print(classification_report(y_test, y_test_pred))
```

	precision	recall	f1-score	support
0	0.80	0.84	0.82	165
1	0.72	0.66	0.69	103
accuracy			0.77	268
macro avg	0.76	0.75	0.76	268
weighted avg	0.77	0.77	0.77	268

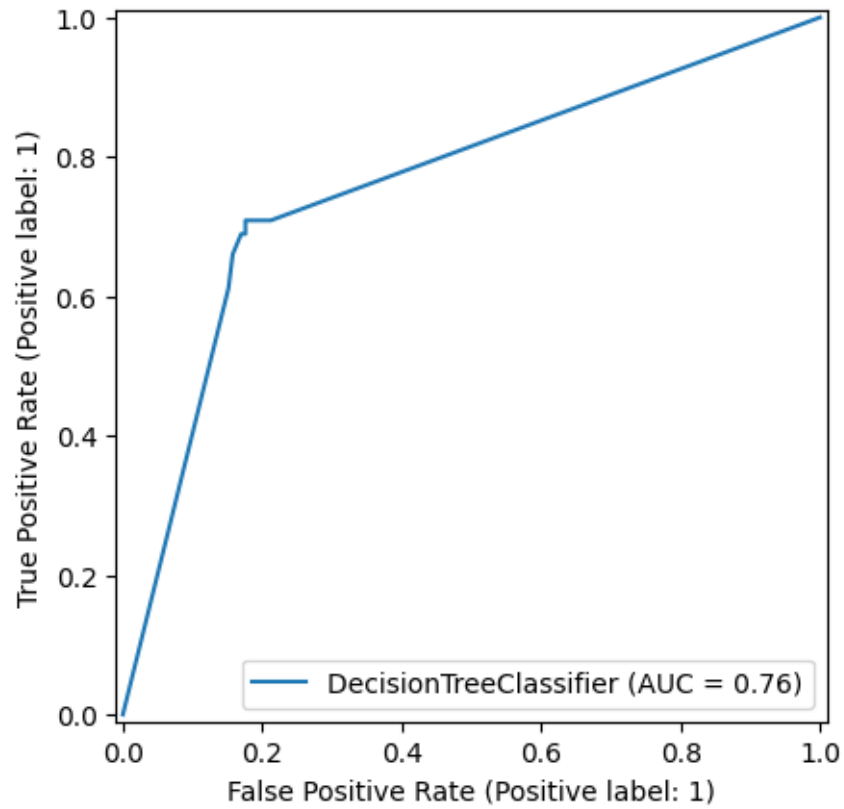
```
[72]: plt.figure(figsize=(20, 8), dpi=300)
plot_tree(dtp, feature_names=attributes, filled=True, fontsize=13)
plt.show()
```



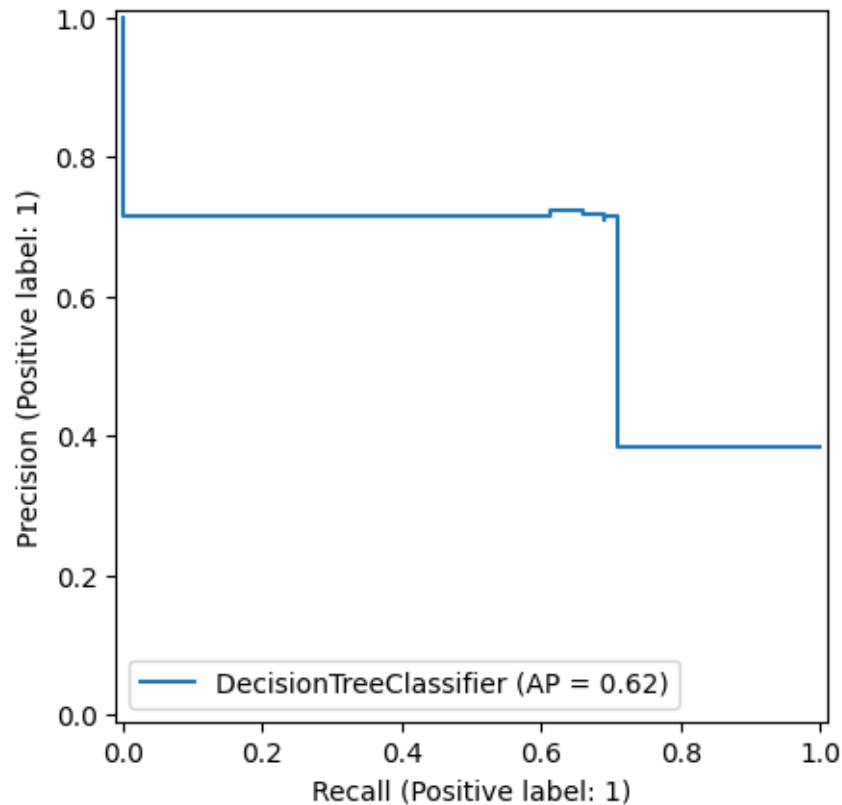
0.0.5 ROC Curves

```
[73]: from sklearn.metrics import RocCurveDisplay, PrecisionRecallDisplay
```

```
[74]: RocCurveDisplay.from_estimator(dtp, X_test, y_test)  
plt.show()
```



```
[75]: PrecisionRecallDisplay.from_estimator(dtp, X_test, y_test)  
plt.show()
```



0.0.6 Pruning Utils

In scikit-learn's decision trees, it can sometimes occur that a node split creates branches where all subsequent splits lead to the same predicted label as the node itself. In such cases, these branches do not contribute additional information to the predictions. Tree pruning functions are designed to simplify the tree by removing such redundant branches, keeping only the root node of the branch. This process reduces the complexity of the tree and is particularly useful for improving interpretability and visualization.

```
[76]: from sklearn.tree._tree import TREE_LEAF
```

```
[77]: def is_leaf(inner_tree, index):
    # Check whether node is leaf node
    return (inner_tree.children_left[index] == TREE_LEAF and
            inner_tree.children_right[index] == TREE_LEAF)

def prune_index(inner_tree, decisions, index=0):
    # Start pruning from the bottom - if we start from the top, we might miss
    # nodes that become leaves during pruning.
    # Do not use this directly - use prune_duplicate_leaves instead.
```

```

if not is_leaf(inner_tree, inner_tree.children_left[index]):
    prune_index(inner_tree, decisions, inner_tree.children_left[index])
if not is_leaf(inner_tree, inner_tree.children_right[index]):
    prune_index(inner_tree, decisions, inner_tree.children_right[index])

# Prune children if both children are leaves now and make the same decision:
if (is_leaf(inner_tree, inner_tree.children_left[index]) and
    is_leaf(inner_tree, inner_tree.children_right[index]) and
    (decisions[index] == decisions[inner_tree.children_left[index]]) and
    (decisions[index] == decisions[inner_tree.children_right[index]])):
    # turn node into a leaf by "unlinking" its children
    inner_tree.children_left[index] = TREE_LEAF
    inner_tree.children_right[index] = TREE_LEAF
    # print("Pruned {}".format(index))

def prune_duplicate_leaves(dt):
    # Remove leaves if both
    decisions = dt.tree_.value.argmax(axis=2).flatten().tolist() # Decision
    ↪ for each node
    prune_index(dt.tree_, decisions)

```

```

[78]: dt = DecisionTreeClassifier(criterion='entropy', max_depth=None,
    ↪ min_samples_split=5, min_samples_leaf=5, random_state=94)

```

```

[79]: dt.fit(X_train, y_train)

```

```

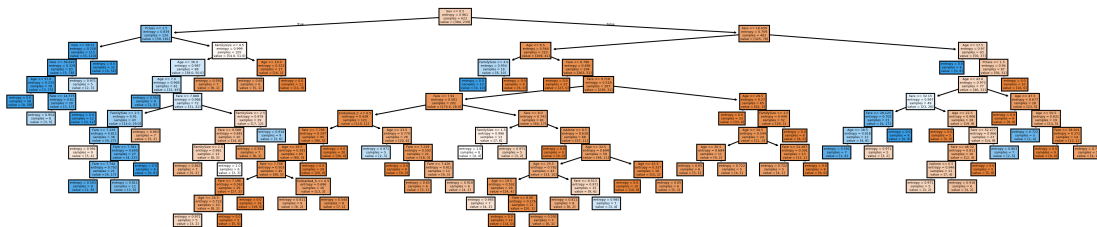
[79]: DecisionTreeClassifier(criterion='entropy', min_samples_leaf=5,
    min_samples_split=5, random_state=94)

```

```

[80]: plt.figure(figsize=(20, 4), dpi=300)
    plot_tree(dt, feature_names=attributes, filled=True)
    plt.show()

```



```

[81]: prune_duplicate_leaves(dt)

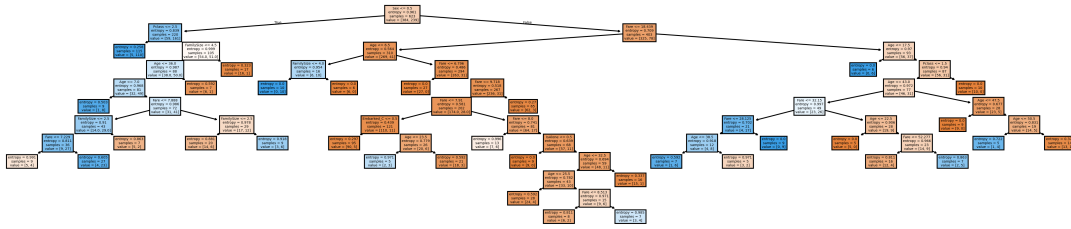
```

```

[82]: # let us visualise it again after pruning

```

```
[83]: plt.figure(figsize=(20, 4), dpi=300)
      plot_tree(dt, feature_names=attributes, filled=True)
      plt.show()
```



[83]:

[83]:

[83]:

[83]:

0.0.7 RuleTree

RuleTree is a sklearn-inspired Decision Tree library developed internally in the KDD lab. It is capable of directly handling the “children with same classes” problem directly in its parameters.

Feel free to explore RuleTrees: using scikit learn implementation is mandatory, while using RuleTree is an additional exploration you can do

For code examples on classification and code implementation, check:

<https://github.com/fismimosa/RuleTree>

https://github.com/fismimosa/RuleTree/blob/main/Examples/RuleTreeClassifier_example.ipynb

The correct import is **RuleTree.tree.RuleTreeClassifier**, not `ruletree.tree.RuleTreeClassifier` as in the repository, which still need to be updated

```
[84]: # Force re-installation of numpy, scipy, and RuleTree to ensure binary
      ↪compatibility
      !pip install --upgrade --force-reinstall numpy scipy RuleTree
      # IMPORTANT: Restart the runtime after this installation for changes to take
      ↪effect.
```

Collecting numpy

Using cached

numpy-2.3.5-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata
(62 kB)

Collecting scipy

```

Using cached
scipy-1.16.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata
(62 kB)
Collecting RuleTree
  Using cached RuleTree-0.0.4.post2-py3-none-any.whl
Collecting numpy
  Using cached
numpy-1.26.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata
(61 kB)
Collecting scikit-learn>=1.5.0 (from RuleTree)
  Using cached scikit_learn-1.7.2-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (11 kB)
Collecting pandas (from RuleTree)
  Using cached pandas-2.3.3-cp312-cp312-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (91 kB)
Collecting category-encoders (from RuleTree)
  Using cached category_encoders-2.9.0-py3-none-any.whl.metadata (7.9 kB)
Collecting threadpoolctl (from RuleTree)
  Using cached threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Collecting tqdm (from RuleTree)
  Using cached tqdm-4.67.1-py3-none-any.whl.metadata (57 kB)
Collecting progress-table (from RuleTree)
  Using cached progress_table-3.1.2-py3-none-any.whl.metadata (5.7 kB)
Collecting graphviz>=0.20.3 (from RuleTree)
  Using cached graphviz-0.21-py3-none-any.whl.metadata (12 kB)
Collecting numba (from RuleTree)
  Using cached
numba-0.62.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata
(2.8 kB)
Collecting psutil (from RuleTree)
  Using cached psutil-7.1.3-cp36-abi3-
manylinux2010_x86_64.manylinux_2_12_x86_64.manylinux_2_28_x86_64.whl.metadata
(23 kB)
Collecting setuptools (from RuleTree)
  Using cached setuptools-80.9.0-py3-none-any.whl.metadata (6.6 kB)
Collecting matplotlib (from RuleTree)
  Using cached matplotlib-3.10.7-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (11 kB)
Collecting tempfile312 (from RuleTree)
  Using cached tempfile312-1.0.1-py3-none-any.whl.metadata (1.7 kB)
Collecting joblib>=1.2.0 (from scikit-learn>=1.5.0->RuleTree)
  Using cached joblib-1.5.2-py3-none-any.whl.metadata (5.6 kB)
Collecting patsy>=0.5.1 (from category-encoders->RuleTree)
  Using cached patsy-1.0.2-py2.py3-none-any.whl.metadata (3.6 kB)
Collecting statsmodels>=0.9.0 (from category-encoders->RuleTree)
  Using cached statsmodels-0.14.5-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
(9.5 kB)

```


Collecting python-dateutil>=2.8.2 (from pandas->RuleTree)
 Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
 Collecting pytz>=2020.1 (from pandas->RuleTree)
 Using cached pytz-2025.2-py2.py3-none-any.whl.metadata (22 kB)
 Collecting tzdata>=2022.7 (from pandas->RuleTree)
 Using cached tzdata-2025.2-py2.py3-none-any.whl.metadata (1.4 kB)
 Collecting contourpy>=1.0.1 (from matplotlib->RuleTree)
 Using cached contourpy-1.3.3-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (5.5 kB)
 Collecting cycycler>=0.10 (from matplotlib->RuleTree)
 Using cached cycycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
 Collecting fonttools>=4.22.0 (from matplotlib->RuleTree)
 Using cached fonttools-4.60.1-cp312-cp312-manylinux1_x86_64.manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_5_x86_64.whl.metadata (112 kB)
 Collecting kiwisolver>=1.3.1 (from matplotlib->RuleTree)
 Using cached kiwisolver-1.4.9-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (6.3 kB)
 Collecting packaging>=20.0 (from matplotlib->RuleTree)
 Using cached packaging-25.0-py3-none-any.whl.metadata (3.3 kB)
 Collecting pillow>=8 (from matplotlib->RuleTree)
 Using cached pillow-12.0.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (8.8 kB)
 Collecting pyparsing>=3 (from matplotlib->RuleTree)
 Using cached pyparsing-3.2.5-py3-none-any.whl.metadata (5.0 kB)
 Collecting llvmlite<0.46,>=0.45.0dev0 (from numba->RuleTree)
 Using cached llvmlite-0.45.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (4.9 kB)
 Collecting colorama (from progress-table->RuleTree)
 Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
 Collecting six>=1.5 (from python-dateutil>=2.8.2->pandas->RuleTree)
 Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
 Using cached
 scipy-1.16.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (35.7 MB)
 Using cached
 numpy-1.26.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (18.0 MB)
 Using cached graphviz-0.21-py3-none-any.whl (47 kB)
 Using cached
 scikit_learn-1.7.2-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (9.5 MB)
 Using cached threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
 Using cached category_encoders-2.9.0-py3-none-any.whl (85 kB)
 Using cached
 pandas-2.3.3-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (12.4 MB)

```

Using cached
matplotlib-3.10.7-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl
(8.7 MB)
Using cached
numba-0.62.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (3.8 MB)
Using cached progress_table-3.1.2-py3-none-any.whl (20 kB)
Using cached psutil-7.1.3-cp36-abi3-
manylinux2010_x86_64.manylinux_2_12_x86_64.manylinux_2_28_x86_64.whl (263 kB)
Using cached setuptools-80.9.0-py3-none-any.whl (1.2 MB)
Using cached tempfile312-1.0.1-py3-none-any.whl (19 kB)
Using cached tqdm-4.67.1-py3-none-any.whl (78 kB)
Using cached
contourpy-1.3.3-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (362
kB)
Using cached cycler-0.12.1-py3-none-any.whl (8.3 kB)
Using cached fonttools-4.60.1-cp312-cp312-
manylinux1_x86_64.manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_5_x86_6
4.whl (4.9 MB)
Using cached joblib-1.5.2-py3-none-any.whl (308 kB)
Using cached
kiwisolver-1.4.9-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (1.5
MB)
Using cached
llvmlite-0.45.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (56.3
MB)
Using cached packaging-25.0-py3-none-any.whl (66 kB)
Using cached patsy-1.0.2-py2.py3-none-any.whl (233 kB)
Using cached
pillow-12.0.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (7.0
MB)
Using cached pyparsing-3.2.5-py3-none-any.whl (113 kB)
Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Using cached pytz-2025.2-py2.py3-none-any.whl (509 kB)
Using cached statsmodels-0.14.5-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (10.4 MB)
Using cached tzdata-2025.2-py2.py3-none-any.whl (347 kB)
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: pytz, tzdata, tqdm, threadpoolctl, tempfile312,
six, setuptools, pyparsing, psutil, pillow, packaging, numpy, llvmlite,
kiwisolver, joblib, graphviz, fonttools, cycler, colorama, scipy, python-
dateutil, progress-table, patsy, numba, contourpy, scikit-learn, pandas,
matplotlib, statsmodels, category-encoders, RuleTree
  Attempting uninstall: pytz
    Found existing installation: pytz 2025.2
    Uninstalling pytz-2025.2:
      Successfully uninstalled pytz-2025.2
  Attempting uninstall: tzdata

```

Found existing installation: tzdata 2025.2
Uninstalling tzdata-2025.2:
 Successfully uninstalled tzdata-2025.2
Attempting uninstall: tqdm
 Found existing installation: tqdm 4.67.1
 Uninstalling tqdm-4.67.1:
 Successfully uninstalled tqdm-4.67.1
Attempting uninstall: threadpoolctl
 Found existing installation: threadpoolctl 3.6.0
 Uninstalling threadpoolctl-3.6.0:
 Successfully uninstalled threadpoolctl-3.6.0
Attempting uninstall: tempfile312
 Found existing installation: tempfile312 1.0.1
 Uninstalling tempfile312-1.0.1:
 Successfully uninstalled tempfile312-1.0.1
Attempting uninstall: six
 Found existing installation: six 1.17.0
 Uninstalling six-1.17.0:
 Successfully uninstalled six-1.17.0
Attempting uninstall: setuptools
 Found existing installation: setuptools 80.9.0
 Uninstalling setuptools-80.9.0:
 Successfully uninstalled setuptools-80.9.0
Attempting uninstall: pyparsing
 Found existing installation: pyparsing 3.2.5
 Uninstalling pyparsing-3.2.5:
 Successfully uninstalled pyparsing-3.2.5
Attempting uninstall: psutil
 Found existing installation: psutil 7.1.3
 Uninstalling psutil-7.1.3:
 Successfully uninstalled psutil-7.1.3
Attempting uninstall: pillow
 Found existing installation: pillow 12.0.0
 Uninstalling pillow-12.0.0:
 Successfully uninstalled pillow-12.0.0
Attempting uninstall: packaging
 Found existing installation: packaging 25.0
 Uninstalling packaging-25.0:
 Successfully uninstalled packaging-25.0
Attempting uninstall: numpy
 Found existing installation: numpy 1.26.4
 Uninstalling numpy-1.26.4:
 Successfully uninstalled numpy-1.26.4
Attempting uninstall: llvmlite
 Found existing installation: llvmlite 0.45.1
 Uninstalling llvmlite-0.45.1:
 Successfully uninstalled llvmlite-0.45.1
Attempting uninstall: kiwisolver

Found existing installation: kiwisolver 1.4.9
Uninstalling kiwisolver-1.4.9:
 Successfully uninstalled kiwisolver-1.4.9
Attempting uninstall: joblib
 Found existing installation: joblib 1.5.2
 Uninstalling joblib-1.5.2:
 Successfully uninstalled joblib-1.5.2
Attempting uninstall: graphviz
 Found existing installation: graphviz 0.21
 Uninstalling graphviz-0.21:
 Successfully uninstalled graphviz-0.21
Attempting uninstall: fonttools
 Found existing installation: fonttools 4.60.1
 Uninstalling fonttools-4.60.1:
 Successfully uninstalled fonttools-4.60.1
Attempting uninstall: cyclcr
 Found existing installation: cyclcr 0.12.1
 Uninstalling cyclcr-0.12.1:
 Successfully uninstalled cyclcr-0.12.1
Attempting uninstall: colorama
 Found existing installation: colorama 0.4.6
 Uninstalling colorama-0.4.6:
 Successfully uninstalled colorama-0.4.6
Attempting uninstall: scipy
 Found existing installation: scipy 1.16.3
 Uninstalling scipy-1.16.3:
 Successfully uninstalled scipy-1.16.3
Attempting uninstall: python-dateutil
 Found existing installation: python-dateutil 2.9.0.post0
 Uninstalling python-dateutil-2.9.0.post0:
 Successfully uninstalled python-dateutil-2.9.0.post0
Attempting uninstall: progress-table
 Found existing installation: progress-table 3.1.2
 Uninstalling progress-table-3.1.2:
 Successfully uninstalled progress-table-3.1.2
Attempting uninstall: patsy
 Found existing installation: patsy 1.0.2
 Uninstalling patsy-1.0.2:
 Successfully uninstalled patsy-1.0.2
Attempting uninstall: numba
 Found existing installation: numba 0.62.1
 Uninstalling numba-0.62.1:
 Successfully uninstalled numba-0.62.1
Attempting uninstall: contourpy
 Found existing installation: contourpy 1.3.3
 Uninstalling contourpy-1.3.3:
 Successfully uninstalled contourpy-1.3.3
Attempting uninstall: scikit-learn

```
Found existing installation: scikit-learn 1.7.2
Uninstalling scikit-learn-1.7.2:
  Successfully uninstalled scikit-learn-1.7.2
Attempting uninstall: pandas
Found existing installation: pandas 2.3.3
Uninstalling pandas-2.3.3:
  Successfully uninstalled pandas-2.3.3
Attempting uninstall: matplotlib
Found existing installation: matplotlib 3.10.7
Uninstalling matplotlib-3.10.7:
  Successfully uninstalled matplotlib-3.10.7
Attempting uninstall: statsmodels
Found existing installation: statsmodels 0.14.5
Uninstalling statsmodels-0.14.5:
  Successfully uninstalled statsmodels-0.14.5
Attempting uninstall: category-encoders
Found existing installation: category_encoders 2.9.0
Uninstalling category_encoders-2.9.0:
  Successfully uninstalled category_encoders-2.9.0
Attempting uninstall: RuleTree
Found existing installation: RuleTree 0.0.4.post2
Uninstalling RuleTree-0.0.4.post2:
  Successfully uninstalled RuleTree-0.0.4.post2
```

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.

ipython 7.34.0 requires jedi>=0.16, which is not installed.

google-colab 1.0.0 requires pandas==2.2.2, but you have pandas 2.3.3 which is incompatible.

opencv-python 4.12.0.88 requires numpy<2.3.0,>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.

jax 0.7.2 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.

opencv-contrib-python 4.12.0.88 requires numpy<2.3.0,>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.

pytensor 2.35.1 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.

jaxlib 0.7.2 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.

cuml-cu12 25.10.0 requires numba<0.62.0a0,>=0.60.0, but you have numba 0.62.1 which is incompatible.

opencv-python-headless 4.12.0.88 requires numpy<2.3.0,>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.

shap 0.50.0 requires numpy>=2, but you have numpy 1.26.4 which is incompatible.

cudf-cu12 25.10.0 requires numba<0.62.0a0,>=0.60.0, but you have numba 0.62.1 which is incompatible.

gradio 5.49.1 requires pillow<12.0,>=8.0, but you have pillow 12.0.0 which is incompatible.

Successfully installed RuleTree-0.0.4.post2 category-encoders-2.9.0
colorama-0.4.6 contourpy-1.3.3 cycler-0.12.1 fonttools-4.60.1 graphviz-0.21
joblib-1.5.2 kiwisolver-1.4.9 llvmlite-0.45.1 matplotlib-3.10.7 numba-0.62.1
numpy-1.26.4 packaging-25.0 pandas-2.3.3 patsy-1.0.2 pillow-12.0.0 progress-
table-3.1.2 psutil-7.1.3 pyparsing-3.2.5 python-dateutil-2.9.0.post0 pytz-2025.2
scikit-learn-1.7.2 scipy-1.16.3 setuptools-80.9.0 six-1.17.0 statsmodels-0.14.5
tempfile312-1.0.1 threadpoolctl-3.6.0 tqdm-4.67.1 tzdata-2025.2

[100]: *#Initialize the RuleTreeClassifier and train it on your dataset*

```

from RuleTree.tree.RuleTreeClassifier import RuleTreeClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

```

```
[106]: X_train.shape, attributes
```

```

[106]: ((623, 9),
        ['Pclass',
         'Sex',
         'Age',
         'Fare',
         'FamilySize',
         'IsAlone',
         'Embarked_C',
         'Embarked_Q',
         'Embarked_S'])

```

```

[107]: rtc = RuleTreeClassifier(
        max_depth=3,
        criterion='gini',
        prune_useless_leaves=True, ##prune_useless_leaves performing the pruning
        ↪ internally for you without the need to access external functions
        random_state=42
    )

```

IMPORTANT DETAIL: for training, both X_train and y_train should be numpy objects, not pandas series

```

[108]: # Fit the RuleTreeClassifier to the training data
        rtc.fit(X_train, np.array(y_train))

```

```

[108]: RuleTreeClassifier(base_stumps=[(1.0,
                                         DecisionTreeStumpClassifier(ccp_alpha=0.0,
                                                                       class_weight=None,
                                                                       criterion='gini',
                                                                       max_depth=1,
                                                                       max_features=None,
                                                                       min_impurity_decrease=0.0,
                                                                       min_samples_leaf=1,
                                                                       min_samples_split=2,
                                                                       min_weight_fraction_leaf=0.0,
                                                                       monotonic_cst=None,
                                                                       random_state=42,
                                                                       splitter='best'))],
                           max_depth=3, prune_useless_leaves=True, random_state=42)

```

```
[109]: # Extract the rules as a dictionary using feature names
rules = rtc.get_rules(columns_names=attributes)
```

```
[110]: # Print textual rules of trained tree
rtc.print_rules(rules, columns_names=attributes)
```

```
|--- Sex <= 0.5 623
|   |--- Pclass <= 2.5 220
|   |   |--- Fare <= 5.75 115
|   |   |   output: 0
|   |   |--- Fare > 5.75
|   |   |   output: 1
|   |--- Pclass > 2.5
|   |   |--- FamilySize <= 4.5 105
|   |   |   output: 1
|   |   |--- FamilySize > 4.5
|   |   |   output: 0
|--- Sex > 0.5
|   |--- Age <= 6.5 403
|   |   |--- FamilySize <= 4.5 20
|   |   |   output: 1
|   |   |--- FamilySize > 4.5
|   |   |   output: 0
|   |--- Age > 6.5
|   |   output: 0
```

```
[111]: y_pred = rtc.predict(X_test)
```

```
[112]: print('Accuracy result\n', accuracy_score(y_test, y_pred))
```

```
Accuracy result
0.8283582089552238
```

```
[113]: # Obtain global feature importances similar to sklearn feature_importances_of_
↳ a trained Tree
feature_importances = rtc.compute_feature_importances()
for attr, imp in zip(attributes, feature_importances):
    print(attr, imp)
```

```
Pclass 0.17992977235110494
Sex 0.6098058869982659
Age 0.07982962656091805
Fare 0.01365006097271372
FamilySize 0.11678465311699747
IsAlone 0.0
Embarked_C 0.0
Embarked_Q 0.0
Embarked_S 0.0
```



```
[114]: # Get local feature importances similar to https://github.com/andosa/
      ↪treeinterpreter implementation
      # Prediction is the sum of bias and feature contributions
      prediction, bias, contributions = rtc.local_interpretation(X_test,
      ↪joint_contribution= True)
```

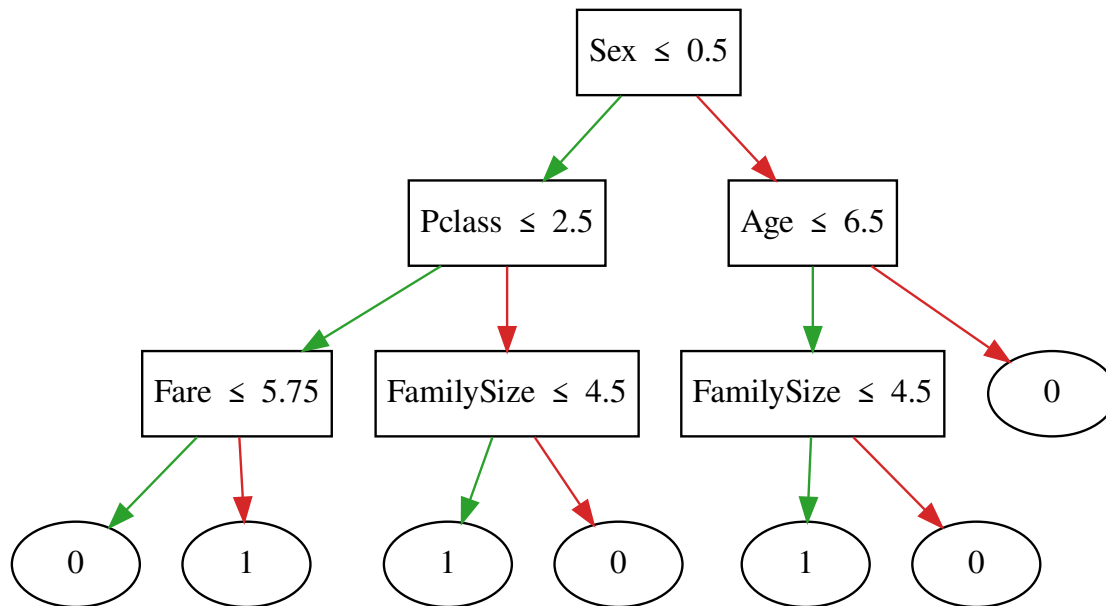
```
[115]: prediction[0], bias[0], contributions[0]
```

```
[115]: (array([0.83289817, 0.16710183]),
      array([0.61637239, 0.38362761]),
      {(1,): array([ 0.19007922, -0.19007922]),
       (1, 2): array([ 0.02644656, -0.02644656])})
```

```
[115]:
```

```
[116]: rtc.export_graphviz(columns_names = attributes)
```

```
[116]:
```



```
[117]: rtc.export_graphviz(columns_names = attributes, filename = 'data/
      ↪rtc_plot_tree') #adding filename will save a pdf of the plot
```

```
[118]: #directly obtain information on the nodes in the tree

      rtc.root.node_to_dict()
```

```
[118]: {'node_id': 'R',
      'is_leaf': False,
      'prediction': 0,
```

```

'prediction_probability': [0.6163723916532905, 0.38362760834670945],
'prediction_classes_': [0, 1],
'features': 9,
'left_node': 'Rl',
'right_node': 'Rr',
'feature_idx': 1,
'threshold': 0.5,
'is_categorical': None,
'samples': 623,
'feature_name': 'X_1',
'textual_rule': 'X_1 <= 0.5\t623',
'blob_rule': 'X_1 <= 0.5',
'graphviz_rule': {'label': 'X_1 0.5'},
'not_textual_rule': 'X_1 > 0.5',
'not_blob_rule': 'X_1 > 0.5',
'not_graphviz_rule': {'label': 'X_1 > 0.5'},
'stump_type': 'RuleTree.stumps.classification.DecisionTreeStumpClassifier',
'impurity': 0.4729149329217863,
'args': {'is_oblique': False,
'is_pivotal': False,
'unique_val_enum': None,
'coefficients': None,
'max_depth': 1,
'criterion': 'gini',
'splitter': 'best',
'min_samples_split': 2,
'min_samples_leaf': 1,
'min_weight_fraction_leaf': 0.0,
'max_features': None,
'random_state': 42,
'min_impurity_decrease': 0.0,
'class_weight': None,
'ccp_alpha': 0.0,
'monotonic_cst': None},
'split': {'args': {}}}

```

```

[119]: #navigate the tree from the root with .node_l and .node_r
rtc.root.node_l.node_to_dict()

```

```

[119]: {'node_id': 'Rl',
'is_leaf': False,
'prediction': 1,
'prediction_probability': [0.2681818181818182, 0.7318181818181818],
'prediction_classes_': [0, 1],
'features': 9,
'left_node': 'Rll',
'right_node': 'Rlr',

```

```

'feature_idx': 0,
'threshold': 2.5,
'is_categorical': None,
'samples': 220,
'feature_name': 'X_0',
'textual_rule': 'X_0 <= 2.5\t220',
'blob_rule': 'X_0 <= 2.5',
'graphviz_rule': {'label': 'X_0 2.5'},
'not_textual_rule': 'X_0 > 2.5',
'not_blob_rule': 'X_0 > 2.5',
'not_graphviz_rule': {'label': 'X_0 > 2.5'},
'stump_type': 'RuleTree.stumps.classification.DecisionTreeStumpClassifier',
'impurity': 0.3925206611570248,
'args': {'is_oblique': False,
'is_pivotal': False,
'unique_val_enum': None,
'coefficients': None,
'max_depth': 1,
'criterion': 'gini',
'splitter': 'best',
'min_samples_split': 2,
'min_samples_leaf': 1,
'min_weight_fraction_leaf': 0.0,
'max_features': None,
'random_state': 42,
'min_impurity_decrease': 0.0,
'class_weight': None,
'ccp_alpha': 0.0,
'monotonic_cst': None},
'split': {'args': {}}}

```

```

[121]: !pip install nbconvert
!apt-get install texlive texlive-xetex texlive-latex-extra pandoc

```

```

Requirement already satisfied: nbconvert in /usr/local/lib/python3.12/dist-
packages (7.16.6)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-
packages (from nbconvert) (4.13.5)
Requirement already satisfied: bleach!=5.0.0 in /usr/local/lib/python3.12/dist-
packages (from bleach[css]!=5.0.0->nbconvert) (6.3.0)
Requirement already satisfied: defusedxml in /usr/local/lib/python3.12/dist-
packages (from nbconvert) (0.7.1)
Requirement already satisfied: Jinja2>=3.0 in /usr/local/lib/python3.12/dist-
packages (from nbconvert) (3.1.6)
Requirement already satisfied: jupyter-core>=4.7 in
/usr/local/lib/python3.12/dist-packages (from nbconvert) (5.9.1)
Requirement already satisfied: jupyterlab-pygments in

```

/usr/local/lib/python3.12/dist-packages (from nbconvert) (0.3.0)
 Requirement already satisfied: markupsafe>=2.0 in
 /usr/local/lib/python3.12/dist-packages (from nbconvert) (3.0.3)
 Requirement already satisfied: mistune<4,>=2.0.3 in
 /usr/local/lib/python3.12/dist-packages (from nbconvert) (3.1.4)
 Requirement already satisfied: nbclient>=0.5.0 in
 /usr/local/lib/python3.12/dist-packages (from nbconvert) (0.10.2)
 Requirement already satisfied: nbformat>=5.7 in /usr/local/lib/python3.12/dist-
 packages (from nbconvert) (5.10.4)
 Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-
 packages (from nbconvert) (25.0)
 Requirement already satisfied: pandocfilters>=1.4.1 in
 /usr/local/lib/python3.12/dist-packages (from nbconvert) (1.5.1)
 Requirement already satisfied: pygments>=2.4.1 in
 /usr/local/lib/python3.12/dist-packages (from nbconvert) (2.19.2)
 Requirement already satisfied: traitlets>=5.1 in /usr/local/lib/python3.12/dist-
 packages (from nbconvert) (5.7.1)
 Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-
 packages (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert) (0.5.1)
 Requirement already satisfied: tinycss2<1.5,>=1.1.0 in
 /usr/local/lib/python3.12/dist-packages (from bleach[css]!=5.0.0->nbconvert)
 (1.4.0)
 Requirement already satisfied: platformdirs>=2.5 in
 /usr/local/lib/python3.12/dist-packages (from jupyter-core>=4.7->nbconvert)
 (4.5.0)
 Requirement already satisfied: jupyter-client>=6.1.12 in
 /usr/local/lib/python3.12/dist-packages (from nbclient>=0.5.0->nbconvert)
 (7.4.9)
 Requirement already satisfied: fastjsonschema>=2.15 in
 /usr/local/lib/python3.12/dist-packages (from nbformat>=5.7->nbconvert) (2.21.2)
 Requirement already satisfied: jsonschema>=2.6 in
 /usr/local/lib/python3.12/dist-packages (from nbformat>=5.7->nbconvert) (4.25.1)
 Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-
 packages (from beautifulsoup4->nbconvert) (2.8)
 Requirement already satisfied: typing-extensions>=4.0.0 in
 /usr/local/lib/python3.12/dist-packages (from beautifulsoup4->nbconvert)
 (4.15.0)
 Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-
 packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (25.4.0)
 Requirement already satisfied: jsonschema-specifications>=2023.03.6 in
 /usr/local/lib/python3.12/dist-packages (from
 jsonschema>=2.6->nbformat>=5.7->nbconvert) (2025.9.1)
 Requirement already satisfied: referencing>=0.28.4 in
 /usr/local/lib/python3.12/dist-packages (from
 jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.37.0)
 Requirement already satisfied: rpds-py>=0.7.1 in /usr/local/lib/python3.12/dist-
 packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.29.0)
 Requirement already satisfied: entrypoints in /usr/local/lib/python3.12/dist-

packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (0.4)
 Requirement already satisfied: nest-asyncio>=1.5.4 in
 /usr/local/lib/python3.12/dist-packages (from jupyter-
 client>=6.1.12->nbclient>=0.5.0->nbconvert) (1.6.0)
 Requirement already satisfied: python-dateutil>=2.8.2 in
 /usr/local/lib/python3.12/dist-packages (from jupyter-
 client>=6.1.12->nbclient>=0.5.0->nbconvert) (2.9.0.post0)
 Requirement already satisfied: pyzmq>=23.0 in /usr/local/lib/python3.12/dist-
 packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (26.2.1)
 Requirement already satisfied: tornado>=6.2 in /usr/local/lib/python3.12/dist-
 packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (6.5.1)
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
 packages (from python-dateutil>=2.8.2->jupyter-
 client>=6.1.12->nbclient>=0.5.0->nbconvert) (1.17.0)
 Reading package lists... Done
 Building dependency tree... Done
 Reading state information... Done
 pandoc is already the newest version (2.9.2.1-3ubuntu2).
 pandoc set to manually installed.

The following additional packages will be installed:

dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
 fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
 libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
 libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
 libsynctex2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzip-0-13
 lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
 ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration t1utils
 teckit tex-common tex-gyre texlive-base texlive-binaries
 texlive-fonts-recommended texlive-latex-base texlive-latex-recommended
 texlive-pictures texlive-plain-generic tipa xfonts-encodings xfonts-utils

Suggested packages:

fonts-noto fonts-freefont-otf | fonts-freefont-ttf libavalon-framework-java
 libcommons-logging-java-doc libexcalibur-logkit-java liblog4j1.2-java
 poppler-utils ghostscript fonts-japanese-mincho | fonts-ipafont-mincho
 fonts-japanese-gothic | fonts-ipafont-gothic fonts-arphic-ukai
 fonts-arphic-uming fonts-nanum ri ruby-dev bundler debhelper gv
 | postscript-viewer perl-tk xpdf | pdf-viewer xzdec
 texlive-fonts-recommended-doc texlive-latex-base-doc python3-pygments
 icc-profiles libfile-which-perl libspreadsheet-parseexcel-perl
 texlive-latex-extra-doc texlive-latex-recommended-doc texlive-luatex
 texlive-pstricks dot2tex prerex texlive-pictures-doc vprerex
 default-jre-headless tipa-doc

The following NEW packages will be installed:

dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
 fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
 libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
 libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
 libsynctex2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzip-0-13

```

lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration tlutils
teckit tex-common tex-gyre texlive texlive-base texlive-binaries
texlive-fonts-recommended texlive-latex-base texlive-latex-extra
texlive-latex-recommended texlive-pictures texlive-plain-generic
texlive-xetex tipa xfonts-encodings xfonts-utils
0 upgraded, 54 newly installed, 0 to remove and 41 not upgraded.
Need to get 182 MB of archives.
After this operation, 571 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-droid-fallback all
1:6.0.1r16-1.1build1 [1,805 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-lato all 2.0-2.1
[2,696 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy/main amd64 poppler-data all
0.4.11-1 [2,171 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tex-common all 6.17
[33.7 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-urw-base35 all
20200910-1 [6,367 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libgs9-common
all 9.55.0~dfsg1-0ubuntu5.13 [753 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libidn12 amd64
1.38-4ubuntu1 [60.0 kB]
Get:8 http://archive.ubuntu.com/ubuntu jammy/main amd64 libijs-0.35 amd64
0.35-15build2 [16.5 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy/main amd64 libjbig2dec0 amd64
0.19-3build2 [64.7 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libgs9 amd64
9.55.0~dfsg1-0ubuntu5.13 [5,032 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libkpathsea6
amd64 2021.20210626.59705-1ubuntu0.2 [60.4 kB]
Get:12 http://archive.ubuntu.com/ubuntu jammy/main amd64 libwoff1 amd64
1.0.2-1build4 [45.2 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy/universe amd64 dvisvgm amd64
2.13.1-1 [1,221 kB]
Get:14 http://archive.ubuntu.com/ubuntu jammy/universe amd64 fonts-lmodern all
2.004.5-6.1 [4,532 kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-noto-mono all
20201225-1build1 [397 kB]
Get:16 http://archive.ubuntu.com/ubuntu jammy/universe amd64 fonts-texgyre all
20180621-3.1 [10.2 MB]
Get:17 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libapache-pom-java
all 18-1 [4,720 B]
Get:18 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libcommons-parent-
java all 43-1 [10.8 kB]
Get:19 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libcommons-logging-
java all 1.2-2 [60.3 kB]
Get:20 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libptexenc1

```

amd64 2021.20210626.59705-1ubuntu0.2 [39.1 kB]
 Get:21 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 rubygems-integration
 all 1.18 [5,336 B]
 Get:22 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby3.0 amd64
 3.0.2-7ubuntu2.11 [50.1 kB]
 Get:23 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-rubygems
 all 3.3.5-2ubuntu1.2 [228 kB]
 Get:24 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby amd64 1:3.0~exp1
 [5,100 B]
 Get:25 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 rake all 13.0.6-2 [61.7
 kB]
 Get:26 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby-net-telnet all
 0.1.1-2 [12.6 kB]
 Get:27 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-webrick
 all 1.7.0-3ubuntu0.2 [52.5 kB]
 Get:28 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-xmlrpc all
 0.3.2-1ubuntu0.1 [24.9 kB]
 Get:29 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libruby3.0
 amd64 3.0.2-7ubuntu2.11 [5,114 kB]
 Get:30 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libsynchronet2
 amd64 2021.20210626.59705-1ubuntu0.2 [55.6 kB]
 Get:31 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libteckit0 amd64
 2.5.11+ds1-1 [421 kB]
 Get:32 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libtexlua53
 amd64 2021.20210626.59705-1ubuntu0.2 [120 kB]
 Get:33 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libtexluajit2
 amd64 2021.20210626.59705-1ubuntu0.2 [267 kB]
 Get:34 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libzip-0-13 amd64
 0.13.72+dfsg.1-1.1 [27.0 kB]
 Get:35 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 xfonts-encodings all
 1:1.0.5-0ubuntu2 [578 kB]
 Get:36 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 xfonts-utils amd64
 1:7.7+6build2 [94.6 kB]
 Get:37 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 lmodern all
 2.004.5-6.1 [9,471 kB]
 Get:38 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 preview-latex-style
 all 12.2-1ubuntu1 [185 kB]
 Get:39 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 t1utils amd64
 1.41-4build2 [61.3 kB]
 Get:40 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 teckit amd64
 2.5.11+ds1-1 [699 kB]
 Get:41 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 tex-gyre all
 20180621-3.1 [6,209 kB]
 Get:42 <http://archive.ubuntu.com/ubuntu> jammy-updates/universe amd64 texlive-
 binaries amd64 2021.20210626.59705-1ubuntu0.2 [9,860 kB]
 Get:43 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-base all
 2021.20220204-1 [21.0 MB]
 Get:44 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-fonts-

```

recommended all 2021.20220204-1 [4,972 kB]
Get:45 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-base
all 2021.20220204-1 [1,128 kB]
Get:46 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-
recommended all 2021.20220204-1 [14.4 MB]
Get:47 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive all
2021.20220204-1 [14.3 kB]
Get:48 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libfontbox-java all
1:1.8.16-2 [207 kB]
Get:49 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libpdfbox-java all
1:1.8.16-2 [5,199 kB]
Get:50 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-pictures
all 2021.20220204-1 [8,720 kB]
Get:51 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-extra
all 2021.20220204-1 [13.9 MB]
Get:52 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-plain-
generic all 2021.20220204-1 [27.5 MB]
Get:53 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tipa all 2:1.3-21
[2,967 kB]
Get:54 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-xetex all
2021.20220204-1 [12.4 MB]
Fetched 182 MB in 8s (22.5 MB/s)
Extracting templates from packages: 100%
Preconfiguring packages ...
Selecting previously unselected package fonts-droid-fallback.
(Reading database ... 121713 files and directories currently installed.)
Preparing to unpack .../00-fonts-droid-fallback_1%3a6.0.1r16-1.1build1_all.deb
...
Unpacking fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Selecting previously unselected package fonts-lato.
Preparing to unpack .../01-fonts-lato_2.0-2.1_all.deb ...
Unpacking fonts-lato (2.0-2.1) ...
Selecting previously unselected package poppler-data.
Preparing to unpack .../02-poppler-data_0.4.11-1_all.deb ...
Unpacking poppler-data (0.4.11-1) ...
Selecting previously unselected package tex-common.
Preparing to unpack .../03-tex-common_6.17_all.deb ...
Unpacking tex-common (6.17) ...
Selecting previously unselected package fonts-urw-base35.
Preparing to unpack .../04-fonts-urw-base35_20200910-1_all.deb ...
Unpacking fonts-urw-base35 (20200910-1) ...
Selecting previously unselected package libgs9-common.
Preparing to unpack .../05-libgs9-common_9.55.0~dfsg1-0ubuntu5.13_all.deb ...
Unpacking libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libidn12:amd64.
Preparing to unpack .../06-libidn12_1.38-4ubuntu1_amd64.deb ...
Unpacking libidn12:amd64 (1.38-4ubuntu1) ...
Selecting previously unselected package libijs-0.35:amd64.

```



```

Preparing to unpack .../07-libijs-0.35_0.35-15build2_amd64.deb ...
Unpacking libijs-0.35:amd64 (0.35-15build2) ...
Selecting previously unselected package libjbig2dec0:amd64.
Preparing to unpack .../08-libjbig2dec0_0.19-3build2_amd64.deb ...
Unpacking libjbig2dec0:amd64 (0.19-3build2) ...
Selecting previously unselected package libgs9:amd64.
Preparing to unpack .../09-libgs9_9.55.0~dfsg1-0ubuntu5.13_amd64.deb ...
Unpacking libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libkpathsea6:amd64.
Preparing to unpack .../10-libkpathsea6_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libwoff1:amd64.
Preparing to unpack .../11-libwoff1_1.0.2-1build4_amd64.deb ...
Unpacking libwoff1:amd64 (1.0.2-1build4) ...
Selecting previously unselected package dvisvgm.
Preparing to unpack .../12-dvisvgm_2.13.1-1_amd64.deb ...
Unpacking dvisvgm (2.13.1-1) ...
Selecting previously unselected package fonts-lmodern.
Preparing to unpack .../13-fonts-lmodern_2.004.5-6.1_all.deb ...
Unpacking fonts-lmodern (2.004.5-6.1) ...
Selecting previously unselected package fonts-noto-mono.
Preparing to unpack .../14-fonts-noto-mono_20201225-1build1_all.deb ...
Unpacking fonts-noto-mono (20201225-1build1) ...
Selecting previously unselected package fonts-texgyre.
Preparing to unpack .../15-fonts-texgyre_20180621-3.1_all.deb ...
Unpacking fonts-texgyre (20180621-3.1) ...
Selecting previously unselected package libapache-pom-java.
Preparing to unpack .../16-libapache-pom-java_18-1_all.deb ...
Unpacking libapache-pom-java (18-1) ...
Selecting previously unselected package libcommons-parent-java.
Preparing to unpack .../17-libcommons-parent-java_43-1_all.deb ...
Unpacking libcommons-parent-java (43-1) ...
Selecting previously unselected package libcommons-logging-java.
Preparing to unpack .../18-libcommons-logging-java_1.2-2_all.deb ...
Unpacking libcommons-logging-java (1.2-2) ...
Selecting previously unselected package libptexenc1:amd64.
Preparing to unpack .../19-libptexenc1_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package rubygems-integration.
Preparing to unpack .../20-rubygems-integration_1.18_all.deb ...
Unpacking rubygems-integration (1.18) ...
Selecting previously unselected package ruby3.0.
Preparing to unpack .../21-ruby3.0_3.0.2-7ubuntu2.11_amd64.deb ...
Unpacking ruby3.0 (3.0.2-7ubuntu2.11) ...
Selecting previously unselected package ruby-rubygems.
Preparing to unpack .../22-ruby-rubygems_3.3.5-2ubuntu1.2_all.deb ...

```

```

Unpacking ruby-rubygems (3.3.5-2ubuntu1.2) ...
Selecting previously unselected package ruby.
Preparing to unpack .../23-ruby_1%3a3.0~exp1_amd64.deb ...
Unpacking ruby (1:3.0~exp1) ...
Selecting previously unselected package rake.
Preparing to unpack .../24-rake_13.0.6-2_all.deb ...
Unpacking rake (13.0.6-2) ...
Selecting previously unselected package ruby-net-telnet.
Preparing to unpack .../25-ruby-net-telnet_0.1.1-2_all.deb ...
Unpacking ruby-net-telnet (0.1.1-2) ...
Selecting previously unselected package ruby-webrick.
Preparing to unpack .../26-ruby-webrick_1.7.0-3ubuntu0.2_all.deb ...
Unpacking ruby-webrick (1.7.0-3ubuntu0.2) ...
Selecting previously unselected package ruby-xmlrpc.
Preparing to unpack .../27-ruby-xmlrpc_0.3.2-1ubuntu0.1_all.deb ...
Unpacking ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Selecting previously unselected package libruby3.0:amd64.
Preparing to unpack .../28-libruby3.0_3.0.2-7ubuntu2.11_amd64.deb ...
Unpacking libruby3.0:amd64 (3.0.2-7ubuntu2.11) ...
Selecting previously unselected package libsyntax2:amd64.
Preparing to unpack .../29-libsyntax2_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libsyntax2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libteckit0:amd64.
Preparing to unpack .../30-libteckit0_2.5.11+ds1-1_amd64.deb ...
Unpacking libteckit0:amd64 (2.5.11+ds1-1) ...
Selecting previously unselected package libtexlua53:amd64.
Preparing to unpack .../31-libtexlua53_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libtexluajit2:amd64.
Preparing to unpack
.../32-libtexluajit2_2021.20210626.59705-1ubuntu0.2_amd64.deb ...
Unpacking libtexluajit2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libzip-0-13:amd64.
Preparing to unpack .../33-libzip-0-13_0.13.72+dfsg.1-1.1_amd64.deb ...
Unpacking libzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Selecting previously unselected package xfonts-encodings.
Preparing to unpack .../34-xfonts-encodings_1%3a1.0.5-0ubuntu2_all.deb ...
Unpacking xfonts-encodings (1:1.0.5-0ubuntu2) ...
Selecting previously unselected package xfonts-utils.
Preparing to unpack .../35-xfonts-utils_1%3a7.7+6build2_amd64.deb ...
Unpacking xfonts-utils (1:7.7+6build2) ...
Selecting previously unselected package lmodern.
Preparing to unpack .../36-lmodern_2.004.5-6.1_all.deb ...
Unpacking lmodern (2.004.5-6.1) ...
Selecting previously unselected package preview-latex-style.
Preparing to unpack .../37-preview-latex-style_12.2-1ubuntu1_all.deb ...

```

```

Unpacking preview-latex-style (12.2-1ubuntu1) ...
Selecting previously unselected package t1utils.
Preparing to unpack .../38-t1utils_1.41-4build2_amd64.deb ...
Unpacking t1utils (1.41-4build2) ...
Selecting previously unselected package teckit.
Preparing to unpack .../39-teckit_2.5.11+ds1-1_amd64.deb ...
Unpacking teckit (2.5.11+ds1-1) ...
Selecting previously unselected package tex-gyre.
Preparing to unpack .../40-tex-gyre_20180621-3.1_all.deb ...
Unpacking tex-gyre (20180621-3.1) ...
Selecting previously unselected package texlive-binaries.
Preparing to unpack .../41-texlive-
binaries_2021.20210626.59705-1ubuntu0.2_amd64.deb ...
Unpacking texlive-binaries (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package texlive-base.
Preparing to unpack .../42-texlive-base_2021.20220204-1_all.deb ...
Unpacking texlive-base (2021.20220204-1) ...
Selecting previously unselected package texlive-fonts-recommended.
Preparing to unpack .../43-texlive-fonts-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-fonts-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-base.
Preparing to unpack .../44-texlive-latex-base_2021.20220204-1_all.deb ...
Unpacking texlive-latex-base (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-recommended.
Preparing to unpack .../45-texlive-latex-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-latex-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive.
Preparing to unpack .../46-texlive_2021.20220204-1_all.deb ...
Unpacking texlive (2021.20220204-1) ...
Selecting previously unselected package libfontbox-java.
Preparing to unpack .../47-libfontbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libfontbox-java (1:1.8.16-2) ...
Selecting previously unselected package libpdfbox-java.
Preparing to unpack .../48-libpdfbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libpdfbox-java (1:1.8.16-2) ...
Selecting previously unselected package texlive-pictures.
Preparing to unpack .../49-texlive-pictures_2021.20220204-1_all.deb ...
Unpacking texlive-pictures (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-extra.
Preparing to unpack .../50-texlive-latex-extra_2021.20220204-1_all.deb ...
Unpacking texlive-latex-extra (2021.20220204-1) ...
Selecting previously unselected package texlive-plain-generic.
Preparing to unpack .../51-texlive-plain-generic_2021.20220204-1_all.deb ...
Unpacking texlive-plain-generic (2021.20220204-1) ...
Selecting previously unselected package tipa.
Preparing to unpack .../52-tipa_2%3a1.3-21_all.deb ...
Unpacking tipa (2:1.3-21) ...
Selecting previously unselected package texlive-xetex.

```

```

Preparing to unpack .../53-texlive-xetex_2021.20220204-1_all.deb ...
Unpacking texlive-xetex (2021.20220204-1) ...
Setting up fonts-lato (2.0-2.1) ...
Setting up fonts-noto-mono (20201225-1build1) ...
Setting up libwoff1:amd64 (1.0.2-1build4) ...
Setting up libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up libijs-0.35:amd64 (0.35-15build2) ...
Setting up libtexluaajit2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up libfontbox-java (1:1.8.16-2) ...
Setting up rubygems-integration (1.18) ...
Setting up libzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Setting up fonts-urw-base35 (20200910-1) ...
Setting up poppler-data (0.4.11-1) ...
Setting up tex-common (6.17) ...
update-language: texlive-base not installed and configured, doing nothing!
Setting up libbig2dec0:amd64 (0.19-3build2) ...
Setting up libteckit0:amd64 (2.5.11+ds1-1) ...
Setting up libapache-pom-java (18-1) ...
Setting up ruby-net-telnet (0.1.1-2) ...
Setting up xfonts-encodings (1:1.0.5-0ubuntu2) ...
Setting up t1utils (1.41-4build2) ...
Setting up libidn12:amd64 (1.38-4ubuntu1) ...
Setting up fonts-texgyre (20180621-3.1) ...
Setting up libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up ruby-webrick (1.7.0-3ubuntu0.2) ...
Setting up fonts-lmodern (2.004.5-6.1) ...
Setting up fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Setting up ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Setting up libsynctex2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...
Setting up teckit (2.5.11+ds1-1) ...
Setting up libpdfbox-java (1:1.8.16-2) ...
Setting up libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Setting up preview-latex-style (12.2-1ubuntu1) ...
Setting up libcommons-parent-java (43-1) ...
Setting up dvisvgm (2.13.1-1) ...
Setting up libcommons-logging-java (1.2-2) ...
Setting up xfonts-utils (1:7.7+6build2) ...
Setting up libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up texlive-binaries (2021.20210626.59705-1ubuntu0.2) ...
update-alternatives: using /usr/bin/xdvi-xaw to provide /usr/bin/xdvi.bin
(xdvi.bin) in auto mode
update-alternatives: using /usr/bin/bibtex.original to provide /usr/bin/bibtex
(bibtex) in auto mode
Setting up lmodern (2.004.5-6.1) ...
Setting up texlive-base (2021.20220204-1) ...
/usr/bin/ucfr
/usr/bin/ucfr

```

```

/usr/bin/ucfr
/usr/bin/ucfr
mktexlsr: Updating /var/lib/texmf/ls-R-TEXLIVEDIST...
mktexlsr: Updating /var/lib/texmf/ls-R-TEXMFMAIN...
mktexlsr: Updating /var/lib/texmf/ls-R...
mktexlsr: Done.
tl-paper: setting paper size for dvips to a4:
/var/lib/texmf/dvips/config/config-paper.ps
tl-paper: setting paper size for dvipdfmx to a4:
/var/lib/texmf/dvipdfmx/dvipdfmx-paper.cfg
tl-paper: setting paper size for xdvi to a4: /var/lib/texmf/xdvi/XDvi-paper
tl-paper: setting paper size for pdftex to a4: /var/lib/texmf/tex/generic/tex-
ini-files/pdftexconfig.tex
Setting up tex-gyre (20180621-3.1) ...
Setting up texlive-plain-generic (2021.20220204-1) ...
Setting up texlive-latex-base (2021.20220204-1) ...
Setting up texlive-latex-recommended (2021.20220204-1) ...
Setting up texlive-pictures (2021.20220204-1) ...
Setting up texlive-fonts-recommended (2021.20220204-1) ...
Setting up tipa (2:1.3-21) ...
Setting up texlive (2021.20220204-1) ...
Setting up texlive-latex-extra (2021.20220204-1) ...
Setting up texlive-xetex (2021.20220204-1) ...
Setting up rake (13.0.6-2) ...
Setting up libruby3.0:amd64 (3.0.2-7ubuntu2.11) ...
Setting up ruby3.0 (3.0.2-7ubuntu2.11) ...
Setting up ruby (1:3.0~exp1) ...
Setting up ruby-rubygems (3.3.5-2ubuntu1.2) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for mailcap (3.70+nmulubuntu1) ...
Processing triggers for fontconfig (2.13.1-4.2ubuntu5) ...
Processing triggers for libc-bin (2.35-0ubuntu3.8) ...
/sbin/ldconfig.real: /usr/local/lib/libumf.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_5.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm_debug.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_0.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libhwloc.so.15 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero_v2.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc.so.2 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbb.so.12 is not a symbolic link

```

```

/sbin/ldconfig.real: /usr/local/lib/libur_loader.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc_proxy.so.2 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_opencl.so.0 is not a symbolic
link

Processing triggers for tex-common (6.17) ...
Running updmap-sys. This may take some time... done.
Running mktexlsr /var/lib/texmf ... done.
Building format(s) --all.
    This may take some time... done.

```

```
[122]: !jupyter nbconvert --to pdf Classification_Decision_Tree.ipynb
```

```

[NbConvertApp] WARNING | pattern 'Classification_Decision_Tree.ipynb' matched no
files
This application is used to convert notebook files (*.ipynb)
    to various other formats.

```

```
WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.
```

Options

```
=====
```

The options below are convenience aliases to configurable class-options, as listed in the "Equivalent to" description-line of the aliases.

To see all configurable class-options for some <cmd>, use:

```
<cmd> --help-all
```

--debug

set log level to logging.DEBUG (maximize logging output)

Equivalent to: [--Application.log_level=10]

--show-config

Show the application's configuration (human-readable format)

Equivalent to: [--Application.show_config=True]

--show-config-json

Show the application's configuration (json format)

Equivalent to: [--Application.show_config_json=True]

--generate-config

```

    generate default config file
    Equivalent to: [--JupyterApp.generate_config=True]
-y
    Answer yes to any questions instead of prompting.
    Equivalent to: [--JupyterApp.answer_yes=True]
--execute
    Execute the notebook prior to export.
    Equivalent to: [--ExecutePreprocessor.enabled=True]
--allow-errors
    Continue notebook execution even if one of the cells throws an error and
    include the error message in the cell output (the default behaviour is to abort
    conversion). This flag is only relevant if '--execute' was specified, too.
    Equivalent to: [--ExecutePreprocessor.allow_errors=True]
--stdin
    read a single notebook file from stdin. Write the resulting notebook with
    default basename 'notebook.*'
    Equivalent to: [--NbConvertApp.from_stdin=True]
--stdout
    Write notebook output to stdout instead of files.
    Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]
--inplace
    Run nbconvert in place, overwriting the existing notebook (only
    relevant when converting to notebook format)
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=]
--clear-output
    Clear output of current file and save in place,
    overwriting the existing notebook.
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=
--ClearOutputPreprocessor.enabled=True]
--coalesce-streams
    Coalesce consecutive stdout and stderr outputs into one stream (within each
    cell).
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=
--CoalesceStreamsPreprocessor.enabled=True]
--no-prompt
    Exclude input and output prompts from converted document.
    Equivalent to: [--TemplateExporter.exclude_input_prompt=True
--TemplateExporter.exclude_output_prompt=True]
--no-input
    Exclude input cells and output prompts from converted document.
    This mode is ideal for generating code-free reports.
    Equivalent to: [--TemplateExporter.exclude_output_prompt=True
--TemplateExporter.exclude_input=True
--TemplateExporter.exclude_input_prompt=True]
--allow-chromium-download

```

Whether to allow downloading chromium if no suitable version is found on the system.

Equivalent to: `[--WebPDFExporter.allow_chromium_download=True]`

`--disable-chromium-sandbox`

Disable chromium security sandbox when converting to PDF..

Equivalent to: `[--WebPDFExporter.disable_sandbox=True]`

`--show-input`

Shows code input. This flag is only useful for dejavu users.

Equivalent to: `[--TemplateExporter.exclude_input=False]`

`--embed-images`

Embed the images as base64 dataurls in the output. This flag is only useful for the HTML/WebPDF/Slides exports.

Equivalent to: `[--HTMLExporter.embed_images=True]`

`--sanitize-html`

Whether the HTML in Markdown cells and cell outputs should be sanitized..

Equivalent to: `[--HTMLExporter.sanitize_html=True]`

`--log-level=<Enum>`

Set the log level by value or name.

Choices: any of [0, 10, 20, 30, 40, 50, 'DEBUG', 'INFO', 'WARN', 'ERROR', 'CRITICAL']

Default: 30

Equivalent to: `[--Application.log_level]`

`--config=<Unicode>`

Full path of a config file.

Default: ''

Equivalent to: `[--JupyterApp.config_file]`

`--to=<Unicode>`

The export format to be used, either one of the built-in formats

['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf']

or a dotted object name that represents the import path for an

``Exporter`` class

Default: ''

Equivalent to: `[--NbConvertApp.export_format]`

`--template=<Unicode>`

Name of the template to use

Default: ''

Equivalent to: `[--TemplateExporter.template_name]`

`--template-file=<Unicode>`

Name of the template file to use

Default: None

Equivalent to: `[--TemplateExporter.template_file]`

`--theme=<Unicode>`

Template specific theme(e.g. the name of a JupyterLab CSS theme distributed as prebuilt extension for the lab template)

Default: 'light'

Equivalent to: `[--HTMLExporter.theme]`

`--sanitize_html=<Bool>`

Whether the HTML in Markdown cells and cell outputs should be sanitized. This should be set to True by nbviewer or similar tools.

Default: False

Equivalent to: [--HTMLExporter.sanitize_html]

--writer=<DottedObjectName>

Writer class used to write the results of the conversion

Default: 'FilesWriter'

Equivalent to: [--NbConvertApp.writer_class]

--post=<DottedOrNone>

PostProcessor class used to write the results of the conversion

Default: ''

Equivalent to: [--NbConvertApp.postprocessor_class]

--output=<Unicode>

Overwrite base name use for output files.

Supports pattern replacements '{notebook_name}'.

Default: '{notebook_name}'

Equivalent to: [--NbConvertApp.output_base]

--output-dir=<Unicode>

Directory to write output(s) to. Defaults to output to the directory of each notebook.

To recover previous default behaviour (outputting to the current working directory) use . as the flag value.

Default: ''

Equivalent to: [--FilesWriter.build_directory]

--reveal-prefix=<Unicode>

The URL prefix for reveal.js (version 3.x).

This defaults to the reveal CDN, but can be any url pointing to a copy of reveal.js.

For speaker notes to work, this must be a relative path to a local copy of reveal.js: e.g., "reveal.js".

If a relative path is given, it must be a subdirectory of the current directory (from which the server is run).

See the usage documentation (<https://nbconvert.readthedocs.io/en/latest/usage.html#reveal-js-html-slideshow>) for more details.

Default: ''

Equivalent to: [--SlidesExporter.reveal_url_prefix]

--nbformat=<Enum>

The nbformat version to write.

Use this to downgrade notebooks.

Choices: any of [1, 2, 3, 4]

Default: 4

Equivalent to: `[--NotebookExporter.nbformat_version]`

Examples

The simplest way to use nbconvert is

```
> jupyter nbconvert mynotebook.ipynb --to html
```

Options include ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf'].

```
> jupyter nbconvert --to latex mynotebook.ipynb
```

Both HTML and LaTeX support multiple output templates. LaTeX includes

'base', 'article' and 'report'. HTML includes 'basic', 'lab' and 'classic'. You can specify the flavor of the format used.

```
> jupyter nbconvert --to html --template lab mynotebook.ipynb
```

You can also pipe the output to stdout, rather than a file

```
> jupyter nbconvert mynotebook.ipynb --stdout
```

PDF is generated via latex

```
> jupyter nbconvert mynotebook.ipynb --to pdf
```

You can get (and serve) a Reveal.js-powered slideshow

```
> jupyter nbconvert myslides.ipynb --to slides --post serve
```

Multiple notebooks can be given at the command line in a couple of different ways:

```
> jupyter nbconvert notebook*.ipynb
> jupyter nbconvert notebook1.ipynb notebook2.ipynb
```

or you can specify the notebooks list in a config file, containing::

```
c.NbConvertApp.notebooks = ["my_notebook.ipynb"]
```

```
> jupyter nbconvert --config mycfg.py
```

To see all available configurables, use `--help-all`.

[124]: !ls -F

data/ sample_data/

[]: